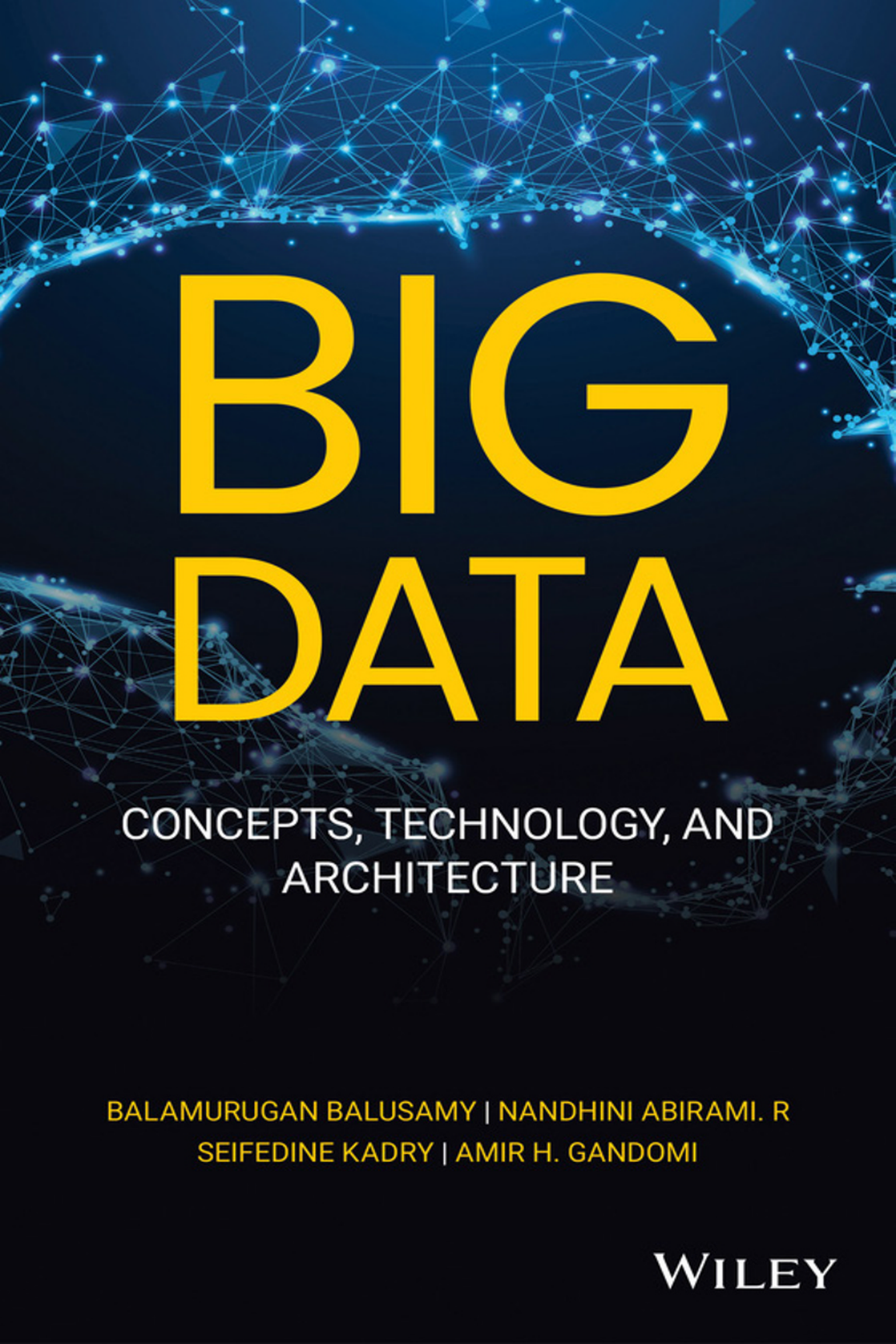




BIG DATA

CONCEPTS, TECHNOLOGY, AND
ARCHITECTURE

BALAMURUGAN BALUSAMY | NANDHINI ABIRAMI. R
SEIFEDINE KADRY | AMIR H. GANDOMI

WILEY

Big Data

Big Data

Concepts, Technology, and Architecture

*Balamurugan Balusamy, Nandhini Abirami. R,
Seifedine Kadry, and Amir H. Gandomi*

WILEY

This first edition first published 2021
© 2021 John Wiley & Sons, Inc.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, except as permitted by law. Advice on how to obtain permission to reuse material from this title is available at <http://www.wiley.com/go/permissions>.

The right of Balamurugan Balusamy, Nandhini Abirami. R, Seifedine Kadry, and Amir H. Gandomi to be identified as the author(s) of this work has been asserted in accordance with law.

Registered Office

John Wiley & Sons, Inc., 111 River Street, Hoboken, NJ 07030, USA

Editorial Office

111 River Street, Hoboken, NJ 07030, USA

For details of our global editorial offices, customer services, and more information about Wiley products visit us at www.wiley.com.

Wiley also publishes its books in a variety of electronic formats and by print-on-demand. Some content that appears in standard print versions of this book may not be available in other formats.

Limit of Liability/Disclaimer of Warranty

While the publisher and authors have used their best efforts in preparing this work, they make no representations or warranties with respect to the accuracy or completeness of the contents of this work and specifically disclaim all warranties, including without limitation any implied warranties of merchantability or fitness for a particular purpose. No warranty may be created or extended by sales representatives, written sales materials or promotional statements for this work. The fact that an organization, website, or product is referred to in this work as a citation and/or potential source of further information does not mean that the publisher and authors endorse the information or services the organization, website, or product may provide or recommendations it may make. This work is sold with the understanding that the publisher is not engaged in rendering professional services. The advice and strategies contained herein may not be suitable for your situation. You should consult with a specialist where appropriate. Further, readers should be aware that websites listed in this work may have changed or disappeared between when this work was written and when it is read. Neither the publisher nor authors shall be liable for any loss of profit or any other commercial damages, including but not limited to special, incidental, consequential, or other damages.

Library of Congress Cataloging-in-Publication Data Applied for:

ISBN 978-1-119-70182-8

Cover Design: Wiley

Cover Image: © Illus_man /Shutterstock

Set in 9.5/12.5pt STIXTwoText by SPi Global, Pondicherry, India

10 9 8 7 6 5 4 3 2 1

To My Dear SAIBABA, IDKM KALIAMMA, My Beloved Wife Dr. Deepa Muthiah, Sweet Daughter Rhea, My dear Mother Mrs. Andal, Supporting father Mr. M. Balusamy, and ever-loving sister Dr. Bhuvaneshwari Suresh. Without all these people, I am no one.

-Balamurugan Balusamy

To the people who mean a lot to me, my beloved daughter P. Rakshita, and my dear son P. Pranav Krishna.

-Nandhini Abirami. R

To My Family, and In Memory of My Grandparents Who Will Always Be In Our Hearts And Minds.

-Amir H. Gandomi

Contents

Acknowledgments *xi*

About the Author *xii*

1 Introduction to the World of Big Data 1

- 1.1 Understanding Big Data 1
- 1.2 Evolution of Big Data 2
- 1.3 Failure of Traditional Database in Handling Big Data 3
- 1.4 3 Vs of Big Data 4
- 1.5 Sources of Big Data 7
- 1.6 Different Types of Data 8
- 1.7 Big Data Infrastructure 11
- 1.8 Big Data Life Cycle 12
- 1.9 Big Data Technology 18
- 1.10 Big Data Applications 21
- 1.11 Big Data Use Cases 21
- Chapter 1 Refresher 24

2 Big Data Storage Concepts 31

- 2.1 Cluster Computing 32
- 2.2 Distribution Models 37
- 2.3 Distributed File System 43
- 2.4 Relational and Non-Relational Databases 43
- 2.5 Scaling Up and Scaling Out Storage 47
- Chapter 2 Refresher 48

3 NoSQL Database 53

- 3.1 Introduction to NoSQL 53
- 3.2 Why NoSQL 54
- 3.3 CAP Theorem 54
- 3.4 ACID 56

- 3.5 BASE 56
- 3.6 Schemaless Databases 57
- 3.7 NoSQL (Not Only SQL) 57
- 3.8 Migrating from RDBMS to NoSQL 76
- Chapter 3 Refresher 77

4 Processing, Management Concepts, and Cloud Computing 83

- Part I: Big Data Processing and Management Concepts 83
 - 4.1 Data Processing 83
 - 4.2 Shared Everything Architecture 85
 - 4.3 Shared-Nothing Architecture 86
 - 4.4 Batch Processing 88
 - 4.5 Real-Time Data Processing 88
 - 4.6 Parallel Computing 89
 - 4.7 Distributed Computing 90
 - 4.8 Big Data Virtualization 90
- Part II: Managing and Processing Big Data in Cloud Computing 93
 - 4.9 Introduction 93
 - 4.10 Cloud Computing Types 94
 - 4.11 Cloud Services 95
 - 4.12 Cloud Storage 96
 - 4.13 Cloud Architecture 101
- Chapter 4 Refresher 103

5 Driving Big Data with Hadoop Tools and Technologies 111

- 5.1 Apache Hadoop 111
- 5.2 Hadoop Storage 114
- 5.3 Hadoop Computation 119
- 5.4 Hadoop 2.0 129
- 5.5 HBASE 138
- 5.6 Apache Cassandra 141
- 5.7 SQOOP 141
- 5.8 Flume 143
- 5.9 Apache Avro 144
- 5.10 Apache Pig 145
- 5.11 Apache Mahout 146
- 5.12 Apache Oozie 146
- 5.13 Apache Hive 149

5.14	Hive Architecture	151
5.15	Hadoop Distributions	152
	Chapter 5 Refresher	153
6	Big Data Analytics	161
6.1	Terminology of Big Data Analytics	161
6.2	Big Data Analytics	162
6.3	Data Analytics Life Cycle	166
6.4	Big Data Analytics Techniques	170
6.5	Semantic Analysis	175
6.6	Visual analysis	178
6.7	Big Data Business Intelligence	178
6.8	Big Data Real-Time Analytics Processing	180
6.9	Enterprise Data Warehouse	181
	Chapter 6 Refresher	182
7	Big Data Analytics with Machine Learning	187
7.1	Introduction to Machine Learning	187
7.2	Machine Learning Use Cases	188
7.3	Types of Machine Learning	189
	Chapter 7 Refresher	196
8	Mining Data Streams and Frequent Itemset	201
8.1	Itemset Mining	201
8.2	Association Rules	206
8.3	Frequent Itemset Generation	210
8.4	Itemset Mining Algorithms	211
8.5	Maximal and Closed Frequent Itemset	229
8.6	Mining Maximal Frequent Itemsets: the GenMax Algorithm	233
8.7	Mining Closed Frequent Itemsets: the Charm Algorithm	236
8.8	CHARM Algorithm Implementation	236
8.9	Data Mining Methods	239
8.10	Prediction	240
8.11	Important Terms Used in Bayesian Network	241
8.12	Density Based Clustering Algorithm	249
8.13	DBSCAN	249
8.14	Kernel Density Estimation	250
8.15	Mining Data Streams	254
8.16	Time Series Forecasting	255

9	Cluster Analysis	259
9.1	Clustering	259
9.2	Distance Measurement Techniques	261
9.3	Hierarchical Clustering	263
9.4	Analysis of Protein Patterns in the Human Cancer-Associated Liver	266
9.5	Recognition Using Biometrics of Hands	267
9.6	Expectation Maximization Clustering Algorithm	274
9.7	Representative-Based Clustering	277
9.8	Methods of Determining the Number of Clusters	277
9.9	Optimization Algorithm	284
9.10	Choosing the Number of Clusters	288
9.11	Bayesian Analysis of Mixtures	290
9.12	Fuzzy Clustering	290
9.13	Fuzzy C-Means Clustering	291
10	Big Data Visualization	293
10.1	Big Data Visualization	293
10.2	Conventional Data Visualization Techniques	294
10.3	Tableau	297
10.4	Bar Chart in Tableau	309
10.5	Line Chart	310
10.6	Pie Chart	311
10.7	Bubble Chart	312
10.8	Box Plot	313
10.9	Tableau Use Cases	313
10.10	Installing R and Getting Ready	318
10.11	Data Structures in R	321
10.12	Importing Data from a File	335
10.13	Importing Data from a Delimited Text File	336
10.14	Control Structures in R	337
10.15	Basic Graphs in R	341
	Index	347

Acknowledgments

Writing a book is harder than I thought and more rewarding than I could have ever imagined. None of this would have been possible without my family. I wish to extend my profound gratitude to my father, Mr. N. J. Rajendran and my mother, Mrs. Mallika Rajendran, for their moral support. I salute you for the selfless love, care, pain, and sacrifice you did to shape my life. Special mention goes to my father, who supported throughout my education, career, and encouraged me to pursue my higher studies. It is my fortune to gratefully acknowledge my sisters, DR. R. Vidhya Lakshmi and Mrs. R. Rajalakshmi Priyanka, for their support and generous care throughout my education and career. They were always beside me during the happy and hard moments to push me and motivate me. With great pleasure, I acknowledge the people who mean a lot to me, my beloved daughter P. Rakshita, and my dear son P. Pranav Krishna without whose cooperation, writing this book would not be possible. I owe thanks to a very special person, my husband, Mr. N. Pradeep, for his continued and unfailing support and understanding. I would like to extend my love and thanks to my dears, Nila Nagarajan, Akshara Nagarajan, Vaibhav Surendran, and Nivin Surendran. I would also like to thank my mother-in-law Mrs. Thenmozhi Nagarajan who supported me in all possible means to pursue my career.

About the Author

Balamurugan Balusamy is the professor of Data Sciences and Chief Research Coordinator at Galgotias University, NCR, India. His research focuses on the role of Data Sciences in various domains. He is the author of over a hundred Journal papers and book chapters on Data Sciences, IoT, and Blockchain. He has chaired many International Conferences and had given multiple Keynote addresses in Top Notch conferences across the Globe. He holds a Doctorate degree, masters, and Bachelors degree in Computer Science and Engineering from Premier Institutions. In his spare time, he likes to do Yoga and Meditation.

Nandhini Abirami R is a first year PhD student and a Research Associate in the School of Information Technology at Vellore Institute of Technology. Her doctoral research investigates the advancement and effectiveness of Generative Adversarial Network in computer vision. She takes a multidisciplinary approach that encompasses the fields of healthcare and human computer interaction. She holds a master's degree in Information Technology from Vellore Institute of Technology, which investigated the effectiveness of machine learning algorithms in predicting heart disease. She worked as Assistant Systems Engineer at Tata Consultancy Services.

Amir H. Gandomi is a Professor of Data Science and an ARC DECRA Fellow at the Faculty of Engineering & Information Technology, University of Technology Sydney. Prior to joining UTS, Prof. Gandomi was an Assistant Professor at Stevens Institute of Technology, USA and a distinguished research fellow in BEACON center, Michigan State University, USA. Prof. Gandomi has published over two hundred journal papers and seven books which collectively have been cited 19,000+ times (H-index = 64). He has been named as one of the most influential scientific mind and Highly Cited Researcher (top 1% publications and 0.1% researchers) for four consecutive years, 2017 to 2020. He also ranked 18th in GP bibliography among more than 12,000 researchers. He has served as associate editor, editor and guest editor in several prestigious journals such as AE of SWEVO, IEEE TBD, and IEEE IoTJ. Prof Gandomi is active in delivering keynotes and invited talks. His research interests are global optimisation and (big) data analytics using machine learning and evolutionary computations in particular.

1

Introduction to the World of Big Data

CHAPTER OBJECTIVE

This chapter deals with the introduction to big data, defining what actually big data means. The limitations of the traditional database, which led to the evolution of Big Data, are explained, and insight into big data key concepts is delivered. A comparative study is made between big data and traditional database giving a clear picture of the drawbacks of the traditional database and advantages of big data. The three Vs of big data (volume, velocity, and variety) that distinguish it from the traditional database are explained. With the evolution of big data, we are no longer limited to the structured data. The different types of human- and machine-generated data—that is, structured, semi-structured, and unstructured—that can be handled by big data are explained. The various sources contributing to this massive volume of data are given a clear picture. The chapter expands to show the various stages of big data life cycle starting from data generation, acquisition, preprocessing, integration, cleaning, transformation, analysis, and visualization to make business decisions. This chapter sheds light on various challenges of big data due to its heterogeneity, volume, velocity, and more.

1.1 Understanding Big Data

With the rapid growth of Internet users, there is an exponential growth in the data being generated. The data is generated from millions of messages we send and communicate via WhatsApp, Facebook, or Twitter, from the trillions of photos taken, and hours and hours of videos getting uploaded in YouTube every single minute. According to a recent survey 2.5 quintillion (2 500 000 000 000 000 000, or 2.5×10^{18}) bytes of data are generated every day. This enormous amount of data generated is referred to as “big data.” Big data does not only mean that the data sets are too large, it is a blanket term for the data that are too large in size, complex in nature, which may be structured or

unstructured, and arriving at high velocity as well. Of the data available today, 80 percent has been generated in the last few years. The growth of big data is fueled by the fact that more data are generated on every corner of the world that needs to be captured.

Capturing this massive data gives only meager value unless this IT value is transformed into business value. Managing the data and analyzing them have always been beneficial to the organizations; on the other hand, converting these data into valuable business insights has always been the greatest challenge. Data scientists were struggling to find pragmatic techniques to analyze the captured data. The data has to be managed at appropriate speed and time to derive valuable insight from it. These data are so complex that it became difficult to process it using traditional database management systems, which triggered the evolution of the big data era. Additionally, there were constraints on the amount of data that traditional databases could handle. With the increase in the size of data either there was a decrease in performance and increase in latency or it was expensive to add additional memory units. All these limitations have been overcome with the evolution of big data technologies that lets us capture, store, process, and analyze the data in a distributed environment. Examples of Big data technologies are Hadoop, a framework for all big data process, Hadoop Distributed File System (HDFS) for distributed cluster storage, and MapReduce for processing.

1.2 Evolution of Big Data

The first documentary appearance of big data was in a paper in 1997 by NASA scientists narrating the problems faced in visualizing large data sets, which were a captivating challenge for the data scientists. The data sets were large enough, taxing more memory resources. This problem is termed big data. Big data, the broader concept, was first put forward by a noted consultancy: McKinsey. The three dimensions of big data, namely, volume, velocity, and variety, were defined by analyst Doug Laney. The processing life cycle of big data can be categorized into acquisition, preprocessing, storage and management, privacy and security, analyzing, and visualization.

The broader term big data encompasses everything that includes web data, such as click stream data, health data of patients, genomic data from biologic research, and so forth.

Figure 1.1 shows the evolution of big data. The growth of the data over the years is massive. It was just 600 MB in the 1950s but has grown by 2010 up to 100 petabytes, which is equal to 100 000 000 000 MB.

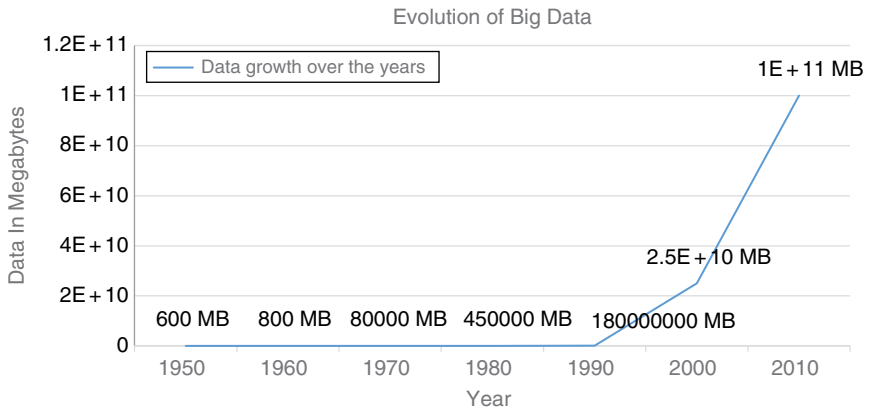


Figure 1.1 Evolution of Big Data.

1.3 Failure of Traditional Database in Handling Big Data

The Relational Database Management Systems (RDBMS) was the most prevalent data storage medium until recently to store the data generated by the organizations. A large number of vendors provide database systems. These RDBMS were devised to store the data that were beyond the storage capacity of a single computer. The inception of a new technology is always due to limitations in the older technologies and the necessity to overcome them. Below are the limitations of traditional database in handling big data.

- Exponential increase in data volume, which scales in terabytes and petabytes, has turned out to become a challenge to the RDBMS in handling such a massive volume of data.
- To address this issue, the RDBMS increased the number of processors and added more memory units, which in turn increased the cost.
- Almost 80% of the data fetched were of semi-structured and unstructured format, which RDBMS could not deal with.
- RDBMS could not capture the data coming in at high velocity.

Table 1.1 shows the differences in the attributes of RDBMS and big data.

1.3.1 Data Mining vs. Big Data

Table 1.2 shows a comparison between data mining and big data.

Table 1.1 Differences in the attributes of big data and RDBMS.

ATTRIBUTES	RDBMS	BIG DATA
Data volume	gigabytes to terabytes	petabytes to zettabytes
Organization	centralized	distributed
Data type	structured	unstructured and semi-structured
Hardware type	high-end model	commodity hardware
Updates	read/write many times	write once, read many times
Schema	static	dynamic

Table 1.2 Data Mining vs. Big Data.

S. No.	Data mining	Big data
1)	Data mining is the process of discovering the underlying knowledge from the data sets.	Big data refers to massive volume of data characterized by volume, velocity, and variety.
2)	Structured data retrieved from spread sheets, relational databases, etc.	Structured, unstructured, or semi-structured data retrieved from non-relational databases, such as NoSQL.
3)	Data mining is capable of processing large data sets, but the data processing costs are high.	Big data tools and technologies are capable of storing and processing large volumes of data at a comparatively lower cost.
4)	Data mining can process only data sets that range from gigabytes to terabytes.	Big data technology is capable of storing and processing data that range from petabytes to zettabytes.

1.4 3 Vs of Big Data

Big data is distinguished by its exceptional characteristics with various dimensions. Figure 1.2 illustrates various dimensions of big data. The first of its dimensions is the size of the data. Data size grows partially because the cluster storage with commodity hardware has made it cost effective. Commodity hardware is a low cost, low performance, and low specification functional hardware with no distinctive features. This is referred by the term “volume” in big data technology. The second dimension is the variety, which describes its heterogeneity to accept all the data types, be it structured, unstructured, or a mix of both. The third dimension is velocity, which relates to the rate at which the data is generated and being processed to derive the desired value out of the raw unprocessed data.

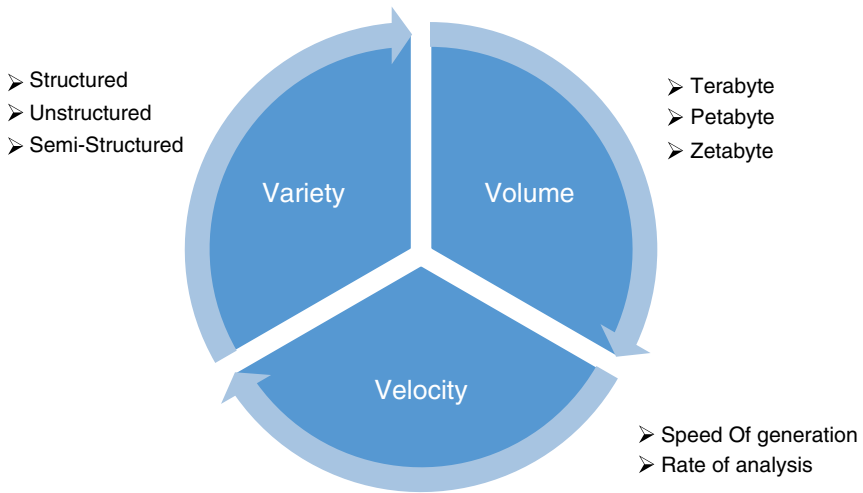


Figure 1.2 3 Vs of big data.

The complexities of the data captured pose a new opportunity as well as a challenge for today's information technology era.

1.4.1 Volume

Data generated and processed by big data are continuously growing at an ever increasing pace. Volume grows exponentially owing to the fact that business enterprises are continuously capturing the data to make better and bigger business solutions. Big data volume measures from terabytes to zettabytes (1024 GB = 1 terabyte; 1024 TB = 1 petabyte; 1024 PB = 1 exabyte; 1024 EB = 1 zettabyte; 1024 ZB = 1 yottabyte). Capturing this massive data is cited as an extraordinary opportunity to achieve finer customer service and better business advantage. This ever increasing data volume demands highly scalable and reliable storage. The major sources contributing to this tremendous growth in the volume are social media, point of sale (POS) transactions, online banking, GPS sensors, and sensors in vehicles. Facebook generates approximately 500 terabytes of data per day. Every time a link on a website is clicked, an item is purchased online, a video is uploaded in YouTube, data are generated.

1.4.2 Velocity

With the dramatic increase in the volume of data, the speed at which the data is generated also surged up. The term “velocity” not only refers to the speed at which data are generated, it also refers to the rate at which data is processed and

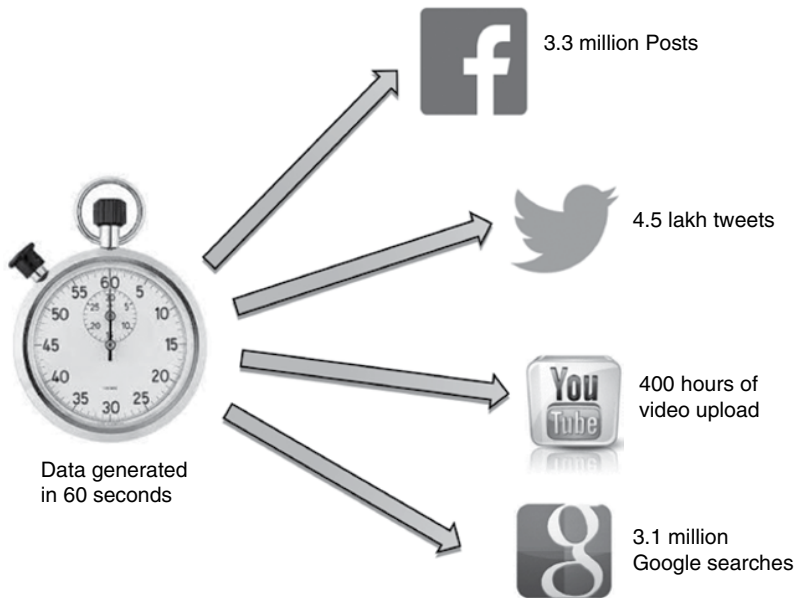


Figure 1.3 High-velocity data sets generated online in 60 seconds.

analyzed. In the big data era, a massive amount of data is generated at high velocity, and sometimes these data arrive so fast that it becomes difficult to capture them, and yet the data needs to be analyzed. Figure 1.3 illustrates the data generated with high velocity in 60 seconds: 3.3 million Facebook posts, 450 thousand tweets, 400 hours of video upload, and 3.1 million Google searches.

1.4.3 Variety

Variety refers to the format of data supported by big data. Data arrives in structured, semi-structured, and unstructured format. Structured data refers to the data processed by traditional database management systems where the data are organized in tables, such as employee details, bank customer details. Semi-structured data is a combination of structured and unstructured data, such as XML. XML data is semi-structured since it does not fit the formal data model (table) associated with traditional database; rather, it contains tags to organize fields within the data. Unstructured data refers to data with no definite structure, such as e-mail messages, photos, and web pages. The data that arrive from Facebook, Twitter feeds, sensors of vehicles, and black boxes of airplanes are all



Figure 1.4 Big data—data variety.

unstructured, which the traditional database cannot process, and here is when big data comes into the picture. Figure 1.4 represents the different data types.

1.5 Sources of Big Data

Multiple disparate data sources are responsible for the tremendous increase in the volume of big data. Much of the growth in data can be attributed to the digitization of almost anything and everything in the globe. Paying E-bills, online shopping, communication through social media, e-mail transactions in various organizations, a digital representation of the organizational data, and so forth, are some of the examples of this digitization around the globe.

- **Sensors:** Sensors that contribute to the large volume of big data are listed below.
 - *Accelerometer sensors* installed in mobile devices to sense the vibrations and other movements.
 - *Proximity Sensors* used in public places to detect the presence of objects without physical contact with the objects.
 - Sensors in vehicles and medical devices.
- **Health care:** The major sources of big data in health care are:
 - *Electronic Health Records (EHRs)* collect and display patient information such as past medical history, prescriptions by the medical practitioners, and laboratory test results.
 - *Patient portals* permit patients to access their personal medical records saved in EHRs.
 - *Clinical data repository* aggregates individual patient records from various clinical sources and consolidates them to give a unified view of patient history.
- **Black box:** Data are generated by the black box in airplanes, helicopters, and jets. The black box captures the activities of flight, flight crew announcements, and aircraft performance information.

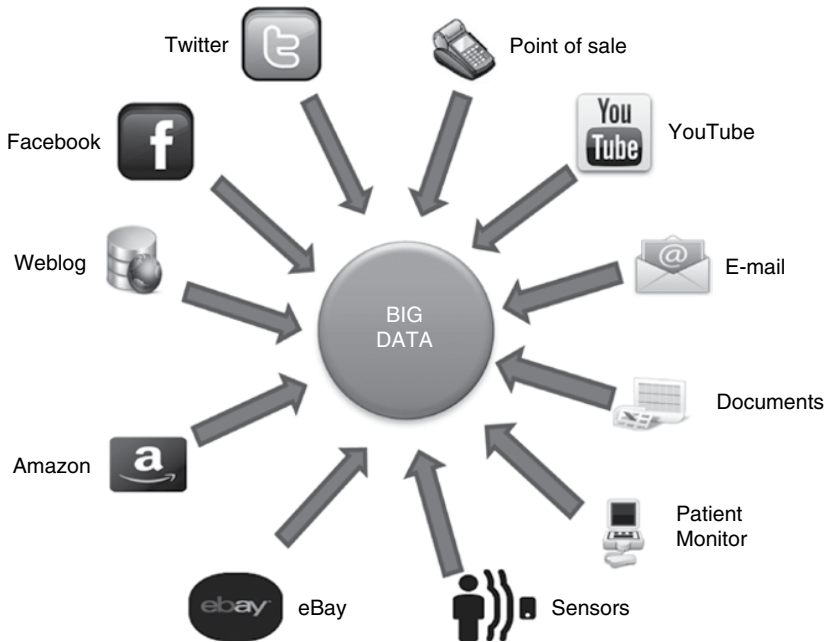


Figure 1.5 Sources of big data.

- **Web data:** Data generated on clicking a link on a website is captured by the online retailers. This is perform click stream analysis to analyze customer interest and buying patterns to generate recommendations based on the customer interests and to post relevant advertisements to the consumers.
- **Organizational data:** E-mail transactions and documents that are generated within the organizations together contribute to the organizational data.

Figure 1.5 illustrates the data generated by various sources that were discussed above.

1.6 Different Types of Data

Data may be machine generated or human generated. Human-generated data refers to the data generated as an outcome of interactions of humans with the machines. E-mails, documents, Facebook posts are some of the human-generated data. Machine-generated data refers to the data generated by computer applications or hardware devices without active human intervention. Data from sensors, disaster warning systems, weather forecasting systems, and satellite data are some of the machine-generated data. Figure 1.6 represents the data generated by a

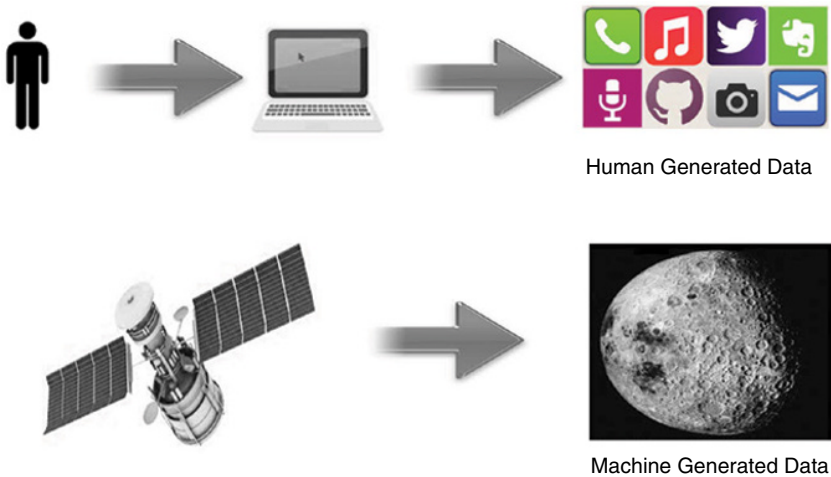


Figure 1.6 Human- and machine-generated data.

human in various social media, e-mails sent, and pictures that were taken by them and machine data generated by the satellite.

The machine-generated and human-generated data can be represented by the following primitive types of big data:

- Structured data
- Unstructured data
- Semi-structured data

1.6.1 Structured Data

Data that can be stored in a relational database in table format with rows and columns is called structured data. Structured data often generated by business enterprises exhibits a high degree of organization and can easily be processed using data mining tools and can be queried and retrieved using the primary key field. Examples of structured data include employee details and financial transactions. Figure 1.7 shows an example of structured data, employee details table with EmployeeID as the key.

1.6.2 Unstructured Data

Data that are raw, unorganized, and do not fit into the relational database systems are called unstructured data. Nearly 80% of the data generated are unstructured. Examples of unstructured data include video, audio, images, e-mails, text files,

Employee ID	Employee Name	Sex	Salary
334332	Daniel	Male	\$2300
334333	John	Male	\$2000
338332	Michael	Male	\$2800
339232	Diana	Female	\$1800
337891	Joseph	Male	\$3800
339876	Agnes	Female	\$4000

Figure 1.7 Structured data—employee details of an organization.

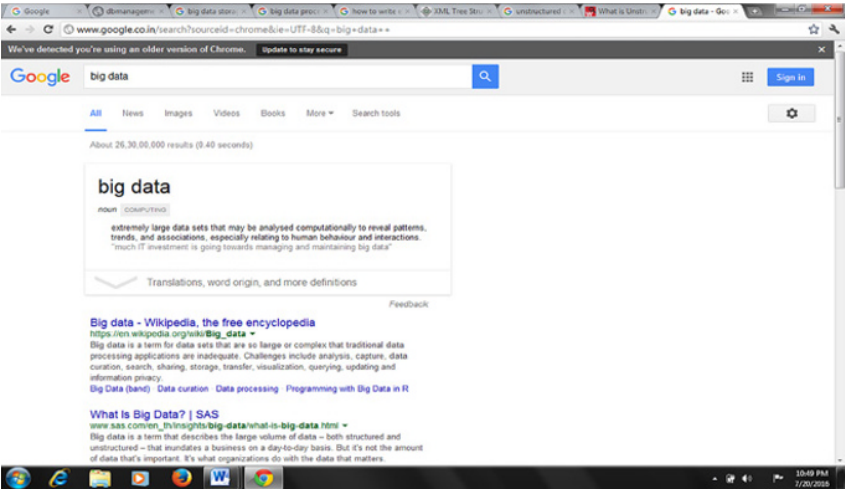


Figure 1.8 Unstructured data—the result of a Google search.

and social media posts. Unstructured data usually reside on either text files or binary files. Data that reside in binary files do not have any identifiable internal structure, for example, audio, video, and images. Data that reside in text files are e-mails, social media posts, pdf files, and word processing documents. Figure 1.8 shows unstructured data, the result of a Google search.

1.6.3 Semi-Structured Data

Semi-structured data are those that have a structure but do not fit into the relational database. Semi-structured data are organized, which makes it easier to analyze when compared to unstructured data. JSON and XML are examples of semi-structured data. Figure 1.9 is an XML file that represents the details of an employee in an organization.

```

<?xml version = "1.0"?>
<Company>
<Employee>
    <EmployeeId>339876</EmployeeId>
    <FirstName>Joseph</FirstName>
    <LastName>Agnes</LastName>
    <Sex>Female</Sex>
    <Salary>$4000</Salary>
</Employee>
</Company>

```

Figure 1.9 XML file with employee details.

1.7 Big Data Infrastructure

The core components of big data technologies are the tools and technologies that provide the capacity to store, process, and analyze the data. The method of storing the data in tables was no longer supportive with the evolution of data with 3 Vs, namely volume, velocity, and variety. The robust RDBMS was no longer cost effective. The scaling of RDBMS to store and process huge amount of data became expensive. This led to the emergence of new technology, which was highly scalable at very low cost.

The key technologies include

- Hadoop
- HDFS
- MapReduce

Hadoop – Apache Hadoop, written in Java, is open-source framework that supports processing of large data sets. It can store a large volume of structured, semi-structured, and unstructured data in a distributed file system and process them in parallel. It is a highly scalable and cost-effective storage platform. Scalability of Hadoop refers to its capability to sustain its performance even under highly increasing loads by adding more nodes. Hadoop files are written once and read many times. The contents of the files cannot be changed. A large number of computers interconnected working together as a single system is called a cluster. Hadoop clusters are designed to store and analyze the massive amount of disparate data in distributed computing environments in a cost effective manner.

Hadoop Distributed File system – HDFS is designed to store large data sets with streaming access pattern running on low-cost commodity hardware. It does not require highly reliable, expensive hardware. The data set is generated from multiple sources, stored in an HDFS file system in a write-once, read-many-times pattern, and analyses are performed on the data set to extract knowledge from it.

MapReduce – MapReduce is the batch-processing programming model for the Hadoop framework, which adopts a divide-and-conquer principle. It is highly scalable, reliable, and fault tolerant, capable of processing input data with any format in parallel and distributed computing environments supporting only batch workloads. Its performance reduces the processing time significantly compared to the traditional batch-processing paradigm, as the traditional approach was to move the data from the storage platform to the processing platform, whereas the MapReduce processing paradigm resides in the framework where the data actually resides.

1.8 Big Data Life Cycle

Big data yields big benefits, starting from innovative business ideas to unconventional ways to treat diseases, overcoming the challenges. The challenges arise because so much of the data is collected by the technology today. Big data technologies are capable of capturing and analyzing them effectively. Big data infrastructure involves new computing models with the capability to process both distributed and parallel computations with highly scalable storage and performance. Some of the big data components include Hadoop (framework), HDFS (storage), and MapReduce (processing).

Figure 1.10 illustrates the big data life cycle. Data arriving at high velocity from multiple sources with different data formats are captured. The captured data is stored in a storage platform such as HDFS and NoSQL and then preprocessed to make the data suitable for analysis. The preprocessed data stored in the storage platform is then passed to the analytics layer, where the data is processed using big data tools such as MapReduce and YARN and analysis is performed on the processed data to uncover hidden knowledge from it. Analytics and machine learning are important concepts in the life cycle of big data. Text analytics is a type of analysis performed on unstructured textual data. With the growth of social media and e-mail transactions, the importance of text analytics has surged up. Predictive analysis on consumer behavior and consumer interest analysis are all performed on the text data extracted from various online sources such as social media, online retailing websites, and much more. Machine learning has made text analytics possible. The analyzed data is visually represented by visualization tools such as Tableau to make it easily understandable by the end user to make decisions.

1.8.1 Big Data Generation

The first phase of the life cycle of big data is the data generation. The scale of data generated from diversified sources is gradually expanding. Sources of this large volume of data were discussed under the Section 1.5, “Sources of Big Data.”

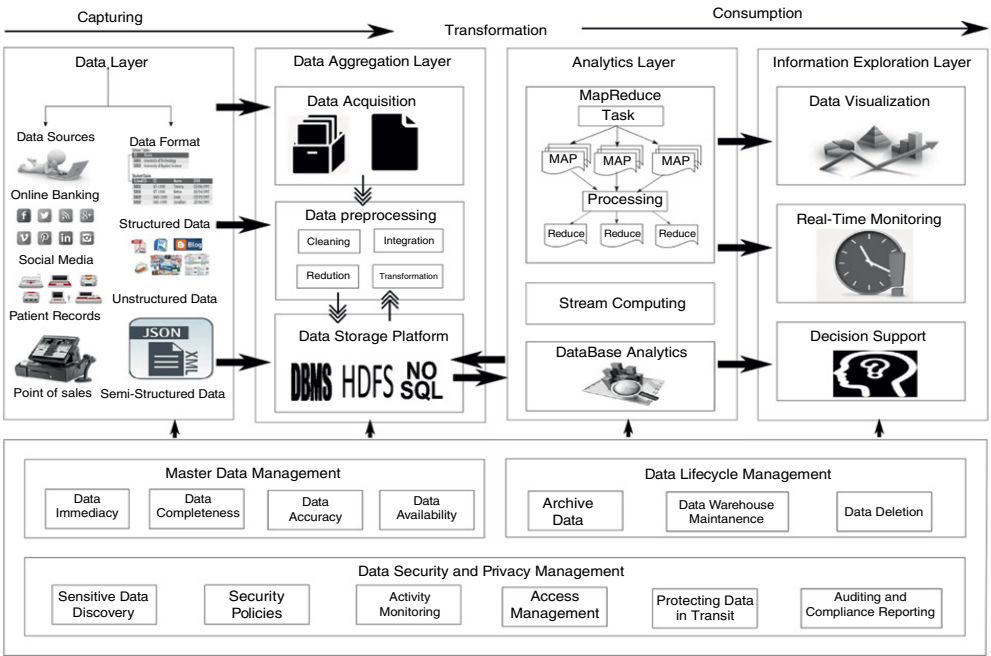


Figure 1.10 Big data life cycle.

1.8.2 Data Aggregation

The data aggregation phase of the big data life cycle involves collecting the raw data, transmitting the data to the storage platform, and preprocessing them. Data acquisition in the big data world means acquiring the high-volume data arriving at an ever-increasing pace. The raw data thus collected is transmitted to a proper storage infrastructure to support processing and various analytical applications. Preprocessing involves data cleansing, data integration, data transformation, and data reduction to make the data reliable, error free, consistent, and accurate. The data gathered may have redundancies, which occupy the storage space and increase the storage cost and can be handled by data preprocessing. Also, much of the data gathered may not be related to the analysis objective, and hence it needs to be compressed while being preprocessed. Hence, efficient data preprocessing is indispensable for cost-effective and efficient data storage. The preprocessed data are then transmitted for various purposes such as data modeling and data analytics.

1.8.3 Data Preprocessing

Data preprocessing is an important process performed on raw data to transform it into an understandable format and provide access to consistent and accurate data. The data generated from multiple sources are erroneous, incomplete, and inconsistent because of their massive volume and heterogeneous sources, and it is meaningless to store useless and dirty data. Additionally, some analytical applications have a crucial requirement for quality data. Hence, for effective, efficient, and accurate data analysis, systematic data preprocessing is essential. The quality of the source data is affected by various factors. For instance, the data may have errors such as a salary field having a negative value (e.g., salary = -2000), which arises because of transmission errors or typos or intentional wrong data entry by users who do not wish to disclose their personal information. Incompleteness implies that the field lacks the attributes of interest (e.g., Education = ""), which may come from a not applicable field or software errors. Inconsistency in the data refers to the discrepancies in the data, say date of birth and age may be inconsistent. Inconsistencies in data arise when the data collected are from different sources, because of inconsistencies in naming conventions between different countries and inconsistencies in the input format (e.g., date field DD/MM when interpreted as MM/DD). Data sources often have redundant data in different forms, and hence duplicates in the data also have to be removed in data preprocessing to make the data meaningful and error free. There are several steps involved in data preprocessing:

- 1) Data integration
- 2) Data cleaning
- 3) Data reduction
- 4) Data transformation

1.8.3.1 Data Integration

Data integration involves combining data from different sources to give the end users a unified data view. Several challenges are faced while integrating data; as an example, while extracting data from the profile of a person, the first name and family name may be interchanged in a certain culture, so in such cases integration may happen incorrectly. Data redundancies often occur while integrating data from multiple sources. Figure 1.11 illustrates that diversified sources such as organizations, smartphones, personal computers, satellites, and sensors generate disparate data such as e-mails, employee details, WhatsApp chat messages, social media posts, online transactions, satellite images, and sensory data. These different types of structured, unstructured, and semi-structured data have to be integrated and presented as unified data for data cleansing, data modeling, data warehousing, and to extract, transform, and load (ETL) the data.

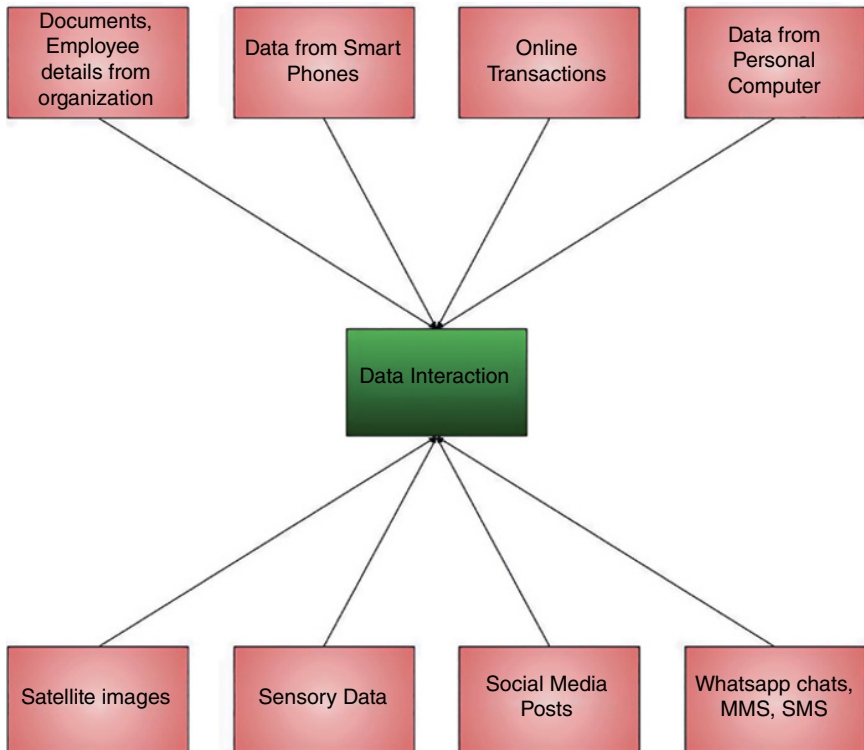


Figure 1.11 Data integration.

1.8.3.2 Data Cleaning

The data-cleaning process fills in the missing values, corrects the errors and inconsistencies, and removes redundancy in the data to improve the data quality. The larger the heterogeneity of the data sources, the higher the degree of dirtiness. Consequently, more cleaning steps may be involved. Data cleaning involves several steps such as spotting or identifying the error, correcting the error or deleting the erroneous data, and documenting the error type. To detect the type of error and inconsistency present in the data, a detailed analysis of the data is required. Data redundancy is the data repetition, which increases storage cost and transmission expenses and decreases data accuracy and reliability. The various techniques involved in handling data redundancy are redundancy detection and data compression. Missing values can be filled in manually, but it is tedious, time-consuming, and not appropriate for the massive volume of data. A global constant can be used to fill in all the missing values, but this method creates issues while integrating the data; hence, it is not a foolproof method. Noisy data can be handled by four methods, namely, regression, clustering, binning, and manual inspection.

1.8.3.3 Data Reduction

Data processing on massive data volume may take a long time, making data analysis either infeasible or impractical. Data reduction is the concept of reducing the volume of data or reducing the dimension of the data, that is, the number of attributes. Data reduction techniques are adopted to analyze the data in reduced format without losing the integrity of the actual data and yet yield quality outputs. Data reduction techniques include data compression, dimensionality reduction, and numerosity reduction. Data compression techniques are applied to obtain the compressed or reduced representation of the actual data. If the original data is retrieved back from the data that is being compressed without any loss of information, then it is called lossless data reduction. On the other hand, if the data retrieval is only partial, then it is called lossy data reduction. Dimensionality reduction is the reduction of a number of attributes, and the techniques include wavelet transforms where the original data is projected into a smaller space and attribute subset selection, a method which involves removal of irrelevant or redundant attributes. Numerosity reduction is a technique adopted to reduce the volume by choosing smaller alternative data. Numerosity reduction is implemented using parametric and nonparametric methods. In parametric methods instead of storing the actual data, only the parameters are stored. Nonparametric methods stores reduced representations of the original data.

1.8.3.4 Data Transformation

Data transformation refers to transforming or consolidating the data into an appropriate format and converting them into logical and meaningful information for data management and analysis. The real challenge in data transformation

comes into the picture when fields in one system do not match the fields in another system. Before data transformation, data cleaning and manipulation takes place. Organizations are collecting a massive amount of data, and the volume of the data is increasing rapidly. The data captured are transformed using ETL tools.

Data transformation involves the following strategies:

Smoothing, which removes noise from the data by incorporating binning, clustering, and regression techniques.

Aggregation, which applies summary or aggregation on the data to give a consolidated data. (E.g., daily profit of an organization may be aggregated to give consolidated monthly or yearly turnover.)

Generalization, which is normally viewed as climbing up the hierarchy where the attributes are generalized to a higher level overlooking the attributes at a lower level. (E.g., street name may be generalized as city name or a higher level hierarchy, namely the country name).

Discretization, which is a technique where raw values in the data (e.g., age) are replaced by conceptual labels (e.g., teen, adult, senior) or interval labels (e.g., 0–9, 10–19, etc.)

1.8.4 Big Data Analytics

Businesses are recognizing the unrevealed potential value of this massive data and putting forward the tools and technologies to capitalize on the opportunity. The key to deriving business value from big data is the potential use of analytics. Collecting, storing, and preprocessing the data creates a little value. It has to be analyzed and the end users must make decisions out of the results to derive business value from the data. Big data analytics is a fusion of big data technologies and analytic tools.

Analytics is not a new concept: many analytic techniques, namely, regression analysis and machine learning, have existed for many years. Intertwining big data technologies with data from new sources and data analytic techniques is a newly evolved concept. The different types of analytics are descriptive analytics, predictive analytics, and prescriptive analytics.

1.8.5 Visualizing Big Data

Visualization makes the life cycle of big data complete assisting the end users to gain insights from the data. From executives to call center employees, everyone wants to extract knowledge from the data collected to assist them in making better decisions. Regardless of the volume of data, one of the best methods to discern relationships and make crucial decisions is to adopt advanced data analysis and visualization tools. Line graphs, bar charts, scatterplots, bubble plots, and pie charts are conventional data visualization techniques. Line graphs are

used to depict the relationship between one variable and another. Bar charts are used to compare the values of data belonging to different categories represented by horizontal or vertical bars, whose heights represent the actual values. Scatterplots are used to show the relationship between two variables (X and Y). A bubble plot is a variation of a scatterplot where the relationships between X and Y are displayed in addition to the data value associated with the size of the bubble. Pie charts are used where parts of a whole phenomenon are to be compared.

1.9 Big Data Technology

With the advancement in technology, the ways the data are generated, captured, processed, and analyzed are changing. The efficiency in processing and analyzing the data has improved with the advancement in technology. Thus, technology plays a great role in the entire process of gathering the data to analyzing them and extracting the key insights from the data.

Apache Hadoop is an open-source platform that is one of the most important technologies of big data. Hadoop is a framework for storing and processing the data. Hadoop was originally created by Doug Cutting and Mike Cafarella, a graduate student from the University of Washington. They jointly worked with the goal of index-

ing the entire web, and the project is called “Nutch.” The concept of MapReduce and GFS were integrated into Nutch, which led to the evolution of Hadoop. The word “Hadoop” is the name of the toy elephant of Doug’s son. The core components of Hadoop are HDFS, Hadoop common, which is a collection of common utilities that support other Hadoop modules, and MapReduce.

Apache Hadoop is an open-source framework for distributed storage and for processing large data sets. Hadoop can store petabytes of structured, semi-structured, or unstructured data at low cost. The low cost is due to the cluster of commodity hardware on which Hadoop runs.

Figure 1.12 shows the core components of Hadoop. A brief overview

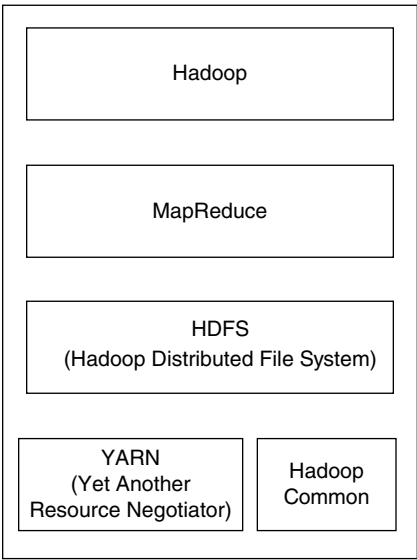


Figure 1.12 Hadoop core components.

about Hadoop, MapReduce, and HDFS was given under Section 1.7, “Big Data Infrastructure.” Now, let us see a brief overview of YARN and Hadoop common.

YARN – YARN is the acronym for Yet Another Resource Negotiator and is an open-source framework for distributed processing. It is the key feature of Hadoop version 2.0 of the Apache software foundation. In Hadoop 1.0 MapReduce was the only component to process the data in distributed environments. Limitations of classical MapReduce have led to the evolution of YARN. The cluster resource management of MapReduce in Hadoop 1.0 was taken over by YARN in Hadoop 2.0. This has lightened up the task of MapReduce and enables it to focus on the data processing part. YARN enables Hadoop to run jobs other than MapReduce jobs as well.

Hadoop common – Hadoop common is a collection of common utilities, which supports other Hadoop modules. It is considered as the core module of Hadoop as it offers essential services. Hadoop common has the scripts and Java Archive (JAR) files that are required to start Hadoop.

1.9.1 Challenges Faced by Big Data Technology

Indeed, we are facing a lot of challenges when it comes to dealing with the data. Some data are structured that could be stored in traditional databases, while some are videos, pictures, and documents, which may be unstructured or semi-structured, generated by sensors, social media, satellite, business transactions, and much more. Though these data can be managed independently, the real challenge is how to make sense by integrating disparate data from diversified sources.

- Heterogeneity and incompleteness
- Volume and velocity of the data
- Data storage
- Data privacy

1.9.2 Heterogeneity and Incompleteness

The data types of big data are heterogeneous in nature as the data is integrated from multiple sources and hence has to be carefully structured and presented as homogenous data before big data analysis. The data gathered may be incomplete, making the analysis much more complicated. Consider an example of a patient online health record with his name, occupation, birth data, medical ailment, laboratory test results, and previous medical history. If one or more of the above details are missing in multiple records, the analysis cannot be performed as it may not turn out to be valuable. In some scenarios a NULL value may be inserted in the place of missing values, and the analysis may be performed if that particular value

does not have a great impact on the analysis and if the rest of the available values are sufficient to produce a valuable outcome.

1.9.3 Volume and Velocity of the Data

Managing the massive and ever increasing volume of big data is the biggest concern in the big data era. In the past, the increase in the data volume was handled by appending additional memory units and computer resources. But the data volume was increasing exponentially, which could not be handled by traditional existing database storage models. The larger the volume of data, the longer the time consumed for processing and analysis.

The challenge faced with velocity does not only mean rate at which data arrives from multiple sources but also the rate at which data has to be processed and analyzed in the case of real-time analysis. For example, in the case of credit card transactions, if fraudulent activity is suspected, the transaction has to be declined in real time.

1.9.4 Data Storage

The volume of data contributed by social media, mobile Internet, online retailers, and so forth, is massive and was beyond the handling capacity of traditional databases. This requires a storage mechanism that is highly scalable to meet the increasing demand. The storage mechanism should be capable of accommodating the growing data, which is complex in nature. When the data volume is previously known, the storage capacity required is predetermined. But in case of streaming data, the required storage capacity is not predetermined. Hence, a storage mechanism capable of accommodating this streaming data is required. Data storage should be reliable and fault tolerant as well.

Data stored has to be retrieved at a later point in time. This data may be purchase history of a customer, previous releases of a magazine, employee details of a company, twitter feeds, images captured by a satellite, patient records in a hospital, financial transactions of a bank customer, and so forth. When a business analyst has to evaluate the improvement of sales of a company, she has to compare the sales of the current year with the previous year. Hence, data has to be stored and retrieved to perform the analysis.

1.9.5 Data Privacy

Privacy of the data is yet another concern growing with the increase in data volume. Inappropriate access to personal data, EHRs, and financial transactions is a social problem affecting the privacy of the users to a great extent. The data has to

be shared limiting the extent of data disclosure and ensuring that the data shared is sufficient to extract business knowledge from it. Whom access to the data should be granted to, limit of access to the data, and when the data can be accessed should be predetermined to ensure that the data is protected. Hence, there should be a deliberate access control to the data in various stages of the big data life cycle, namely data collection, storage, and management and analysis. The research on big data cannot be performed without the actual data, and consequently the issue of data openness and sharing is crucial. Data sharing is tightly coupled with data privacy and security. Big data service providers hand over huge data to the professionals for analysis, which may affect data privacy. Financial transactions contain the details of business processes and credit card details. Such kind of sensitive information should be protected well before delivering the data for analysis.

1.10 Big Data Applications

- Banking and Securities – Credit/debit card fraud detection, warning for securities fraud, credit risk reporting, customer data analytics.
- Healthcare sector – Storing the patient data and analyzing the data to detect various medical ailments at an early stage.
- Marketing – Analyzing customer purchase history to reach the right customers in order market their newly launched products.
- Web analysis – Social media data, data from search engines, and so forth, are analyzed to broadcast advertisements based on their interests.
- Call center analytics – Big data technology is used to identify the recurring problems and staff behavior patterns by capturing and processing the call content.
- Agriculture—Sensors are used by biotechnology firms to optimize crop efficiency. Big data technology is used in analyzing the sensor data.
- Smartphones—Facial recognition feature of smart phones is used to unlock their phones, retrieve information about a person with the information previously stored in their smartphones.

1.11 Big Data Use Cases

1.11.1 Health Care

To cope up with the massive flood of information generated at a high velocity, medical institutions are looking around for a breakthrough to handle this digital flood to aid them to enhance their health care services and create a successful

business model. Health care executives believe adopting innovative business technologies will reduce the cost incurred by the patients for health care and help them provide finer quality medical services. But the challenges in integrating patient data that are so large and complex growing at a faster rate hampers their efforts in improving clinical performance and converting the assets to business value.

Hadoop, the framework of big data, plays a major role in health care making big data storage and processing less expensive and highly available, giving more insight to the doctors. It has become possible with the advent of big data technologies that doctors can monitor the health of the patients who reside in a place that is remote from the hospital by making the patients wear watch-like devices. The devices will send reports of the health of the patients, and when any issue arises or if patients' health deteriorates, it automatically alerts the doctor.

With the development of health care information technology, the patient data can be electronically captured, stored, and moved across the universe, and health care can be provided with increased efficiency in diagnosing and treating the patient and tremendously improved quality of service. Health care in recent trend is evidence based, which means analyzing the patient's healthcare records from heterogeneous sources such as EHR, clinical text, biomedical signals, sensing data, biomedical images, and genomic data and inferring the patient's health from the analysis. The biggest challenge in health care is to store, access, organize, validate, and analyze this massive and complex data; also the challenge is even bigger for processing the data generated at an ever increasing speed. The need for real-time and computationally intensive analysis of patient data generated from ICU is also increasing. Big data technologies have evolved as a solution for the critical issues in health care, which provides real-time solutions and deploy advanced health care facilities. The major benefits of big data in health care are preventing disease, identifying modifiable risk factors, and preventing the ailment from becoming very serious, and its major applications are medical decision supporting, administrator decision support, personal health management, and public epidemic alert.

Big data gathered from heterogeneous sources are utilized to analyze the data and find patterns which can be the solution to cure the ailment and prevent its occurrence in the future.

1.11.2 Telecom

Big data promotes growth and increases profitability across telecom by optimizing the quality of service. It analyzes the network traffic, analyzes the call data in real-time to detect any fraudulent behavior, allows call center representatives to modify subscribers plan immediately on request, utilizes the insight gained by analyzing

the customer behavior and usage to evolve new plans and services to increase profitability, that is, provide personalized service based on consumer interest.

Telecom operators could analyze the customer preferences and behaviors to enable the recommendation engine to match plans to their price preferences and offer better add-ons. Operators lower the costs to retain the existing customers and identify cross-selling opportunities to improve or maintain the average revenue per customer and reduce churn. Big data analytics can further be used to improve the customer care services. Automated procedures can be imposed based on the understanding of customers' repetitive calls to solve specific issues to provide faster resolution. Delivering better customer service compared to its competitors can be a key strategy in attracting customers to their brand. Big data technology optimizes business strategy by setting new business models and higher business targets. Analyzing the sales history of products and services that previously existed allows the operators to predict the outcome or revenue of new services or products to be launched.

Network performance, the operator's major concern, can be improved with big data analytics by identifying the underlying issue and performing real-time troubleshooting to fix the issue. Marketing and sales, the major domain of telecom, utilize big data technology to analyze and improve the marketing strategy and increase the sales to increase revenue.

1.11.3 Financial Services

Financial services utilize big data technology in credit risk, wealth management, banking, and foreign exchange to name a few. Risk management is of high priority for a finance organization, and big data is used to manage various types of risks associated with the financial sector. Some of the risks involved in financial organizations are liquidity risk, operational risk, interest rate risk, the impact of natural calamities, the risk of losing valuable customers due to existing competition, and uncertain financial markets. Big data technologies derive solutions in real time resulting in better risk management.

Issuing loans to organizations and individuals is the major sector of business for a financial institution. Issuing loans is primarily done on the basis of creditworthiness of an organization or individual. Big data technology is now being used to find the credit worthiness based on latest business deals of an organization, partnership organizations, and new products that are to be launched. In the case of individuals, the credit worthiness is determined based on their social activity, their interest, and purchasing behavior.

Financial institutions are exposed to fraudulent activities by consumers, which cause heavy losses. Predictive analytics tools of big data are used to identify new patterns of fraud and prevent them. Data from multiples sources such as shopping

patterns and previous transactions are correlated to detect and prevent credit card fraud by utilizing in-memory technology to analyze terabytes of streaming data to detect fraud in real time.

Big data solutions are used in financial institutions call center operations to predict and resolve customer issues before they affect the customer; also, the customers can resolve the issues via self-service giving them more control. This is to go beyond customer expectations and provide better financial services. Investment guidance is also provided to consumers where wealth management advisors are used to help out consumers for making investments. Now with big data solutions these advisors are armed with insights from the data gathered from multiple sources.

Customer retention is becoming important in the competitive markets, where financial institutions might cut down the rate of interest or offer better products to attract customers. Big data solutions assist the financial institutions to retain the customers by monitoring the customer activity and identify loss of interest in financial institutions personalized offers or if customers liked any of the competitors' products on social media.

Chapter 1 Refresher

- 1 Big Data is _____.
- A Structured
 - B Semi-structured
 - C Unstructured
 - D All of the above

Answer: d

Explanation: Big Data is a blanket term for the data that are too large in size, complex in nature, and which may be structured, unstructured, or semi-structured and arriving at high velocity as well.

- 2 The hardware used in big data is _____.
- A High-performance PCs
 - B Low-cost commodity hardware
 - C Dumb terminal
 - D None of the above

Answer: b

Explanation: Big data uses low-cost commodity hardware to make cost-effective solutions.

3 What does commodity hardware in the big data world mean?

- A** Very cheap hardware
- B** Industry-standard hardware
- C** Discarded hardware
- D** Low specifications industry-grade hardware

Answer: d

Explanation: Commodity hardware is a low-cost, low performance, and low specification functional hardware with no distinctive features.

4 What does the term “velocity” in big data mean?

- A** Speed of input data generation
- B** Speed of individual machine processors
- C** Speed of ONLY storing data
- D** Speed of storing and processing data

Answer: d

5 What are the data types of big data?

- A** Structured data
- B** Unstructured data
- C** Semi-structured data
- D** All of the above

Answer: d

Explanation: Machine-generated and human-generated data can be represented by the following primitive types of big data

- Structured data
- Unstructured data
- Semi-Structured data

6 JSON and XML are examples of _____.

- A** Structured data
- B** Unstructured data
- C** Semi-structured data
- D** None of the above

Answer: c

Explanation: Semi-structured data are that which have a structure but do not fit into the relational database. Semi-structured data are organized, which makes it easier for analysis when compared to unstructured data. JSON and XML are examples of semi-structured data.

7 _____ is the process that corrects the errors and inconsistencies.

- A Data cleaning
- B Data Integration
- C Data transformation
- D Data reduction

Answer: a

Explanation: The data-cleaning process fills in the missing values, corrects the errors and inconsistencies, and removes redundancy in the data to improve the data quality.

8 _____ is the process of transforming data into an appropriate format that is acceptable by the big data database.

- A Data cleaning
- B Data Integration
- C Data transformation
- D Data reduction

Answer: c

Explanation: Data transformation refers to transforming or consolidating the data into an appropriate format that is acceptable by the big data database and converting them into logical and meaningful information for data management and analysis.

9 _____ is the process of combining data from different sources to give the end users a unified data view.

- A Data cleaning
- B Data integration
- C Data transformation
- D Data reduction

Answer: b

10 _____ is the process of collecting the raw data, transmitting the data to a storage platform, and preprocessing them.

- A Data cleaning
- B Data integration
- C Data aggregation
- D Data reduction

Answer: c

Conceptual Short Questions with Answers

1 What is big data?

Big data is a blanket term for the data that are too large in size, complex in nature, which may be structured or unstructured, and arriving at high velocity as well.

2 What are the drawbacks of traditional database that led to the evolution of big data?

Below are the limitations of traditional databases, which has led to the emergence of big data.

- Exponential increase in data volume, which scales in terabytes and petabytes, has turned out to become a challenge to the RDBMS in handling such a massive volume of data.
- To address this issue, the RDBMS increased the number of processors and added more memory units, which in turn increased the cost.
- Almost 80% of the data fetched were of semi-structured and unstructured format, which RDBMS could not deal with.
- RDBMS could not capture the data coming in at high velocity.

3 What are the factors that explain the tremendous increase in the data volume?

Multiple disparate data sources are responsible for the tremendous increase in the volume of big data. Much of the growth in data can be attributed to the digitization of almost anything and everything in the globe. Paying e-bills, online shopping, communication through social media, e-mail transactions in various organizations, a digital representation of the organizational data, and so forth, are some of the examples of this digitization around the globe.

4 What are the different data types of big data?

Machine-generated and human-generated data can be represented by the following primitive types of big data

- Structured data
- Unstructured data
- Semi-Structured data

5 What is semi-structured data?

Semi-structured data are that which have a structure but does not fit into the relational database. Semi-structured data are organized, which makes it easier for analysis when compared to unstructured data. JSON and XML are examples of semi-structured data.

6 What does the three Vs of big data mean?

Volume–Size of the data

- 1) Velocity–Rate at which the data is generated and is being processed
- 2) Variety–Heterogeneity of data: structured, unstructured, and semi-structured

7 What is commodity hardware?

Commodity hardware is a low-cost, low-performance, and low-specification functional hardware with no distinctive features. Hadoop can run on commodity hardware and does not require any high-end hardware or supercomputers to execute its jobs.

8 What is data aggregation?

The data aggregation phase of the big data life cycle involves collecting the raw data, transmitting the data to a storage platform, and preprocessing them. Data acquisition in the big data world means acquiring the high-volume data arriving at an ever increasing pace.

9 What is data preprocessing?

Data preprocessing is an important process performed on raw data to transform it into an understandable format and provide access to a consistent and an accurate data. The data generated from multiple sources are erroneous, incomplete, and inconsistent because of their massive volume and heterogeneous sources, and it is pointless to store useless and dirty data. Additionally, some analytical applications have a crucial requirement for quality data. Hence, for effective, efficient, and accurate data analysis, systematic data preprocessing is essential.

10 What is data integration?

Data integration involves combining data from different sources to give the end users a unified data view.

11 What is data cleaning?

The data-cleaning process fills in the missing values, corrects the errors and inconsistencies, and removes redundancy in the data to improve the data quality. The larger the heterogeneity of the data sources, the higher the degree of dirtiness. Consequently, more cleaning steps may be involved.

12 What is data reduction?

Data processing on massive data volume may take a long time, making data analysis either infeasible or impractical. Data reduction is the concept of reducing the volume of data or reducing the dimension of the data, that is, the number of attributes. Data reduction techniques are adopted to analyze the data in reduced format without losing the integrity of the actual data and yet yield quality outputs.

13 What is data transformation?

Data transformation refers to transforming or consolidating the data into an appropriate format that is acceptable by the big data database and converting them into logical and meaningful information for data management and analysis.

Frequently Asked Interview Questions**1 Give some examples of big data.**

Facebook is generating approximately 500 terabytes of data per day, about 10 terabytes of sensor data are generated every 30 minutes by airlines, the New York Stock Exchange is generating approximately 1 terabyte of data per day. These are examples of big data.

2 How is big data analysis useful for organizations?

Big data analytics is useful for the organizations to make better decisions, find new business opportunities, compete against business rivals, improve performance and efficiency, and reduce cost by using advanced data analytics techniques.

2

Big Data Storage Concepts

CHAPTER OBJECTIVE

The various storage concepts of big data, namely, clusters and file system are given a brief overview. The data replication, which has made big the data storage concept a fault tolerant system is explained with master-slave and peer-peer types of replications. Various storage types of on-disk storage are briefed. Scalability techniques, namely, scaling up and scaling out, adopted by various database systems are overviewed.

In big data storage, architecture data reaches users through multiple organization data structures. The big data revolution provides significant improvements to the data storage architecture. New tools such as Hadoop, an open-source framework for storing data on clusters of commodity hardware, are developed, which allows organizations to effectively store and analyze large volumes of data.

In Figure 2.1 the data from the source flow through Hadoop, which acts as an online archive. Hadoop is highly suitable for unstructured and semi-structured data. However, it is also suitable for some structured data, which are expensive to be stored and processed in traditional storage engines (e.g., call center records). The data stored in Hadoop is then fed into a data warehouse, which distributes the data to data marts and other systems in the downstream where the end users can query the data using query tools and analyze the data.

In modern BI architecture the raw data stored in Hadoop can be analyzed using MapReduce programs. MapReduce is the programming paradigm of Hadoop. It can be used to write applications to process the massive data stored in Hadoop.

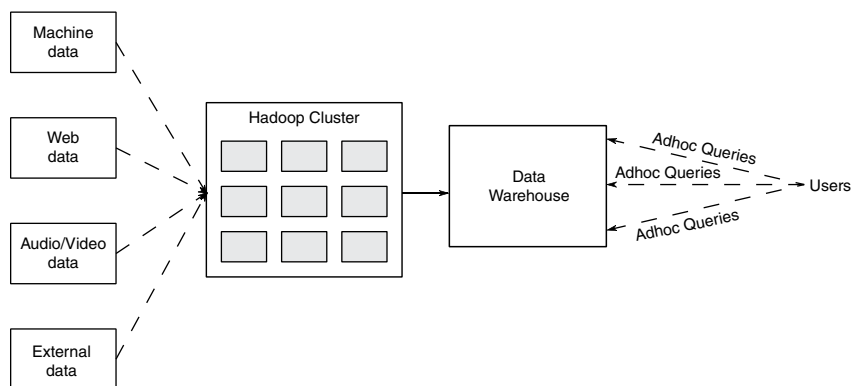


Figure 2.1 Big data storage architecture.

2.1 Cluster Computing

Cluster computing is a distributed or parallel computing system comprising multiple stand-alone PCs connected together working as a single, integrated, highly available resource. Multiple computing resources are connected together in a cluster to constitute a single larger and more powerful virtual computer with each computing resource running an instance of the OS. The cluster components are connected together through local area networks (LANs). Cluster computing technology is used for high availability as well as load balancing with better system performance and reliability. The benefits of massively parallel processors and cluster computers are high availability, scalable performance, fault tolerance, and the use of cost-effective commodity hardware. Scalability is achieved by removing nodes or adding additional nodes as per the demand without hindering the system operation. A cluster of systems connects together a group of systems to share critical computational tasks. The servers in a cluster are called nodes. Cluster computing can be client-server architecture or a peer-peer model. It provides high-speed computational power for processing data-intensive applications related to big data technologies. Cluster computing with distributed computation infrastructure provides fast and reliable data processing power to gigantic-sized big data solutions with integrated and geographically separated autonomous resources. They make a cost-effective solution to big data as they do allow multiple applications to share the computing resources. They are flexible to add more computing resources as required by the big data technology. The clusters are capable of changing the size dynamically, they shrink when any server shuts down or grow in size when additional servers are added to handle more load. They survive the failures with no or minimal impact. Clusters adopt a failover mechanism to eliminate the service interruptions. Failover is the process of switching to a redundant node upon the abnormal termination or failure of a previously

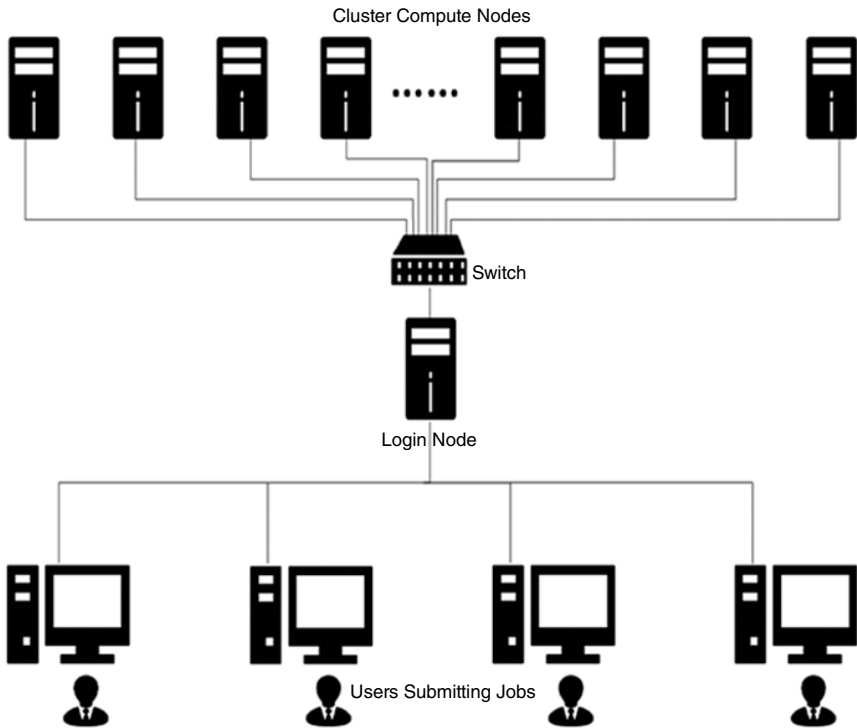


Figure 2.2 Cluster computing.

active node. Failover is an automatic mechanism that does not require any human intervention, which differentiates it from the switch-over operation.

Figure 2.2 shows the overview of cluster computing. Multiple stand-alone PCs connected together through a dedicated switch. The login node acts as the gateway into the cluster. When the cluster has to be accessed by the users from a public network, the user has to login to the login node. This is to prevent unauthorized access by the users. Cluster computing has a master-slave model and a peer-to-peer model. There are two major types of clusters, namely, high-availability cluster and load-balancing cluster. Cluster types are briefed in the following section.

2.1.1 Types of Cluster

Clusters may be configured for various purposes such as web-based services or computational-intensive workloads. Based on their purpose, the clusters may be classified into two major types:

- High availability
- Load balancing

When the availability of the system is of high importance in case of failure of the nodes, high-availability clusters are used. When the computational workload has to be shared among the cluster nodes, load-balancing clusters are used to improve the overall performance. Thus, computer clusters are configured based on the business purpose needs.

2.1.1.1 High Availability Cluster

High availability clusters are designed to minimize downtime and provide uninterrupted service when nodes fail. Nodes in a highly available cluster must have access to a shared storage. Such systems are often used for failover and backup purposes. Without clustering the nodes if the server running an application goes down, the application will not be available until the server is up again. In a highly available cluster, if a node becomes inoperative, continuous service is provided by failing over service from the inoperative cluster node to another, without administrative intervention. Such clusters must maintain data integrity while failing over the service from one cluster node to another. High availability systems consist of several nodes that communicate with each other and share information. High availability makes the system highly fault tolerant with many redundant nodes, which sustain faults and failures. Such systems also ensure high reliability and scalability. The higher the redundancy, the higher the availability. A highly available system eliminates single point of failures.

Highly available systems are essential for an organization that has to protect its business against loss of transactional data or incomplete data and overcome the risk of system outage. These risks, under certain circumstances, are bound to cause millions of dollars of losses to the business. Certain applications such as online platforms may face sudden increase in traffic. To manage these traffic spikes a robust solution such as cluster computing is required. Billing, banking, and e-commerce demand a system that is highly available with zero loss of transactional data.

2.1.1.2 Load Balancing Cluster

Load-balancing clusters are designed to distribute workloads across different cluster nodes to share the service load among the nodes. If a node in a load-balancing cluster goes down, the load from that node is switched over to another node. This is achieved by having identical copies of data across all the nodes, so the remaining nodes can share the increase in load. The main objective of load balancing is to optimize the use of resources, minimize response time, maximize throughput, and avoid overload on any one of the resources. The resources are used efficiently in this kind of cluster algorithm as there is a good amount of control over the way in which the requests are routed. This kind of routing is

essential when the cluster is composed of machines that are not equally efficient; in that case, low-performance machines are assigned a lesser share of work. Instead of having a single, very expensive and very powerful server, load balancing can be used to share the load across several inexpensive, low performing systems for better scalability.

Round robin load balancing, weight-based load balancing, random load balancing, and server affinity load balancing are examples of load balancing. Round robin load balancing chooses server from the top server in the list in sequential order until the last server in the list is chosen. Once the last server is chosen it resets back to the top. The weight-based load balancing algorithm takes into account the previously assigned weight for each server. The weight field will be assigned a numerical value between 1 and 100, which determines the proportion of the load the server can bear with respect to other servers. If the servers bear equal weight, an equal proportion of the load is distributed among the servers. Random load balancing routes requests to servers at random. Random load balancing is suitable only for homogenous clusters, where the machines are similarly configured. A random routing of requests does not allow for differences among the machines in their processing power. Server affinity load balancing is the ability of the load balancer to remember the server where the client initiated the request and to route the subsequent requests to the same server.

2.1.2 Cluster Structure

In a basic cluster structure, a group of computers are linked and work together as a single computer. Clusters are deployed to improve performance and availability. Based on how these computers are linked together, cluster structure is classified into two types:

- Symmetric clusters
- Asymmetric clusters

Symmetric cluster is a type of cluster structure in which each node functions as an individual computer capable of running applications. The symmetric cluster setup is simple and straightforward. A sub-network is created with individual machines or machines can be added to an existing network and cluster-specific software can be installed to it. Additional machines can be added as needed. Figure 2.3 shows a symmetric cluster.

Asymmetric clusters are a type of cluster structure in which one machine acts as the head node, and it serves as the gateway between the user and the remaining nodes. Figure 2.4 shows an asymmetric cluster.

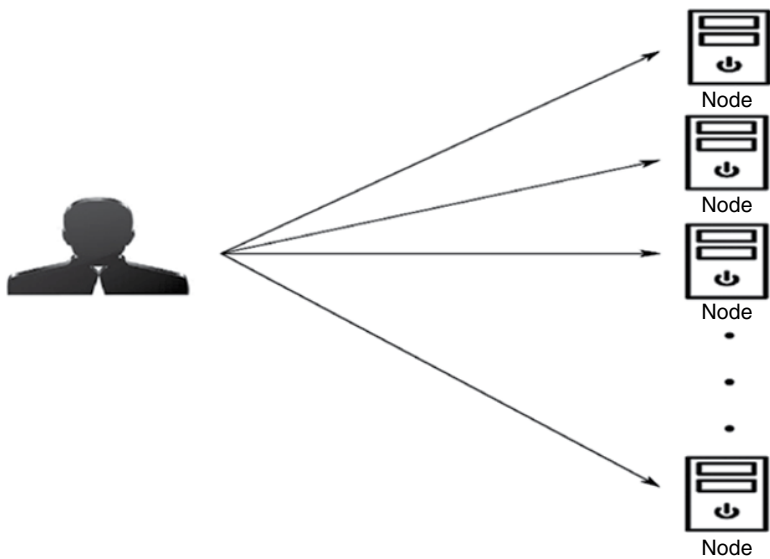


Figure 2.3 Symmetric clusters.

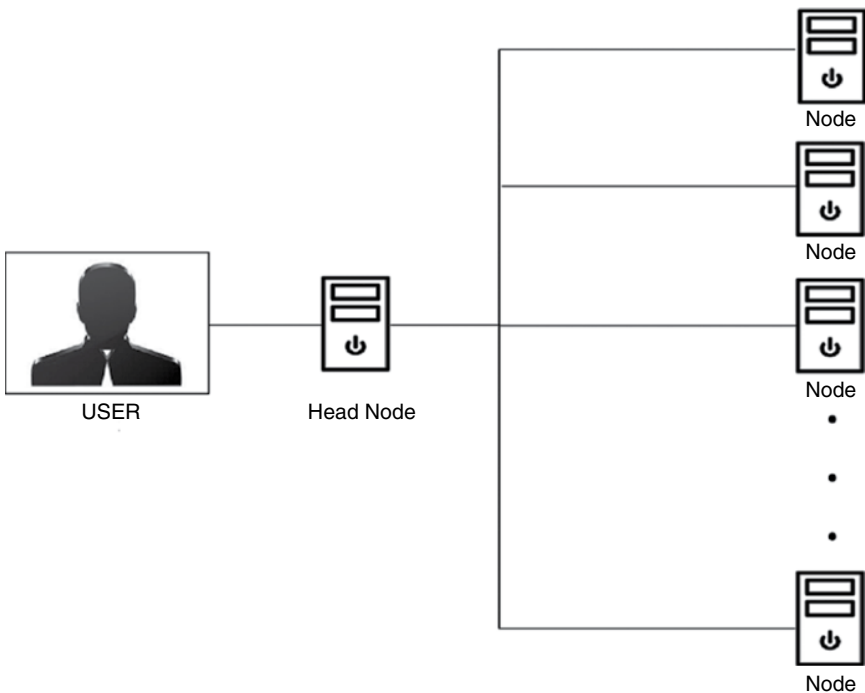


Figure 2.4 Asymmetric cluster.

2.2 Distribution Models

The main reason behind distributing data over a large cluster is to overcome the difficulty and to cut the cost of buying expensive servers. There are several distribution models with which an increase in data volume and large volumes of read or write requests can be handled, and the network can be made highly available. The downside of this type of architecture is the complexity it introduces with the increase in the number of computers added to the cluster. Replication and sharding are the two major techniques of data distribution. Figure 2.5 shows the distribution models.

- Replication—Replication is the process of placing the same set of data over multiple nodes. Replication can be performed using a peer-to-peer model or a master-slave model.
- Sharding—Sharding is the process of placing different sets of data on different nodes.
- Sharding and Replication—Sharding and replication can either be used alone or together.

2.2.1 Sharding

Sharding is the process of partitioning very large data sets into smaller and easily manageable chunks called shards. The partitioned shards are stored by distributing them across multiple machines called nodes. No two shards of the same file are stored in the same node, each shard occupies separate nodes, and the shards spread across multiple nodes collectively constitute the data set.

Figure 2.6a shows that a 1 GB data block is split up into four chunks each of 256 MB. When the size of the data increases, a single node may be insufficient to store the data. With sharding more nodes are added to meet the demands of the

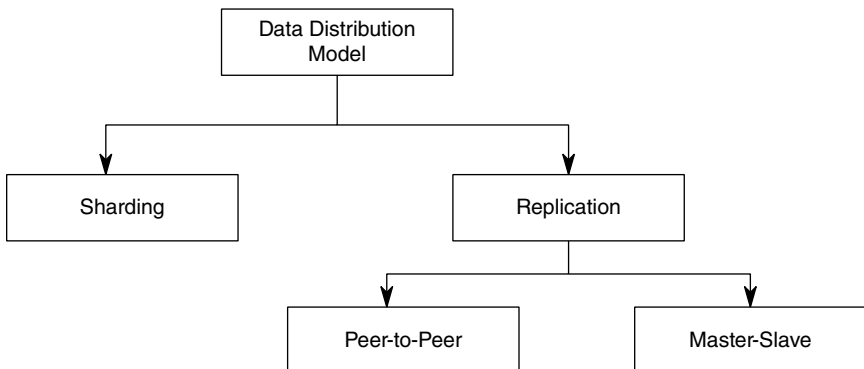
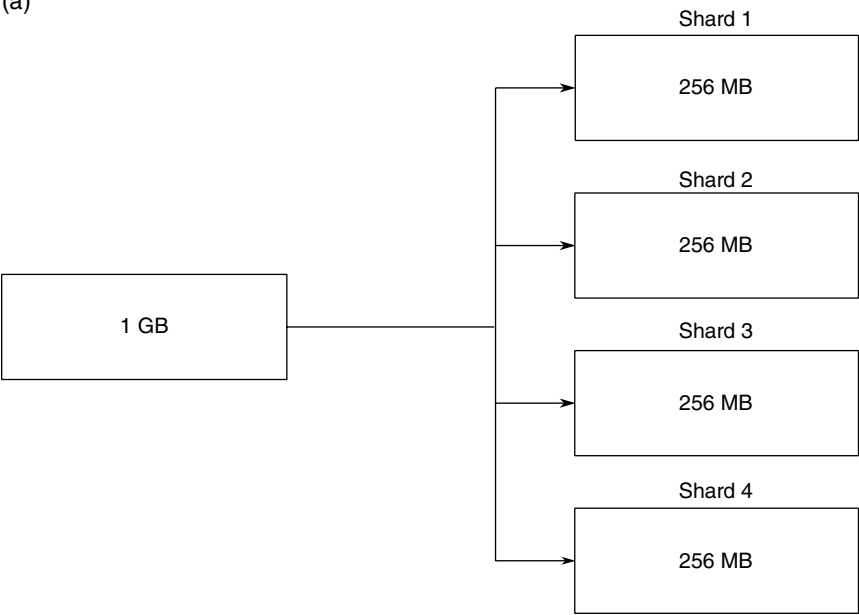


Figure 2.5 Distribution model.

(a)



(b)

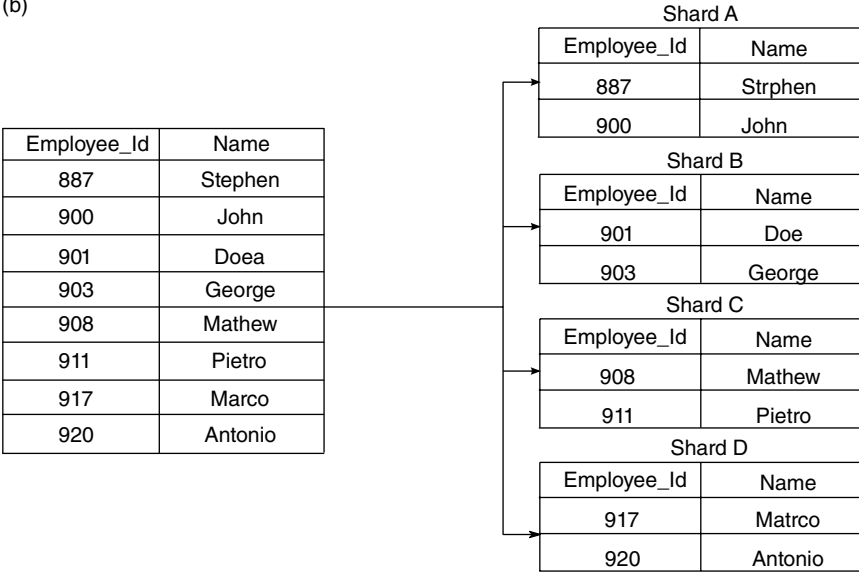


Figure 2.6 (a) Sharding. (b) Sharding example.

massive data growth. Sharding reduces the number of transaction each node handles and increases throughput. It reduces the data each node needs to store.

Figure 2.6b shows an example as how a data block is split up into shards across multiple nodes. A data set with employee details is split up into four small blocks: shard A, shard B, shard C, shard D and stored across four different nodes: node A, node B, node C, and node D. Sharding improves the fault tolerance of the system as the failure of a node affects only the block of the data stored in that particular node.

2.2.2 Data Replication

Replication is the process of creating copies of the same set of data across multiple servers. When a node crashes, the data stored in that node will be lost. Also, when a node is down for maintenance, the node will not be available until the maintenance process is over. To overcome these issues, the data block is copied across multiple nodes. This process is called data replication, and the copy of a block is called replica. Figure 2.7 shows data replication.

Replication makes the system fault tolerant since the data is not lost when an individual node fails as the data is redundant across the nodes. Replication increases the data availability as the same copy of data is available across multiple nodes. Figure 2.8 illustrates that the same data is replicated across node A, node B, and node C. Data replication is achieved through the master-slave and peer-peer models.

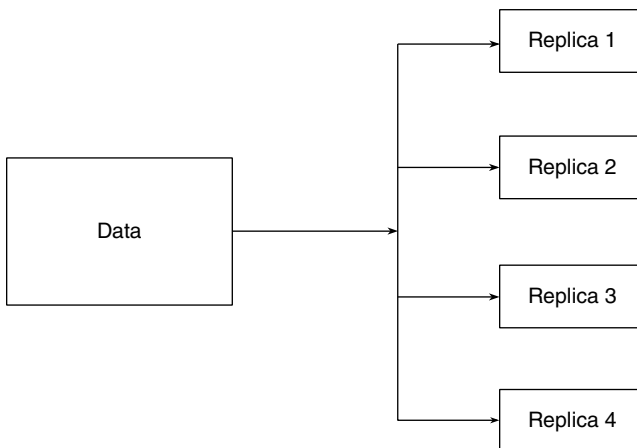


Figure 2.7 Replication.

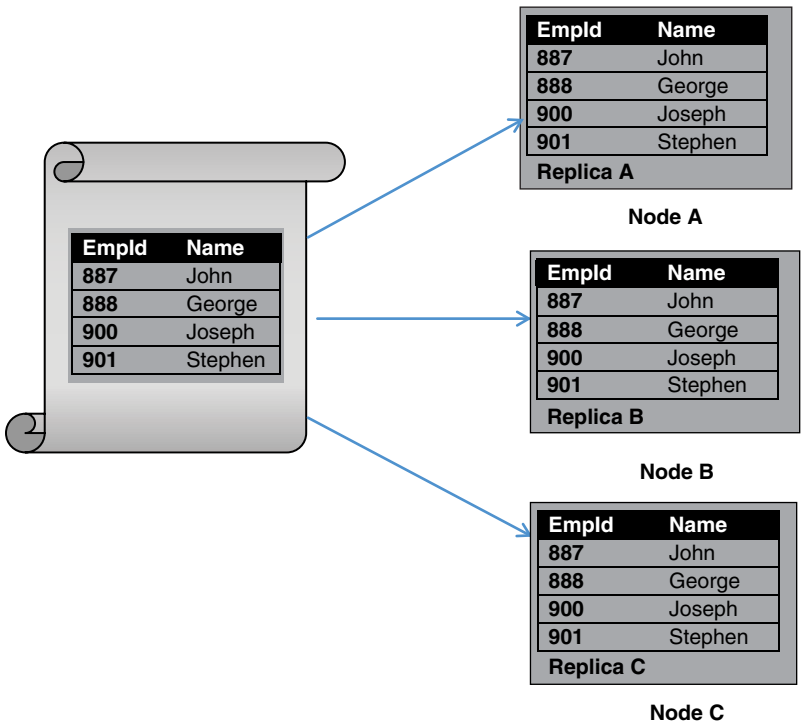


Figure 2.8 Data replication.

2.2.2.1 Master-Slave Model

Master-slave configuration is a model where one centralized device known as the master controls one or more devices known as slaves. In a master-slave configuration a replica set constitutes a master node and several slave nodes. Once the relationship between master and slave is established, the flow of control is only from master to the slaves. In master-slave replication, all the incoming data are written on the master node, and the same data is replicated over several slave nodes. All the write requests are handled by the master node, and the data update, insert, or delete occurs in the master node, while the read requests are handled by slave nodes. This architecture supports intensive read requests as the increasing demands can be handled by appending additional slave nodes. If a master node fails, write requests cannot be fulfilled until the master node is resumed or a new master node is created from one of the slave nodes. Figure 2.9 shows data replication in a master-slave configuration.

2.2.2.2 Peer-to-Peer Model

In the master-slave model only the slaves are guaranteed against single point of failure. The cluster still suffers from single point of failure, if the master fails. Also, the writes are limited to the maximum capacity that a master can handle;

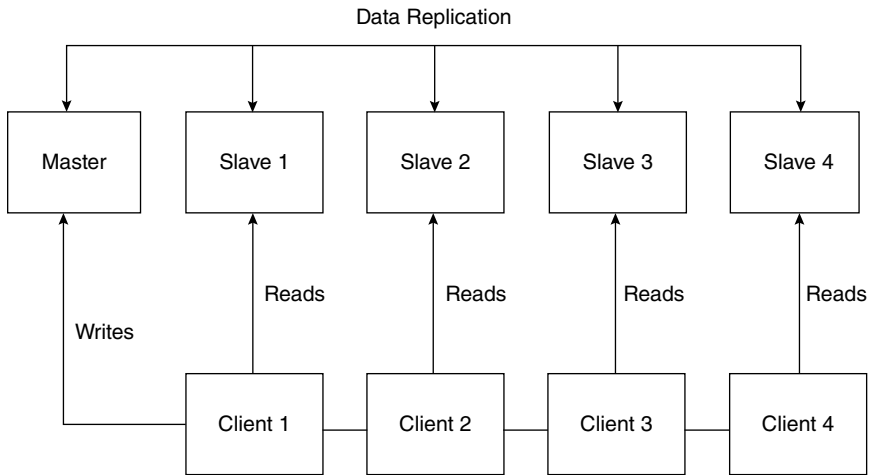


Figure 2.9 Master-Slave model.

hence, it provides only read scalability. These drawbacks in the master-slave model are overcome in the peer-to-peer model. In a peer-to-peer configuration there is no master-slave concept, all the nodes have the same responsibility and are at the same level. The nodes in a peer-to-peer configuration act both as client and the server. In the master-slave model, communication is always initiated by the master, whereas in a peer-to-peer configuration, either of the devices involved in the process can initiate communication. Figure 2.10 shows replication in the peer-to-peer model.

In the peer-to-peer model the workload or the task is partitioned among the nodes. The nodes consume as well as donate the resources. Resources such as disk storage space, memory, bandwidth, processing power, and so forth, are shared among the nodes.

Reliability of this type of configuration is improved through replication. Replication is the process of sharing the same data across multiple nodes to avoid single point of failure. Also, the nodes connected in a peer-to-peer configuration are geographically distributed across the globe.

2.2.3 Sharding and Replication

In sharding when a node goes down, the data stored in the node will be lost. So it provides only a limited fault tolerance to the system. Sharding and replication can be combined to make the system fault tolerant and highly available. Figure 2.11 illustrates the combination of sharding and replication where the data set is split up into shard A and shard B. Shard A is replicated across node A and node B; similarly shard B is replicated across node C and node D.

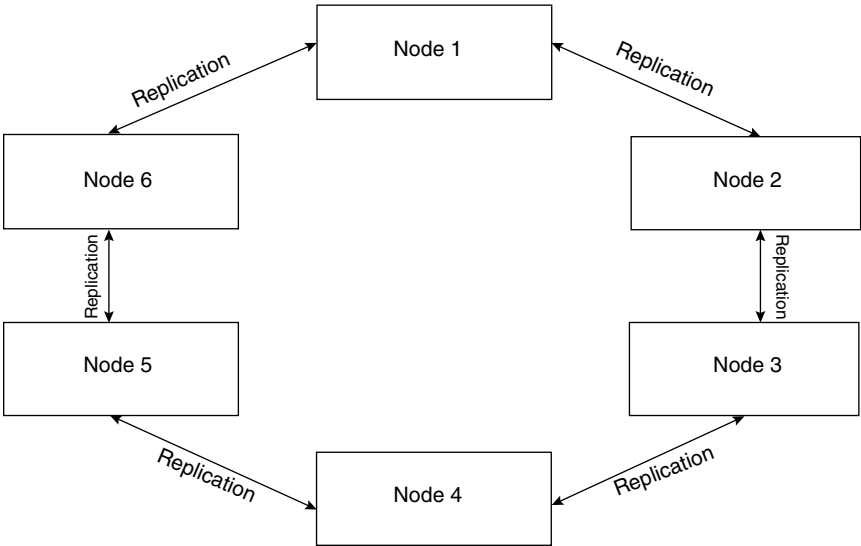


Figure 2.10 Peer-to-peer model.

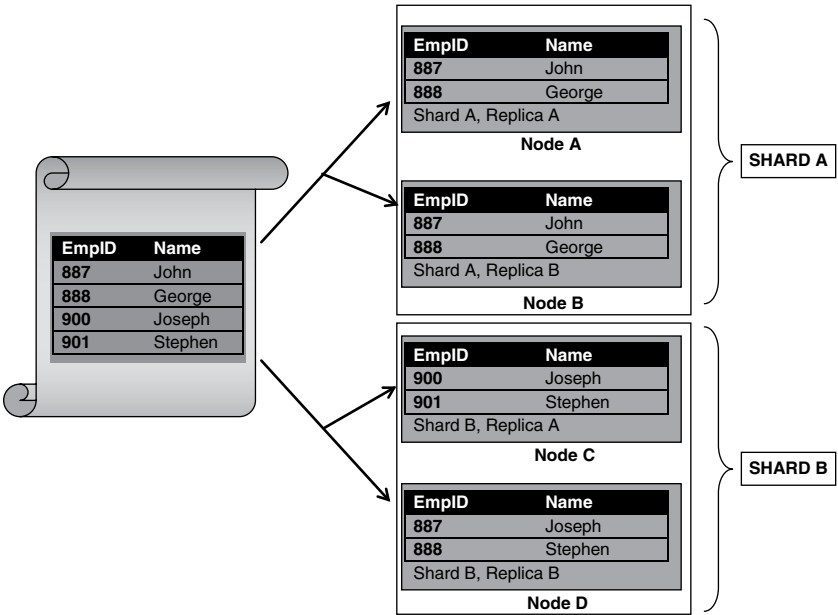


Figure 2.11 Combination of sharding and replication.

2.3 Distributed File System

A file system is a way of storing and organizing the data on storage devices such as hard drives, DVDs, and so forth, and to keep track of the files stored on them. The file is the smallest unit of storage defined by the file system to pile the data. These file systems store and retrieve data for the application to run effectively and efficiently on the operating systems. A distributed file system stores the files across cluster nodes and allows the clients to access the files from the cluster. Though physically the files are distributed across the nodes, logically it appears to the client as if the files are residing on their local machine. Since a distributed file system provides access to more than one client simultaneously, the server has a mechanism to organize updates for the clients to access the current updated version of the file, and no version conflicts arise. Big data widely adopts a distributed file system known as Hadoop Distributed File System (HDFS).

The key concept of a distributed file system is the data replication where the copies of data called replicas are distributed on multiple cluster nodes so that there is no single point of failure, which increases the reliability. The client can communicate with any of the closest available nodes to reduce latency and network traffic. Fault tolerance is achieved through data replication as the data will not be lost in case of node failure due to the redundancy in the data across nodes.

2.4 Relational and Non-Relational Databases

Relational databases organize data into tables of rows and columns. The rows are called records, and the columns are called attributes or fields. A database with only one table is called a flat database, while a database with two or more tables that are related is called a relational database. Table 2.1 shows a simple table that stores the details of the students registering for the courses offered by an institution.

In the above example, the table holds the details of the students and CourseId of the courses for which the students have registered. The above table meets the basic needs to keep track of the courses for which each student has registered. But it has some serious flaws in accordance with efficiency and space utilization. For example, when a student registers for more than one course, then details of the student has to be entered for every course he registers. This can be overcome by dividing the data across multiple related tables. Figure 2.12 represents the data in the above table is divided among multiple related tables with unique primary and foreign keys.

Relational tables have attributes that uniquely identify each row. The attributes which uniquely identify the tuples are called primary key. StudentId is the primary key, and hence its value should be unique. Attribute in one table that references to

Table 2.1 Student course registration database.

StudentName	Phone	DOB	CourseId	Faculty
James	541 754 3010	03/05/1985	1	Dr.Jeffrey
John	415 555 2671	05/01/1992	2	Dr.Lewis
Richard	415 570 2453	09/12/1999	2	Dr.Philips
Michael	555 555 1234	12/12/1995	3	Dr.Edwards
Richard	415 555 2671	02/05/1989	4	Dr.Anthony

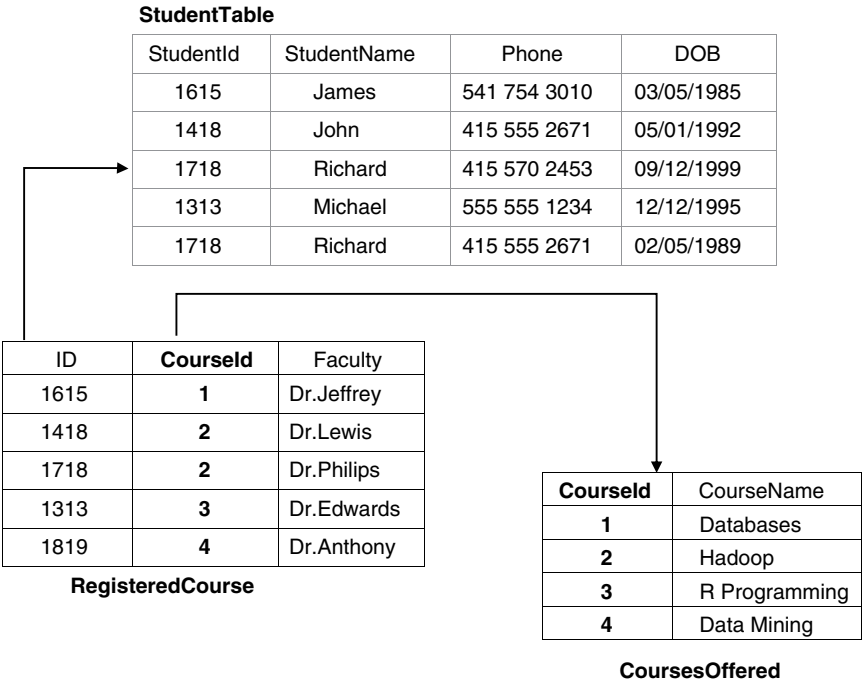


Figure 2.12 Data divided across multiple related tables.

the primary key in another table is called foreign key. `CourseId` in `RegisteredCourse` is a foreign key, which references to `CourseId` in the `CoursesOffered` table.

Relational databases become unsuitable when organizations collect vast amount of customer databases, transactions, and other data, which may not be structured to fit into relational databases. This has led to the evolution of non-relational databases, which are schema-less. NoSQL is a non-relational database and a few frequently used NoSQL databases are Neo4J, Redis, Cassandra, and MongoDB. Let us have a quick look at the properties of RDBMS and NoSQL databases.

2.4.1 RDBMS Databases

RDBMS is vertically scalable and exhibits ACID (atomicity, consistency, isolation, durability) properties and support data that adhere to a specific schema. This schema check is made at the time of inserting or updating data, and hence they are not ideal for capturing and storing data arriving at high velocity. The architectural limitation of RDBMS makes it unsuitable for big data solutions as a primary storage device.

For the past decades, relational database management systems that were running in corporate data centers have stored the bulk of the world's data. But with the increase in volume of the data, RDBMS can no longer keep pace with the volume, velocity, and variety of data being generated and consumed.

Big data, which is typically a collection of data with massive volume and variety arriving at a high velocity, cannot be effectively managed with traditional data management tools. While conventional databases are still existing and used in a large number of applications, one of the key advancements in resolving the problems with big data is the emergence of modern alternate database technologies that do not require any fixed schema to store data; rather, the data is distributed across the storage paradigm. The main alternative databases are NoSQL and NewSQL databases.

2.4.2 NoSQL Databases

A NoSQL (Not Only SQL) database includes all non-relational databases. Unlike RDBMS, which exhibits ACID properties, a NoSQL database follows the CAP theorem (consistency, availability, partition tolerance) and exhibits the BASE (basically, available, soft state, eventually consistent) model, where the storage devices do not provide immediate consistency; rather, they provide eventual consistency. Hence, these databases are not appropriate for implementing large transactions.

The various types of NoSQL databases, namely, Key-value databases, document databases, column-oriented databases, graph databases, were discussed in detail in Section 2.3. Table 2.2 shows examples of various types of NoSQL databases.

Table 2.2 Popular NoSQL databases.

Key-value databases	Document databases	Column databases	Graph databases
Redis	MongoDB	DynamoDB	Neo4j
Riak	CouchDB	Cassandra	OrientDB
SimpleDB	RethinkDB	Accumulo	ArangoDB
BerkeleyDB Oracle	MarkLogic	Big Table	FlockDB

2.4.3 NewSQL Databases

NewSQL databases provide scalable performance similar to that of NoSQL systems combining the ACID properties of a traditional database management system. VoltDB, NuoDB, Clustrix, MemSQL, and TokuDB are some of the examples of NewSQL database.

NewSQL databases are distributed in nature, horizontally scalable, fault tolerant, and support relational data model with three layers: the administrative layer, transactional layer, and storage layer. NewSQL database is highly scalable and operates in shared nothing architecture. NewSQL has SQL compliant syntax and uses relational data model for storage. Since it supports SQL compliant syntax, transition from RDBMS to the highly scalable system is made easy.

The applications targeting these NewSQL systems are those that execute the same queries repeatedly with different inputs and have a large number of transactions. Some of the commercial products of NewSQL databases are briefed below.

2.4.3.1 Clustrix

Clustrix is a high performance, fault tolerant, distributed database. Clustrix is used in applications with massive, high transactional volume.

2.4.3.2 NuoDB

NuoDB is a cloud based, scale-out, fault tolerant, distributed database. They support both batch and real-time SQL queries.

2.4.3.3 VoltDB

VoltDB is a scale-out, in-memory, high performance, fault tolerant, distributed database. They are used to make real-time decisions to maximize business value.

2.4.3.4 MemSQL

MemSQL is a high performance, in-memory, fault tolerant, distributed database. MemSQL is known for its blazing fast performance and used for real-time analytics.

2.5 Scaling Up and Scaling Out Storage

Scalability is the ability of the system to meet the increasing demand for storage capacity. A system capable of scaling delivers increased performance and efficiency. With the advent of the big data era there is an imperative need to scale data storage platforms to make them capable of storing petabytes of data. The storage platforms can be scaled in two ways:

- Scaling-up (vertical scalability)
- Scaling-out (horizontal scalability)

Scaling-up. The vertical scalability adds more resources to the existing server to increase its capacity to hold more data. The resources can be computation power, hard drive, RAM, and so on. This type of scaling is limited to the maximum scaling capacity of the server. Figure 2.13 shows a scale-up architecture where the RAM capacity of the same machine is upgraded from 32 GB to 128 GB to meet the increasing demand.

Scaling-out. The horizontal scalability adds new servers or components to meet the demand. The additional component added is termed as node. Big data technologies work on the basis of scaling out storage. Horizontal scaling enables the system to scale wider to meet the increasing demand. Scaling out storage uses low cost commodity hardware and storage components. The components can be added as required without much complexity. Multiple components connect together to work as a single entity. Figure 2.14 shows the scale-out architecture where the capacity is increased by adding additional commodity hardware to the cluster to meet the increasing demand.

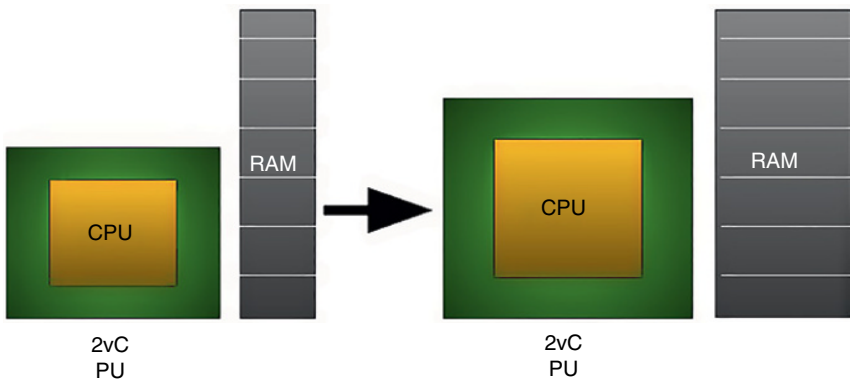


Figure 2.13 Scale-up architecture.

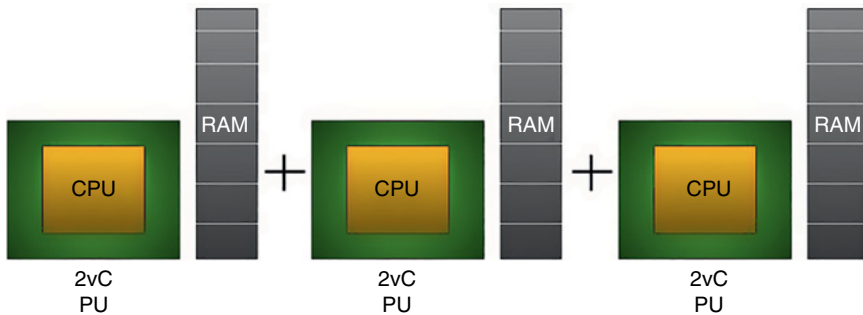


Figure 2.14 Scale-out architecture.

Chapter 2 Refresher

1 The set of loosely connected computers is called ____.

- A LAN
- B WAN
- C Workstation
- D Cluster

Answer: d

Explanation: In a computer cluster all the participating computers work together on a particular task.

2 Cluster computing is classified into

- A High-availability cluster
- B Load-balancing cluster
- C Both a and b
- D None of the above

Answer: c

3 The computer cluster architecture emerged as a result of ____.

- A ISA
- B Workstation
- C Supercomputers
- D Distributed systems

Answer: d

Explanation: A distributed system is a computer system spread out over a geographic area.

4 Cluster adopts _____ mechanism to eliminate the service interruptions.

- A Sharding

- B** Replication
- C** Failover
- D** Partition

Answer: c

- 5** _____ is the process of switching to a redundant node upon the abnormal termination or failure of a previously active node.
- A** Sharding
 - B** Replication
 - C** Failover
 - D** Partition

Answer: c

- 6** _____ adds more storage resources and CPU to increase capacity.
- A** Horizontal scaling
 - B** Vertical scaling
 - C** Partition
 - D** All of the mentioned

Answer: b

Explanation: When the primary steps down, the MongoDB closes all client connections.

- 7** _____ is the process of copying the same data blocks across multiple nodes.
- A** Replication
 - B** Partition
 - C** Sharding
 - D** None of the above

Answer: a

Explanation: Replication is the process of copying the same data blocks across multiple nodes to overcome the loss of data when a node crashes.

- 8** _____ is the process of dividing the data set and distributing the data over multiple servers.
- A** Vertical
 - B** Sharding
 - C** Partition
 - D** All of the mentioned

Answer: b

Explanation: Sharding is the process of partitioning very large data sets into smaller and easily manageable chunks called shards.

- 9 A sharded cluster is _____ to provide high availability.
- A Replicated
 - B Partitioned
 - C Clustered
 - D None of the above

Answer: a

Explanation: Replication makes the system fault tolerant since the data is not lost when an individual node fails as the data is redundant across the nodes.

- 10 NoSQL databases exhibit _____ properties.
- A ACID
 - B BASE
 - C Both a and b
 - D None of the above

Answer: b

Conceptual Short Questions with Answers

1 What is a distributed file system?

A distributed file system is an application that stores the files across cluster nodes and allows the clients to access the files from the cluster. Though physically the files are distributed across the nodes, logically it appears to the client as if the files are residing on their local machine.

2 What is failover?

Failover is the process of switching to a redundant node upon the abnormal termination or failure of a previously active node.

3 What is the difference between failover and switch over?

Failover is an automatic mechanism that does not require any human intervention. This differentiates it from the switch over operation, which essentially requires human intervention.

4 What are the types of cluster?

There are types of cluster

- High-availability cluster
- Load-balancing cluster

5 What is a high-availability cluster?

High availability clusters are designed to minimize downtime and provide uninterrupted service when nodes fail. Nodes in a highly available cluster must have

access to a shared storage. Such systems are often used for failover and backup purposes.

6 What is a load-balancing cluster?

Load balancing clusters are designed to distribute workloads across different cluster nodes to share the service load among the nodes. The main objective of load balancing is to optimize the use of resources, minimize response time, maximize throughput, and avoid overload on any one of the resources.

7 What is a symmetric cluster?

Symmetric cluster is a type of cluster structure in which each node functions as an individual computer capable of running applications.

8 What is an asymmetric cluster?

Asymmetric cluster is a type of cluster structure in which one machine acts as the head node, and it serves as the gateway between the user and the remaining nodes.

9 What is sharding?

Sharding is the process of partitioning very large data sets into smaller and easily manageable chunks called shards. The partitioned shards are stored by distributing them across multiple machines called nodes. No two shards of the same file are stored in the same node, each shard occupies separate nodes, and the shards spread across multiple nodes collectively constitute the data set.

10 What is Replication?

Replication is the process of copying the same data blocks across multiple nodes to overcome the loss of data when a node crashes. The copy of a data block is called replica. Replication makes the system fault tolerant since the data is not lost when an individual node fails as the data is redundant across the nodes.

11 What is the difference between replication and sharding?

Replication copies the same data blocks across multiple nodes whereas sharding copies different data across different nodes.

12 What is the master-slave model?

Master-slave configuration is a model where one centralized device known as the master controls one or more devices known as slaves.

13 What is the peer-to-peer model?

In a peer-to-peer configuration there is no master-slave concept, all the nodes have the same responsibility and are at the same level.

14 What is scaling up?

Scaling-up, the vertical scalability, adds more resources to the existing server to increase its capacity to hold more data. The resources can be computation power, hard drive, RAM, and so on. This type of scaling is limited to the maximum scaling capacity of the server.

15 What is Scaling-out?

Scaling out, the horizontal scalability, adds new servers or components to meet the demand. The additional component added is termed as node. Big data technologies work on the basis of scaling out storage. Horizontal scaling enables the system to scale wider to meet the increasing demand. Scaling out storage uses low cost commodity hardware and storage components. The components can be added as required without much complexity. Multiple components connect together to work as a single entity.

16 What is a NewSQL database?

A NewSQL database is designed to provide scalable performance similar to that of NoSQL systems combining the ACID (atomicity, consistency, isolation, and durability), properties of a traditional database management system.

3

NoSQL Database

CHAPTER OBJECTIVE

This chapter answers the question of what NoSQL is and its advantage over RDBMS. Cap theorem, ACID, and BASE properties exhibited by various database systems are explained. We also make a comparison explaining the drawbacks of SQL database and advantages of NoSQL database, which led to the switch over from SQL to NoSQL. It also explains various NoSQL technologies such as key-value database, column store database, document database, and graph database. This chapter expands to show the NoSQL CRUD (create, update, and delete) operations.

3.1 Introduction to NoSQL

In day-to-day operations massive data is generated from all sources in different formats. Bringing together this data for processing and analysis demands a flexible storage system that can accommodate this massive data with varying formats. The NoSQL database is designed in a way that it is best suitable to meet the big data processing demands.

NoSQL is a technology that represents a class of products that does not follow RDBMS principles and are often related to storage and retrieval of massive volumes of data. They find their applications in big data and other real-time web applications. Horizontal scalability, flexible schema, reliability, and fault tolerance are some of the features of NoSQL databases. NoSQL databases are structured in one of the following ways: key-value pairs, document-oriented database, graph database, or column-oriented database.

3.2 Why NoSQL

RDBMS has been the one solution in the past decades for all the database needs. In recent years massive volumes of data are generated most of which are not organized and well structured. RDBMS supports only structured data such as tables with predefined columns. This created the problem for the traditional database management systems of handling these unstructured and voluminous data. The NoSQL database has been adopted in recent years to overcome the drawbacks of traditional RDBMS. NoSQL databases support large volumes of structured, unstructured, and semi-structured data. It supports horizontal scaling on inexpensive commodity hardware. As NoSQL databases are schemaless, integrating huge data from different sources becomes very easy for developers, thus, making NoSQL databases suitable for big data storage demands, which require different data types to be brought into one shell.

3.3 CAP Theorem

CAP is the acronym for consistency, availability, and partition tolerance formulated by Eric Brewer.

Consistency—On performing a read operation the retrieved data is the same across multiple nodes. For example, if three users are performing a read operation on three different nodes, all the users get the same value for a particular column across all the nodes.

Availability—The acknowledgment of success or failure of every read/write request is referred to as the availability of the system. If two users perform a write operation on two different nodes, but in one of the nodes the update has failed, then, in that case the user is notified about the failure.

Partition tolerance—Partition tolerance is the tolerance of the database system to a network partition, and each partition should have at least one node alive, that is, when two nodes cannot communicate with each other, they still service read/write requests so that clients are able to communicate with either one or both of those nodes.

According to Brewer, a database cannot exhibit more than two of the three properties of the CAP theorem. Figure 3.1 depicts different properties of the CAP theorem that a system can exhibit at the same time: consistency and availability (CA), consistency and partition tolerance (CP), or availability and partition tolerance (AP).

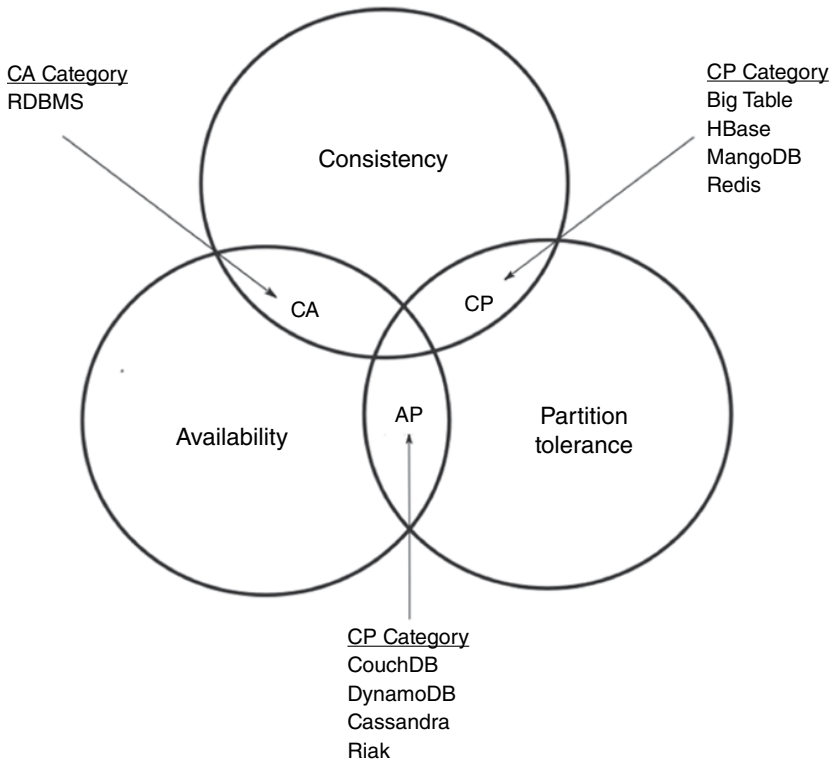


Figure 3.1 Properties of a system following CAP theorem.

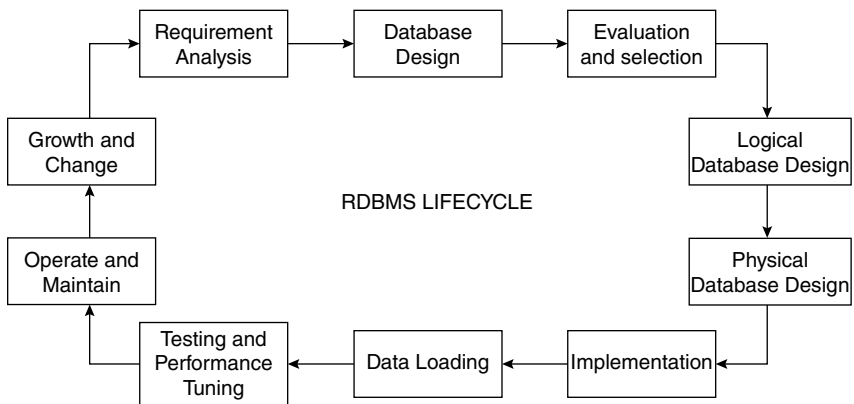


Figure 3.2 RBDMS life cycle.

Consistency and availability (CA)—If the system requires consistency (C) and availability (A), then the available nodes have to communicate to guarantee consistency (C) in the system; hence, network partitioning is not possible.

Consistency and partition tolerance (CP)—If the system requires consistency (C) and partition tolerance (P), availability of the system is affected while consistency is being achieved.

Availability and partition tolerance (AP)—If the system requires availability (A) and partition tolerance (P), consistency (C) of the system is forfeited as the communication between the nodes is broken so the data will be available but with inconsistency.

Relational databases achieve CA (consistency and availability). NoSQL databases are designed to achieve either CP (consistency and partition tolerance) or AP (availability and partition tolerance), that is, NoSQL databases exhibit partition tolerance at the cost of sacrificing either consistency or availability.

3.4 ACID

ACID is the acronym for a set of properties related to database transactions. The properties are atomicity (A), consistency (C), isolation (I), and durability (D). Relational database management systems exhibit ACID properties.

Atomicity (A)—Atomicity (A) is a property that states each transaction should be considered as an atomic unit where either all the operations of a transaction are executed or none are executed. There should not be any intermediary state where operations are partially completed. In the case of partial transactions, the system will be rolled back to its previous state.

Consistency (C)—Consistency is a property that ensures the database will remain in a consistent state after a successful transaction. If the database remained consistent before the transaction executed, it must remain consistent even after the successful execution of the transaction. For example, if a user tries to update a column of a table of type float with a value of type varchar, the update is rejected by the database as it violates the consistency property.

Isolation (I)—Isolation is a property that prevents the conflict between concurrent transactions, where multiple users access the same data, and ensures that the data updated by one user is not overwritten by another user. When two users are attempting to update a record, they should be able to work in isolation without the intervention of each other, that is, one transaction should not affect the existence of another transaction.

Durability (D)—Durability is a property that ensures the database will be durable enough to retain all the updates even if the system crashes, that is, once a

transaction is completed successfully, it becomes permanent. If a transaction attempts to update a data in a database and completes successfully, then the database will have the modified data. On the other hand, if a transaction is committed, but the system crashes before the data is written to the disk, then the data will be updated when the system is brought back into action again.

3.5 BASE

BASE is the acronym for a set of properties related to database design based on the CAP theorem. The set of properties are basically available, soft state, and eventually consistent. NoSQL databases exhibit the BASE properties.

Basically available—A database is said to be basically available if the system is always available despite a network failure.

Soft state—Soft state means database nodes may be inconsistent when a read operation is performed. For example, if a user updates a record in node A before updating node B, which contains a copy of the data in node A, and if a user requests to read the data in node B, the database is now said to be in the soft state, and the user receives only stale data.

Eventual consistency—The state that follows the soft state is eventual consistency. The database is said to have attained consistency once the changes in the data are updated on all nodes. Eventual consistency states that a read operation performed by a user immediately followed by a write operation may return inconsistent data. For example, if a user updates a record in node A, and another user requests to read the same record from node B before the record gets updated, resulting data will be inconsistent; however, after consistency is eventually attained, the user gets the correct value.

3.6 Schemaless Databases

Schemaless databases are those that do not require any rigid schema to store the data. They can store data in any format, be it structured or unstructured. When data has to be stored in RDBMS a schema has to be designed first. A schema is a predefined structure for a database that provides the details about the tables and columns existing in the table and the data types that each column can hold. Before the data can be stored in such a database, the schema has to be defined for it. With a schema database what type of data needs to be stored has to be known

in advance, whereas with a schemaless database it is easy to store any type of data without prior knowledge about the data; also, it allows storing data with each record holding different set fields. Storing this kind of data in a schema database will make the table a total mess with either a lot of null or meaningless columns.

NoSQL database is a schemaless database, where storing data is much easier compared to the traditional database. A key-value type of NoSQL database allows the user to store any data under a key. A document-oriented database does not put forward any restrictions on the internal structure of the document to be stored. A column-store database allows the user to store any data under a column. A graph database allows the user to add edges and add properties to them without any restrictions.

3.7 NoSQL (Not Only SQL)

NoSQL database is a non-relational database designed to store and retrieve semi-structured and unstructured data. It was designed to overcome big data's scalability and performance issues, which traditional databases were not designed to address. It is specifically used when organizations need to access, process, and analyze a large volume of unstructured data. Unlike the traditional database systems, which organize the data in tables, the NoSQL database organizes data in key/value pairs, or tuples. As the name suggests, a NoSQL database supports not only SQL but also other query languages, namely, HQL to query structured data, XQuery to query XML files, SPARQL to query RDF data, and so forth. The most popular NoSQL database implementations are Cassandra, SimpleDB, and Google Bigtable.

3.7.1 NoSQL vs. RDBMS

RDBMS are schema-based database systems as they first create a relation or table structure of the given data to store them in rows and columns and use primary key and foreign key. It takes a significant amount of time to define a schema, but the response time to the query is faster. The schema can be changed later, but that also requires a significant amount of time.

Unlike RDBMS, NoSQL (Not Only SQL) databases don't have a stringent requirement for the schema. They have the capability to store the data in HDFS as it arrives and later a schema can be defined using Hive to query the data from the database. Figure 3.3 illustrates the differences between RDBMS and NoSQL. Figure 3.2 shows the life cycle of RDBMS.

RDBMS	NoSQL
Structured data with a rigid schema.	Structured, Unstructured, Semi-Structured data with a flexible schema.
Extract, Transform, Load (ETL) required.	ETL is not required.
Storage in rows and columns.	Data are stored in Key/Value pairs database, Columnar database, Document database, Graph Database.
RDBMS is based on ACID transactions. ACID stands for Atomic, Consistent, Isolated and Durable.	NoSQL is based on BASE transactions. BASE stands for Basically available, Soft state, Eventual consistency.
RDBMS Scale up when the data load increases, i.e., expensive servers are brought to handle the additional load.	NoSQL is highly scalable at low cost. NoSQL scales out to meet the extra load. i.e., low-cost commodity servers are distributed across the cluster.
SQL server, Oracle, and MySQL are some of the examples.	MongoDB, HBase, Cassandra are some of the examples.
Structured Query Language is used to query the data stored in the data warehouse.	Hive Query Language (HQL) is used to query the data stored in HDFS.
Matured and stable. Matured indicates that it is in existence for a number of years.	Flexible, Incubation. Incubation indicates that it is in existence from the recent past.

Figure 3.3 RDBMS vs. NoSQL databases.

3.7.2 Features of NoSQL Databases

Schemaless—NoSQL database is a schemaless database where storing data is much easier compared to the traditional database. Since SQL databases have a rigid schema a lot of upfront work has to be done before storing the data in the database, while in a NoSQL database, which is schemaless, the data can be stored without previous knowledge about the schema.

Horizontal scalability—Unlike SQL databases, which have vertical scalability, NoSQL databases have horizontal scalability. They have the capability to grow dynamically with rapidly changing requirements. Horizontal scalability is implemented through sharding and replication where the database files are shared across multiple servers, and the files are replicated to make the system fault tolerant in the event of planned maintenance or events or outages. NoSQL supports both manual and automatic sharding. Also NoSQL databases support automatic replication across multiple geographic locations to withstand regional failures.

Distributed computing—Distributed computing allows the data to be stored in more than one device, increasing the reliability. A single large data set is split and stored across multiple nodes, and the data can be processed in parallel.

Lower cost—SQL databases use highly reliable and high performance servers since they are vertically scalable, whereas NoSQL databases can work on low-cost, low-performing commodity hardware, since they are horizontally scalable. They allow adding cheap servers to meet the increasing demand for storage and processing hardware.

Non-Relational—Relational databases are designed to recognize how the tables stored relate to each other. For example, in online retailing a single product row relates to many customer rows. Similarly each customer row can relate to multiple product rows. This concept of relationship is eliminated in a non-relational database. Here each product has the customer embedded with it. It means the customer is duplicated with every product row that uses it. Doing so will require additional space but has the advantage of easy storage and retrieval.

Handles large a volume of data—Relational databases are capable of handling tables with even millions of records, which were considered massive data in the past. But today in the digital world this data is meager, and tables have grown to billions and trillions of rows. Also the RDBMS were confined to handle only the data that would fit into the table structure. NoSQL databases are capable of handling this massive growth in data more efficiently. They are capable of handling a massive volume of structured, unstructured, and semi-structured data.

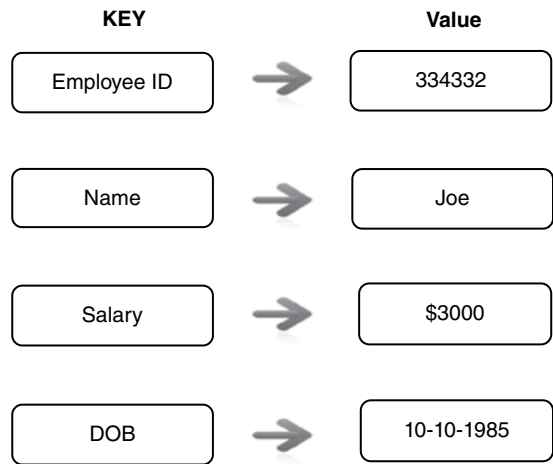
3.7.3 Types of NoSQL Technologies

- 1) Key-value store database
- 2) Column-store database
- 3) Document database
- 4) Graph database

3.7.3.1 Key-Value Store Database

A key-value store database is the simplest and most efficient database that can be implemented easily. It allows the user to store data in key-value pairs without any schema. The data is usually split into two parts: key and value. The key is a string, and the value is the actual data; hence the reference key-value pair. The implementation of the key-value database is similar to hash tables. The retrieval of data is with the key as the index. There are no alternate keys or foreign keys as with the case of RDBMS, and they are much faster than RDBMS. A practical application of key-value data store includes online shopping cart and store session information in online gaming in the case of multiplayer games. Figure 3.4 illustrates a

Figure 3.4 A key-value store database.



key-value database where Employee id, Name, Salary, and Date of birth are the key, and the data corresponding to it is the value. Amazon DynamoDB is a NoSQL database released by Amazon. Other key-value databases are Riak, Redis, Berkeley DB, Memcached, and Hamster DB. Every database is created to handle new challenges, and each of them is used to solve different challenges.

3.7.3.1.1 Amazon DynamoDB Amazon DynamoDB was developed by Amazon to meet the business needs of its e-commerce platform, which serves millions of customers. Amazon requires highly reliable and scalable storage technology that is always available. The customers should not have any interruption in case of any failure in the system, that is, they should be able to shop and add items to their cart even if there is a failure. So Amazon's systems should be built in such a way that they handle a failure without having any impact on their performance and their availability to the customers.

To meet this reliability and scalability requirements, Amazon has developed a highly available and cost-effective storage platform: Amazon DynamoDB. Some of the key features of Amazon DynamoDB are: it is schemaless, simple, and fast, pays only for the space consumed, it is fault-tolerant, and has automatic data replication. Amazon DynamoDB meets most of the Amazon service requirements such as customer shopping cart management, top selling product list, customer session management, and product catalog for which the use of a traditional database would be inefficient and limits scalability and availability. Amazon DynamoDB provides a simple primary key access to the data store to meet the requirements.

3.7.3.1.2 Microsoft Azure Table Storage There are several cloud computing platforms provided by different organizations. Microsoft Azure Table Storage is one such platform developed by Microsoft, intended to store a large amount of unstructured data. It is a non-relational, schemaless, cost-effective, massively scalable, easy to adopt, key-value pair storage system that provides fast access to the data. Here the key-value pairs are named as properties, useful for retrieving the data based on specific selection criteria. A collection of properties are called entities, and a group of entities forms the table. Unlike a traditional database, entities of the Azure table need not hold similar properties. There is no limit for the data to be stored in a single table, and the restriction is only on the entire azure storage account, which is 200 terabytes.

3.7.3.2 Column-Store Database

A column-oriented database stores the data as columns instead of storing them as rows. For better understanding, here the column database is compared with the row-oriented database and explained how just a difference in the physical layout of the same data improves performance.

Employee_Id	Name	Salary	City	Pin_code
3623	Tony	\$6000	Huntsville	35801
3636	Sam	\$5000	Anchorage	99501
3967	Williams	\$3000	Phoenix	85001
3987	Andrews	\$2000	Little Rock	72201

Row store database

Employee_Id	Name	Salary	City	Pincode
3623	Tony	\$6000	Huntsville	35801
3636	Sam	\$5000	Anchorage	99501

Column store database

The working method of a column store database is that it saves data into sections of columns rather than sections of rows. Choosing how the data is to be stored, row-oriented or column-oriented, depends on the data retrieval needs. OLTP (online transaction processing) data retrieves less number of rows and more columns, so the row-oriented database is suitable. OLAP (online analytical processing) retrieves fewer columns and more rows, so the column-oriented

database is suitable. Let us consider an example of online shopping to have a better understanding of this concept.

Table: Order

Product_ID	Total_Amount	Product_desc
1000	\$250	AA
1023	\$800	BB
1900	\$365	CC

Internally row database will be stored like this,

1000 = 1000,\$250,AA; 1023 = 1023,\$800,BB; 1900 = 1900,\$365,CC;

Column Database will be stored like this,

Product_ID = 1000:1000, 1023:1023, 1900:1900; Total_Amount = 1000:\$250, 1023:\$800, 1900:\$365;

Product_desc = 1000:AA, 1023:BB, 1900:CC;

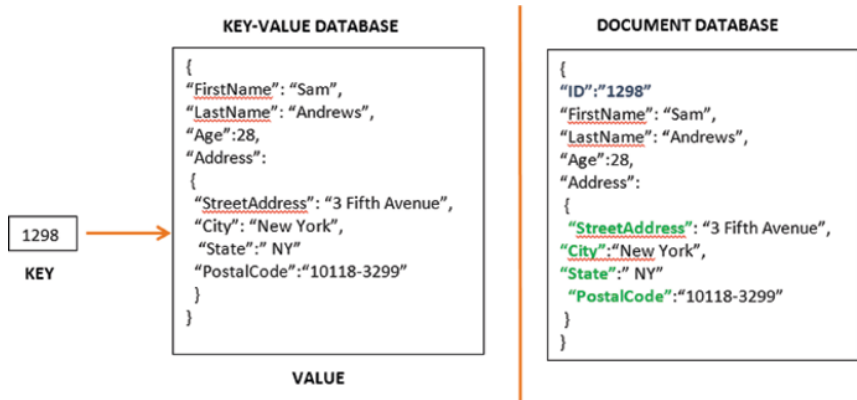
This is a simple order table with a key and a value, where data are stored in rows. If the customer wishes to see one of the products, it is easy to retrieve if the data is stored row oriented, since only very few rows are to be retrieved. If it is stored in the column-oriented database, to retrieve the data for such a query, the database system has to traverse all the columns and check the Product_ID against each individual column. This shows the difference in style of data retrieval between OLAP and OLTP. Apache Cassandra and Apache HBase are examples of column-oriented databases.

3.7.3.2.1 Apache Cassandra Apache Cassandra was developed by Facebook. It is distributed, fault tolerant, and handles the massive amount of data across several commodity servers, providing high scalability and availability without compromising the performance. It is both a key-value and column-oriented database. It is based on Amazon DynamoDB and Google's Bigtable. The Cassandra database is suitable for applications that cannot afford to lose their data. It automatically replicates its data across nodes of the cluster for fault tolerance. Its features such as continuous availability, linear scalability (increased throughput with the increase in the number of nodes), flexibility in data storage, and data distribution cannot be matched with other NoSQL databases.

Cassandra adopts a ring design that is easy to set up and maintain. In the Cassandra architecture, all the nodes are playing identical roles, that is, there is no master-slave architecture. Cassandra is best suitable to handle a large number of

concurrent users and handling the massive amount of data. It is used by Facebook, Twitter, eBay, and others.

3.7.3.3 Document-Oriented Database



A document-oriented database is horizontally scalable. When data load increases, more commodity hardware is added to distribute the data load. This database is designed by adopting the concept of a document. Documents encapsulate data in XML, JSON, YAML, or binary format (PDF, MS Word). In a document-oriented database the entire document will be treated as a single, complete unit.

DOCUMENT DATABASE

KEY-VALUE DATABASE

```
{
  "ID": "1298"
  "FirstName": "Sam",
  "LastName": "Andrews",
  "Age": 28,
  "Address":
  {
    "StreetAddress": "3 Fifth Avenue",
    "City": "New York",
    "State": " NY"
    "PostalCode": "10118-3299"
  }
}
```

```

"FirstName": "Sam",
"LastName": "Andrews",
"Age": 28,
"Address":
{
  "StreetAddress": "3 Fifth Avenue",
  "City": "New York",
  "State": " NY"
  "PostalCode": "10118-3299"
}
}

```

VALUE

The data stored in a document-oriented database can be queried using any key instead of querying only with a primary key. Similar to RDBMS, document-oriented databases allow creating, reading, updating, and deleting the data.

Examples of document-oriented databases are MongoDB, CouchDB, and Microsoft DocumentDB.

3.7.3.3.1 CouchDB CouchDB, the acronym for a cluster of unreliable commodity hardware, is a semi-structured document-oriented NoSQL database that uses JavaScript as the query language and JSON to store data. The CouchDB database has a flexible structure to store documents where the structure of the data is not a constraint, and each database is considered as a collection of documents. It exhibits ACID properties, and it can handle multiple readers and writers concurrently without exhibiting any conflict. Any number of users can read the documents without any kind of interruption from concurrent updates. The database readers are never put in a wait state or locked out for other readers or writers to complete their current action. CouchDB never overwrites the data that are committed, which ensures that the data are always in a consistent state. Multi-version concurrency control (MCC) is the concurrency method adopted by CouchDB, where each user has the flexibility to see a snapshot of the database at an instant of time.

3.7.3.4 Graph-Oriented Database

The graph-oriented database stores the entities also known as nodes and the relationships between them. Each node has properties, and the relationships between the nodes are known as edges. The relationships have properties and directional significance. The properties of the relationships are used to query the graph database. These properties may represent the distance between the nodes, or for example, a relationship between a company and an employee may represent the properties, namely, the number of years of experience of the employee, the role of the employee, and so on.

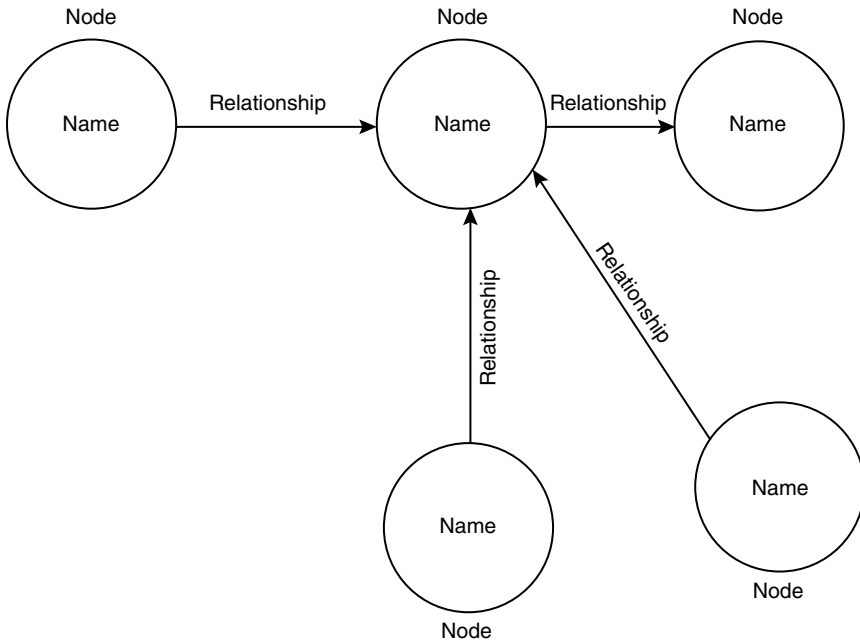


Figure 3.5 General representation of graph database.

Different types of graph databases are Neo4J, InfiniteGraph, HypherGraphDB, AllegroGraph, GraphBase, and orientDB. Figure 3.5 represents a graph database.

3.7.3.4.1 Neo4J Neo4J is an open-source, schemaless NoSQL graph database written in Java and Cypher query language (CQL) is used to query the database. In Neo4J all the input data are represented by a node, relationships, and their properties. These properties are represented as key-value pairs. All the nodes have an id, which may be a name, employee id, date of birth, age, and so on. A Neo4J database handles unstructured or semi-structured data easily as the properties of all the nodes are constrained to be the same.

Consider an example of a graph database that represents a company, the employees, and the relationship between the employees and the relationship between the employees and the company. Figure 3.6 shows ABC Company with its employees. The nodes represent the name of the company and the employees of the company. The relationship has properties, which describe the role of the employee, number of years of experience, and the relationships that exist among the employees.

3.7.3.4.2 Cypher Query Language (CQL) Cypher query language is Neo4j's graph query language. CQL is simple yet powerful. Some of the basic commands of CQL are given below.

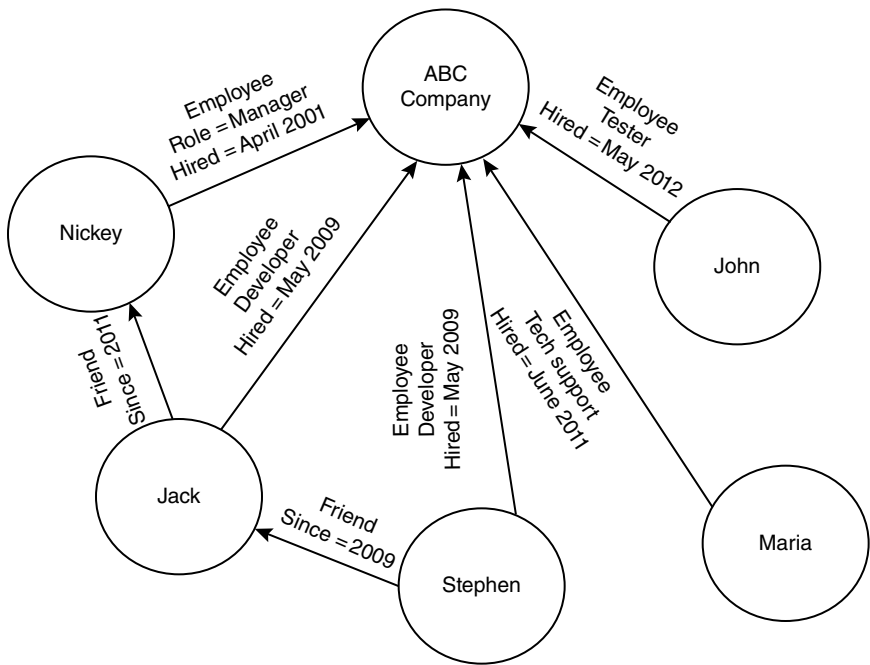


Figure 3.6 Neo4J Relationships with properties.

A node can be created using the CREATE clause. The basic syntax for the CREATE clause is:

```
CREATE (node_name)
```

Let employee be the name of the node.

```
CREATE (employee)
```

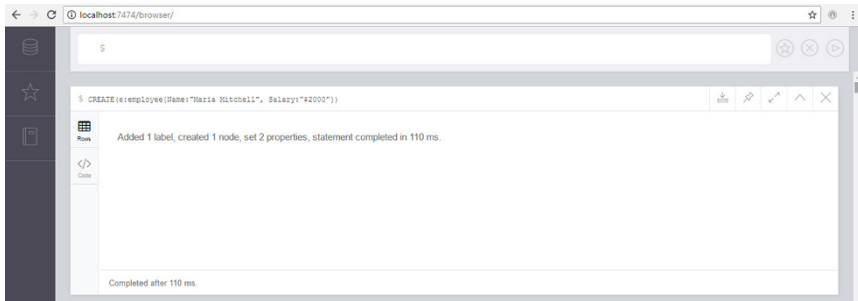
To create a node with a label 'e' the following syntax is used:

```
CREATE (e:employee)
```

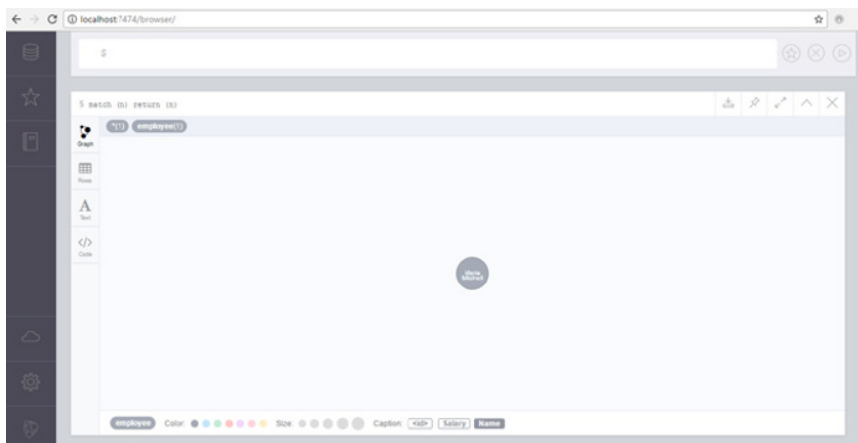


A node can be created along with properties. For example, Name, Salary are the properties of the node employee.

```
CREATE (employee{Name:"Maria Mitchell", Salary:$2000})
```



Match (n) Return (n) command is used to view the created node.



Relationships are created using the CREATE clause:

```
CREATE (node1) -[r:relationship]-> (node2)
```

Relationship flows from node1 to node2.

```
CREATE (c:Course{Name:"Computer Science"})
CREATE (e:employee{Name:"Maria Mitchell"}) -[r:Teaches]->
(c:Course{Name:"Computer Science"})
```



Neo4J Relationship example.

Let us consider an example where three tables that hold the details of employees, location of department, and the courses with the name of the faculty teaching the course. The steps below are taken to establish the relationship among the employee table, Dept_Location Table, Courses Table. The established relationship is depicted through neo4J graph.

Step 1: Create Emp node with properties Name, Salary, Gender, Address, Department.

Step 2: Create Course node with properties Name and Course

Step 3: Here step 1 and step 2 are merged where node is created as well as the relationship between them is established. Relationship “Teaches” is established between employee and course (e.g., Gary teaches Big Data).

Step 4: Create Dept node with properties Name and Location.

Step 5: Establish the relationship “worksfor” between the Emp node and the Dept node (e.g., Mitchell worksfor Computer Science).

Employee Table

Name	Salary	Gender	Address	Department
Mitchell	\$2000	Male	Miami	Computer Science
Gary	\$6000	Male	San Francisco	Information Technology
Jane	\$3000	Female	Orlando	Electronics
Tom	\$4000	Male	Las Vegas	Computer Science

Dept_Location Table

Name	Location
Computer Science	A Block
Information Technology	B Block
Electronics	C Block

Courses Table

Name	Course
Mitchell	Databases
Mitchell	R language
Gary	Big Data
Jane	NoSQL
Tom	Machine Learning

Below commands are used to create the relationship table between Employee table and Coures table.

```
create(e:Emp{Name:"Mitchell",Salary:2000,Gender:"Male",
Address:"Miami",Department:"Computer Science"})-
[r:Teaches]->(c:Course{Name:"Mitchell",
Course:"Databases"})

create(e:Emp{Name:"Gary",Salary:6000,Gender:"Male",
Address:"San Francisco", Department:"Information
Technology"})-[r:Teaches]->(c:Course{Name:"Gary",
Course:"Big Data"})

create(e:Emp{Name:"Jane",Salary:3000,Gender:"Fale",
Address:"Orlando",Department:"Electronics"})-
[r:Teaches]->(c:Course{Name:"Jane",Course:"NoSQL"})

create(e:Emp{Name:"Tom",Salary:4000,Gender:"Male",
Address:"Las Vegas",Department:"Computer Science"})-
[r:Teaches]->(c:Course{Name:"Tom",Course:"Machine
Learning"})

create(c:Course{Name:"Mitchell",Course:"R Language"})

Match(e:Emp{Name:"Mitchell"},(c:Course{Course:
"R Language"})) create(e) -[r:teaches]->(c)
```




Figure 3.7 Relationship graph between course and employee.

Figure 3.7 shows Neo4j graph after creating emp node and course node and establishing the relationship “Teaches” between them. Match(n) return (n) - command returns the graph below.

Below commands are used to create Dept Node with properties Name and Location;

```
create(d:Dept{Name:"Computer Science", Location:
"A Block"})
create(d:Dept{Name:"Information Technology",Location:
"B Block"})
create(d:Dept{Name:"Electronics",Location:"C Block"})
```

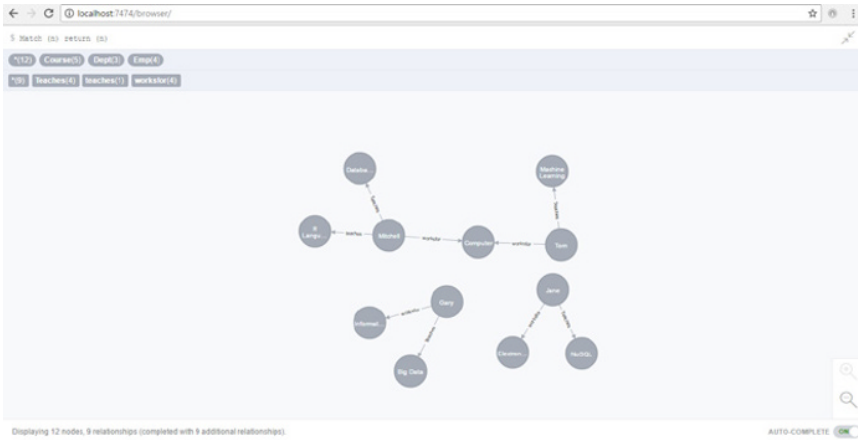
Below commands are used to create relationship between dept node and emp node;

```
Match(e:Emp{Name:"Mitchell"}, (d:Dept{Name:"Co
mputer Science", Location:"A Block"}) create(e)
-[r:worksfor]->(d)

Match(e:Emp{Name:"Gary"}, (d:Dept{Name:"Informat
ion Technology", Location:"B Block"}) create(e)
-[r:worksfor]->(d)

Match(e:Emp{Name:"Jane"}, (d:Dept{Name:"Electronics",
Location:"C Block"}) create(e) -[r:worksfor]->(d)

Match(e:Emp{Name:"Tom"}, (d:Dept{Name:"Computer Science",
Location:"A Block"}) create(e) -[r:worksfor]->(d)
```



3.7.4 NoSQL Operations

The set of NoSQL operations is known as CRUD, which is the acronym for create, read, update, and delete. Creating a record for the first time involves creating a new entry. Before creating a new entry, the record has to be identified to find out if the record already exists. These records are stored within the table, and a unique key called primary key can be used to identify the records uniquely. The primary key of the record that has to be checked whether it already exists is retrieved and checked if it already exists. If the record already exists, it will be updated instead of recreated. The various commands used in the MongoDB database are explained below.

Create database—The command `use DATABASE_NAME` creates a database. This command does two operations alternatively. It creates a new database if it does not exist; alternatively the command will return the existing database if a database already exists in the same name.

Syntax:

```
use DATABASE_NAME
```

Example: If a database has to be created with a name `studentdb`, the command given below is used.

```
>use studentdb
```

Few other commands to show the selected database and to see the list of databases are:

Command to show the database that has been selected

```
>db
```

Command to show the list of available databases

```
>show dbs
```

This command will show the databases that are currently available. It will not list the database without any record in it. To display a database a record has to be inserted into it. The command given below is used to insert a document into a database.

```
>db.studCollection.insert
(
{
  "StudentId":15,
  "StudentName": "George Mathew"
}
)
```

The first part of the command is used to insert a document into a database where studCollection is the name of the collection. A collection with a name studCollection will be created and document will be inserted. The statements within the curly braces are used to add field name and their corresponding values. On successful execution of the command the document will be inserted into the database.

Drop Database -The command db.dropDatabase() drops an existing database. If a data database exists, then it will be deleted; otherwise, the default database, test, will be deleted. To delete a database it has to be first retrieved and then the dropDatabase command has to be executed.

Syntax:

```
Db.dropDatabase()
```

Example

```
>usestudentdb
>db.dropDatabase()
```

Create collection—The command db.createCollection(name, options) is used to create a collection, where name is the name of the collection and is of type string, and options is the memory size, indexing, maximum number of documents, and so forth, which are optional to be mentioned and is of type document.

Another method of creating a collection is to insert a record into a collection. An insert command will automatically create a new collection if the collection in the statement does not exist.

Syntax:

```
db.createCollection(name, {capped: <Boolean>,
                           size: <number>,
                           max: <number>
                          }
)
```

Capped—Capped collection is type of collection where older entries are automatically overwritten when the maximum size specified is reached. It is mandatory to specify the maximum size in the size field if the collection is capped. If a capped collection is to be created, the Boolean value should be true.

Size—Size is the maximum size of the capped collection. Once the capped collection reaches the maximum size, older files are overwritten. Size is specified for a capped collection and ignored for other types of collections.

Max—Max is the maximum number of documents allowed in a capped collection. Here the size limit is given priority. When the size reaches the maximum limit before the maximum number of documents is reached, the older documents are overwritten.

Example:

```
>use studentdb
>db.createCollection("firstcollection", { capped : true,
                                         size : 1048576,
                                         max : 5000
                                       }
)
```

On successful execution of the command, a new 'firstcollection' will be created, and the collection will be capped with maximum size of 1 MB and the maximum number of documents allowed will be 5000.

Drop collection—The command `db.collection_name.drop()` drops a collection from the database.

Syntax

```
db.collection_name.drop()
```

Example

```
>db.firstcollection.drop()
```

The above command will drop the 'firstcollection' from the studentdb database.

Insert Document – The command `insert()` is used to insert a document into a database.

Syntax:

```
db.collection_name.insert (document)
```

Example

```
>db.studCollection.insert
([
{
  "StudentId":15,
  "StudentName": "George Mathew",
  "CourseName": "NoSQL",
  "Fees":5000
},
{
  "StudentId":17,
  "StudentName": "Richard",
  "CourseName": "DataMining",
  "Fees":6000
},
{
  "StudentId":21,
  "StudentName": "John",
  "CourseName": "Big Data",
  "Fees":10000
},
])
```

Update Document—The command `update()` is used to update the values in a document.

Syntax:

```
db.collection_name.update(criteria, update)
```

‘Criteria’ is to fetch the record that has to be updated, and ‘update’ is the replacement value for the existing value.

Example:

```
db.firstcollection.update(
    { "coursename":
      "Big Data" }
)
```

Delete Document—The command `remove()` is used to delete a document from a collection.

Syntax:

```
db.collection_name.remove(criteria)
```

Example:

```
db.firstcollection.remove(
    { "StudentId":15 }
)
```

Query Document – The command `db.collection.find()` is used to query data from a collection.

Syntax:

```
db.collection.find()
```

Example:

```
db.collection.find
(
  {
    "StudentName": "George Mathew"
  }
)
```

3.8 Migrating from RDBMS to NoSQL

Data generated in recent times has a broader profile in terms of size and shape. This tremendous volume has to be harnessed to extract the underlying knowledge and make business decisions. The global ease of the Internet is the major reason for the generation of massive volumes of unstructured data. Classical relational databases no longer support the profile of the data being generated. The boom in the data of huge volume and highly unstructured is one of the major reasons why the relational databases are no more the only databases to be relied on. One of the contributing factors to this boom is social media, where everybody wants to share with others the happenings related to them by means of audio, video, pictures, and textual data. We can very well see that data created by the web has no specific structural boundaries. This has mandated the invention of a database that is non-relational and schemaless.

It is evident that there is a need for an efficient mechanism to deal with such data. Here comes into picture the non-relational and schemaless database NoSQL. It differs from traditional relational database management systems in some significant aspects. Drawbacks of traditional relational database are:

- Entire schema should be known upfront
- Rigid structure where the properties of every record should be the same

- Scalability is expensive
- Fixed schema makes it difficult to adjust to needs of the applications
- Altering schema is expensive

Below are the advantages of NoSQL, which led to the migration from RDBMS to NoSQL:

- Open-source and distributed
- High scalability
- Handles structured, unstructured, and semi-structured data
- Flexible schema
- No complex relationships

Chapter 3 Refresher

1 Which among the following databases is not a NoSQL database?

- A MongoDB
- B SQL Server
- C Cassandra
- D None of the above

Answer: a

Explanation: SQL Server is an RDBMS developed by Microsoft.

2 NoSQL databases are used mainly for handling large volumes of _____ data.

- A unstructured
- B structured
- C semi-structured
- D All of the above

Answer: a

Explanation: MongoDB is a typical choice for unstructured data storage.

3 Which of the following is a column store database?

- A Cassandra
- B Riak
- C MongoDB
- D Redis

Answer: a

Explanation: Column-store databases such as Hbase and Cassandra are optimized for queries over very large data sets and store data in columns, instead of rows.

- 4 Which of the following is a NoSQL database type?
- A SQL
 - B Document databases
 - C JSON
 - D All of the above

Answer: b

Explanation: Document databases pair each key with a complex data structure known as a document.

- 5 The simplest of all the databases is _____.
- A key-value store database
 - B column-store database
 - C document-oriented database
 - D graph-oriented database

Answer: a

Explanation: Key-value store database is the simplest and most efficient database that can be implemented easily. It allows the user to store data in key-value pairs without any schema.

- 6 Many of the NoSQL databases support auto _____ for high availability.
- A scaling
 - B partition
 - C replication
 - D sharding

Answer: c

- 7 A _____ database stores the entities also known as nodes and the relationships between them.
- A key-value store
 - B column-store
 - C document-oriented
 - D graph-oriented

Answer: d

Explanation: A graph-oriented database stores the entities also known as nodes and the relationships between them. Each node has properties, and the relationships between the nodes are known as the edges.

- 8 Point out the wrong statement.
- A CRUD is the acronym for create, read, update, and delete.
 - B NoSQL databases exhibit ACID properties.

- C NoSQL is a schemaless database.
- D All of the above.

Answer: b

Explanation: NoSQL exhibits BASE properties.

- 9 Which of the following operations create a new collection if the collection does not exist?
- A Insert
 - B Update
 - C Read
 - D All of the above.

Answer: a

Explanation: An insert command will automatically create a new collection if the collection in the statement does not exist.

- 10 The maximum size of a capped collection is determined by which of the following factors?
- A Capped
 - B Max
 - C Size
 - D None of the above

Answer: b

Explanation: Size is the maximum size of the capped collection. Once the capped collection reaches the maximum size, older files are overwritten. Size is specified for a capped collection and ignored for other types of collections.

Conceptual Short Questions with Answers

1 What is a schemaless database?

Schemaless databases are those that do not require any rigid schema to store the data. They can store data in any format, be it structured or unstructured.

2 What is a NoSQL Database?

A NoSQL, or Not Only SQL, database is a non-relational database designed to store and retrieve semi-structured and unstructured data. It was designed to overcome big data's scalability and performance issues, which traditional databases were not designed to address. It is specifically used when organizations need to access, process, and analyze a large volume of unstructured data.

3 What is the difference between NoSQL and a traditional database?

RDBMS is a schema-based database system as it first creates a relation or table structure of the given data to store them in rows and columns and uses primary key and foreign key. It takes a significant amount of time to define a schema, but the response time to the query is faster. The schema can be changed later, but this requires a significant amount of time.

Unlike RDBMS, NoSQL databases don't have a stringent requirement for the schema. They have the capability to store the data in HDFS as it arrives and later a schema can be defined using Hive to query the data from the database.

4 What are the features of NoSQL database?

- Schemaless
- Horizontal scalability
- Distributed computing
- Low cost
- Non-relational
- Handles large volume of data

5 What are the types of NoSQL databases?

The four types of NoSQL databases are:

- Key-value store database
- Column-store database
- Document database
- Graph database

6 What is a key-value store database?

A key-value store database is the simplest and most efficient database that can be implemented easily. It allows the user to store data in key-value pairs without any schema. The data is usually split into two parts: key and value. The key is a string, and the value is the actual data; hence the reference key-value pair.

7 What is a graph-oriented database?

A graph-oriented database stores the entities also known as nodes and the relationships between them. Each node has properties and the relationships between the nodes are known as edges. The relationships have properties and directional significance. The properties of the relationships are used to query the graph database.

8 What is a column-store database?

A column-oriented database stores the data as columns instead of rows. A column store database saves data into sections of columns rather than sections of rows.

9 What is a document-oriented database?

This database is designed by adopting the concept of a document. Documents encapsulate data in XML, JSON, YAML, or binary format (PDF, MS Word). In a document-oriented database the entire document will be treated as a record.

10 What are the various NoSQL operations?

The set of NoSQL operations is known as CRUD, which is the acronym for create, read, update, and delete.

4

Processing, Management Concepts, and Cloud Computing

Part I: Big Data Processing and Management Concepts

CHAPTER OBJECTIVE

This chapter deals with concepts behind the processing of big data such as parallel processing, distributed data processing, processing in batch mode, and processing in real time. Virtualization, which has provided an added level of efficiency to big data technologies, is explained with various attributes and its types, namely, server, desktop, and storage virtualization.

4.1 Data Processing

Data processing is defined as the process of collecting, processing, manipulating, and managing the data to generate meaningful information to the end user. Data becomes information only when it undergoes a process by which it is manipulated and organized. There is no specific point to determine when the data becomes information. A set of numbers and letters may appear meaningful to one person, while it doesn't carry any meaning to another. Information is identified, defined, and analyzed by the users based on its purpose.

Data may be originated from diversified sources in the form of transactions, observations, and so forth. Data may be recorded in paper form and then converted into a machine readable form or may be recorded directly in a machine readable form. This collection of data is termed as data capture.

Once data is captured, data processing begins. There are basically two different types of data processing, namely, centralized and distributed data processing. Centralized data processing is a processing technique that requires minimal resources and is suitable for organizations with one centralized location for service. Figure 4.1 shows the data processing cycle.

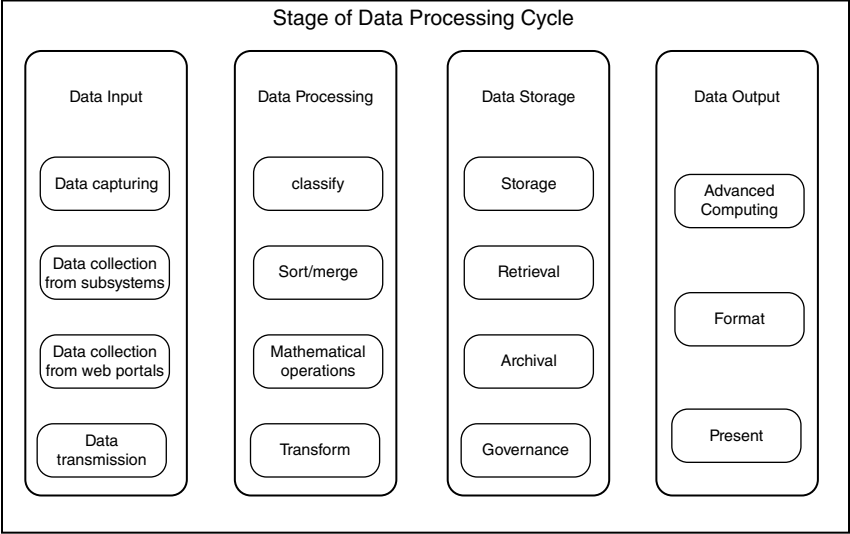


Figure 4.1 Data processing cycle.

Distributed processing is a processing technique where data collection and processing are distributed across different physical locations. This type of processing overcomes the shortcomings of centralized data processing, which mandates data collection to be at one central location. Distributed processing is implemented by several architectures, namely, client-server architecture, three-tier architecture, n-tier architecture, cluster architecture, and peer-to-peer architecture. In client-server architecture, client manages the data collection and its presentation while data processing and management are handled by the server. But this kind of architecture introduces a latency and overhead in carrying the data between the client and the server. Three-tier architecture and n-tier architecture isolate servers, applications, and middleware into different tiers for better scalability and performance. This kind of architectural design enables each tier to be scaled independently of others based on demand. The cluster is an architecture where machines are connected together to form a network and process the computation in parallel fashion to reduce latency. Peer-to-peer is a type of architecture where all the machines have equal responsibilities in data processing.

Once data is captured, it is converted into a form that is suitable for further processing and analysis. After conversion, data with similar characteristics are categorized into similar groups. After classifying, the data is verified to ensure accuracy. The data is then sorted to arrange them in a desired sequence. Data are usually sorted as it becomes easier to work with the data if they are arranged in a logical sequence. Arithmetic manipulations are performed on the data if required. Records of the data may be added, subtracted, multiplied, or divided. Based on the

requirements, mathematical operations are performed on the data and then it is transformed into a machine sensible form.

After capturing and manipulating the data, it is stored for later use. The storing activity involves storing the information or data in an organized manner to facilitate the retrieval. Of course data has to be stored only if the value of storing them for future use exceeds the storage cost. The data may be retrieved for further analysis. For example, business analysts may compare current sales figures with the previous year's to analyze the performance of the company. Hence, storage of data and its retrieval is necessary to make any further analysis. But with the increase in big data volume, moving the data between the computing and the storage layers for storage and manipulation has always been a challenging task.

4.2 Shared Everything Architecture

Shared everything architecture is a type of system architecture sharing all the resources such as storage, memory, and processor. But this type of architecture limits scalability. Figure 4.2 shows the shared everything architecture. Distributed shared memory and symmetric multiprocessing are the types of shared everything architecture.

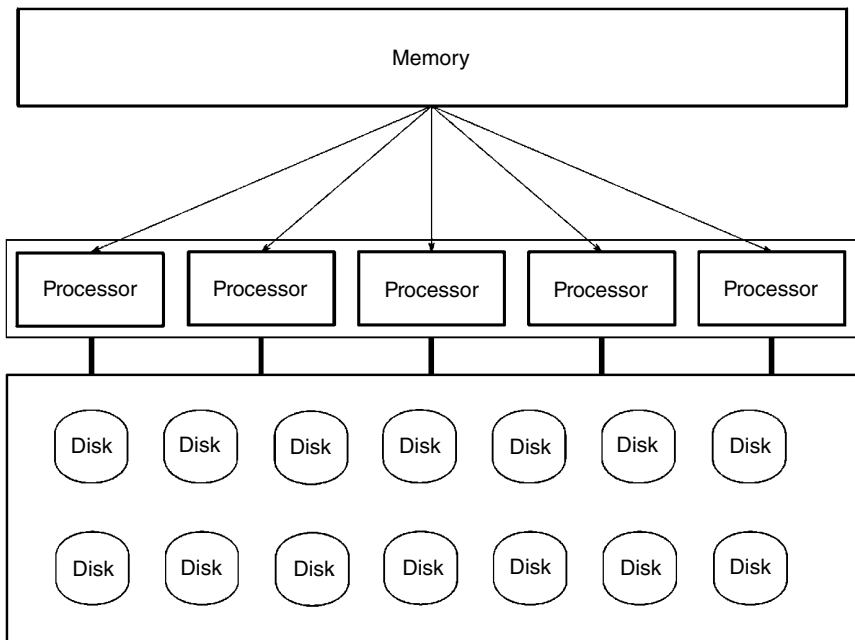


Figure 4.2 Shared everything architecture.

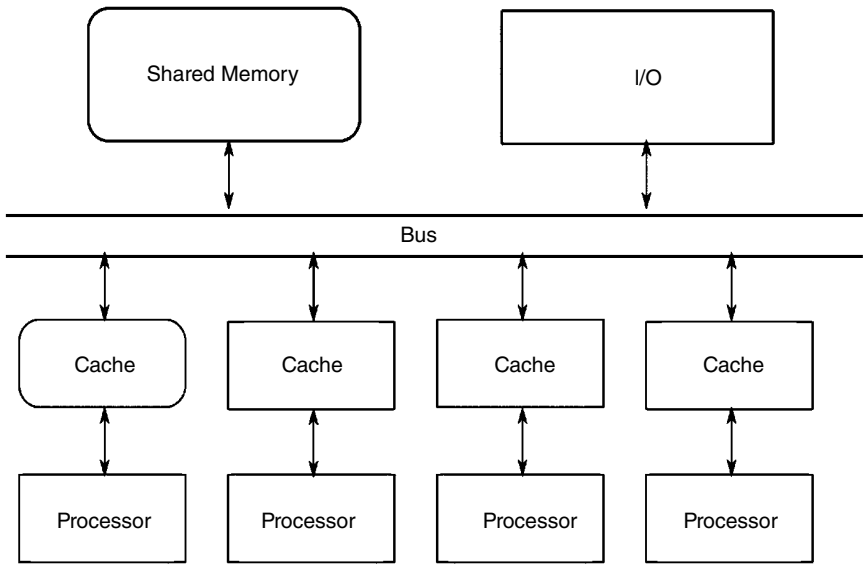


Figure 4.3 Symmetric multiprocessing memory.

4.2.1 Symmetric Multiprocessing Architecture

In the symmetric multiprocessing architecture, a single memory pool is shared by all the processors for concurrent read-write access. This is also referred to as uniform memory access. When multiple processors share a single bus, it results in bandwidth choking. This drawback is overcome in distributed shared memory architecture.

4.2.2 Distributed Shared Memory

Distributed shared memory is a type of memory architecture that provides multiple memory pools for the processors. This is also called non-uniform memory access architecture. Latency in this architecture depends on the distances between the processors and their corresponding memory pools. Figure 4.4 shows distributed shared memory.

4.3 Shared-Nothing Architecture

Shared-nothing architecture is a type of distributed system architecture that has multiple systems interconnected to make the system scalable. Each system in the network is called a node and has its own dedicated memory, storage, and disks independent of other nodes in the network, thus making it a shared-nothing architecture. The infinite scalability of this architecture makes it suitable for Internet and web applications. Figure 4.5 shows a shared-nothing architecture.

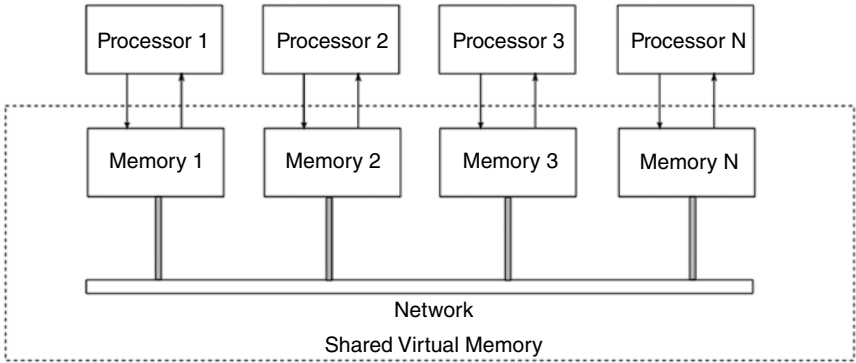


Figure 4.4 Distributed shared memory.

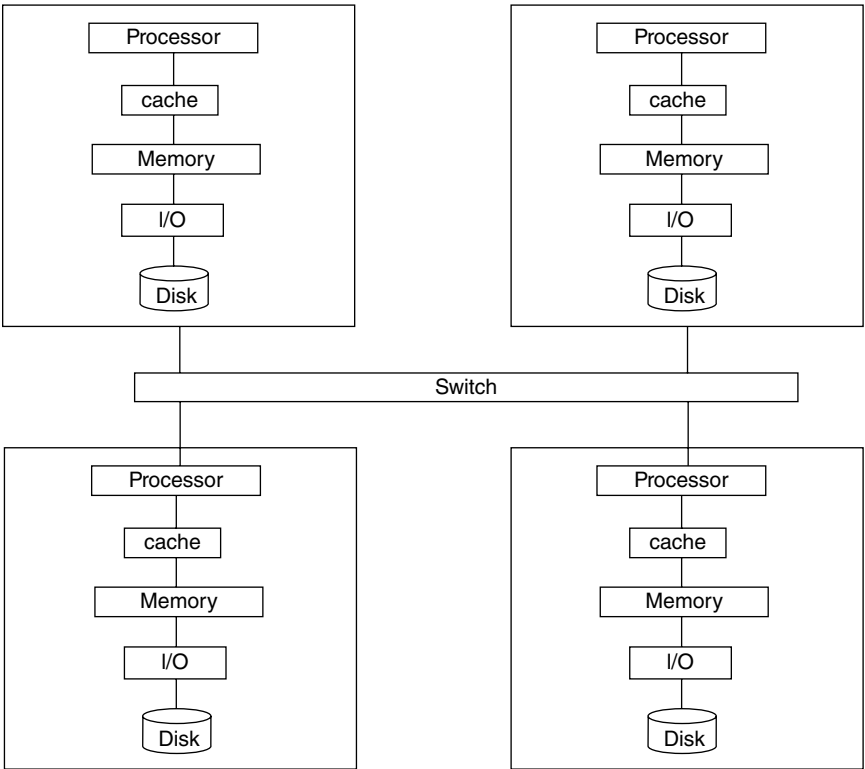


Figure 4.5 Shared-nothing architecture.

4.4 Batch Processing

Batch processing is a type of processing, where series of jobs that are logically connected are executed sequentially or in parallel, and then the output of all the individual jobs are put together to give a final output. Batch processing is implemented by collecting the data in batches and processing them to produce the output, which can be the input for another process. It is suitable for applications with terabytes or petabytes of data where the response time is not critical.

Batch processing is used in log analysis where the data are collected over a time and analysis is performed. They are also used in payroll, billing systems, data warehouses, and so on.

Figure 4.6 shows batch processing. Batch processing jobs are implemented using the Hadoop MapReduce architecture. The main objectives of these jobs are to aggregate the data and keep them available for analysis when required. The early trend in big data was to adopt a batch processing technique by extracting the data and scheduling the jobs later. Compared to a streaming system, batch processing systems are always cost-effective and easy to execute.

4.5 Real-Time Data Processing

Real-time data processing involves processing continual data flow producing the results. Here data are processed in-memory due to the requirement to analyze the data while it is streaming. Hence data are stored on the disk after the data is being

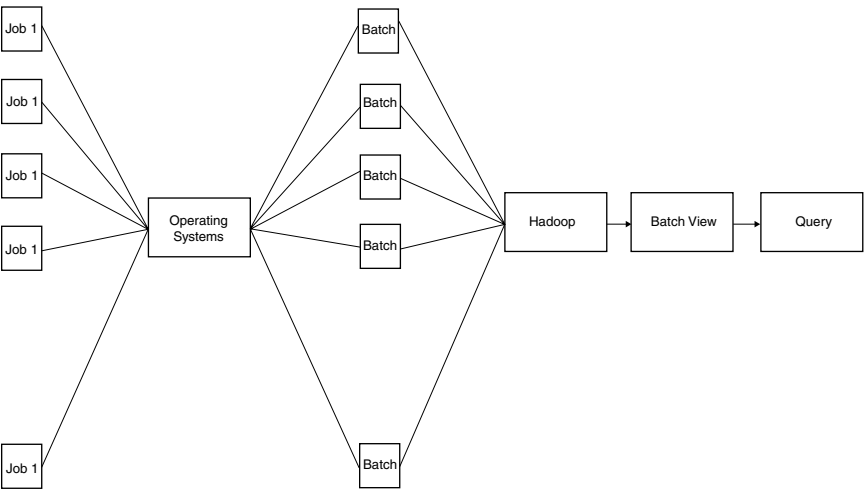


Figure 4.6 Batch processing.

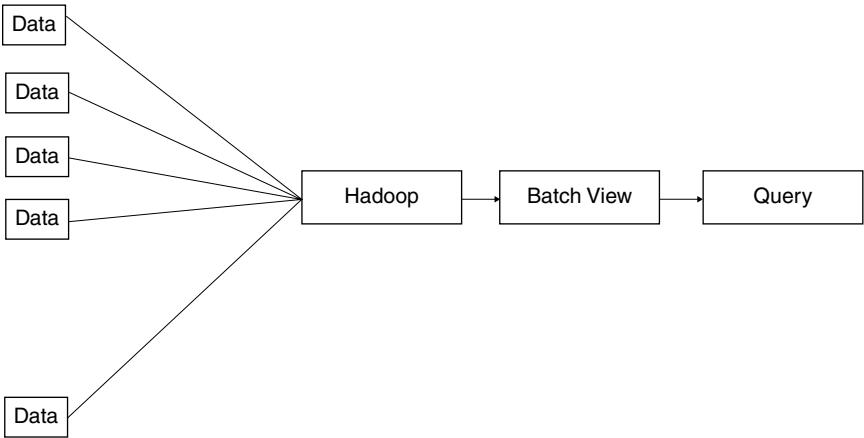


Figure 4.7 Real-time processing.

Solution	Developer	Type	Description
Storm	Twitter	Streaming	Framework for stream processing
S4	Yahoo	Streaming	Distributed stream computing platform
Mill Wheel	Google	Streaming	Fault tolerant stream processing framework
Hadoop	Apache	Batch	First open source framework for implementation of Map Reduce
Disco	Nokia	Batch	MapReduce framework by Nokia

Figure 4.8 Real-time and batch computation systems example.

processed. Online transactions, ATM transactions, the point of sales transactions are some examples which have to be processed in real time. Real-time data processing enables the organizations to respond with low latency where immediate actions are required for detecting transaction fraud in near real time. Storms, S4, Mill wheel are all real time computation platforms that process streaming data. Figure 4.7 shows the real time data processing, and Figure 4.8 shows an example of time and batch computation systems.

4.6 Parallel Computing

Parallel computing is the process of splitting up a larger task into multiple subtasks and executing them simultaneously to reduce the overall execution time. The execution of subtasks is carried out on multiple processors within a single

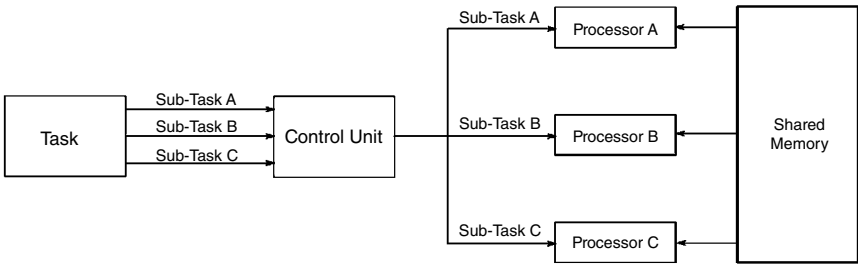


Figure 4.9 Parallel computing.

machine. Figure 4.9 shows parallel computing where the task is split into subtask A, subtask B, and subtask C and executed by processor A, processor B, and processor C running on the same machine.

4.7 Distributed Computing

Distributed computing, similar to parallel computing, splits up larger tasks into subtasks, but the execution takes place in separate machines networked together forming a cluster. Figure 4.10 shows distributed computing where the task is split into subtask A, subtask B, and subtask C and executed by processor A, processor B, and processor C running on different machines that are interconnected.

4.8 Big Data Virtualization

Data virtualization is a technology where data can be accessed from a heterogeneous environment, treating it as a single logical entity. The main purpose of virtualization in big data is to provide a single point of access to the data aggregated from multiple sources. Data virtualization benefits data integration in big data to a greater extent. Virtualization is a technique that uses PC components (both hardware and

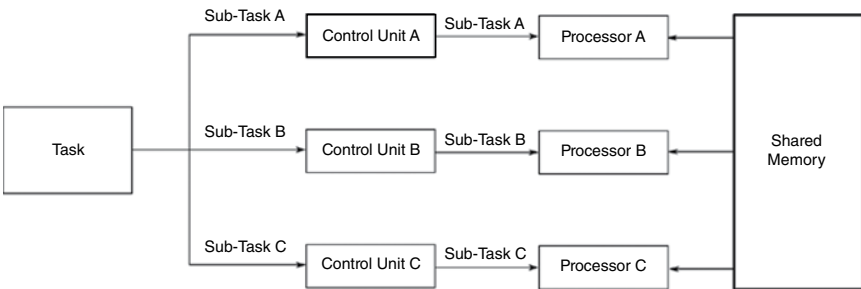


Figure 4.10 Distributed computing.

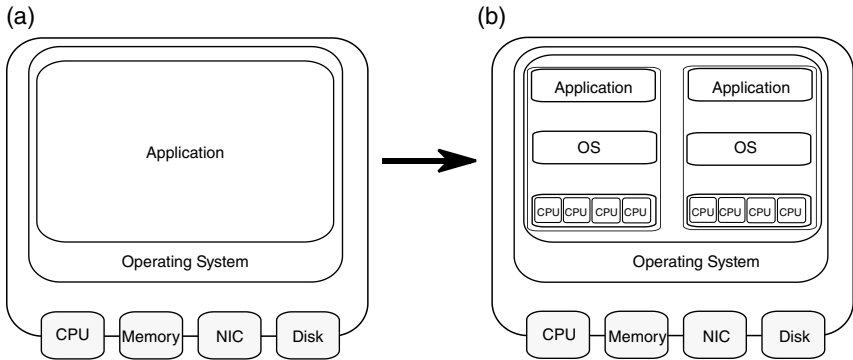


Figure 4.11 System architecture before and after virtualization.

software) to imitate other PC components. Earlier server virtualizations were prominent; today entire IT infrastructure software, storage, memory is virtualized to improve performance and efficiency and cost savings. Virtualization lays the foundation for cloud computing. Virtualization significantly reduces the framework cost by assigning a set of virtual resources to each application rather than allocating dedicated physical resources.

Figure 4.11 illustrates system architecture before and after virtualization. Figure 4.11a illustrates a traditional system with a host operating system and Figure 4.11b illustrates that a virtualization layer is inserted between the host operating system and the virtual machines (VMs). The virtualization layer is called Virtual Machine Monitor (VMM) or hypervisor. The VMs are run by the guest operating systems independent of the host operating system. Physical hardware of a host system is virtualized into virtual resources by the hypervisor to be exclusively used by the VMs.

4.8.1 Attributes of Virtualization

Three main attributes of virtualization are

- Encapsulation;
- Partitioning; and
- Isolation.

4.8.1.1 Encapsulation

A VM is a software representation of a physical machine that can perform functions similar to a physical machine. Encapsulation is a technique where the VM is stored or represented as a single file, and hence it can be identified easily based on the service it provides. This encapsulated VM can be used as a complete entity and presented to an application. Since each application is given a dedicated VM, one application does not interfere with another application.

4.8.1.2 Partitioning

Partitioning is a technique that partitions the physical hardware of a host machine into multiple logical partitions to be run by the VMs each with separate operating systems.

4.8.1.3 Isolation

Isolation is a technique in which VMs are isolated from each other and from the host physical system. A key feature of this isolation is if one VM crashes, other instances of the VM and the host physical system are not affected. Figure 4.12 illustrates that VMs are isolated from physical machines.

4.8.2 Big Data Server Virtualization

Virtualization works by inserting a layer of software on computer hardware on the host operating system. Multiple operating systems can run simultaneously on the single system. Each OS is independent, and it is not aware of other OS or VM running on the same machine.

In server virtualization, the server is partitioned into several VMs (servers). The PC assets CPU, memory are all virtualized running separate applications. Hence from a single server, several applications can be run. Server virtualization enables handling a large volume of data in Big Data analysis. In real time analysis, the volume of data is not known prior due to this uncertainty server virtualization is much needed in providing an environment with the ability to handle the unforeseen demand for processing huge dataset.

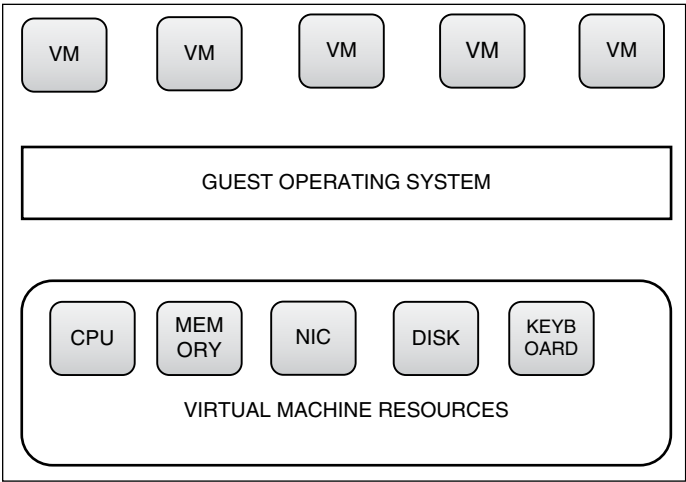


Figure 4.12 Isolation.

Part II: Managing and Processing Big Data in Cloud Computing

4.9 Introduction

Big data and cloud computing are the two fast evolving paradigms that are driving a revolution in various fields of computing. Big data promotes the development of e-finance, e-commerce, intelligent transportation, telematics, and smart cities. The potential to cross-relate consumer preferences with data gathered from tweets, blogs, and other social networks opens up a wide range of opportunities to the organizations to understand the customer needs and demands. But putting them into practice is complex and time consuming. Big data presents significant value to the organizations that adopt it; on the other hand, it poses several challenges to extract the business value from the data. So the organizations acquire expensive licenses and use large, complex, and expensive computing infrastructure that lacks flexibility. Cloud computing has modified the conventional ways of storing, accessing, and manipulating the data by adopting new concepts of storage and moving computing and data closer.

Cloud computing, simply called the cloud, is the delivery of shared computing resources and the stored data on demand. Cloud computing provides a cost-effective alternative by adding flexibility to the storage paradigm enabling the IT industry and organizations to pay only for the resources consumed and services utilized. To substantially reduce the expenditures, organizations are using cloud computing to deliver the resources required. The major benefit of the cloud is that it offers resources in a cost-effective way by offering the liberty to the organizations to pay as you go. Cloud computing has improved storage capacity tremendously and has made data gathering cheaper than ever making the organizations prefer buying more storage space than deciding on what data to be deleted. Also, cloud computing has reduced the overhead of IT professionals by dynamically allocating the computing resources depending on the real-time computational needs. Cloud computing provides large-scale distributed computing and storage in service mode to the users with flexibility to use them on demand improving the efficiency of resource utilization and reducing the cost. This kind of flexibility and sophistication offered by cloud services giants such as Amazon, Microsoft, and Google attracts more companies to migrate toward cloud computing. Cloud data centers provide large-scale physical resources while cloud computing platforms provide efficient scheduling and management to big data solutions. Thus, cloud computing basically provides infrastructure support to big data. It solves the growing computational and storage issues of big data.

The tools evolved to solve the big data challenges. For example, NoSQL modified the storage and retrieval pattern adopted by the traditional database management systems into a pattern that solves the big data issues, and Hadoop adopted

distributed storage and parallel processing that can be deployed under cloud computing. Cloud computing allows deploying a cluster of machines for distributing the load among them.

One of the key aspects of improving the performance of big data analytics is the locality of the data. This is because of the massive volume of big data, which prohibits it from transferring the data for processing and analyzing since the ratio of data transfer and processing time will be large in such scenarios. Since moving data to the computational node is not feasible, a different approach is adopted where the computational nodes are moved to the area where the actual data is residing.

Though cloud computing is a cost-effective alternative for the organizations in terms of operation and maintenance, the major drawback with cloud are privacy and security. As the data resides in the vendor's premise, the security and privacy of the data always becomes a doubtful aspect. This is specifically important in case of sensitive departments such as banks and government. In case there is a security issue for the customer information such as debit card or credit card details, it will have a crucial impact on the consumer, the financial institution, and the cloud service providers.

4.10 Cloud Computing Types

Cloud computing makes sharing of resources dramatically simpler. With the development of cloud computing technology, resources are connected either via public or private networks to provide highly scalable infrastructures for storage and other applications. Clients opting for cloud services need not worry about updating to the latest version of software, which will be taken care of by the cloud service providers. Cloud computing technology is broadly classified into three types based on its infrastructure:

- Public cloud;
- Private cloud; and
- Hybrid cloud.

Public cloud: In a public cloud, services are provided over the Internet by third-party vendors. Resources such as storage are made available to the clients via the Internet. Clients are allowed to use the services on a pay-as-you-go model, which significantly reduces the cost. In a pay-as-you-go model the clients are required to pay only for the resources consumed. Advantages of public cloud are availability, reduced investment, and reduced maintenance as all the maintenance activities including hardware and software are performed by the cloud service providers. The clients are provided with the updated versions of the software and any unforeseen increase in the hardware capacity requirements are handled by the service providers. Public cloud services are larger in scale, which provides on-demand

scalability to its clients. A few examples of public cloud are IBM's blue cloud, Amazon Elastic compute cloud, and Windows Azure services platform. Public clouds may not be a right choice for all the organizations because of limitations on configurations and security as these factors are completely managed by the service providers. Saving documents to the iCloud, Google Drive, and playing music from Amazon's cloud player are all public cloud services.

Private Cloud: A private cloud is also known as corporate cloud or internal cloud. These are owned exclusively by a single company with the control of maintaining its own data center. The main purpose of a private cloud is not to sell the service to external customers but to acquire the benefits of cloud architecture. Private clouds are comparatively more expensive than public clouds. In spite of the increased cost and maintenance of a private cloud, companies prefer a private cloud to address the concern regarding the security of the data and keep the assets within the firewall, which is lacking in a private cloud. Private clouds are not a best fit for small- to medium-sized business, but they are better suitable for larger enterprises. The two variations of a private cloud are on-premise private cloud and externally hosted private cloud. On-premise private cloud is the internal cloud hosted within the data center of an organization. It provides more security but often with a limit on its size and scalability. These are best fit for businesses that require complete control over security. An externally hosted private cloud is hosted by external cloud service providers with full guarantee of privacy. In an externally hosted private cloud the clients are provided with an exclusive cloud environment. This kind of cloud architecture is preferred by organizations that are not interested in using a public cloud because of the security issues and the risk involved in sharing the resources.

Hybrid Cloud: Hybrid clouds are a combination of public and private clouds where the advantages of both types of cloud environments are clubbed. A hybrid cloud uses third-party cloud service providers either fully or partially. A hybrid cloud has at least one public cloud and one private cloud. Hence, some resources are managed in-house and some are acquired from external sources. It is specifically beneficial during scheduled maintenance windows. It has increased flexibility of computing and is also capable of providing on-demand scalability.

4.11 Cloud Services

The cloud offers three different services, namely, software as a service (SaaS), platform as a service (PaaS), and infrastructure as a service (IaaS). Figure 4.13 illustrates the cloud computing service-oriented architecture.

SaaS provides license to an application to a customer through subscription or in a pay-as-you-go basis on-demand. The software and data provided are shared

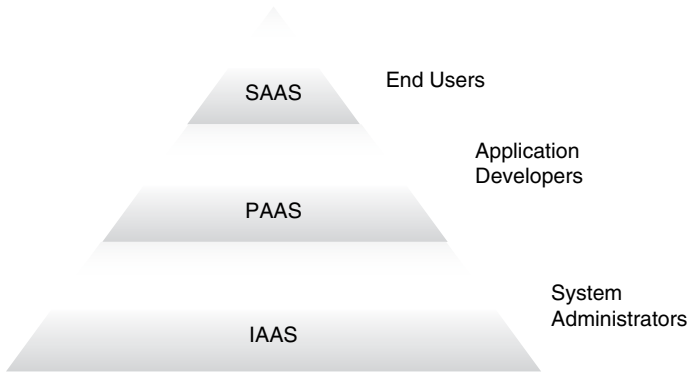


Figure 4.13 Service-oriented architecture.

securely simultaneously by multiple users. Some of the SaaS providers are salesforce.com, Microsoft, Oracle, and IBM.

PaaS provides platform to the users to develop, run, and maintain their applications. PaaS is accessed through a web browser by the users. The users will then be charged on pay-per-use basis. Some of the PaaS providers are Amazon, Google, AppFog, and Heroku.

IaaS provides consumers with computing resources, namely, servers, networking, data center space, and storage on a pay-per-use and self-service basis. Rather than purchasing these computing resources, clients use them as an outsourced service on-demand. The resources are provided to the users either as dedicated or shared (virtual) resources. Some of the IaaS providers are Amazon, Google, IBM, Oracle, Fujitsu, and Hewlett-Packard.

4.12 Cloud Storage

To meet the exponentially growing demand for storage, big data requires a highly scalable, highly reliable and highly available, cost-effective, decentralized, and fault-tolerant system. Cloud storage adopts a distributed file system and a distributed database. A distributed file system adopts distributed storage to store a large amount of files and the processing and analysis of a large volume of data is supported by a distributed NoSQL database.

To overcome the problems faced with the storage and analysis of Google web pages, Google developed Google File System and MapReduce distributed programming model based on Google File System. Google also built a high performance database system called Bigtable. Since Google's file system and database were not open-source, an open-source system called Hadoop was developed by Yahoo for the implementation of MapReduce. The underlying file system of Hadoop, the HDFS, is consistent

with GFS, and HBase, an open-source distributed database similar to Bigtable, is also provided. Hadoop and HBase, managed by Apache, have been widely adopted since their evolution.

4.12.1 Architecture of GFS

A Google File System (GFS) follows a master-chunkserver relationship. A GFS cluster consists of a single primary server, which is the master, and multiple chunkservers. Large files are divided into chunks of predefined size 64 Mb by default, and these chunks are stored as Linux files on the hard drive of the chunkserver. These chunks are identified by 64 bit unique chunk handles, which are assigned at the time of creation by the master server. For reliability, chunks are replicated on chunkservers, and by default the chunks have three replicas. Metadata of the entire file system is managed by the master along with namespace, location of chunks in the chunkserver, and access control. Communication between the master and the chunkserver takes place through heartbeat signal. The heartbeat signal gives instructions to the chunkserver, gathers the state of the chunkserver, and passes it back to the master.

The client interacts with the master to gather metadata and interacts with chunkservers for read/write operations. Figure 4.14 shows the Google File System architecture. The basic operation in GFS is:

- Master holds the metadata;
- Client contacts master for metadata about the chunks;

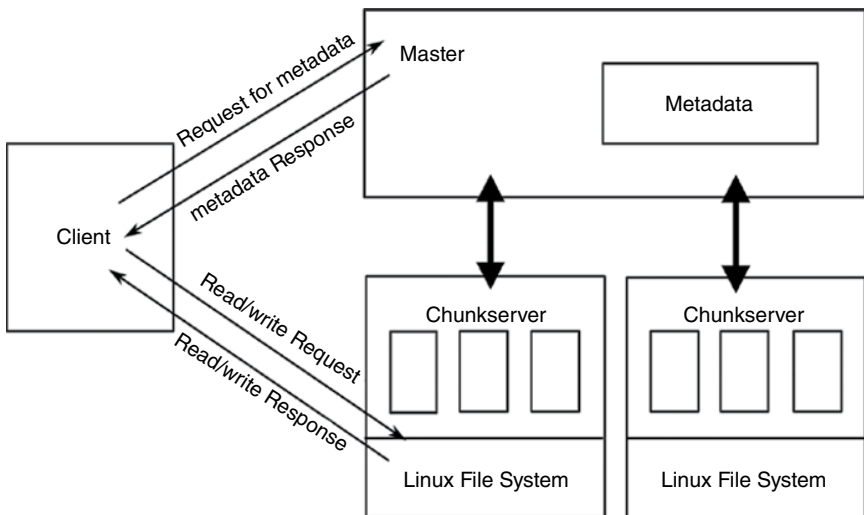


Figure 4.14 Google File System architecture.

- Client retrieves metadata about chunks stored in chunkservers; and
- Client send read/write request to the chunkservers.

4.12.1.1 Master

The major role of the master is to maintain the metadata. This includes mapping from files to chunks, details of location of each chunk's replica, and managing file, access control information, and chunk namespaces. Generally metadata for each 64MB chunk will be less than 64 bytes. Besides maintaining metadata, master is also responsible for managing the chunks, deleting the stale replicas. Master gives periodical instructions to chunkservers, gathers information about their state, and track cluster health.

4.12.1.2 Client

The role of the client is to communicate with master to gather information about which chunkserver to contact. Once the metadata are retrieved, all the data-bearing operations are performed with the chunkservers.

4.12.1.3 Chunk

Chunk in GFS is similar to block in file system. But chunks are comparatively larger in size than blocks. The average size of blocks ranges in KBs while the default size of chunks in GFS is 64MB. Since in Google's world terabytes of data and GBs of files are common, 64MB was a mandated size. Also the size of metadata is reduced with the increase in the size of the chunk. For example, if the size of the chunk is 10MB, and 1000MB of data is to be stored, it is necessary to store metadata for 100 chunks. If the size of the chunks is 64MB, metadata of only 16 chunks are stored, which makes a huge difference. So the lower the number of chunks, the smaller the metadata. Also, it reduces the number of times a client needs to contact the master.

4.12.1.4 Read Algorithm

The read algorithm follows the sequence below:

Step 1: Read request is initiated by the application.

Step 2: Filename and byte range are translated by the GFS client and sent to the master. Byte range is translated into chunk index while the filename remains the same.

Step 3: Replica location and chunk handle is sent by the master

Figure 4.15a shows the first three steps of the read algorithm.

Step 4: Location of the replica is picked by the client and request is sent

Step 5: Requested data is the sent by the chunkserver

Step 6: Data received from the chunkserver is sent to application by the client.

Figure 4.15b shows the last three steps of the read algorithm.

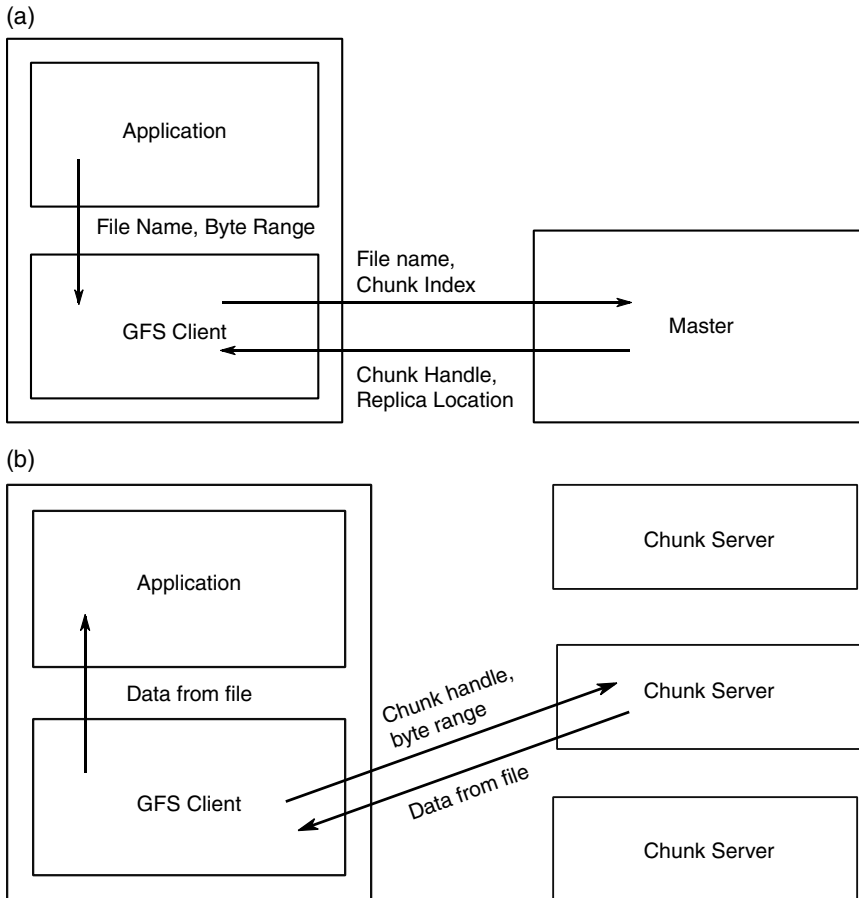


Figure 4.15 Read algorithm: (a) The first three steps. (b) The last three steps.

4.12.1.5 Write Algorithm

The write algorithm follows the sequence below:

Step 1: Read request is initiated by the application.

Step 2: Filename and data are translated by the GFS client and sent to the master. Data is translated into chunk index while the filename remains the same.

Step 3: Primary and secondary replica locations along with chunk handle are sent by the master

Figure 4.16a shows the first three steps of the write algorithm.

Step 4: The data to be written is pushed by the client to all locations. Data is stored in the internal buffers of the chunkservers.

Step 5: Write command is sent to the primary by the client.

Figure 4.16(b) shows step 4 and 5 of the write algorithm.

Step 6: Serial order for the data instances is determined by the primary.

Step 7: Serial order is sent to the secondary and write operations are performed.

Figure 4.16(c) shows steps 6 and 7 of the write algorithm.

Step 8: Secondaries respond to primary.

Step 9: Primary in turn respond to client

Figure 4.16(d) shows steps 8 and 9 of the write algorithm.

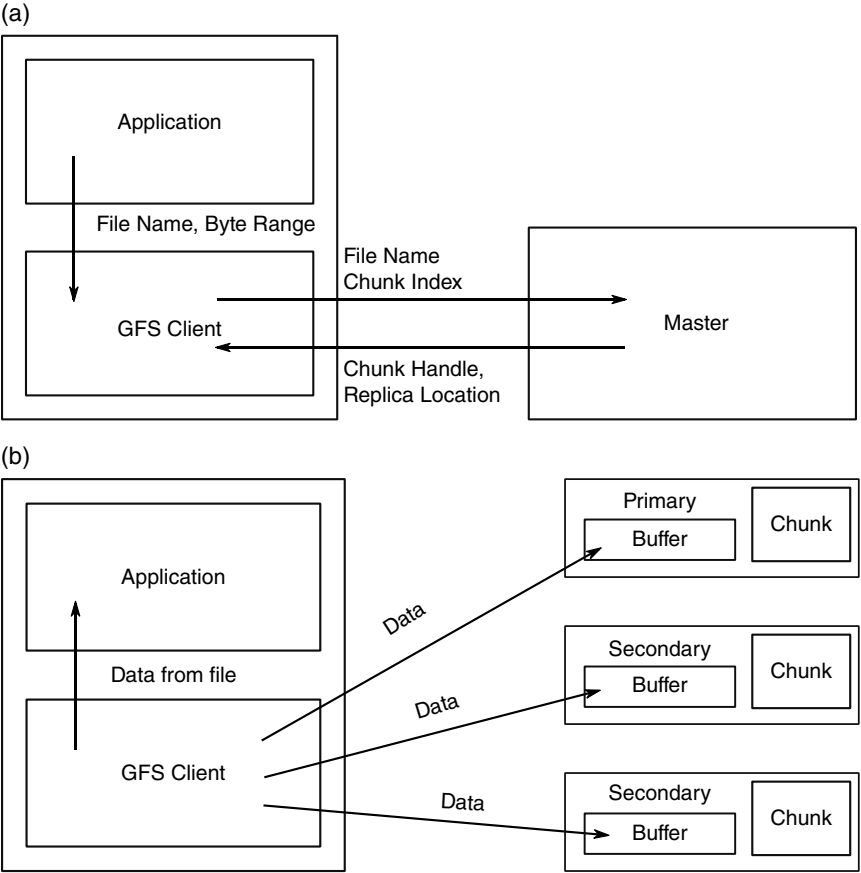


Figure 4.16 Write algorithm: (a) The first three steps. (b) Steps 4 and 5. (c) Steps 6 and 7 (d) Steps 8 and 9.

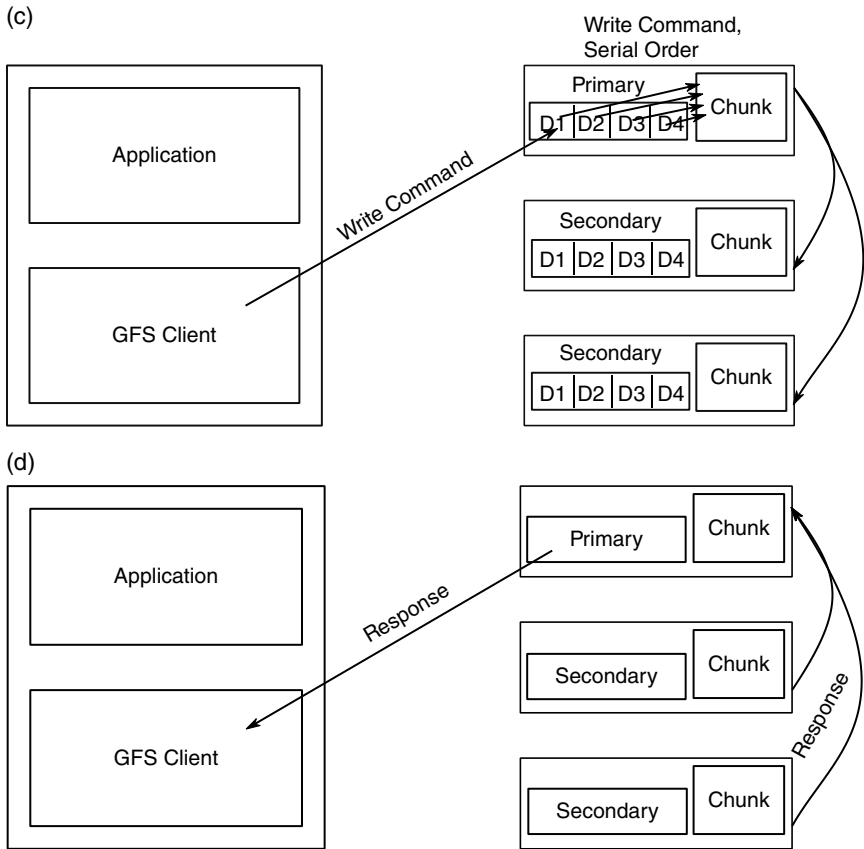


Figure 4.16 (Continued)

4.13 Cloud Architecture

Cloud architecture has a front end and back end connected through a network. The network is usually the Internet. The front end is the client infrastructure consisting of applications that require access to a cloud computing platform. The back end is the cloud infrastructure consisting of the resources, namely, data storage, servers, and network required to provide services to the clients. The back end is responsible to provide security, privacy, protocol, and traffic control. The server employs middleware for the connected devices to communicate with each other.

Figure 4.17 shows the cloud architecture. The key component of the cloud infrastructure is the network. In cloud computing, the Internet-based computing is connected to the Internet through the network. Cloud servers are the virtual

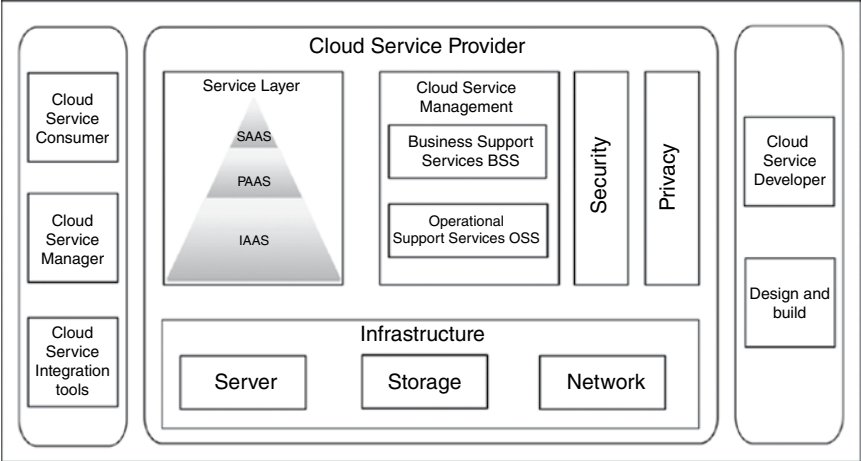


Figure 4.17 Cloud architecture.

servers, which work as physical servers do but the functions of virtual servers are different from the physical servers. Cloud servers are responsible for resource allocation, de-allocation, providing security, and more. The clients pay for the hours of usage of the resource. Clients may opt for either shared or dedicated hosting. Shared hosting is the cheaper alternative compared to a dedicated hosting. In a shared hosting, servers are shared between the clients, but this kind of hosting cannot cope up with heavy traffic. Dedicated hosting overcomes the drawbacks of shared hosting, since the entire server is dedicated to a single client without any sharing. Clients may require more than one dedicated server, and they pay for the resources they have used according to their demand. The resources can be scaled up according to the demand, making it more flexible and cost effective. Cost effectiveness, ease of set-up, reliability, flexibility, and scalability are the benefits of cloud services.

Cloud storage has multiple replicas of the data. If any of the resources holding the data fails, then the data can be recovered from the replicas stored in another storage resource.

IaaS provides access to resources, namely, servers, networking, data center space, load balancers, and storage on pay-per-use and self-service basis. These resources are provided to the clients through server visualization, and to the clients it appears as if they own the resources. IaaS provides full control over the resources, and flexible, efficient, and cost-effective renting of resources. SaaS provides license to an application to a customer through subscription or in a pay-as-you-go basis on-demand. PaaS provides a platform to the users to develop, run, and maintain their applications. PaaS is accessed through a web browser by the users.

Business support services (BSS) and operational support services (OSS) of cloud service management help enable automation.

4.13.1 Cloud Challenges

Cloud Computing is posed with multiple challenges in data and information handling. Some of the challenges are:

- Security and Privacy;
- Portability;
- Computing performance;
- Reliability and availability; and
- Interoperability.

Security and privacy—Security and privacy of the data is the biggest challenge posed on cloud computing specifically when the resources are shared and the data resides in the cloud service provider's storage platform outside the corporate firewall. Hacking would attack many clients even if only one site of the cloud service provider is attacked. This can be overcome by employing security applications and security hardware that tracks unusual activities across the server.

Portability—Portability is yet another challenge on cloud computing where the applications are to be easily migrated from one cloud computing platform to another without any lock-in period.

Computing performance—High network performance is required for data intensive applications on the cloud, which results in a high cost. Desired computing performance cannot be met with low bandwidth.

Reliability and availability—The cloud computing platform has to be reliable and robust and provide round the clock service. Lack of round the clock services results in frequent outages, which reduce the reliability of the cloud service.

Interoperability—Interoperability is the ability of the system to provide services to the applications from other platforms.

Chapter 4 Refresher

- 1 In a distributed system if one site fails, _____.
 - A the remaining sites continue operating
 - B all the systems stop working
 - C working of directly connected sites will be stopped
 - D none of the above

Answer: a

- 2 A distributed file system disperses _____ among the machines of distributed system.
- A clients
 - B storage devices
 - C servers
 - D all of the above

Answer: d

- 3 Teradata is a _____.
- A shared-nothing architecture
 - B shared-everything architecture
 - C distributed shared memory architecture
 - D none of the above

Answer: a

- 4 The attributes of virtualization is/are _____.
- A encapsulation
 - B partitioning
 - C isolation
 - D all of the above

Answer: d

- 5 The process of collecting, processing, manipulating, and managing the data to generate meaningful information to the end user is called _____.
- A data acquisition
 - B data Processing
 - C data integration
 - D data transformation

Answer: b

- 6 The architecture sharing all the resources such as storage, memory, and processor is called _____.
- A shared-everything architecture
 - B shared-nothing architecture
 - C shared-disk architecture
 - D none of the above

Answer: a

- 7 The process of splitting up a larger task into multiple subtasks and executing them simultaneously to reduce the overall execution time is called _____.
- A parallel computing
 - B distributed computing

- C both a and b
- D none of the above

- 8 _____ is/are the type/types of virtualization
- A Desktop virtualization
 - B Storage virtualization
 - C Network virtualization
 - D All of the above

Answer: d

- 9 _____ is also called uniform memory access.
- A Shared-nothing architecture
 - B Symmetric multiprocessing
 - C Distributed shared memory architecture
 - D Shared-everything architecture

Answer: b

- 10 _____ is used in log analysis where the data are collected over a time and analysis is performed.
- A Batch processing
 - B Real-time processing
 - C Parallel processing
 - D None of the above

Answer: a

- 11 _____ refers to the applications that run on a distributed network and uses virtualized resources.
- A Cloud computing
 - B Distributed computing
 - C Parallel computing
 - D Data processing

Answer: a

- 12 Which of the following concepts is related to sharing of resources?
- A Abstraction
 - B Virtualization
 - C Reliability
 - D Availability

Answer: b

13 Which of the following is/are cloud deployment model/models?

- A** Public
- B** Private
- C** Hybrid
- D** All of the above

Answer: d

14 Which of the following is/are cloud service model/models?

- A** IaaS
- B** PaaS
- C** SaaS
- D** All of the above

Answer: d

15 A cloud architecture within an enterprise data center is called ____.

- A** public cloud
- B** private cloud
- C** hybrid cloud
- D** none of the above

Answer: b

16 Partitioning a normal server to behave as multiple servers is called ____.

- A** server splitting
- B** server virtualization
- C** server partitioning
- D** none of the above

Answer: b

17 Google is one of the types of cloud computing.

- A** True
- B** False

Answer: a

18 Amazon web service is a/an ____ type of cloud computing distribution model.

- A** software as a service
- B** infrastructure as a service
- C** platform as a service
- D** none of the above

Answer: b

Conceptual Short Questions with Answers

1 What is data processing?

Data processing is defined as the process of collecting, processing, manipulating, and managing the data to generate meaningful information to the end user.

2 What are the types of data processing?

There are basically two different types of data processing, namely, centralized and distributed data processing. Centralized processing is a processing technique that requires minimal resources and is suitable for organizations with one centralized location of service. Distributed processing is a processing technique where data collection and processing are distributed across different physical locations. This type of processing overcomes the shortcomings of centralized data processing, which mandates data collection to be at one central location.

3 What is shared-everything architecture and what are its types?

Shared-everything architecture is a type of system architecture sharing all the resources such as storage, memory, and processor. Distributed shared memory and symmetric multiprocessing are the types of shared-everything architecture.

4 What is shared-nothing architecture?

Shared-nothing architecture is a type of distributed system architecture that has multiple systems interconnected to make the system scalable. Each system in the network is called a node and has its own dedicated memory, storage, and disks independent of other nodes in the network, thus making it a shared-nothing architecture.

5 What is batch processing?

Batch processing is a type of processing where series of jobs that are logically connected are executed sequentially or in parallel, and then the output of all the individual jobs are put together to give a final output. Batch processing is implemented by collecting the data in batches and processing them to produce the output, which can be the input for another process.

6 What is real-time data processing?

Real-time data processing involves processing a continual data flow producing the results. Here data are processed in-memory due to the requirement to analyze the data while it is streaming. Hence, data are stored on the disk after the data is being processed.

7 What is parallel computing?

Parallel computing is the process of splitting up a larger task into multiple subtasks and executing them simultaneously to reduce the overall execution time. The execution of subtasks is carried out on multiple processors within a single machine.

8 What is distributed computing?

Distributed computing, similar to parallel computing, splits up larger tasks into subtasks, but the execution takes place in separate machines networked together forming a cluster.

9 What is virtualization? What is the advantage of virtualization in big data? What are the attributes of virtualization?

Data virtualization is a technology where data can be accessed from a heterogeneous environment treating it as a single logical entity. The main purpose of virtualization in big data is to provide a single point of access to the data aggregated from multiple sources. The attributes of virtualization are encapsulation, partitioning, and isolation.

10 What are the different types of virtualization?

The following are the types of virtualization:

- Server virtualization;
- Desktop virtualization;
- Network virtualization;
- Storage virtualization; and
- Application virtualization.

11 What are the benefits of cloud computing?

The major benefit of the cloud is that it offers resources in a cost-effective way by offering the liberty to the organizations to pay-as-you-go. Cloud computing has improved storage capacity tremendously and has made data gathering cheaper than ever making the organizations prefer buying more storage space than deciding on what data to delete. With cloud computing the resources can be scaled up according to the demand making it more flexible and cost effective. Cost effectiveness, ease of set-up, reliability, flexibility, and scalability are the benefits of cloud services.

12 What are the cloud computing types?

- Public cloud;
- Private cloud; and
- Hybrid cloud.

13 What is a public cloud?

In a public cloud, services are provided over the Internet by third-party vendors. Resources such as storage are made available to the clients via the Internet. Clients are allowed to use the services on a pay-as-you-go model, which significantly reduces the cost. In a pay-as-you-go model the clients are required to pay only for the resources consumed. Advantages of public cloud are availability, reduced

investment, and reduced maintenance as all the maintenance activities including hardware and software are performed by the cloud service providers.

14 What is a private cloud?

A private cloud is also known as corporate cloud or internal cloud. These are owned exclusively by a single company with the control of maintaining its own data center. The main purpose of private cloud is not to sell the service to external customers but to acquire the benefits of cloud architecture. Private clouds are comparatively more expensive than public clouds. In spite of the increased cost and maintenance of a private cloud, companies prefer a private cloud to address the concern regarding the security of the data and keep the assets within the firewall, which is lacking in a private cloud.

15 What is a hybrid cloud?

Hybrid clouds are a combination of the public and the private cloud where the advantages of both types of cloud environments are clubbed. A hybrid cloud uses third-party cloud service providers either fully or partially. A hybrid cloud has at least one public cloud and one private cloud. Hence, some resources are managed in-house and some are acquired from external sources. It is specifically beneficial during scheduled maintenance windows. It has increased flexibility of computing and is also capable of providing on-demand scalability.

16 What are the services offered by the cloud?

The cloud offers three different services, namely, SaaS, PaaS, and IaaS.

17 What is SaaS?

SaaS provides license to an application to a customer through subscription or in a pay-as-you-go basis on-demand. The software and data provided are shared securely simultaneously by multiple users. Some of the SaaS providers are salesforce.com, Microsoft, Oracle, and IBM.

18 What is PaaS?

PaaS provides platform to the users to develop, run, and maintain their applications. PaaS is accessed through a web browser by the users. The users will then be charged on pay-per-use basis. Some of the PaaS providers are Amazon, Google, AppFog, and Heroku.

19 What is IaaS?

IaaS provides consumers with computing resources, namely, servers, networking, data center space, and storage on pay-per-use and self-service basis. Rather than purchasing these computing resources, clients use them as an outsourced service

on-demand. The resources are provided to the users either as dedicated or shared (virtual) resources. Some of the IaaS providers are Amazon, Google, IBM, Oracle, Fujitsu, and Hewlett-Packard.

Cloud Computing Interview Questions

1 What are the advantages of cloud computing?

1. Data storage;
2. Cost effective and time saving;
3. Powerful server capabilities.

2 What is the difference between computing for mobiles and cloud computing?

Cloud computing becomes active with the Internet and allows the users to access the data, which they can retrieve on demand, whereas cloud computing for mobile applications run on a remote server and provides the users access for storage.

3 What are the security aspects of cloud computing?

1. Identity management;
2. Access control;
3. Authentication and authorization.

4 Expand EUCALYPTUS and what is its use in cloud computing?

EUCALYPTUS stands for “Elastic Utility Computing Architecture for Linking Your Programs To Useful Systems.” It is an open-source software used to execute clusters in cloud computing.

5 Name some of the open-source cloud computing databases.

A few open-source cloud computing databases are

1. MangoDB;
2. LuciDB;
3. CouchDB.

6 What is meant by on-demand functionality in cloud computing? How this functionality is provided in cloud computing?

Cloud computing technology provides an on-demand access to its virtualized resources. A shared pool is provided to the consumers, which contains servers, network, storage, applications, and services.

7 List the basic clouds in cloud computing.

1. Professional cloud;
2. Personal cloud;
3. Performance cloud.

5

Driving Big Data with Hadoop Tools and Technologies

CHAPTER OBJECTIVE

The core components of Hadoop, namely HDFS (Hadoop Distributed File System), MapReduce, and YARN (Yet Another Resource Negotiator) are explained in this chapter. This chapter also examines the features of HDFS such as its scalability, reliability, and its robust nature. The HDFS architecture and its storage techniques are also explained.

Deep insight is provided into the various big data tools that are used in various stages of the big data life cycle. Apache HBase, a non-relational database especially designed for the large volume of sparse data is briefed. An SQL-like query language called Hive Query Language (HQL) used to query unstructured data is explained in this segment of the book. Similarly Pig, a platform for a high-level language called Pig Latin used to write MapReduce programs; Mahout, a machine learning algorithm; Avro, the data serialization system; SQOOP, a massive tool for transferring bulk data between RDBMS and Hadoop; and Oozie, a workflow scheduler system which manages Hadoop jobs are all well explained.

5.1 Apache Hadoop

Apache Hadoop is an open-source framework written in Java that supports processing of large data sets in streaming access pattern across clusters in a distributed computing environment. It can store a large volume of structured, semi-structured, and unstructured data in a distributed file system (DFS) and process them in parallel. It is a highly scalable and cost-effective storage platform. Scalability of Hadoop refers to its capability to sustain its performance even under highly increasing loads by adding more nodes. Hadoop files are written once and read many times. The contents of the files cannot be changed. Large number of computers interconnected and working together as a single

system is called a cluster. Hadoop clusters are designed to store and analyze the massive amount of disparate data in a distributed computing environment in a cost-effective manner.

5.1.1 Architecture of Apache Hadoop

Figure 5.1 illustrates that the Hadoop architecture consists of two layers, the storage layer is the HDFS layer, and on the top of it is the MapReduce engine. The details of each of the components in the Hadoop architecture are explained in the following sections in this chapter.

5.1.2 Hadoop Ecosystem Components Overview

Hadoop ecosystem comprises four different layers

- 1) Data storage layer;
- 2) Data Processing layer;
- 3) Data access layer;
- 4) Data management layer.

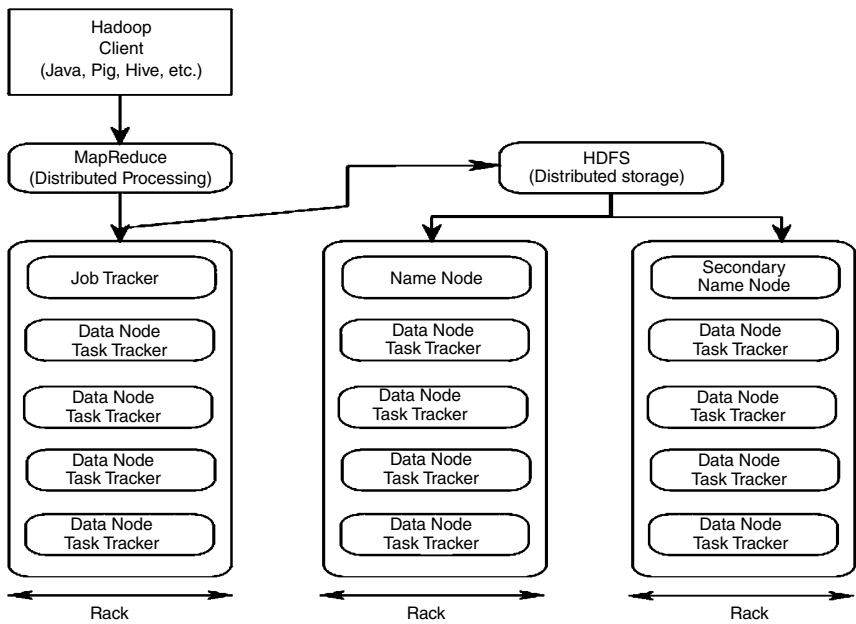


Figure 5.1 Hadoop architecture.

Figure 5.2 shows the Hadoop ecosystem with four layers. The data storage layer comprises HDFS and HBase. In HDFS data is stored in a distributed environment. HBase is a column-oriented database to store a structured database.

The data processing layer comprises MapReduce and YARN. Job processing is handled by MapReduce while the resource allocation and job scheduling and monitoring is handled by YARN.

The data access layer comprises Hive, Pig, Mahout, Avro, and SQOOP. Hive is a query language to access the data in HDFS. Pig is a data analysis high-level scripting language. Mahout is a machine learning platform. Avro is a data serialization framework. SQOOP is a tool transfer data from the traditional database to HDFS and vice versa.

The data management layer interacts with the end user. It comprises Oozie, Chukwa, Flume, and Zookeeper. Oozie is a workflow scheduler. Chukwa is used for data collection and monitoring. Flume is used to direct the data flow from a source to HDFS.

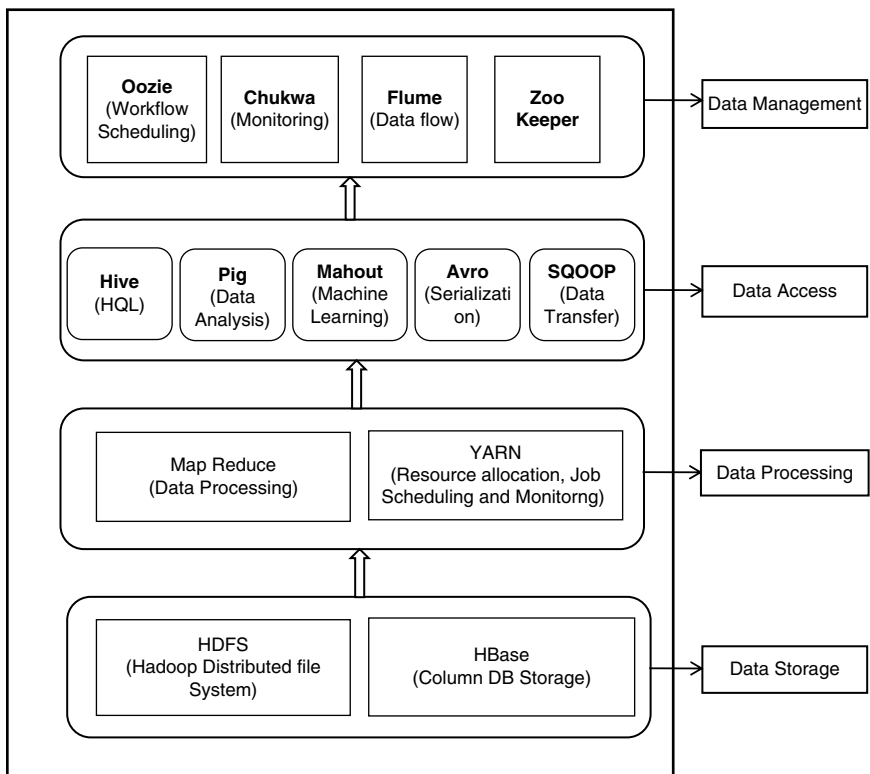


Figure 5.2 Hadoop ecosystem.

5.2 Hadoop Storage

5.2.1 HDFS (Hadoop Distributed File System)

The Hadoop distributed file system is designed to store large data sets with streaming access pattern running on low-cost commodity hardware. It does not require highly reliable expensive hardware. The data set generated from multiple sources is stored in a HDFS in a write once, read many times pattern and analysis is performed on the data set to extract knowledge from it. HDFS is not suitable for applications that require low latency access to the data. HBase is a suitable alternative for such applications requiring low latency.

An HDFS stores the data by partitioning the data into small chunks. Blocks of a single file are replicated to provide fault tolerance and availability. If the blocks are corrupt or if the disk or machine fails, the blocks can be retrieved by replicating the blocks across physically separate machines.

5.2.2 Why HDFS?

Figure 5.3 shows a DFS vs. a single machine. With a single machine, to read 500GB of data it takes approximately 22.5 minutes when the machine has four I/O channels and each channel is capable of processing the task at a speed of 100 MB/s. On top of it, data analysis has to be performed, which will still increase the overall time consumed. If the same data is distributed over 100 machines with the same number of I/O channels in each machine, then the time taken would be 13.5 seconds approximately. This is essentially what Hadoop does, instead of storing the data at a single location, Hadoop stores in a distributed fashion in DFS, where the data is stored in hundreds of data nodes, and the data retrieval occurs in parallel. This approach eliminates the bottleneck and improves performance.

Single Machine



1 Machine

Distributed File System



100 Machines

Figure 5.3 Distributed file system vs. single machine.

5.2.3 HDFS Architecture

HDFS is highly fault-tolerant designed to be deployed on commodity hardware. The applications that run on HDFS typically range from terabytes to petabytes as it is designed to support such large files. It is also designed in a way that it is easy to port HDFS from one platform to another. It basically adopts master/slave architecture wherein one machine in the cluster acts as a master and all other machines serve as the slaves. Figure 5.4 shows the HDFS architecture. The master node has the NameNode and the associated daemon called JobTracker. NameNode manages the namespace of the entire file system, supervises the health of the DataNode through the Heartbeat signal, and controls the access to the files by the end user. The NameNode does not hold the actual data, it is the directory for DataNode holding the information of which blocks together constitute the file and location of those blocks. NameNode is the single point of failure in the entire system, and if it fails, it needs manual intervention. Also, HDFS is not suitable for storing a large number of small files. This is because the file system metadata is stored in NameNode; the total number of files that can be stored in HDFS is governed by the memory capacity of the NameNode. If a large number of small files has to be stored, more metadata will have to be stored, which occupies more memory space.

The set of all slave nodes with the associated daemon, which is called TaskTracker, comprises the DataNode. DataNode is the location where the actual data reside, distributed across the cluster. The distribution occurs by splitting up the file that has the user data into blocks of size 64 Mb by default, and these blocks are then stored in the DataNodes. The mapping of the block to the DataNode is performed by the NameNode, that is, the NameNode decides which block of the file has to be placed in a specific DataNode. Several blocks of the same file are

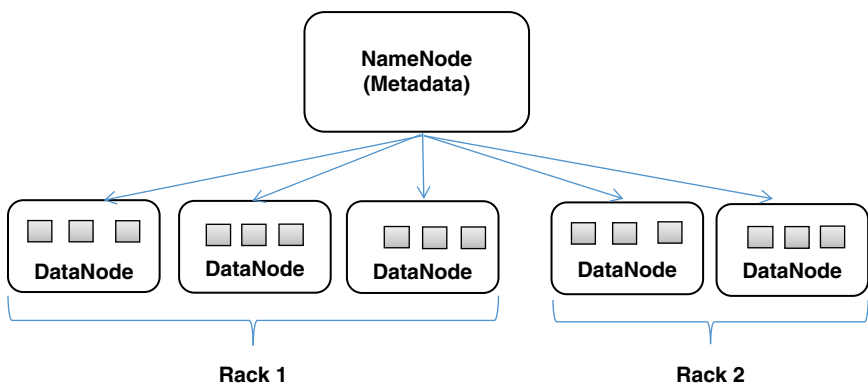


Figure 5.4 HDFS architecture.

stored in different DataNodes. Each block is mapped to three DataNodes by default to provide reliability and fault tolerance through data replication. The number of replicas that a file should have in HDFS can also be specified by the application. NameNode has the location of each block in DataNode. It also does several other operations such as opening or closing files and renaming files and directories. NameNode also decides which block of the file has to be written to which DataNode within a specific rack. The rack is a storage area where multiple DataNodes are put together. The three replicas of the block are written in such a way that the first block is written on a separate rack and blocks 2 and 3 are always written on the same rack on two different DataNodes, but blocks 2 and 3 cannot be written on the same rack where the block 1 is written. This approach is to overcome rack failure. The placement of these blocks decided by the NameNode is based on proximity between the nodes. The closer the proximity, the faster is the communication between the DataNodes.

HDFS has a secondary NameNode, which periodically backs up all the data that resides in the RAM of the NameNode. The secondary NameNode does not act as the NameNode if it fails; rather, it acts as a recovery mechanism in case of its failure. The secondary NameNode runs on a separate machine because it requires memory space equivalent to NameNode to back up the data residing in the NameNode. Despite the presence of the secondary NameNode, the system does not guarantee high availability: NameNode still remains a single point of failure. Failure of NameNode makes the filesystem unavailable to read or write until a new NameNode is brought into action.

HDFS federation is introduced since the limitation on the memory size of the NameNode, which holds the metadata and the reference to each block in the file system, limits cluster scaling. Under HDFS federation, additional NameNodes are added and each individual NameNode manages Namespace independent of the other NameNode. Hence NameNodes do not communicate with each other, and failure of one NameNode does not affect the Namespace of another NameNode.

5.2.4 HDFS Read/Write Operation

The HDFS client initiates a read request to the Distributed File System, and DFS, in turn, connects with NameNode. NameNode creates a new record for storing the metadata about the new block, and a new file creation operation is initiated after placing a check for file duplication. The DataNodes are identified based on the number of replicas, which is by default three. The input file is split up into blocks of default size 64 MB, and then the blocks are sent to DataNodes in packets. The writing is done in a pipelined fashion. The client sends the packet to a DataNode that is of close proximity among the three DataNodes identified by the NameNode, and that DataNode will send the packet received to the second DataNode; the

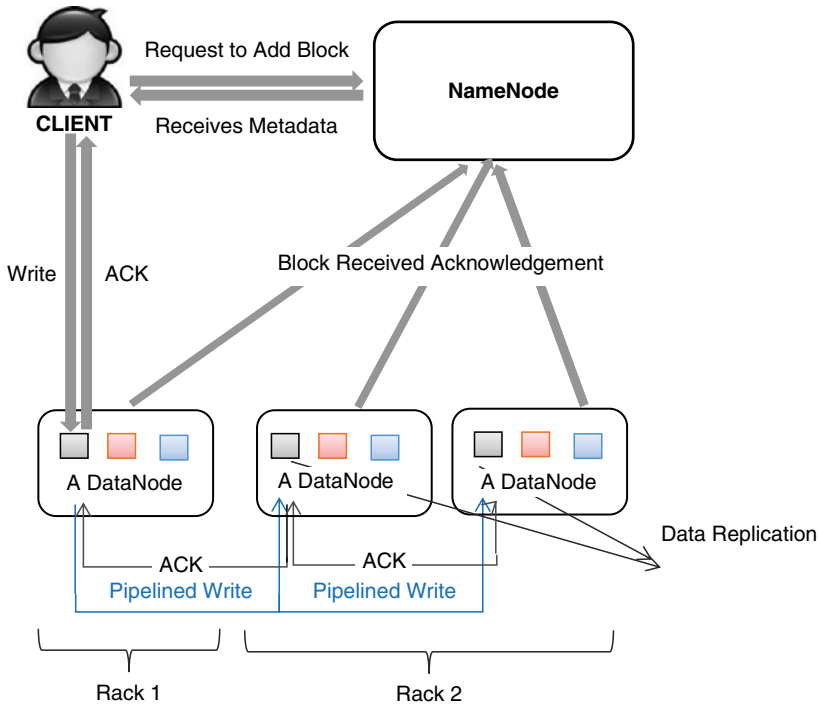


Figure 5.5 File write.

second DataNode, in turn, sends the packet received to a third one. Upon receiving a complete data block, the acknowledgment is sent from the receiver DataNode to the sender DataNode and finally to the client. If the data are successfully written on all identified DataNodes, the connection established between the client and the DataNodes is closed. Figure 5.5 illustrates the file write in HDFS.

The client initiates the read request to DFS, and the DFS, in turn, interacts with NameNode to receive the metadata, that is, the block location of the data file to be read. NameNode returns the location of all the DataNode holding the copy of the block in a sorted order by placing the nearest DataNode first. This metadata is then passed on from DFS to the client; the client then picks the DataNode with close proximity first and connects to it. The read operation is performed, and the NameNode is again called to get the block location for the next batch of files to be read. This process is repeated until all the necessary data are read, and a close operation is performed to close the connection established between client and DataNode. Meanwhile, if any of the DataNodes fails, data is read from the block where the same data is replicated. Figure 5.6 illustrates the file read in HDFS.

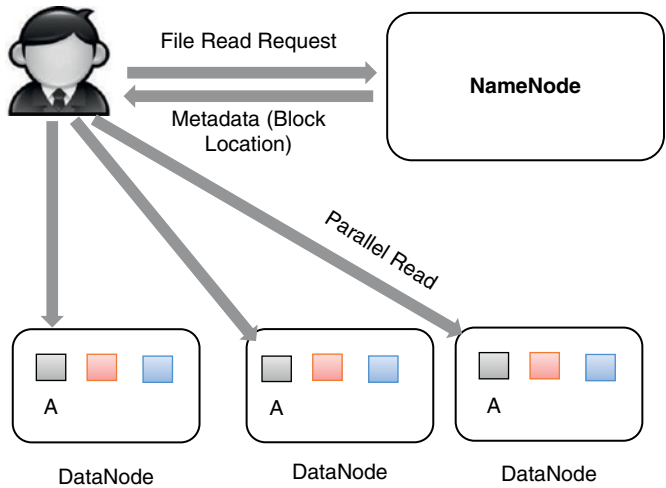


Figure 5.6 File read.

5.2.5 Rack Awareness

HDFS has its DataNodes spanned across different racks, and the racks are identified by the rack IDs, the details of which are stored in NameNode. The three replicas of a block are placed such that the first block is written on a separate rack and blocks 2 and 3 are always written on the same rack on two different DataNodes, but blocks 2 and 3 cannot be placed on the same rack where the block 1 is placed to make the DFS highly available and fault tolerant. Thus, when the rack where block 1 is placed goes down, the data can still be fetched from the rack where blocks 2 and 3 are placed. The logic here is not to place more than two blocks on the DataNodes of the same rack, and each block is placed on different DataNodes. The number of racks involved in replication should be less than the total number of replicas of the block as the rack failure is less common than DataNode failure. The second and third blocks are placed in the different DataNodes of the same rack as the availability and fault tolerance issues are already handled by placing blocks on two unique racks. The placement of blocks 2 and 3 on the same rack is due to the fact that writing the replicas on the DataNode of the same rack is remarkably faster than writing on DataNodes of different racks. The overall concept is placing the blocks into two separate racks and three different nodes to address both rack failure and node failure.

5.2.6 Features of HDFS

5.2.6.1 Cost-Effective

HDFS is an open-source storage platform; hence, it is available free of cost to the organizations that choose to adopt it as its storage tool. HDFS does not require high-end hardware for storage. It uses commodity hardware for storage, which

has made it cost effective. If HDFS used a specialized, high-end version of hardware, handling and storing big data would be expensive.

5.2.6.2 Distributed Storage

HDFS splits the input files into blocks, each of size 64 MB by default, and then stores in HDFS. A file of size 200 MB will be split into three 64 MB blocks and one 8 MB block. Three 64 MB files occupy three blocks completely, and the 8 MB file does not occupy a full block. This block can be shared to store other files to make the 64 MB utilized fully.

5.2.6.3 Data Replication

HDFS by default makes three copies of all the data blocks and stores them in different nodes in the cluster. If any node crashes, the node carrying the copy of the data that is lost is identified and the data is retrieved.

5.3 Hadoop Computation

5.3.1 MapReduce

MapReduce is the batch-processing programming model for the Hadoop framework, which adopts a divide-and-conquer principle. It is highly scalable, reliable, fault tolerant, and capable of processing input data with any format. It processes the data in a parallel and distributed computing environment, which supports only batch workloads. Its performance reduces the processing time significantly compared to the traditional batch-processing paradigm, as the traditional approach moves the data from storage platform to the processing platform, whereas the MapReduce processing paradigm resides in the framework where the data actually reside. Figure 5.7 shows the MapReduce model.

The processing of data in MapReduce is implemented by splitting up the entire process into two phases, namely, the map phase and the reduce phase. There are several stages in MapReduce processing where the map phase includes map, combine, and partition, and the reduce phase includes shuffle and sort and reduce. Combiner and partitioner are optional depending on the processing to be performed on the input data. The job of the programmer ends up with providing the MapReduce program and the input data, and rest of the processing is carried out by the framework, thus simplifying the use of the MapReduce paradigm.

5.3.1.1 Mapper

Map is the first stage of the map phase, during which a large data set is broken down into multiple small blocks of data. Each data block is resolved into multiple key-value pairs (K1, V1) and processed using the mapper or the map job. Each data block is processed by individual map jobs. The mapper executes the logic

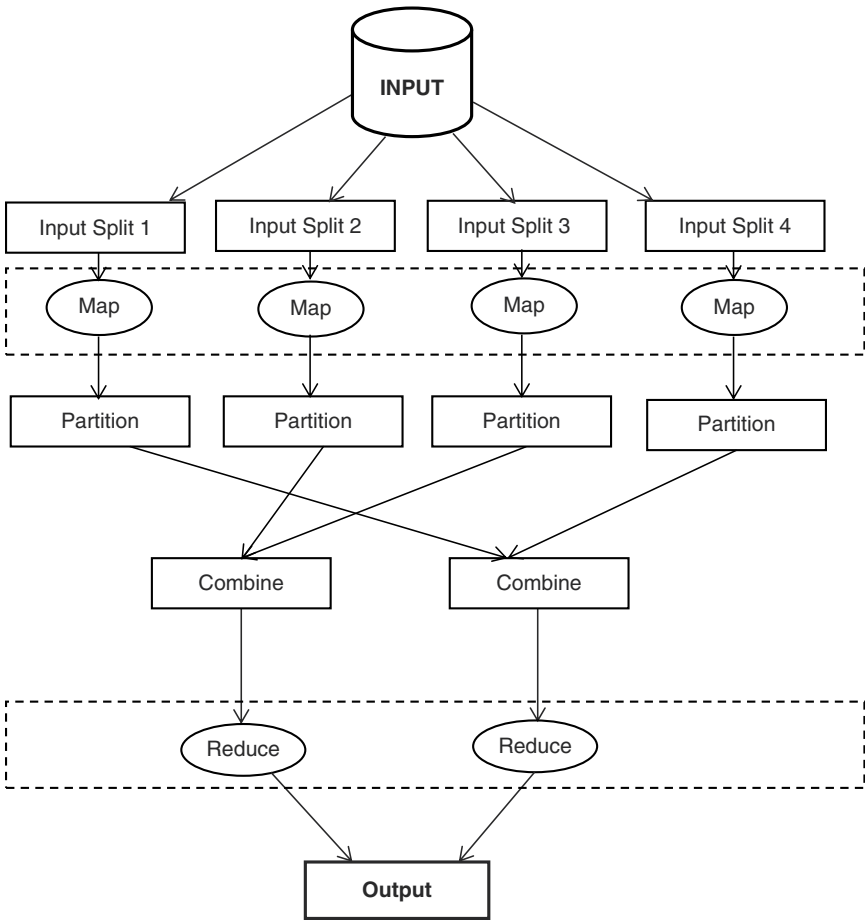


Figure 5.7 MapReduce model.

defined by the user in the MapReduce program and produces another intermediate key and value pair as the output. The processing of all the data blocks is done in parallel and the same key can have multiple values. The output of the mapper is represented as list (K2, V2).

5.3.1.2 Combiner

The output of the mapper is optimized before moving the data to the reducer. This is to reduce the overhead time taken to move larger data sets between the mapper and the reducer. The combiner is essentially the reducer of the map job and logically groups the output of the mapper function, which are multiple

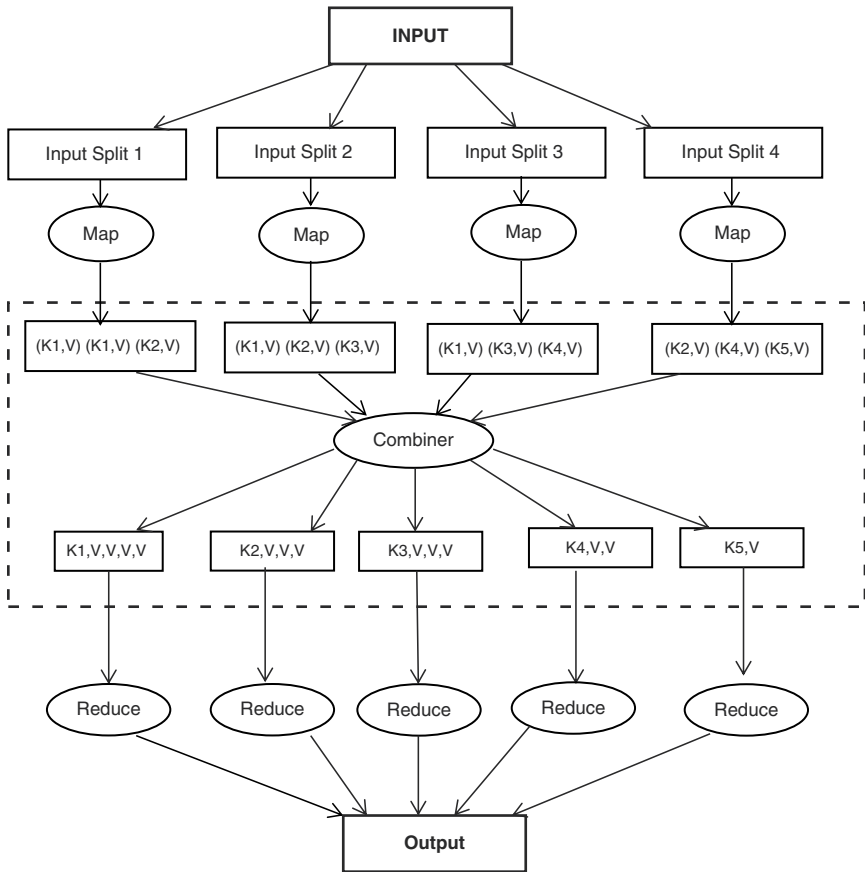


Figure 5.8 Combiner illustration.

key-value pairs. In combiner the keys that are repeated are combined, and the values corresponding to the key are listed. Figure 5.8 illustrates how processing is done in combiner.

5.3.1.3 Reducer

Reducer performs the logical function specified by the user in the MapReduce program. Each reducer runs in isolation from other reducers, and they do not communicate with each other. The input to the reducer is sorted based on the key. Reducer processes the value of each key, value-pairs it, and receives and produces another key-value pair as the output. The output key-value pair may be either the same as the input key-value pair or modified based on the user-defined function. The output of the reducer is written back to the DFS.

5.3.1.4 JobTracker and TaskTracker

Hadoop MapReduce has one JobTracker and several TaskTrackers in a master/slave architecture. Job tracker runs on the master node, and TaskTracker runs on the slave node. There is always only one TaskTracker per slave node. TaskTracker and NameNode run in one machine while JobTracker and DataNode run in another machine, making each node perform both computing and storage tasks. TaskTracker is responsible for workflow management and resource management. Parallel processing of data using MapReduce is handled by JobTracker. Figure 5.9 illustrates a JobTracker as the master and TaskTracker as the slaves executing the tasks assigned by the JobTracker. The two-way arrow indicates that communication flows in both directions. JobTracker communicates with TaskTracker to assign tasks, and TaskTracker periodically updates the progress of the tasks.

JobTracker accepts requests from client for job submissions, schedules tasks that are to be run by the slave nodes, administers the health of the slave nodes, and monitors the progress of tasks that are assigned to TaskTracker. JobTracker is a single point of failure, and if it fails, all the tasks running on the cluster will eventually fail; hence, the machine holding the JobTracker should be highly reliable. The communication between TaskTracker and the client as well as between TaskTracker and JobTracker is established through remote procedure calls (RPC). TaskTracker sends a Heartbeat signal to JobTracker to indicate that the node is alive. Additionally it sends the information about the task that it is handling if it is processing a task or its availability to process a task otherwise. After a specific time interval if the Heartbeat signal is not received from TaskTracker, it is assumed to be dead.

Upon submission of a job, the details about the individual tasks that are in progress are stored in memory. The progress of the task is updated with each heartbeat signal received from the JobTracker giving the end user a real-time view of the task in progress. On an active MapReduce cluster where multiple jobs are

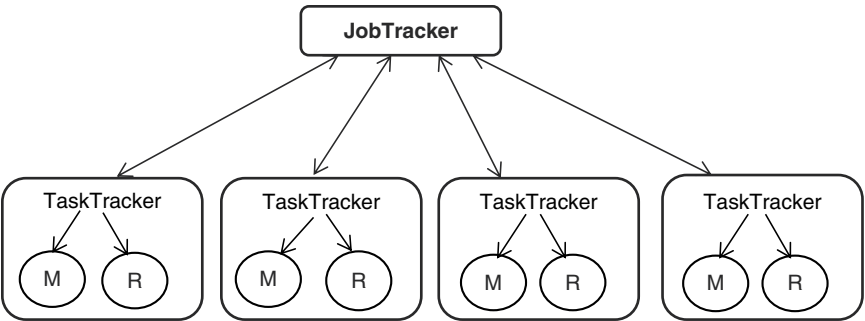


Figure 5.9 JobTracker and TaskTracker.

running, it is hard to estimate the RAM memory space it would consume, so it is highly critical to monitor the memory utilization by the JobTracker.

TaskTracker accepts the tasks from the JobTracker, executes the user code, and sends periodical updates back to the JobTracker. When processing of a task fails, it is detected by the TaskTracker and reported to the JobTracker. The JobTracker reschedules the task to run again either on the same node or on another node of the same cluster. If multiple tasks of the same job on a single TaskTracker fail, then the TaskTracker is refrained from executing other tasks corresponding to a specific job. On the other hand, if tasks from different jobs on the same TaskTracker fail, then the TaskTracker is refrained from executing any task for the next 24 hours.

5.3.2 MapReduce Input Formats

The primitive data types in Hadoop are

- BooleanWritable
- ByteWritable
- IntWritable
- VIntWritable
- FloatWritable
- LongWritable
- VLongWritable
- DoubleWritable

The MapReduce can handle the following formats of file:

- 1) TextInputFormat
- 2) KeyValueTextInputFormat
- 3) NLineInputFormat
- 4) SequenceFileInputFormat
- 5) SequenceFileAsTextInputFormat

TextInputFormat is the default Mapreduce InputFormat. The given input file is broken into lines, and each line is divided into key-value pairs. The key is of LongWritable type, and it is the byte offset of the starting of the line within the entire file. The corresponding value is the line of input excluding the line terminators, which may be a newline or carriage return. For example, consider the following file:

This is the first line of the input file,
 This is the second line of the input file,
 And this is the last line of the input file.

The input file is split up into three records, and the key-value pair of the above input is:

```
(0, This is the first line of the input file.)
(41, This is the second line of the input file.)
(82, And this is the last line of the input file.)
```

The offset acts as the key and is sufficient for applications requiring a unique identifier for each record. The offset along with the file name is unique for each file.

KeyValueTextInputFormat is the *InputFormat* for plain text. Similar to *TextInputFormat*, the input file in *KeyValueInputFormat* is also broken into lines of text, and each line is interpreted as a key-value pair by a separator byte. The default separator is a tab. For better understanding, a comma is taken as separator in the example below:

```
Line1, First line of input,
Line2, Second line of input,
Line3, Third line of input.
```

Everything up to the first separator is considered as the key. In the above example where a comma is the separator, the key in the first line is *Line1* and the text followed by the separator is the value corresponding to the key.

```
(Line1, First line of input,)
(Line2, Second line of input,)
(Line3, Third line of input.)
```

NLineInputFormat

In case of *TextInputFormat* and *KeyValueTextInputFormat*, the number of lines received by mapper as input varies depending on how the input file is split. Splitting the input file varies with the length of each line and size of each split. If the mapper has to receive a fixed number of lines as input, then *NLineInputFormat* is used.

SequenceFileInputFormat

SequenceFileInputFormat stores binary key-value pairs in sequence. *SequenceFileInputFormat* is used to read data from sequence files as well as a map file.

SequenceFileAsTextInputFormat

SequenceFileAsTextInputFormat is used to convert the key-value pairs of sequence files to text.

5.3.3 MapReduce Example

Consider the example below with four files and each file with two columns showing the temperature recorded at different cities on each day. This example handles very small data just to explain the MapReduce concept, but in an actual scenario, MapReduce handles terabytes to petabytes of data. Here the key is the city, and the value is the temperature.

File 1

City	Temperature Recorded
Leeds	20
Bexley	17
Bradford	11
Bradford	15
Bexley	19
Bradford	21

File 2

City	Temperature Recorded
Leeds	16
Bexley	12
Bradford	11
Leeds	13
Bexley	18
Bradford	17

File 3

City	Temperature Recorded
Leeds	19
Bexley	15
Bradford	12
Bexley	13
Bexley	14
Bradford	15

File 4

City	Temperature Recorded
Leeds	22
Bexley	15
Bradford	12
Leeds	18
Leeds	21
Bradford	20

Result After the Map job

Bradford,21	Bexley,19	Leeds,20
Bradford,17	Bexley,18	Leeds,16
Bradford,15	Bexley,15	Leeds,19
Bradford,20	Bexley,15	Leeds,22

Result After the reduce job

Bradford,21	Bexley,19	Leeds,22
-------------	-----------	----------

5.3.4 MapReduce Processing

Each input file that is broken into blocks is read by the `RecordReader`. The `RecordReader` checks the format of the file. If the format of the file is not specified, it takes the format as `TextInputFormat` by default. The `RecordReader` reads one record from the block at a time. Consider an example of a file with `TextInputFormat` to count the number of words in the file.

Hi how are you
How is your Job
How is your family
How is your brother
How is your sister
How is your mother
How is the climate there in your city

Let us consider the size of the file is 150 MB. The file will be split into 64 Mb blocks.

Hi how are you How is your Job How is your family	64MB	}
How is your brother How is your sister How is your mother	64MB	}
How is the climate there in your city	22MB	

The `RecordReader` will read the first record from the block “Hi how are you.” It will give (`byteOffset`, `Entireline`) as output to the mapper. Here in this case (0, Hi how are you) will be given as input to the mapper. When the second record is processed, the offset will be 15 as “Hi how are you” counts to a total of 14. The mapper will make key-value pair as its output.

A simple word count example is illustrated where the algorithm processes the input and counts the number of times each word occurs in the given input data. The given input file is split up into blocks and then processed to organize the data into key-value pairs. Here the actual word acts as the key, and the number of occurrences acts as the value. The MapReduce framework brings together all the values associated with identical keys. Therefore, in the current scenario all the values associated with identical keys are summed up to bring the word count, which is done by the reducer. After the reduce job is done the final output is produced, which is again a key-value pair with the word as the key and the total number of occurrences as value. This output is written back into the DFS, and the number of files written into the DFS depends on the number of reducers, one file for each reducer.

Figure 5.10 illustrates a simple MapReduce word count algorithm where the input file is split up into blocks. For simplicity an input file with a very small number of words is taken, each row here is considered as a block, and the occurrences of the words in each block are calculated individually and finally summed up. The number of times each word occurred in the first block is organized into key-value pairs.

After this process is done the key-value pairs are sorted in alphabetical order. Each Mapper has a combiner, which acts as a mini reducer. It does the job of the reducer for an individual block. Since there is only one reducer, it would be time consuming to process all the key-value pairs coming as output from mappers in parallel fashion. So the combiner is used to increase performance by reducing the traffic. Combiner combines all the key-value pairs of individual mappers and passes them as input to the reducer. The above output from the combiner is then passed to the reducer, and it combines the words from all the blocks and gives a single output file.

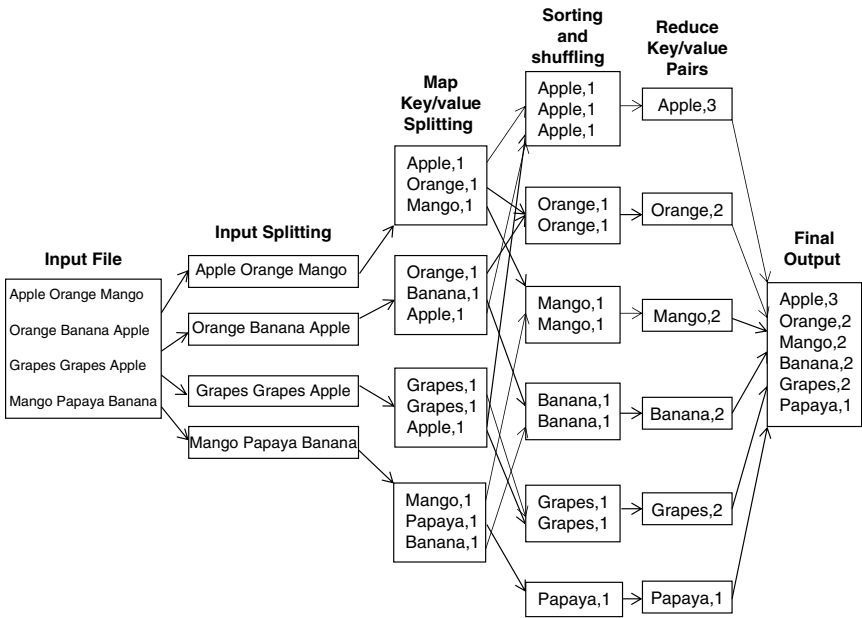


Figure 5.10 Word count algorithm.

5.3.5 MapReduce Algorithm

A MapReduce task has a mapper and a reducer class. The mapper class performs tokenizing the input, mapping, shuffling, and sorting while the reducer class takes the output of the mapper class as its input and performs a searching task to find the matching pairs and reduce them. MapReduce uses various algorithms to divide a task into multiple smaller tasks and assign them to multiple nodes. MapReduce algorithms are essential in assigning map and reduce tasks to appropriate nodes in the cluster. Some of the mathematical algorithms used by the MapReduce paradigm to implement the tasks are sorting, searching, indexing, and Term Frequency–Inverse Document Frequency (TF-IDF).

A *sorting* algorithm is used by the MapReduce algorithm to process the data and analyze them. The key-value pair from the mapper is sorted with the sorting algorithm. The *RawComparator* class is used by the mapper class to gather similar key-value pairs. These are intermediate key-value pairs, and they are sorted by Hadoop automatically to form $(K1, \{V1, V1 \dots\})$ before presenting them to the reducer.

A *searching* algorithm is used to find a match from the given pattern if the filename and text are passed as input. For example, in a given file with employee name and corresponding salary, a searching algorithm with the file name as input

to find out the employee with maximum salary will output the employee name with highest salary and corresponding salary.

Indexing in MapReduce points to a data and its corresponding address. The indexing technique used in MapReduce is called inverted index. Search engines such as Google use an inverted indexing technique.

TF-IDF is the acronym for Term Frequency–Inverse Document Frequency. It is a text-processing algorithm, and the term frequency indicates the number of times a term occurs in a file. Inverse Document Frequency is calculated by dividing the number of files in a database by the number of files where a particular term appears.

5.3.6 Limitations of MapReduce

The MapReduce daemon is indeed the most successful parallel processing framework. MapReduce is used by the research community to solve data-intensive problems in environmental science, finance, and bioinformatics. However, MapReduce also has its limitations. The intrinsic limitation of MapReduce is one-way scalability in its design. It is designed in a way to scale up to process typically large data sets but with a restriction to process smaller data sets. The reason is the fact that the NameNode memory space cannot be wasted to hold the metadata of a large number of smaller data sets. Also NameNode is the single point of failure, and when the NameNode goes down, the cluster becomes unavailable, restricting the system from being highly available. While high availability of the system is the major requirement of many of the applications, it became imperative to design a system that is not only scalable but also highly available.

The reduce phase cannot start until the map task is complete. Similarly, starting a new map task before the completion of the reduce task in the previous application is not possible in standard MapReduce. Hence, each application has to wait until the previous application is complete. When map tasks are executing, the reducer nodes are idle; similarly, when reduce a task is in execution, the mapper nodes are idle, which results in improper utilization of resources. Also there may be a requirement for resources to execute map task while the resources are idle waiting for the map task to be complete and to start the execution of the reduce task.

5.4 Hadoop 2.0

The architectural design of Hadoop 2.0 made HDFS a highly available filesystem where NameNodes are available in active and standby configuration. In case of failure of the active NameNode, standby NameNode takes up the responsibilities of the active NameNode and continues to respond to clients requests without interruption. Figure 5.11 shows Hadoop 1.0 vs. Hadoop 2.0.

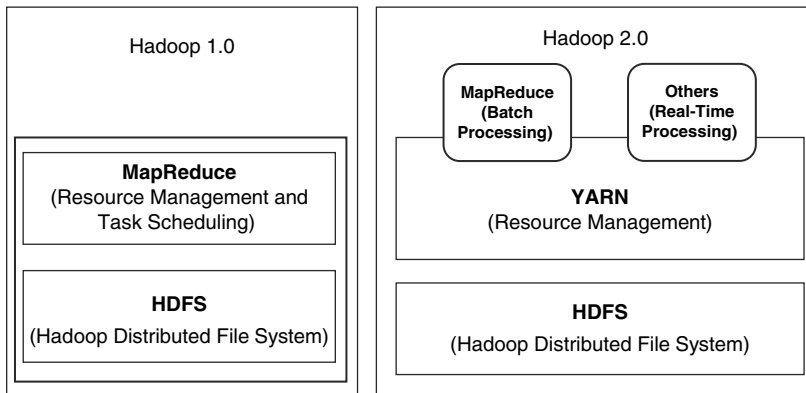


Figure 5.11 Hadoop 1.0 vs Hadoop 2.0.

5.4.1 Hadoop 1.0 Limitations

Limitations on scalability – JobTracker running on a single machine does several tasks including:

- Task scheduling;
- Resource Management;
- Administers the progress of the task; and
- Monitors the health of TaskTracker.

Single Point of Failure – JobTracker and NameNode are the single point of failure. If it fails, the entire Job will fail.

Limitation in running applications – Hadoop 1.0 is limited to run only a MapReduce application and supports only the batch mode of processing.

Imbalance in Resource Utilization – Each TaskTracker is allocated predefined numbers of map and reduce slots, and hence resources may not be utilized completely when the map slots are performing tasks and might be full while the reduce slots are available to perform tasks and vice versa. The resources allocated to perform a reducer function could be sitting idle in spite of an immediate requirement for resources to perform a mapper function.

5.4.2 Features of Hadoop 2.0

High availability of NameNode – NameNode, which stores all the metadata, is highly crucial because if the NameNode crashes, the entire Hadoop cluster goes down. Hadoop 2.0 solves this critical issue by running two NameNodes on the same cluster, namely, the Active NameNode and the standby NameNode. In case of failure of the active NameNode the standby NameNode acts as the active NameNode.

Figure 5.12 illustrates the active and standby NameNodes.

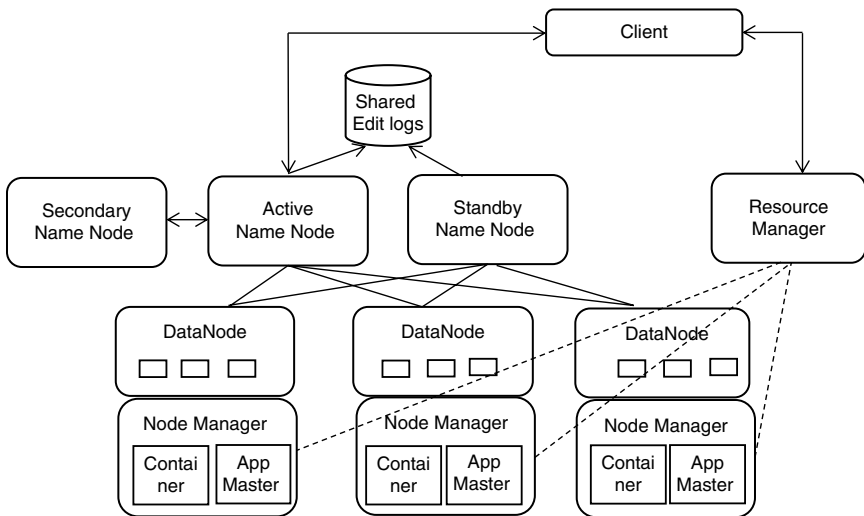


Figure 5.12 Active NameNode and standby NameNode.

Run Non MapReduce applications – Hadoop 1.0 is capable of running only the MapReduce jobs to process HDFS data. For processing the data stored in HDFS by some other processing paradigm, the data has to be transferred to some other storage mode such as HBase or Cassandra and further processing has to be done. Hadoop 2.0 has a framework called YARN, which runs non-MapReduce applications on the Hadoop framework. Spark, Giraph, and Hama are some of the applications that run on Hadoop 2.0.

Improved resource utilization – In Hadoop 1.0 resource management and monitoring the execution of MapReduce tasks are administered by the JobTracker. In Hadoop 2.0 YARN splits up job scheduling and resource management, the two major functions of JobTacker into two separate daemons:

- Global resource manager – resource management; and
- per-application application master – job scheduling and monitoring.

Beyond Batch processing – Hadoop 1.0, which was limited to running batch-oriented applications, is now upgraded to Hadoop 2.0 with the capability to run real-time and near-real time applications. Figure 5.13 shows Hadoop 2.0.

5.4.3 Yet Another Resource Negotiator (YARN)

To overcome the drawbacks of the Hadoop MapReduce architecture, the Hadoop Yarn architecture was developed. In Hadoop Yarn the responsibilities of JobTracker, that is, resource management and job scheduling, are split up to improve performance. Each job request has its own ApplicationMaster.

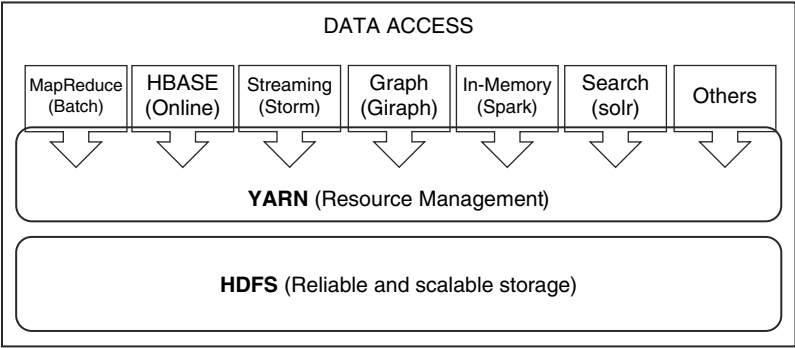


Figure 5.13 Hadoop 2.0.

The main purpose of evolution of the YARN architecture is to reinforce more data processing models such as Apache Storm, Apache Spark, Apache Giraph, and more than just supporting MapReduce. YARN splits the responsibilities of JobTracker into two daemons, a global ResourceManager and per-application ApplicationMaster. ResourceManager takes care of resource management while per-application ApplicationMaster takes care of job scheduling and monitoring. ResourceManager is a cluster level component managing resource allocation for applications running in the entire cluster. The responsibility of the TaskTracker is taken up by the ApplicationMaster, which is application specific and negotiates resources for the applications from the ResourceManager and works with NodeManager to execute the tasks. Hence, in the architecture of YARN the JobTracker and TaskTracker are replaced by ResourceManager and ApplicationMaster respectively.

5.4.4 Core Components of YARN

- ResourceManager;
- ApplicationMaster; and
- NodeManager.

5.4.4.1 ResourceManager

A ResourceManager is a one-per-cluster application that manages the allocation of resources to various applications. Figure 5.14 illustrates various components of ResourceManager. The two major components of ResourceManager are ApplicationsManager and scheduler. ApplicationsManager manages the ApplicationMasters across the cluster and is responsible for accepting or rejecting the applications, and upon accepting an application, it provides resources to the ApplicationMaster for the execution of the application, monitors the status

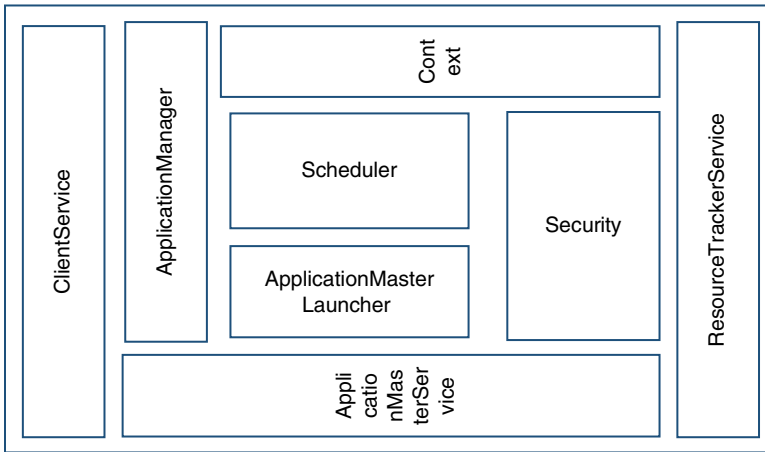


Figure 5.14 ResourceManager.

of the running applications, and restarts the applications in case of failures. Scheduler allocates resources based on FIFO, Fair, and capacity policies to the applications that are submitted to the cluster. It does not monitor the job status whereas its job is only to allocate resources to the applications based on its requirements. ClientService is the interface through which the client interacts with the ResourceManager and handles all the application submission, termination, and so forth. ApplicationMasterService responds to RPC from the ApplicationMaster and interacts with applications. ResourceTrackerService interacts with ResourceManager for resource negotiation by ApplicationMaster.

ApplicationMasterLauncher launches a container to ApplicationMaster when a client submits a job. Security component generates ContainerToken and ApplicationToken to access container and application respectively.

5.4.4.2 NodeManager

Figure 5.15 illustrates various components of NodeManager. The NodeStatusUpdater establishes the communication between ResourceManager and NodeManager and updates ResourceManager about the status of the containers running on the node. The ContainerManager manages all the containers running on the node. The ContainerExecutor interacts with the operating system to launch or cleanup container processes. NodeHealthCheckerService monitors the health of the node and sends the Heartbeat signal to ResourceCheckerManager. Security component verifies all the incoming requests are authorized by ResourceManager.

The MapReduce framework of Hadoop 1.0 architecture supports only batch processing. To process the applications in real time and near-real time the data

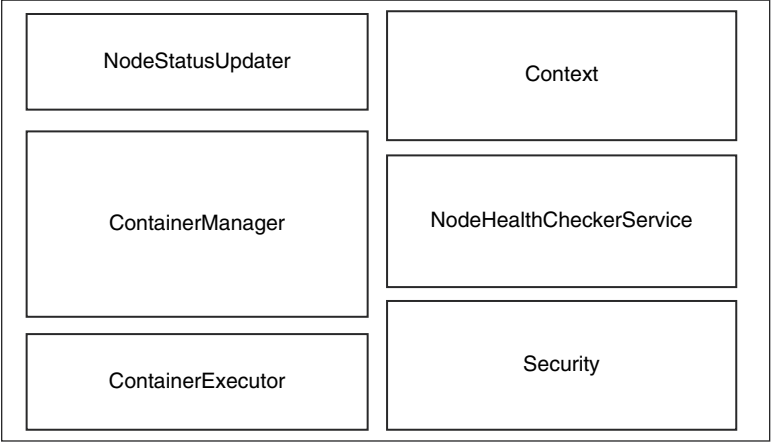


Figure 5.15 NodeManager.

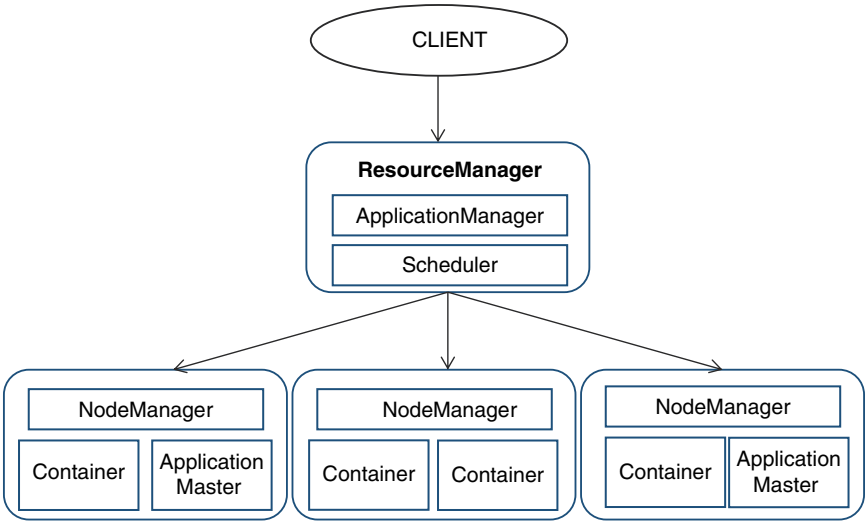


Figure 5.16 YARN architecture.

has to be taken out from Hadoop into other databases. To overcome the limitations of Hadoop 1.0, Yahoo has developed YARN.

Figure 5.16 shows the YARN architecture. In YARN there is no JobTracker and TaskTracker. ResourceManager, ApplicationMaster, and NodeManager together constitute YARN. The responsibilities of JobTracker, that is, resource allocation, job scheduling, and monitoring is split up among ResourceManager

and ApplicationMaster in YARN. ResourceManager allocates all available cluster resources to applications. ApplicationMaster and NodeManager together execute and monitor the applications. ResourceManager has a pluggable scheduler that allocates resources among running applications. ResourceManager does only the scheduling job and does not monitor the status of the tasks. Unlike JobTracker in Hadoop MapReduce, the ResourceManager does not have fault tolerance and it does not restart any tasks that failed due to hardware or application failure. ApplicationMaster negotiates the resources from ResourceManager and tracks their status. NodeManager monitors the resource usage and reports it to ResourceManager. Applications request resources via ApplicationMaster, and the scheduler responds to the request and grants a container. Container is the resource allocation in response to the ResourceRequest. In other words, container is the rights of the applications to use the resources.

Since all the resource allocation, job scheduling, and monitoring is handled by ResourceManager, ApplicationMaster, and NodeManager, YARN uses MapReduce for processing without any major changes. MapReduce is used only for processing. It performs the processing through YARN, and other similar tools also perform the processing through YARN. Thus, YARN is more generic than the earlier Hadoop MapReduce architecture. Hence, non-MapReduce tasks also can be processed using YARN, which was not supported in Hadoop MapReduce.

5.4.5 YARN Scheduler

The YARN architecture has a scheduler that allocates resources according to the applications' requirements depending on some scheduling policies. The different scheduling policies available in YARN architecture are:

- FIFO scheduler;
- Capacity scheduler; and
- Fair scheduler.

5.4.5.1 FIFO Scheduler

The FIFO (first in, first out) scheduler is a simple and easy-to-implement scheduling policy. It executes the jobs in the order of submission: jobs submitted first will be executed first. The priorities of the applications will not be taken into consideration. The jobs are placed in queue, and the first job in the queue will be executed first. Once the job is completed, the next job in the queue will be served, and the subsequent jobs in the queue will be served in a similar fashion. FIFO works efficiently for smaller jobs as the previous jobs in the queue will be completed in a short span of time, and other jobs in the queue will get their turn after a small wait. But in case of long running jobs, FIFO might not be efficient as most of the

resources will be consumed, and the other smaller jobs in the queue may have to wait their turn for a longer span of time.

5.4.5.2 Capacity Scheduler

The capacity scheduler allows multiple applications to share the cluster resources securely so that each running application is allocated resources. This type of scheduling is implemented by configuring one or more queues, with each queue assigned a calculated share of total cluster capacity. The queues are further divided in a hierarchical way, so there may be different applications sharing the cluster capacity allocated to that queue. Within each queue the scheduling is based on the FIFO policy. The queue has Access Control List, which manages the task of deciding which user has to submit a job to which queue. If more than one job is running in a specific queue and if there are idle resources available, the scheduler may assign the resource to other jobs in the queue.

5.4.5.3 Fair Scheduler

Fair scheduling policy is the efficient way of sharing cluster resources. Allocation of resources are done in a way that all the applications running on a cluster get a fairly equal share of the resources on a given time period. If an application running on a cluster requests all the resources and simultaneously if another job is submitted, the fair scheduler allocates the resources that are free in the previous application so that all the running applications are allocated a fairly equal share of resources. Preemption of applications is also adopted where a running application might be temporarily stopped executing and the resource containers may be got back from the ApplicationMaster. In this type of scheduling each queue is assigned a weight depending on which resources are allocated to the queue. A light-weight queue is assigned a minimum number of resources, while a heavy-weight queue would receive higher number of resources compared to that of a light-weight queue. At the time of submitting the application, the users can choose the queue based on the requirement. The user may specify the name of a heavy-weight queue if the application requires a large number of resources, and a light-weight queue may be specified if the application requires a minimal number of resources.

In the fair scheduling policy, if a large job is started and it is the only job currently running, all the available cluster resources will be allocated to the job, and after a certain period of time, a small job is submitted, half the resources are freed from the first large job and is allocated to the second small job so that to each job is allocated a fairly equal share of resources. Once the small job is completed, the large job is again allocated the full cluster capacity. Thus, the cluster resources are used efficiently and the jobs are completed in a timely manner.

5.4.6 Failures in YARN

A successful completion of an application running in Hadoop 2.0 depends on the coordination of various YARN components, namely, ResourceManager, NodeManager, ApplicationMaster, and the containers. Any failure in the YARN components may result in the failure of the application. Hadoop is a distributed framework and hence dealing with failures in such distributed system is comparatively challenging and time consuming. The various YARN components failures are:

- ResourceManager failure;
- NodeManager failure;
- ApplicationMaster failure; and
- Container failure.

5.4.6.1 ResourceManager Failure

In the earlier versions of YARN the ResourceManager is the single point of failure, and if the ResourceManager fails, it has to be manually intervened, debugged, and the ResourceManager has to be restarted. During this time when the ResourceManager is down the whole cluster is unavailable, and once the ResourceManager is active again, all the jobs that were running in the ApplicationMaster have to be restarted. So the YARN is upgraded in two ways to overcome these issues. In the latest version of the YARN architecture, one way is to have an active and a passive ResourceManager. So when the active ResourceManager goes down, the passive ResourceManager becomes active and takes responsibility. Another way is to have a zookeeper, which holds the state of the ResourceManager: when the active ResourceManager goes down, the failure condition is shared with the passive ResourceManager, and it changes its state to active and takes up the responsibility of managing the cluster.

5.4.6.2 ApplicationMaster Failure

The ApplicationMaster failure is detected by the ResourceManager, and another container is started with a new instance of the ApplicationMaster running in it for another attempt of execution of the application. The new ApplicationMaster is responsible for recovering the state of the failed ApplicationMaster. The recovery is possible only if the state of the ApplicationMaster is available in any external location. If recovery is not possible, the ApplicationMaster starts running the application from scratch.

5.4.6.3 NodeManager Failure

The NodeManager runs in all the slave nodes and is a per-node application. The NodeManager is responsible for executing a portion of a job. NodeManager sends a Heartbeat signal to the ResourceManager periodically to update its status. If the

Heartbeat is not received for a specific period of time, the ResourceManager assumes that the NodeManager is dead and then removes that NodeManager from the cluster. The failure is reported to the ApplicationManager, and the container running in the NodeManager is killed. The ApplicationManager reruns the portion of the job running within that NodeManager.

5.4.6.4 Container Failure

Containers are responsible for executing the map and the reduce task. ApplicationMaster detects the failure of container when it does not receive the response from the container for a certain period of time. ApplicationMaster then attempts to re-execute the task. If the task again fails for a certain number of times, the entire task is considered to be failed. The number of attempts to rerun the task can be configured by the user individually for both map and reduce tasks. The configuration can be either based on the number of attempts or based on the percentage of tasks failed during the execution of the job.

5.5 HBASE

HBase is a column-oriented NoSQL database that is a horizontally scalable open-source distributed database built on top of the HDFS. Since it is a NoSQL database, it does not require any predefined schema. HBase supports both structured and unstructured data. It provides real-time access to the data in HDFS. HBase provides random access to massive amounts of structured data sets. Hadoop can access data sets only in sequential fashion. A huge data set when accessed in sequential manner for a simple job may take a long time to give the desired output, which results in high latency. Hence, HBase came into picture to access the data randomly. Hadoop stores data in flat files, while HBase stores data in key-value pairs in a column-oriented fashion. Also Hadoop supports write once and read many times while HBase supports read and write many times. HBase was designed to support the storage of structured data based on Google's Bigtable.

Figure 5.17 shows HBase master-slave architecture with HMaster, Region Server, HFile, MemStore, Write-ahead log (WAL) and Zookeeper. The HBase Master is called HMaster and coordinates the client application with the Region Server. HBase slave is the HRegionServer, and there may be multiple HRegions in an HRegionServer. Each region is used as database and contains the distribution of tables. Each HRegion has one WAL, multiple HFiles, and its associated MemStore. WAL is the technique used in storing logs. HMaster and HRegionServer work in coordination to serve the cluster.

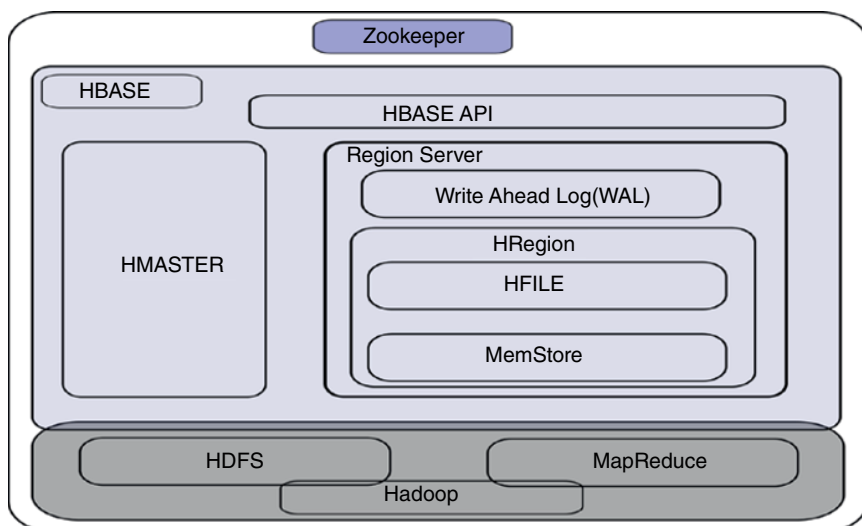


Figure 5.17 HBase architecture.

HBase has no additional features to replicate data, which has to be provided by the underlying file system. HDFS is the most commonly used file system because of its fault tolerance, built-in replication, and scalability. HBase finds its application in medical, sports, web, e-commerce, and so forth.

HMaster – HMaster is the master node in the HBase architecture similar to NameNode in Hadoop. It is the master for all the RegionServers running on several machines, and it holds the metadata. Also, it is responsible for RegionServer failover and auto sharding of regions. To provide high availability, an HBase cluster can have more than one HMaster, but only one HMaster will be active at a time. Other than the active HMaster, all other HMasters are passive until the active HMaster goes down. If the Master goes down, the cluster may continue to work as the clients will communicate directly to the RegionServers. However, since region splits and RegionServer failover are performed by HMaster, it has to be started as soon as possible. In HBase HBase:meta is the catalog table where the list of all the regions is stored.

Zookeeper – Zookeeper provides a centralized service and manages the coordination between the components of a distributed system. It facilitates better reachability to the system components.

RegionServer – RegionServer has a set of regions. RegionServers hold the actual data, similar to a Hadoop cluster where the NameNode holds the metadata and the DataNode holds the actual data. RegionServers serves the regions assigned to it, handles the read/write requests, and maintains Hlogs. Figure 5.18 shows a RegionServer.

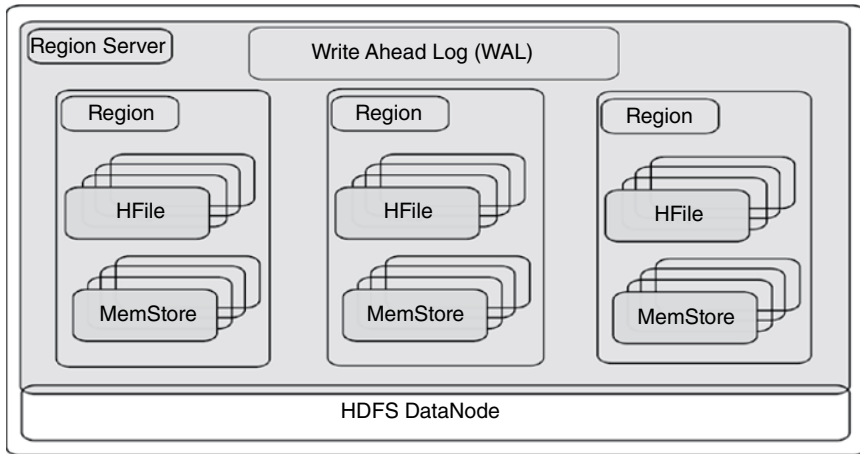


Figure 5.18 RegionServer architecture.

Region – The tables in HBase are split into smaller chunks, which are called regions, and these regions are distributed across multiple RegionServers. The distribution of regions across the RegionServers is handled by the Master. There are two types of files available for data storage in the region, namely, HLog, the WAL, and the Hfile, which is the actual data storage file.

WAL – Data write is not performed directly on the disk; rather, it is placed in the MemStore before it is written on to the disk. Before the MemStore is being flushed if the RegionServer fails the data may be lost as the MemStore is volatile. So, to avoid the data loss it is written into the log first and then written into the MemStore. So if the RegionServer goes down data can be effectively recovered from the log.

HFile – HFiles are the files where the actual data are stored on the disk. The file contains several data blocks, and the default size of each data block is 64KB. For example, a 100 MB file can be split up into multiple 64KB blocks and stored in HFile.

MemStore – Data that has to be written to the disk are first written to the MemStore and WAL. When the MemStore is full, a new HFile is created on HDFS, and the data from the MemStore are flushed in to the disk.

5.5.1 Features of HBase

- Automatic Failover – HBase failover is supported through HRegionServer replication.
- Auto sharding – HBase Regions has contiguous rows that are split by the system into smaller regions when a threshold size is reached. Initially a table has only one region when data are added, and if the configured maximum size is

exceeded, the region is split up, each region is served by an HRegionServer, and each HRegionServer can serve more than one region at a time.

- Horizontal scalability – HBase is horizontally scalable, which enables the system to scale wider to meet the increasing demand where the server need not be upgraded as in the case of vertical scalability. More nodes can be added to the cluster on the fly. Since scaling out storage uses low-cost commodity hardware and storage components, HBase is cost effective.
- Column oriented – In contrast with a relational database, which is row-oriented, HBase is column-oriented. The working method of a column-store database is that it saves data into sections of columns rather than sections of rows.
- HDFS is the most common file system used by HBase. Since HBase has a pluggable file system architecture, it can run on any other supported file system as well. Also, HBase provides massive parallel processing through the MapReduce framework.

5.6 Apache Cassandra

Cassandra is a highly available, linearly scalable, distributed database. It has a ring architecture with multiple nodes where all the nodes are equal, so there are no master or slave nodes. The data is partitioned among all the nodes in a Cassandra cluster, which can be accessed by a partition key. These data are replicated among cluster nodes to make the cluster highly available. In Cassandra when load increases, additional nodes can be added to the cluster to share the load, as the load will be distributed automatically among the newly added nodes. Since the data is replicated across multiple nodes in the cluster, data can be read from any node, and write can be performed on any node. The node on which a read or write request is performed is called coordinator node. Upon performing a write request, the data in the cluster becomes eventually consistent and retrieves the updated data irrespective of the node on which the data write is performed. Since the data is replicated across multiple nodes, there is no single point of failure.

If a node in the cluster goes down, Cassandra will continue the read/write operation on the other nodes of the cluster. The operations that are to be performed are queued and updated once the failed node is up again.

5.7 SQOOP

When the structured data is huge and RDBMS is unable to support the huge data, the data is transferred to HDFS through a tool called SQOOP (**SQL to Hadoop**). To access data in databases outside HDFS, map jobs use external APIs. Organizational data that are stored in relational databases are extracted and stored into Hadoop

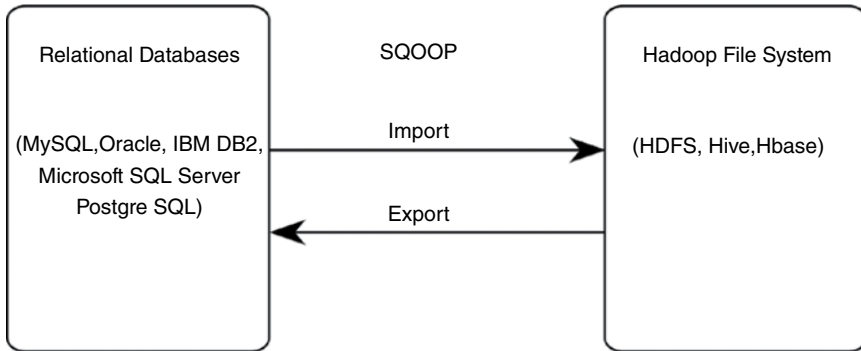


Figure 5.19 SQOOP import and export.

using SQOOP for further processing. SQOOP can also be used to move data from relational databases to HBase. The final results after the analysis is done are exported back to the database for future use by other clients. Figure 5.19 shows SQOOP import and export of data between a Hadoop file system and relational databases. It imports data from traditional databases such as MySQL to Hadoop and exports data from Hadoop to traditional databases. Input to the SQOOP is from a database table or another structured data repository. The input to SQOOP is read row by row into HDFS. Additionally SQOOP can also import data into HBase and Hive. Initially SQOOP was developed to transfer data from Oracle, Tetradata, Netezza, and postgres. Data from a database table are read in parallel, and hence the output is a set of files. Output of SQOOP may be a text file (fields are separated by a comma or a space) or binary Avro, which contains the copy of the data imported from the database table or mainframe systems.

The tables from RDBMS are imported into HDFS where each row is treated as a record and is then processed in Hadoop. The output is then exported back to the target database for further analysis. This export process involves parallel reading of a set of binary files from HDFS, and then the set is split up into individual records and the records are inserted as rows in database tables. If a specific row has to be updated, instead of inserting it as a new row, the column name has to be specified.

Figure 5.20 shows the SQOOP architecture. Importing data in SQOOP is executed in two steps:

- 1) Gather metadata (column name, type, etc.) of the table from which data is to be imported;
- 2) Transfer the data with the map only job to the Hadoop cluster and databases in parallel.

SQOOP exports the file from HDFS back to RDBMS. The files are passed as input to the SQOOP where input is read and parsed into records using the delimiters specified by the users.

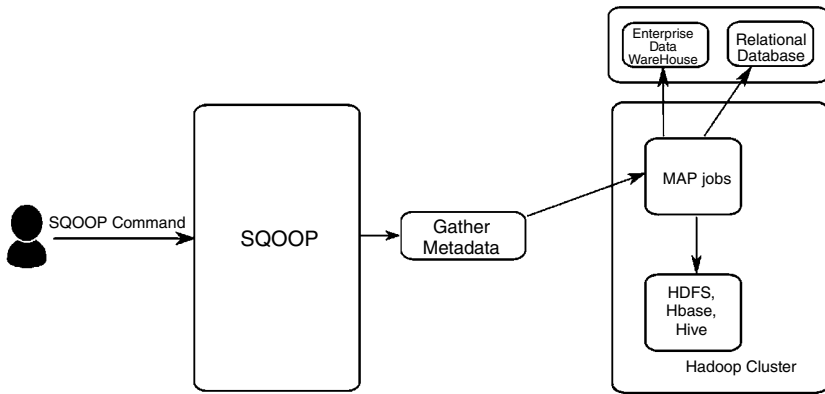


Figure 5.20 SQOOP 1.0 architecture.

5.8 Flume

Flume is a distributed and reliable tool for collecting large amount of streaming data from multiple data sources. The basic difference flume and SQOOP is that SQOOP is used in ingesting structured data into Hive, HDFS, and HBase, whereas Flume is used to ingest large amounts of streaming data into Hive, HDFS, and HBase. Apache flume is a perfect fit for aggregating the high volume of streaming data, storing, and analyzing them using Hadoop. It is fault tolerant with failover and a recovery mechanism. It collects data from a streaming data source such as a sensor, social media, log files from web servers, and so forth, and moves them into HDFS for processing. Flume is also capable of moving data to systems other than HDFS such as HBase and Solr. Flume has a flexible architecture to capture data from multiple data sources and adopts a parallel processing of data.

5.8.1 Flume Architecture

Figure 5.21 shows the Flume architecture. Core concepts and components of flume are described below.

5.8.1.1 Event

The unit of data in the data flow model of the flume architecture is called event. Data flow is the flow of data from the source to the destination. The flow of events is through an Agent.

5.8.1.2 Agent

The three components residing in an Agent are Source, Channel, and Sink, which are the building blocks of the flume architecture. The Source and the Sink are connected through the Channel. An Agent receives events from a Source, directs

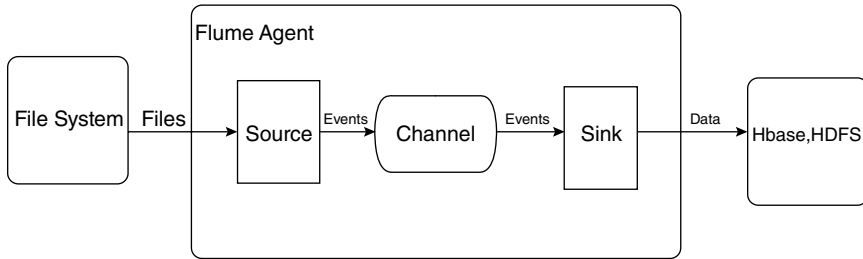


Figure 5.21 Flume architecture.

them to a Channel, and the Channel stores the data and directs them to the destination through a Sink. A Sink collects the events that are forwarded from the Channel, which in turn forwards it to the next destination.

The Channels are the temporary stores to hold the events from the sources until they are transferred to the sink. There are two types of channels, namely, in-memory queues and disk-based queues. In in-memory queues, the data is not persisted in case of Agent failure and hence provides high throughputs, but the events cannot be recovered, whereas disk-based queues are slower than in-memory queues as the events are persisted, and the events can be recovered in case of failure of Agents.

The events are transferred to the destination in two separate transactions. The events are transferred from Source to Channel in one transaction, and another transaction is used to transfer the events from the Channel to the destination. The transaction is marked complete only when the event transfer from the Source to the Channel is successful. When the event transfer from the Source to the Channel is successful, the event is then forwarded to the Sink using another transaction. If there is any failure in event transfer, the transaction will be rolled back, and the events will remain in the Channel for delivery at a later time.

5.9 Apache Avro

Apache Avro is an open-source data serialization framework. Data serialization is a technique that translates data in the memory into binary or textual format to transport it over a network or to store on a disk. Upon retrieving the data from the disk, the data has to be de-serialized again for further processing. It was designed to overcome Hadoop's drawback, the lack of portability. The data format, which can be processed by multiple languages such as C, C++, Java, Perl, and Python, can be easily shared with a large number of end users when compared to a data format that can be processed by a single language. Avro has a data format that can be processed by multiple languages and is usually written JavaScript Object Notation (JSON).

Avro is a language-independent, schema-based system. Avro can process the data without prior knowledge about the schema. The schema of the serialized data is written in JSON and stored with the data in a file called Avro data file for further processing. Since Avro schemas are defined in JSON, it facilitates easy implementation of the data in the languages that already has JSON libraries. The Avro schema has the details about the type of the record, name of the record, location of the record, fields in the record, and data types of the fields in the record. Avro also finds its application in remote procedure calls (RPC) where the schemas are exchanged by the client and server.

Avro sample schema:

```
{
  "type": "record",
  "namespace": "example",
  "name": "StudentName",
  "fields": [
    { "name": "first", "type": "string" },
    { "name": "last", "type": "string" }
  ]
}
```

- Type – document type, which is record in this case.
- Namespace – name of the namespace where the object resides.
- Name – Name of the schema. The combination of name with the namespace is unique, and it is used to identify the schema within the storage platform.
- Fields –it defines the fields of the record.
 - Name –Name of the field.
 - Type – data type of the field. Data types can be simple as well as complex data types. Simple data types include null, string, int, long, float, double, and bytes. Complex data types include Records, Arrays, Enums, Maps, Unions, and Fixed.

5.10 Apache Pig

Pig is developed at Yahoo. Pig has two components. The first component is the pig language, called Pig Latin, and the second is the environment where the Pig Latin scripts are executed. Unlike HBase and HQL, which can handle only structured data, Pig can handle any type of data sets, namely, structured, semi-structured, and unstructured. Pig scripts are basically focused on analyzing large data sets reducing the time consumed to write code for Mapper and Reducer. Programmers with no basic knowledge about the Java language can

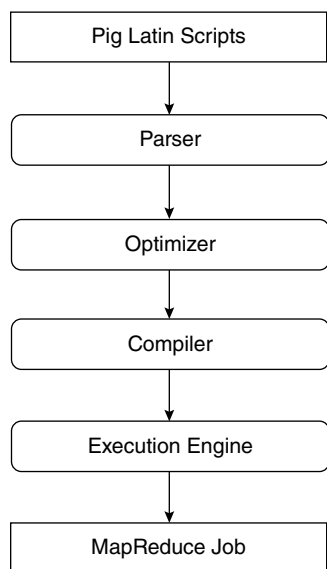


Figure 5.22 Pig – internal process.

perform MapReduce tasks using the Pig Latin language. Hundreds of lines coded in Java can be executed using fewer Pig Latin scripts. Internally Pig Latin scripts are converted into MapReduce jobs and executed on a Hadoop distributed environment. This conversion is carried out by the Pig Engine, which accepts Pig Latin scripts as input and produces MapReduce jobs as output. Pig scripts pass through several steps to be converted to MapReduce jobs. Figure 5.22 depicts the internal process of Pig.

Parser checks the syntax of the script. Optimizer carries out logical optimization. Compiler compiles the logically optimized code into MapReduce jobs. The execution Engine submits the MapReduce jobs to Hadoop, and then these MapReduce jobs are executed in a Hadoop distributed environment.

5.11 Apache Mahout

Apache Mahout is an open-source machine learning algorithm that implements clustering, classification, and recommendation algorithms. But it is flexible to implement other algorithms too. Apache mahout primarily finds its application when the data sets are too large to be handled by other machine learning algorithms. Mahout fulfills the needs of machine learning tools for the big data era. The scalability of Mahout differentiates it from other machine learning tools such as R.

5.12 Apache Oozie

Tasks in the Hadoop environment in some cases may require multiple jobs to be sequenced to complete its goal, which requires the component Oozie in the Hadoop ecosystem. Oozie allows multiple Map/Reduce jobs to combine into a logical unit of work to accomplish the larger task.

Apache Oozie is a tool that manages the workflow of the programs at a desired order in the Hadoop environment. Oozie is capable of configuring jobs to run on demand or periodically. Thus, it provides greater control over jobs allowing them

to be repeated at predetermined intervals. By definition, Apache Oozie is an open-source workflow management engine and scheduler system to run and manage jobs in the Hadoop distributed environment. It acts as a job coordinator to complete multiple jobs. Multiple jobs are run in sequential order to complete a task as a whole. Jobs under a single task can also be scheduled to run in parallel. Oozie supports any type of Hadoop jobs, which includes MapReduce, Hive, Pig, SQOOP, and others.

There are three types of Oozie jobs:

- **Workflow jobs**—These jobs are represented as directed acyclic graphs (DAGs) and run on demand.
- **Coordinator Jobs**—These jobs are scheduled to execute periodically based on frequency or availability of input data.
- **Bundle Jobs**—These are a collection of coordinator jobs run and managed as a single job.

Oozie job definitions for workflow jobs, coordinator jobs, and bundle jobs are written in XML. The Oozie workflow is created when the workflow definition is placed in a file named `workflow.xml`.

5.12.1 Oozie Workflow

An Oozie workflow has multiple stages. A workflow is a collection of actions that are Hadoop Map/Reduce jobs, Pig, Hive, or Sqoop jobs in a control dependency DAGs. Action can also be non-Hadoop jobs such as an email notification or a java application. Control dependency between actions is that the second action cannot start until the first action has been completed. Oozie workflow has control nodes and action nodes. Action nodes specify the actions. Actions are the jobs, namely, a MapReduce job, a Hive job, a Pig job, and so forth. Control nodes determine the order of execution of the actions. The actions in a workflow are dependent on each other and an action will not start until its preceding action in the workflow has been completed. Oozie workflows are initiated on demand, but the majority of times they are run at regular time intervals or based on data availability or external events. Workflow execution schedules are defined based on these parameters. The various control nodes in a workflow are:

- Start and end control nodes;
- Fork and join control nodes; and
- Decision control nodes.

The start and end of the workflow are defined by the start and end control nodes. Parallel executions of the actions are performed by the fork and join

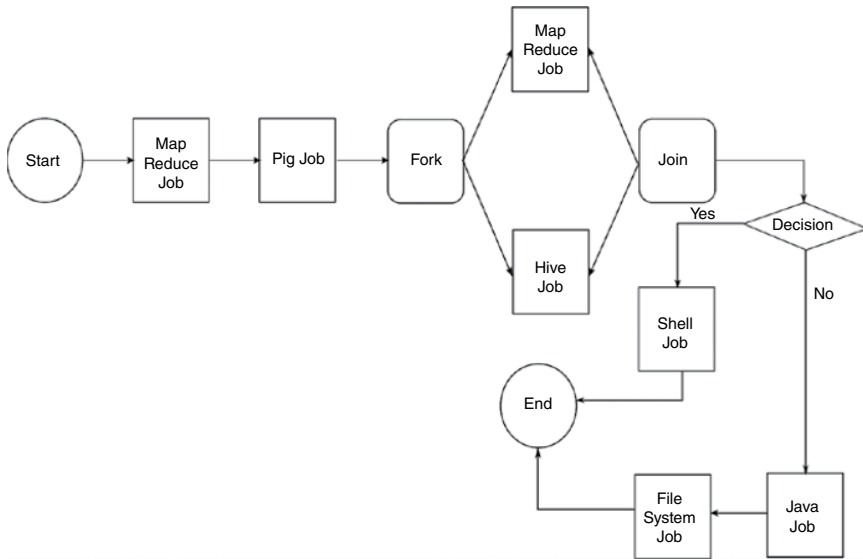


Figure 5.23 Oozie workflow.

control nodes. The decision control node is used to select an execution path within the workflow with the information provided in the job. Figure 5.23 shows an Oozie workflow.

5.12.2 Oozie Coordinators

The Oozie workflow schedules the jobs in a specified sequence. The workflows that have been previously created and stored need to be scheduled, which is done through Oozie coordinators. Oozie coordinators schedule a workflow based on a frequency parameter, that is, jobs are executed at a specific time interval or based on the availability of all the necessary input data. In case of unavailability of input data, the workflow is delayed until all the necessary input data becomes available. Unlike workflow, a coordinator does not have any execution logic, it simply starts and runs a workflow based on the time specified or upon the availability of the input data. An Oozie coordinator is defined with the entities, namely:

- the start and end time;
- Frequency of execution;
- Input data; and
- workflow.

Oozie coordinators are created based on time when jobs have to run daily or weekly to accomplish certain tasks such as generating reports for the organization

periodically. Oozie coordinators created based on time needs three important parameters, namely, the start time, end time, and frequency of execution. Start time specifies the execution of the workflow for the first time, end time specifies the execution of the workflow for the last time, and frequency specifies how often the workflow needs to be executed. When a coordinator is created based on time, it starts and runs automatically until the defined end time is reached; for example, an Oozie coordinator can be created to run a workflow at 8 p.m. every day for seven days starting from November 4, 2016, to November 10, 2016.

An Oozie coordinator created based on the availability of data usually checks the availability of input data for triggering a workflow. The input data may be the output of another workflow or may be passed from an external source. When the input data is available, the workflow is started to process the data to produce the corresponding output data on completion. A data-based coordinator can also be created to run based on the frequency parameter. For example a coordinator set to run at 8 a.m. will trigger the workflow if the data are available at that time. If the data are not available at 8 a.m., the coordinator waits until the data are available, and then it triggers the workflow.

5.12.3 Oozie Bundles

Oozie bundles are a collection of coordinators that specifies the run time of each coordinator. Thus a bundle has one or more coordinators, and a coordinator in turn has one or more workflows. Bundles are specifically useful to group two or more related coordinators where the output of one coordinator becomes the input of another and also useful in an environment where there are hundreds or thousands of workflows scheduled to run on a daily basis.

5.13 Apache Hive

The Hive tool interacts with the Hadoop framework by sending query through an interface such as ODBC or JDBC. The query is sent to a compiler to check syntax. The compiler requests metastore for metadata. The metastore sends metadata in response to the request from compiler.

Hive is a tool to process structured data in the Hadoop environment. It is a platform to develop scripts similar to SQL to perform MapReduce operations. The language for querying is called HQL. The semantics and functions of HQL are similar to SQL. Hive can be run on different computing frameworks. The primitive data types supported by Hive are int., smallint, bigint, float, double, string, Boolean, and decimal, and the complex data types supported by hive are union, struct, array, and map. Hive has a Data Definition Language (DDL) similar to the SQL DDL. DDL is used to create, delete, or alter the schema objects such as tables, partitions, and buckets.

Data in Hive is organized into:

- Tables;
- Partitions; and
- Buckets.

Tables—Tables in Hive are similar to the tables in a relational database. The tables in Hive are associated with the directories in HDFS. Hive tables are referred to as internal tables. Hive also supports external tables where the tables can be created to describe the data that already exists in HDFS.

Partitions—A query in Hive searches the whole Hive table, which slows down the performance in case of large-sized tables. This is resolved by organizing tables into partitions, where the tables are partitioned into related parts that are based on the data of the partitioned columns. When a table is queried, only the required partition in the table is queried so that the performance is greatly improved and the response time is reduced. For example, suppose that a table named EmpTab has employee details such as employee name, employee ID, and year of joining. If the details of the employees who joined in a particular year need to be retrieved, then the whole table has to be scanned for the required information. If the table is partitioned by year, the query processing time will be reduced.

EmpName	Empld	Year of Joining
George	98742	2016
John	98433	2016
Joseph	88765	2015
Mathew	74352	2014
Richard	87927	2015
Williams	76439	2014

The above table can be partitioned as shown below with the year of joining

EmpName	Empld	Year of Joining
George	98742	2016
John	98433	2016

EmpName	Empld	Year of Joining
Joseph	88765	2015
Richard	87927	2015

EmpName	EmpId	Year of Joining
Mathew	74352	2014
Williams	76439	2014

Buckets—Partitions are in turn divided into buckets based on the hash of a column in a table. This is another technique to improve query performance by grouping data sets into more manageable parts.

5.14 Hive Architecture

Figure 5.24 shows the Hive architecture, and it has the following components: *Metastore*—The Hive metastore stores the schema or the metadata of the tables, and the clients are provided access to this data through the metastore API.

Hive Query Language—HQL is similar to SQL in syntax and functions such as loading and querying the tables. HQL is used to query the schema information stored in the metastore. HQL allows users to perform multiple queries on the same data with a single HQL query.

JDBC/ODBC—The Hive tool interacts with the Hadoop framework by sending queries through an interface such as ODBC or JDBC.

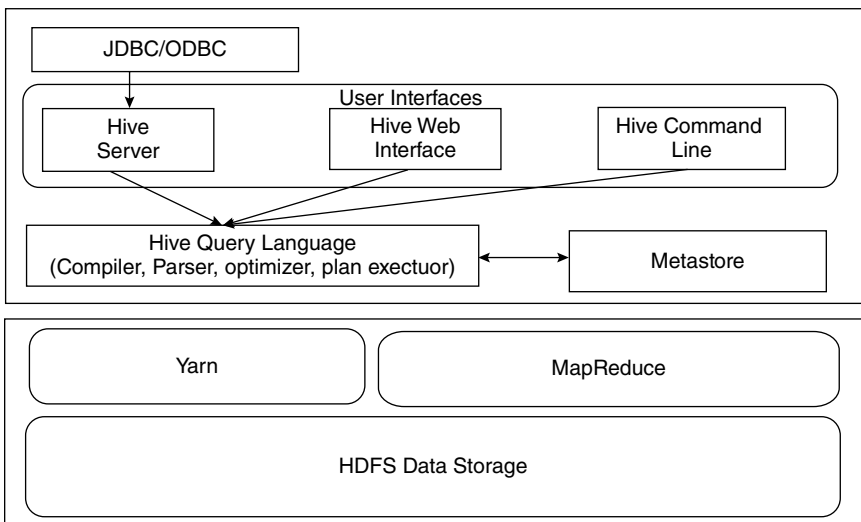


Figure 5.24 Apache Hive architecture.

Compiler—The query is sent to the compiler to check the syntax. The compiler requests metadata from the metastore. The metastore sends metadata in response to the request from the compiler.

Parser—The query is transformed into a parse tree representation with the parser.

Plan executor—Once compiling and parsing is complete, the compiler sends the plan to JDBC/ODBC. The plan is then received by the plan executor, and a MapReduce job is executed. The result is then sent back to the Hive interface.

5.15 Hadoop Distributions

Hadoop has different versions and different distributions available from many companies. Hadoop distributions provide software packages to the users. The different Hadoop distributions available are:

- Cloudera Hadoop distribution (CDH);
- Hortonworks data platform; and
- MapR.

CDH—CDH is the oldest and one of the most popular open-source Hadoop distributions. The primary objective of CHD is to provide support and services to Apache Hadoop software. The Cloudera also comes as a paid distribution with a Cloudera manager, the proprietary maintenance software.

Impala, one of the projects of Cloudera, is an open-source query engine. With Impala, Hadoop queries can be performed in real time and access the data that are stored in HDFS or other databases such as HBase. In contrast to Hive, which is another open-source tool provided by Apache for querying, Impala is a bit faster and eliminates the network bottleneck.

Hortonworks data platform—Hortonworks data platform is another popular open-source, Apache-licensed Hadoop distribution for storing, processing, and analyzing massive data. Hortonworks data platform provides actual Apache released, latest, and stable versions of the components. The components provided by Hortonworks data platform are YARN, HDFS, Pig, HBase, Hive, Zookeeper, SQOOP, Flume, Storm, and Ambari.

MapR—MapR provides a Hadoop-based platform with different versions. M3 is a free version where the features are limited. M5 and M7 are the commercial versions. Unlike Cloudera and Hortonworks, MapR is not an open-source Hadoop distribution. MapR provides enterprise-grade reliability, security, and real-time performance while on the other hand dramatically reduces operational costs. MapR modules include MapR-FS, MapR-DB, and MapR streams and provide high availability, data protection, real-time performance, disaster recovery and global namespace.,

Amazon Elastic MapReduce (Amazon EMR)—Amazon EMR is used to analyze and process massive data by distributing the work across virtual servers in the amazon cloud. Amazon EMR is easy to use, low cost, reliable, secure, and flexible. Amazon EMR finds its application in:

- Clickstream analysis to segment users under different categories to understand the preferences of the users and also advertisers analyze the click stream data to deliver more effective ads to the users;
- Genomics to process large amount of genomic data. Genomics is the study of genes in all living things including humans, animals, and plants; and
- Log processing where large amount of logs generated by web applications are processed.

Chapter 5 Refresher

1 What is the default block size of HDFS?

- A 32 MB
- B 64 MB
- C 128 MB
- D 16 MB

Answer: b

Explanation: The input file is split up into blocks of size 64 Mb by default, and these blocks are then stored in the DataNodes.

2 What is the default replication factor of HDFS?

- A 4
- B 1
- C 3
- D 2

Answer: c

Explanation: The input file is split up into blocks, and each block is mapped to three DataNodes by default to provide reliability and fault tolerance through data replication.

3 Can HDFS data blocks be read in parallel?

- A Yes
- B No

Answer: a

Explanation: HDFS read operations are done in parallel, and write operations are done in pipelined fashion.

- 4 In Hadoop there exists _____.
A one JobTracker per Hadoop job
B one JobTracker per Mapper
C one JobTracker per node
D one JobTracker per cluster

Answer: d

Explanation: Hadoop executes a master/slave architecture where there is one master node and several slave nodes. JobTracker resides in the master node, and TaskTrackers reside in slave nodes one per node.

- 5 Task assigned JobTracker is executed by the _____, which acts as the slave.
A MapReduce
B Mapper
C TaskTracker
D JobTracker

Answer: c

Explanation: JobTracker sends the necessary information for executing a task to the TaskTracker, which executes the task and sends back the results to JobTracker.

- 6 What is the default number of times a Hadoop task can fail before the job is killed?
A 3
B 4
C 5
D 6

Answer: b

Explanation: If a task running on TaskTracker fails, it will be restarted on some other TaskTracker. If the task fails for more than four times, the job will be killed. Four is the default number of times a task can fail, and it can be modified.

- 7 Input key-value pairs are mapped by the _____ into a set of intermediate key-value pairs.
A Mapper
B Reducer
C both Mapper and Reducer
D none of the above

Answer: a

Explanation: Maps are the individual tasks that transform the input records into a set of intermediate records.

- 8 The _____ is a framework-specific entity that negotiates resources from the ResourceManager
- A NodeManager
 - B ResourceManager
 - C ApplicationMaster
 - D all of the above

Answer: c

Explanation: ApplicationMaster has the responsibility of negotiating the resource containers from the ResourceManager.

- 9 Hadoop YARN stands for _____.
- A Yet Another Resource Network
 - B Yet Another Reserve Negotiator
 - C Yet Another Resource Negotiator
 - D all of the mentioned

Answer: c

- 10 _____ is used when the NameNode goes down in Hadoop 1.0.
- A Rack
 - B DataNode
 - C Secondary NameNode
 - D None of the above

Answer: c

Explanation: NameNode is the single point of failure in Hadoop 1.0, and when NameNode goes down, the entire system crashes until a new NameNode is brought into action again.

- 11 _____ is used when the active NameNode goes down in Hadoop 2.0.
- A Standby NameNode
 - B DataNode
 - C Secondary NameNode
 - D None of the above

Answer: a

Explanation: When active NameNode goes down in the Hadoop YARN architecture, the standby NameNode comes into action and takes up the tasks of active NameNode.

Conceptual Short Questions with Answers

1 What is a Hadoop framework?

Apache Hadoop, written in the Java language, is an open-source framework that supports processing of large data sets in streaming access pattern across clusters in a distributed computing environment. It can store a large volume of structured, semi-structured, and unstructured data in a DFS and process them in parallel. It is a highly scalable and cost-effective storage platform.

2 What is fault tolerance?

Fault tolerance is the ability of the system to work without interruption in case of system hardware or software failure. In Hadoop, fault tolerance is the ability of the system to recover the data even if the node where the data is stored fails. This is achieved by data replication where the same data gets replicated across multiple nodes; by default it is three nodes in HDFS.

3 Name the four components that make up the Hadoop framework.

- Hadoop Common: Hadoop common is a collection of common utilities that support other Hadoop modules.
- Hadoop Distributed File System (HDFS): HDFS is a DFS to store large data sets in a distributed cluster and provides high-throughput access to the data across the cluster.
- Hadoop YARN: YARN is the acronym for Yet Another Resource Negotiator and does the job-scheduling and resource-management tasks in the Hadoop cluster.
- Hadoop MapReduce: MapReduce is a framework that performs parallel processing of large unstructured data sets across the clusters.

4 If replication across nodes in HDFS causes data redundancy occupying more memory, then why is it implemented?

HDFS is designed to work on commodity hardware to make it cost effective. The commodity hardware are low-performance machines, which can increase the possibility crashing; thus, to make the system fault tolerant, the data are replicated across three nodes. Hence, if the first node crashes and second node is not available for any reason, the data can be retrieved from the third node, making the system highly fault tolerant.

5 What is a master node and slave node in Hadoop?

Slaves are the Hadoop cluster daemons that are responsible for storing the actual data and the replicated data and processing of the MapReduce jobs. A slave node in Hadoop has the DataNode and TaskTracker. Masters are responsible for monitoring the storage of data across the slaves and the status of the task assigned to slaves. A master node has a NameNode and the JobTracker.

6 What is a NameNode?

NameNode manages the namespace of the entire file system, supervises the health of the DataNode through the Heartbeat signal, and controls the access to the files by the end user. The NameNode does not hold the actual data; it is the directory for DataNode holding the information of which blocks together constitute the file and the location of those blocks. This information is called metadata, which is data about data.

7 Is the NameNode also commodity hardware?

No, the NameNode is the single point of failure, and it cannot be commodity hardware as the entire file system relies on it. NameNode has to be a highly available system.

8 What is MapReduce?

MapReduce is the batch-processing programming model for the Hadoop framework, which adopts a divide-and-conquer principle. It is highly scalable, reliable, and fault tolerant, capable of processing input data with any format in parallel, supporting only batch workloads.

9 What is a DataNode?

A slave node has a DataNode and an associated daemon the TaskTracker. DataNodes are deployed on each slave machine, which provide the actual storage and are responsible for serving read/write requests from clients.

10 What is a JobTracker?

JobTracker is a daemon running on the master that tracks the MapReduce jobs. It assigns the tasks to the different task trackers. A Hadoop cluster has only one JobTracker, and it is the single point of failure. If it goes down, all the running jobs are halted. It receives a Heartbeat from the task tracker based on which JobTracker receives the Heartbeat signal from the TaskTracker, which in turn indicates the health of the JobTracker and the status of the MapReduce jobs.

11 What is a TaskTracker?

TaskTracker is a daemon running on the slave that manages the execution of tasks on slave node. When a job is submitted by a client, the JobTracker will divide and assign the tasks to different TaskTrackers to perform MapReduce tasks. The task tracker will simultaneously communicate with the JobTracker by sending the Heartbeat signal to update the status of the job and to indicate the TaskTracker is alive. If the Heartbeat is not received by the JobTracker for a specified period of time, then the JobTracker assumes that the TaskTracker has crashed.

12 Why is HDFS used for applications with large data sets and not for the applications having large number of small files?

HDFS is suitable for large data sets typically of size 64 MB when compared to a file with large number of small files because NameNode is an expensive, high-performance system; hence, the space cannot be filled with a large volume of metadata generated from large number of small files. So when the file size is large, the metadata will be occupying less space in the NameNode for a single file. Thus, for optimized performance, large data sets are supported by HDFS instead of large number of small files.

13 What is a Heartbeat signal in HDFS?

TaskTracker sends a Heartbeat signal to JobTracker to indicate that the node is alive and additionally the information about the task that it is handling if it is processing a task or its availability to process a task. After a specific time interval, if the Heartbeat signal is not received from TaskTracker, it is assumed dead.

14 What is a secondary NameNode? Is the secondary NameNode a substitute for NameNode?

The secondary NameNode periodically backs up all the data that reside in the RAM of the NameNode. The secondary NameNode does not act as the NameNode if it fails; rather, it acts as a recovery mechanism in case of its failure. The secondary NameNode runs on a separate machine because it requires memory space equivalent to NameNode to back up the data residing in the NameNode.

15 What is a rack?

The rack is a storage area where multiple DataNodes are put together. These DataNodes can be located at different places. Rack is a collection of DataNodes that are stored at a single location.

16 What is a combiner?

The combiner is essentially the reducer of the map job and logically groups the output of the mapper function, which is multiple key-value pairs. In combiner the keys that are repeated are combined, and the values corresponding to the key are listed. Instead of passing the output of the mapper directly to the reducer, it is first sent to the combiner and then to the reducer to optimize the MapReduce job.

17 If a file size is 500 MB, block size is 64MB, and the replication factor is 1, what is the total number of blocks it occupies?

$$\begin{aligned}\text{No of blocks} &= 500/64 * 1 \\ &= 7.8125\end{aligned}$$

So, the number of blocks it occupies is 8

- 18 If a file size is 800 MB, block size is 128 MB, and the replication factor is 3, what is the total number of blocks it occupies? What is the size of each block?**

$$\begin{aligned}
 \text{Total number of blocks} &= 800/128 \\
 &= 6.25 \\
 \text{So, the total number of blocks} &= 7 \\
 \text{Size of 6 blocks} &= 128 \\
 \text{Size of 7th block} &= 800 - (128 * 6) \\
 &= 32
 \end{aligned}$$

Frequently Asked Interview Questions

- 1 In Hadoop why reading process is performed in parallel and writing is not performed in parallel?**

In Hadoop MapReduce, a file is read in parallel for faster data access. But writing operation is not performed in parallel since it will result in data inconsistency. For example, when two nodes are writing data into a file in parallel, then neither of the nodes may be aware of what the other node has written into the file, which results in data inconsistency.

- 2 What is replication factor?**

Replication factor is the number of times a data block is stored in the Hadoop cluster. The default replication factor is 3. This means that three times the storage needed to store the actual data is required.

- 3 Since the data is replicated on three nodes, will the calculations be performed on all the three nodes?**

On execution of MapReduce programs, calculations will be performed only on the original data. If the node on which the calculations are performed fails, then the required calculations will be performed on the second replica.

- 4 How can a running job be stopped in Hadoop?**

The jobid will be killed to stop a running Hadoop job.

- 5 What if all the DataNode of all the three replications fail?**

If DataNodes of all the replications fail, then the data cannot be recovered. If the job is of high priority, then the data can be replicated more than three times by changing the replication factor value, which is 3 by default.

6 What is the difference between input split and HDFS block?

Input split is the logical division of data, and HDFS block is the physical division of data.

7 Is Hadoop suitable for handling streaming data?

Yes, Hadoop handles streaming data with technologies such as Apache flume and Apache Spark.

8 Why are the data replications performed in different racks?

The first replication of a data is placed in a rack, and replications 2 and 3 are placed in the same rack other than the rack where the first replication is placed. This is to overcome rack failure.

9 What are the write types in HDFS? And what is the difference between them?

There are two types of writes in HDFS, namely, posted and non-posted. A posted write does not require acknowledgement, whereas in case of a non-posted write, acknowledgement is required.

10 What happens when a JobTracker goes down in Hadoop 1.0?

When a JobTracker fails, all the jobs in the JobTracker will be restarted, interrupting the overall execution.

11 What is a storage node and compute node?

The storage node is the computer or the machine where the actual data resides, and the compute node is the machine where the business logic is executed.

12 What happens when 100 tasks are spawned for a job and one task fails?

If a task running on TaskTracker fails, it will be restarted on some other TaskTracker. If the task fails for more than four times, the job will be killed. Four is the default number of times a task can fail, but it can be modified.

6

Big Data Analytics

CHAPTER OBJECTIVE

This chapter begins to reap the benefits of the big data era. Anticipating the best time of price fall to make purchases or going in line with current trends by catching up with social media is all possible with big data analysis. A deep insight is given on the various methods with which this massive flood of data can be analyzed, the entire life cycle of big data analysis, and various practical applications of capturing, processing, and analyzing this huge data.

Analyzing the data is always beneficial and the greatest challenge for the organizations. This chapter examines the existing approaches to analyze the stored data to assist organizations in making big business decisions to improve business performance and efficiency, to compete with their business rivals and find new approaches to grow their business. It delivers insight to the different types of data analysis techniques (descriptive analysis, diagnostic analysis, predictive analysis, prescriptive analysis) used to analyze big data. The data analytics life cycle starting from data identification to utilization of data analysis results are explained. It unfolds the techniques used in big data analysis, that is, quantitative analysis, qualitative analysis, and various types of statistical analysis such as A/B testing, correlation, and regression. Earlier the analysis on big data was made by querying this huge data set, and analysis were done in batch mode. Today's trend has made big data analysis possible in real time, and all the tools and technologies that made this possible are all well explained in this chapter.

6.1 Terminology of Big Data Analytics

6.1.1 Data Warehouse

Data warehouse, also termed as Enterprise Data Warehouse (EDW), is a repository for the data that various organizations and business enterprises collect. It gathers the data from diverse sources to make the data available for unified access and analysis by the data analysts.

Big Data: Concepts, Technology, and Architecture, First Edition. Balamurugan Balusamy, Nandhini Abirami. R, Seifedine Kadry, and Amir H. Gandomi.

© 2021 John Wiley & Sons, Inc. Published 2021 by John Wiley & Sons, Inc.

6.1.2 Business Intelligence

Business intelligence (BI) is the process of analyzing the data producing a desirable output to the organizations and end users to make decisions. The benefit of big data analytics is to increase revenue, increase efficiency and performance, and to compete with the rivals of the business by identifying the market trends. BI data comprises both data from the storage (data that are captured and stored previously) and data that are streaming, supporting the organizations to make strategic decisions.

6.1.3 Analytics

Data analytics is the process of analyzing the raw data by the data scientists to make business decisions. Business intelligence is more focused. The way of focus brings out the difference between data analytics and business Intelligence. Both are used to meet the challenges in the business and pave way for new business opportunities.

6.2 Big Data Analytics

Big data analytics is the science of examining or analyzing large data sets with a variety of data types, that is, structured, semi-structured, or unstructured data, which may be streaming or batch data. Big data analytics allows to make better decisions, find new business opportunities, compete against business rivals, improve performance and efficiency, and reduce cost by using advanced data analytics techniques.

Big data, the data-intensive technology, is the booming technology in science and business. Big data plays a crucial role in every facet of human activities empowered by the technological revolution.

Big data technology assists in:

- Tracking the link clicked on a website by the consumer (which is being tracked by many online retailers to perceive the interests of consumers to take their business enterprises to a different altitude);
- Monitoring the activities of a patient;
- Providing enhanced insight; and
- Process control and business solutions to large enterprises manifesting its ubiquitous nature.

Big data technologies are targeted in processing high-volume, high-variety, and high-velocity data sets to extricate the required data value. The role of researchers

in the current scenario is to perceive the essential attributes of big data, the feasibility of technological development with big data, and spot out the security and privacy issues with big data. Based on a comprehensive understanding of big data, researchers propose the big data architecture and present the solutions to existing issues and challenges.

The advancement in the emerging big data technology is tightly coupled with the data revolution in social media, which urged the evolution of analytical tools with high performance and scalability and global infrastructure.

Big data analytics is focused on extracting meaningful information using efficient algorithms on the captured data to process, analyze, and visualize the data. This comprises framing the effective algorithm and efficient system to integrate data, analyzing the knowledge thus produced to make business solutions. For instance, in online retailing analyzing the enormous data generated from online transactions is the key to enhance the perception of the merchants into customer behavior and purchasing patterns to make business decisions. Similarly in Facebook pages advertisements appear by analyzing Facebook posts, pictures, and so forth. When using credit cards the credit card providers use a fraud detection check to confirm that the transaction is legitimate. Customers credit scoring is analyzed by financial institutions to predict whether the applicant will default on a loan. To summarize, the impact and importance of analytics have reached a great height with more data being collected. Analytics will still continue to grow until there is a strategic impact in perceiving the hidden knowledge from the data.

The applications of analytics in various sectors involve:

- Marketing (response modeling, retention modeling);
- Risk management (credit risk, operational risk, fraud detection);
- Government sector (money laundering, terrorism detection);
- Web (social media analytics) and more.

Figure 6.1 shows the types of analytics. The four types of analytics are:

- 1) Descriptive Analytics—Insight into the past;
- 2) Diagnostic Analytics—Understanding what is happening and why did it happen;
- 3) Predictive Analytics—Understanding the future; and
- 4) Prescriptive Analytics—Advice on possible outcomes.

6.2.1 Descriptive Analytics

Descriptive analytics describe, summarize, and visualize massive amounts of raw data into a form that is interpretable by end users. It describes the events that occurred at any point in past and provides insight into what actually has happened in the past. In descriptive analysis, past data are mined to understand the

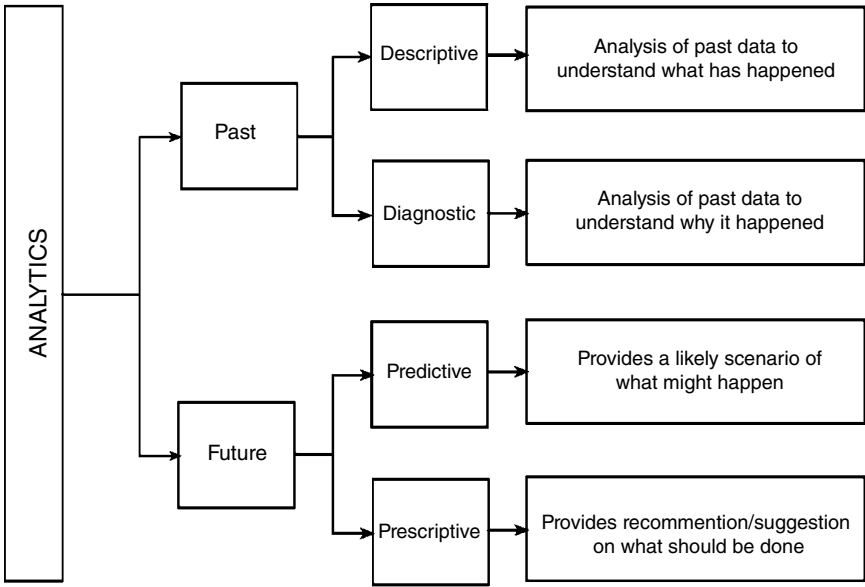


Figure 6.1 Data analytics.

reason behind the failure or success. It allows users to learn from past performance or behavior and interpret how they could influence future outcomes. Any kind of historical data can be analyzed to predict future outcome; for example, past usage of electricity can be analyzed to generate power and set the optimal charge per unit for electricity. Also they can be used to categorize consumers based on their purchasing behavior and product preferences. Descriptive analysis finds its application in sales, marketing, finance, and more.

6.2.2 Diagnostic Analytics

Diagnostic analytics is a form of analytics that enables the users to understand what is happening and why did it happen so that a corrective action can be taken if something went wrong. It benefits the decision-makers of the organizations by giving them actionable insights. It is a type of root-cause analysis, investigative, and detective, which determines the factors that contributed to a certain outcome. Diagnostic analytics is performed using data mining and drill down techniques. The analysis is used to analyze social media, web data, or click-stream data to find a hidden pattern and consumer data. It provides insights into the behavior of profitable as well as non-profitable customers.

6.2.3 Predictive Analytics

Predictive analytics provides valuable and actionable insights to companies based on the data by predicting what might happen in the future. It analyzes the data to determine possible future outcomes. Predictive analytics uses many statistical techniques such as machine learning, modeling, artificial intelligence, and data mining to make predictions. It exploits patterns from historical data to determine risks and opportunities. When applied successfully, predictive analytics allows the business to efficiently interpret big data and derive business value from IT assets. Predictive analytics is applied in health care, customer relationship management, cross-selling, fraud detection, and risk management. For example, it is used to optimize customer relationship management by analyzing customer data and thereby predicting customer behavior. Also, in an organization that offers multiple products to consumers, predictive analytics is used to analyze customer interest, spending patterns, and other behavior through which the organization can effectively cross-sell their products or sell more products to current customers.

6.2.4 Prescriptive Analytics

Prescriptive analytics provides decision support to benefit from the outcome of the analysis. Thus, prescriptive analytics goes beyond just analyzing the data and predicting future outcomes by providing suggestions to extract the benefits and take advantage of the predictions. It provides the organizations with the best option when dealing with a business situation by optimizing the process of decision-making in choosing between the options that are available. It optimizes business outcomes by combining mathematical models, machine learning algorithms, and historical data. It anticipates what will happen in the future, when will it happen, and why it will happen. Prescriptive analytics are implemented using two primary approaches, namely, simulation and optimization. Predictive analytics as well as prescriptive analytics provide proactive optimization of the best action for the future based on the analysis of a variety of past scenarios. The actual difference lies in the fact that predictive analytics helps the users to model future events, whereas prescriptive analytics guide users on how different actions will affect business and suggest them the optimal choice. Prescriptive analytics finds its applications in pricing, production planning, marketing, financial planning, and supply chain optimization. For example, airline pricing systems use prescriptive analytics to analyze purchase timing, demand level, and other travel factors to present the customers with a pricing list to optimize profit but not losing the customers and deter sales.

Figure 6.2 shows data analytics where customer behavior is analyzed using the four techniques of analysis. Initially with descriptive analytics customer behavior

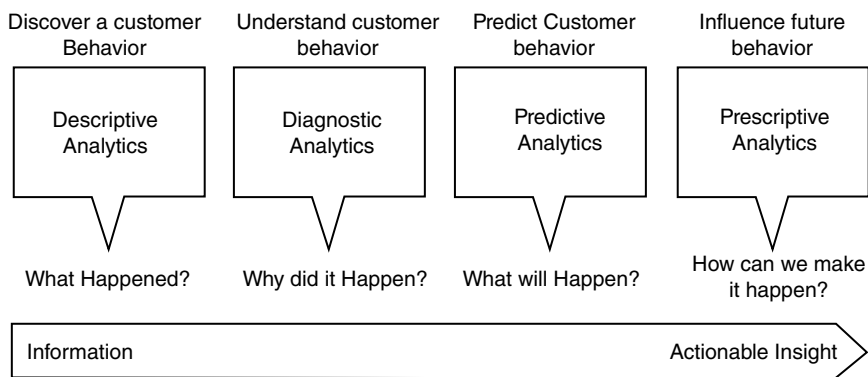


Figure 6.2 Analyzing a customer behavior.

is analyzed with past data. Diagnostic analytics is used to analyze and understand customer behavior while predictive analytics is used to predict customer future behavior, and prescriptive analytics is used to influence this future behavior.

6.3 Data Analytics Life Cycle

The first step in data analytics is to define the business problem that has to be solved with data analytics. The next step in the process is to identify the source data necessary to solve the issue. This is a crucial step as the data is the key to any analytical process. Then the selection of data is performed. Data selection is the most time-consuming step. All the data will then be gathered in a data mart. The data from the data mart will be cleansed to remove the duplicates and inconsistencies. This will be followed by a data transformation, which is transforming the data to the required format, such as converting the data from alphanumeric to numeric. Next is the analytics on the preprocessed data, which may be fraud detection, churn prediction, and so forth. After this the model can be used for analytics applications such as decision-making. This analytical process is iterative, which means data scientists may have to go to previous stages or steps to gather additional data. Figure 6.3 shows various stages of the data analytics life cycle.

6.3.1 Business Case Evaluation and Identification of the Source Data

The big data analytics process begins with the evaluation of the business case to have a clear picture of the goals of the analysis. This assists data scientists to interpret the resources required to arrive to the analysis objective and help them

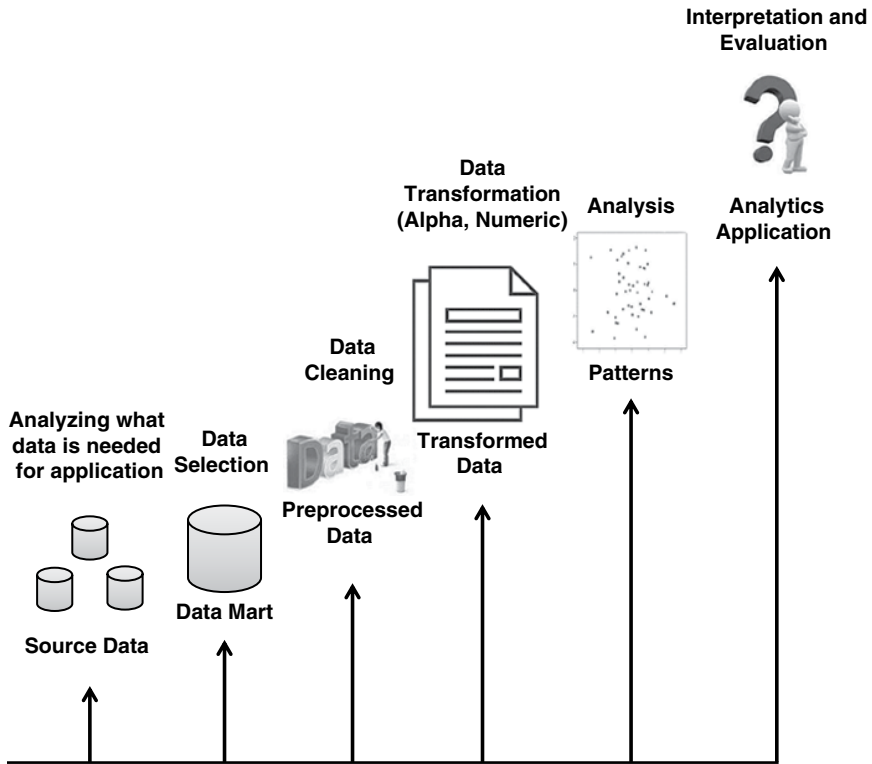


Figure 6.3 Analytics life cycle.

perceive if the issue in hand really pertains to big data. For a problem to be classified as a big data problem, it needs to be associated with one or more of the characteristics of big data, that is, volume, variety, and velocity. The data scientists need to assess the source data available to carry out the analysis in hand. The data set may be accessible internally to the organization or it may be available externally with third-party data providers. It is to be determined if the data available is adequate to achieve the target analysis. If the data available is not adequate, either additional data have to be collected or available data have to be transformed. If the data available is still not sufficient to achieve the target, the scope of the analysis is constrained to work within the limits of the data available. The underlying budget, availability of domain experts, tools, and technology needed and the level of analytical and technological support available within the organization is to be evaluated. It is important to weigh the estimated budget against the benefits of obtaining the desired objective. In addition the time required to complete the project is also to be evaluated.

6.3.2 Data Preparation

The required data could possibly be spread across disparate data sets that have to be consolidated via fields that exist in common between the data sets. Performing this integration might be complicated because of the difference in their data structure and semantics. Semantics is the same value having different labels in different datasets, such as DOB and date of birth. Figure 6.4 illustrates a simple data integration using the EmpId field.

The data gathered from various sources may be erroneous, corrupt, and inconsistent and thus have no significant value to the analysis problem in hand. Thereby the data have to be preprocessed before using it for analysis to make the analysis effective and meaningful and to gain the required insight from the business data. Data that may be considered as unimportant for one analysis could be important for a different type of problem analysis, so a copy of the original data set, be it an internal data set or a data set external to the organization, has to be persisted before filtering the data set. In case of batch analysis, data have to be preserved before analysis and in case of real-time analysis, data have to be preserved after the analysis.

Unlike a traditional database, where the data is structured and validated, the source data for big data solutions may be unstructured, invalid, and complex in nature, which further complicates the analysis. The data have to be cleansed to validate it and to remove redundancy. In case of a batch system, the cleansing can be handled by a traditional ETL (Extract, Transform and Load) operation. In case of real-time analysis, the data must be validated and cleansed through complex in-memory database systems. In-memory data storage systems load the data in main memory, which bypasses the data being written to and read from a disk to lower the CPU requirement and to improve the performance.

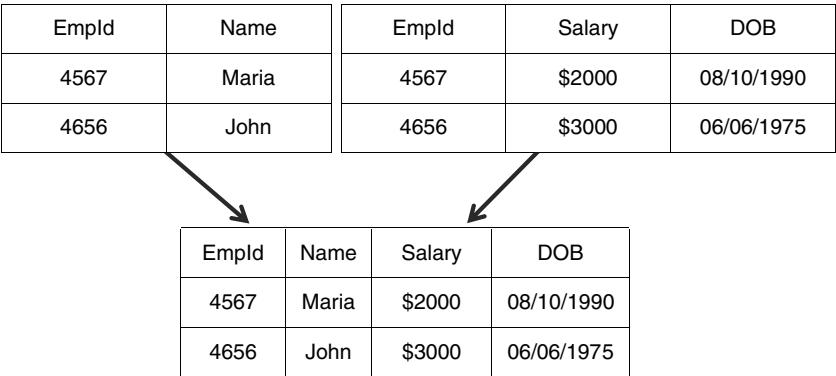


Figure 6.4 Data integration with EmpId field.

6.3.3 Data Extraction and Transformation

The data arriving from disparate sources may be in a format that is incompatible for big data analysis. Hence, the data must be extracted and transformed into a format acceptable by the big data solution and can be utilized for acquiring the desired insight from the data. In some cases, extraction and transformation may not be necessary if the big data solution can directly process the source data, while some cases may demand extraction wherein transformation may not be necessary. Figure 6.5 illustrates the extraction of Computer Name and User Id from the XML file, which does not require any transformation.

6.3.4 Data Analysis and Visualization

Data analysis is the phase where actual analysis on the data set is carried out. The analysis could be iterative in nature, and the task may be repeated until the desired insight is discovered from the data. The analysis could be simple or complex depending on the target to be achieved.

Data analysis falls into two categories, namely, confirmatory analysis and exploratory analysis. Confirmatory data analysis is deductive in nature wherein the data analysts will have the proposed outcome called hypothesis in hand and the evidence must be evaluated against the facts. Exploratory data analysis is inductive in nature where the data scientists do not have any hypotheses or assumptions; rather, the data set is explored and iterated until an appropriate pattern or result is achieved.

Data visualization is a process that makes the analyzed data results to be visually presented to the business users for effective interpretation. Without data visualization tools and techniques, the entire analysis life cycle carries only a meager value as the analysis results could only be interpreted by the analysts. Organizations

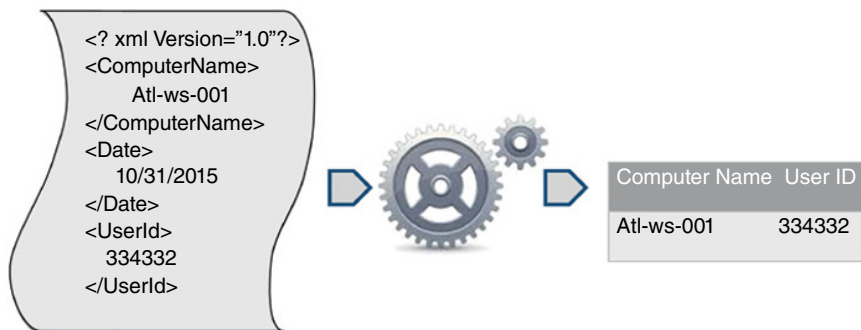


Figure 6.5 Illustration of extraction without transformation.

should be able to interpret the analysis results to obtain value from the entire analysis process and to perform visual analysis and derive valuable business insights from the massive data.

6.3.5 Analytics Application

The analysis results can be used to enhance the business process and increase business profits by evolving a new business strategy. For example, a customer analysis result when fed into an online retail store may deliver the recommendations list that the consumer may be interested in purchasing, thus making the online shopping customer friendly and revamping the business as well.

6.4 Big Data Analytics Techniques

Various analytics techniques involved in big data are:

- Quantitative analysis;
- Qualitative analysis; and
- Statistical analysis.

6.4.1 Quantitative Analysis

Quantitative data is the data based on numbers. Quantitative analysis in big data is the analysis of quantitative data. The main purpose of this type of statistical analysis is quantification. Results from a sample population can be generalized over the entire population under study. Different types of quantitative data on which quantitative analysis is performed are:

- **Nominal data**—It is a type of categorical data where the data is described based on categories. This type of data does not have any numerical significance. Arithmetic operations cannot be performed on this type of data. Examples are: gender (male, female) and height (tall, short).
- **Ordinal data**—The order or the ranking of the data is what matters in ordinal data, rather than the difference between the data. Arithmetic operators $>$ and $<$ are used. For example, when a person is asked to express his happiness in the scale of 1–10, a score of 8 means the person is happier than a score of 5, which is more than a score of 3. These values simply express the order of happiness. Other examples are the ratings that range from one star to five stars, which are used in several applications such as movie rating, current consumption of an electronic device, and performance of android application.

- Interval data—In case of interval data, not only the order of the data matters, but the difference between them also matters. One of the common examples of ordinal data is the difference in temperature in Celsius. The difference between 50°C and 60°C is the same as the difference between 70°C and 80°C. In time scale the increments are consistent and measurable.
- Ratio data—A ratio variable is essentially an interval data with the additional property that the values can have absolute zero. Zero value in ratio indicates that the variable does not exist. Height, weight, and age are examples of ratio data. For example 40 of 10 years. Whereas those data such as temperature are ratio variables since 0°C does not mean that the temperature does not exist.

6.4.2 Qualitative Analysis

Qualitative analysis in big data is the analysis of data in their natural settings. Qualitative data are those that cannot be easily reduced to numbers. Stories, articles, survey comments, transcriptions, conversations, music, graphics, art, and pictures are all qualitative data. Qualitative analysis basically answers to “how,” “why,” and “what” questions. There are basically two approaches in qualitative data analysis, namely, the deductive approach and the inductive approach. A deductive analysis is performed by using the research questions to group the data under study and then look for similarities or differences in them. An inductive approach is performed by using the emergent framework of the research to group the data and then look for the relationships in them.

A qualitative analysis has the following basic types:

- 1) Content analysis—Content analysis is used for the purpose of classification, tabulation, and summarization. Content analysis can be descriptive (what is actually the data?) or interpretive (what does the data mean?).
- 2) Narrative analysis—Narrative analyses are used to transcribe the observation or interview data. The data must be enhanced and presented to the reader in a revised shape. Thus, the core activity of a narrative analysis is reformulating the data presented by people in different contexts based on their experiences.
- 3) Discourse analysis—Discourse analysis is used in analyzing data such as written text or a naturally occurring conversation. The analysis focuses mainly on how people use languages to express themselves verbally. Some people speak in a simple and straightforward way while some other people speak in a vague and indirect way.
- 4) Framework analysis—Framework analysis is used in identifying the initial framework, which is developed from the problem in hand.
- 5) Grounded theory—Grounded theory basically starts with examining one particular case from the population and formulating a general theory about the entire population.

6.4.3 Statistical Analysis

Statistical analysis uses statistical methods for analyzing data. The statistical analysis techniques described are:

- A/B testing;
- Correlation; and
- Regression.

6.4.3.1 A/B Testing

A/B testing, also called split testing or bucket testing, is a method that compares two versions of an object under interest to determine which among the two versions performs better. The element subjected to analysis may be a web page or online deals on products. The two versions are version A, which is the current version and is called control version, and the modified version, version B, is called as the treatment. Both version A and version B are tested simultaneously, and the results are analyzed to determine the successful version. For example, two different versions of a web page to visitors with similar interests. The successful version is the one that has higher conversion rates. When an e-commerce website versions are compared, a version with more of buyers will be considered successful. Similarly, new websites that win a larger number of paid subscriptions is considered the successful version. Anything on the website such as a headline, an image, links, paragraph text, and so forth, can be tested.

6.4.3.2 Correlation

Correlation is a method used to determine if there exists a relationship between two variables, that is, to determine whether they are correlated. If they are correlated, the type of correlation between the variables is determined. The type of correlation is determined by monitoring the second variable when the first variable increases or decreases. It is categorized into three types:

- Positive correlation—When one variable increases, the other variable increases. Figure 6.6a shows positive correlation. Examples of positive correlation are:
 - 1) The production of cold beverages and ice cream increases with the increase in temperature.
 - 2) The more a person exercises, the more the calories burnt.
 - 3) With the increased consumption of food, the weight gain of a person increases.
- Negative correlation—When one variable increases, the other variable decreases. Figure 6.6b shows negative correlation.

Examples of negative correlation are:

- 1) As weather gets colder, the cost of air conditioning decreases.
- 2) The working capability decreases with the increase in age.

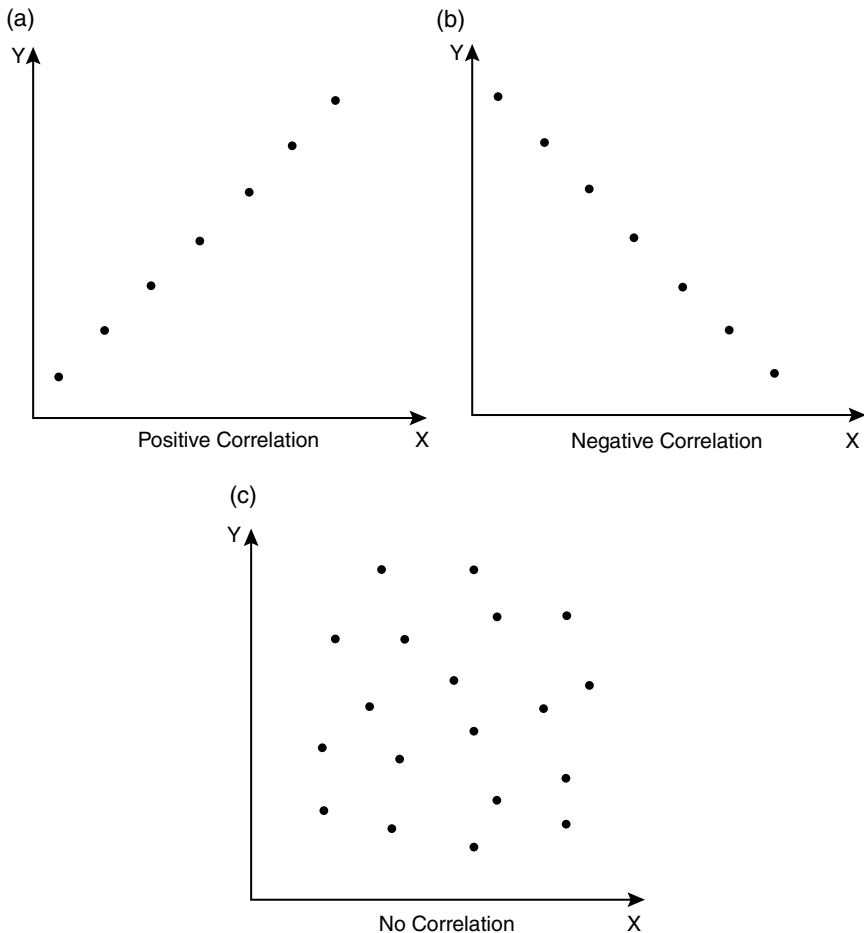


Figure 6.6 (a) Positive correlation. (b) negative correlation. (c) No correlation.

3) With the increase in the speed of the car, time taken to travel decreases.

- No correlation—When one variable increases, the other variable does not change. Figure 6.6c shows no correlation. An example of no correlation between two variables is:

1) There is no correlation between eating Cheetos and speaking better English.

With the scatterplots given above, it is easy to determine whether the variables are correlated. However, to quantify the correlation between two variables, Pearson's correlation coefficient r is used. This technique used to calculate the

correlation coefficient is called Pearson product moment correlation. The formula to calculate the correlation coefficient is

$$\text{Correlation coefficient, } r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

To compute the value of r , the mean is subtracted from each observation for the x and y variables.

The value of the correlation coefficient ranges between -1 to $+1$. A value $+1$ or -1 for the correlation coefficient indicates perfect correlation. If the value of the correlation coefficient is less than zero, it essentially means that there is a negative correlation between the variables, and the increase of one variable will lead to the decrease of the other variable. If the value of the correlation coefficient is greater than zero, it means that there is a positive correlation between the variables, and the increase of one variable leads to the increase of the other variable. The higher the value of the correlation coefficient, the stronger the relationship, be it a positive or negative correlation, and the value closer to zero depicts a weak relationship between the variables. If the value of the correlation coefficient is zero, it means that there is no relationship between the variables. If the value of the correlation coefficient is close to $+1$, it indicates high positive correlation. If the value of the correlation coefficient is close to -1 , it indicates high negative correlation.

The Pearson product moment correlation is the most widely adopted technique to determine the correlation coefficient. Other techniques used to calculate the correlation coefficient are Spearman rank order correlation, PHI correlation, and point biserial.

6.4.3.3 Regression

Regression is a technique that is used to determine the relationship between a dependent variable and an independent variable. The dependent variable is the outcome variable or the response variable or predicted variable, denoted by “ Y ,” and the independent variable is the predictor or the explanatory or the carrier variable or input variable, denoted by “ X .” The regression technique is used when a relationship exists between the variables. The relationship can be determined with the scatterplots. The relationship can be modeled by fitting the data points on a linear equation. The linear equation is

$$Y = a + bX,$$

where,

X = independent variable,
 Y = dependent variable,
 a = intercept, the value of Y when $X = 0$, and
 b = slope of the line.

The major difference between regression and correlation is that correlation does not imply causation. A change in a variable does not cause the change in another variable even if there is a strong correlation between the two variables. While regression, on the other hand, implies a degree of causation between the dependent and the independent variable. Thus correlation can be used to determine if there is a relationship between two variables and if a relationship exists between the variables, regression can be used further to explore and determine the value of the dependent variable based on the independent variable whose value is previously known.

In order to determine the extra stock of ice creams required, the analysts feed the value of temperature recorded based on the weather forecast. Here, the temperature is treated as independent variable and the ice cream stock is treated as the dependent variable. Analysts frame a percentage of increase in stock for a specific decrease in temperature. For example, 10% of the total stock may be required to be increased for every 5°C decrease in temperature. The regression may be linear or nonlinear.

Figure 6.7a shows a linear regression. When there is a constant rate of change, then it is called linear regression.

Figure 6.7b shows nonlinear regression. When there is a variable rate of change, then it is called nonlinear regression.

6.5 Semantic Analysis

Semantic analysis is the science of extracting meaningful information from speech and textual data. For the machines to extract meaningful information from the data, the machines should interpret the data as humans do.

Types of semantics analysis:

- 1) Natural Language Processing (NLP)
- 2) Text analytics
- 3) Sentiment analysis

6.5.1 Natural Language Processing

NLP is a field of artificial intelligence that helps the computers understand human speech and text as understood by humans. NLP is needed when an intelligent system is required to perform according to the instructions provided. Intelligent

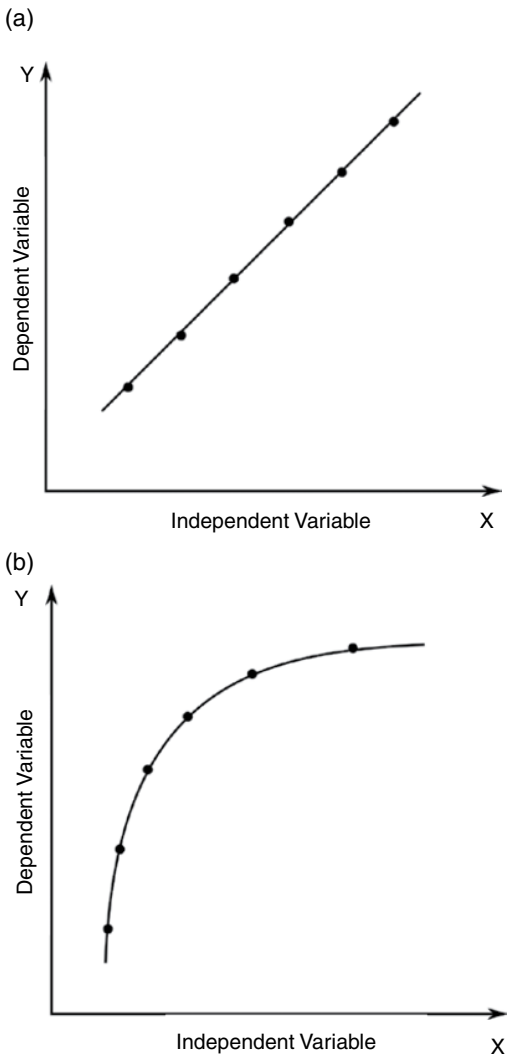


Figure 6.7 (a) Linear regression. (b) Nonlinear regression.

systems can be made to perform useful tasks by interpreting the natural language that humans use. The input to the system can be either speech or written text. There are two components in NLP, namely, Natural Language Understanding (NLU) and Natural Language Generation (NLG).

NLP is performed in different stages, namely, lexical analysis, syntactic analysis, semantic analysis, and pragmatic analysis.

Lexical analysis involves dividing the whole input text data into paragraphs, sentences, and words. It then identifies and analyzes the structure of words.

Syntactic analysis involves analyzing the input data for grammar and arranging the words in the data in a manner that makes sense.

Semantic analysis involves checking the input text or speech for meaningfulness by extracting the dictionary meaning for the input or interpreting the actual meaning from the context. For instance, colorless red glass. This is a meaningless sentence, which would be rejected as colorless red does not make any sense.

Pragmatic analysis involves the analysis of what is intended to be spoken by the speaker. It basically focuses on the underlying meaning of the words spoken by the speaker to interpret what was actually meant.

6.5.2 Text Analytics

Text analytics is the process of transforming the unstructured data into meaningful data by applying machine learning, text mining, and NLP techniques. Text mining is the process of discovering patterns in massive text collection. The steps involved in text analysis are:

- Parsing;
- Searching and retrieval; and
- Text mining.

Parsing—Parsing is the process that transforms unstructured text data into structured data for further analysis. The unstructured text data could be a weblog, a plain text file, an HTML file, or a Word document.

Searching and retrieval—It is the process of identifying the document that contains the search item. The search item may be a word or a phrase or a topic, which are generally called key term.

Text mining—Text mining uses the key terms to derive meaningful insights corresponding to the problem in hand.

6.5.3 Sentiment Analysis

Sentiment Analysis is analyzing a piece of writing and determining whether it is positive, negative, or neutral. Sentiment analysis is also known as opinion mining as it is the process of determining the opinion or attitude of the writer. A common application of sentiment analysis is to determine what people feel about a particular item or incident or a situation. For example, if the analyst wants to know about how people think about the taste of pizza in Papa John's, Twitter sentiment analysis will answer this question. The analyst can even learn why people think that the taste of pizza is good or bad, by extracting the words that indicate why people liked or disliked the taste.

6.6 Visual analysis

Visual analysis is the process of analyzing the results of data analysis integrated with data visualization techniques to understand the complex system in a better way. Various data visualization techniques are explained in Chapter 10. Figure 6.6 shows the data analysis cycle.

6.7 Big Data Business Intelligence

Business intelligence (BI) is the process of analyzing the data and producing a desirable output to the organizations and end users to assist them in decision-making. The benefit of big data analytics is to increase revenue, increase efficiency and performance, and outcompete business rivals by identifying market trends. BI data comprises both data from the storage (previously captured and stored data) and data that are streaming, supporting the organizations to make strategic decisions.

6.7.1 Online Transaction Processing (OLTP)

Online transaction processing (OLTP) is used to process and manage transaction-oriented applications. The applications are processed in real time and not in batch; hence the name OLTP. They are used in transactions where the system is required to respond immediately to the end-user requests. As an example, OLTP technology is used in commercial transaction processing application such as automated teller machines (ATM). OLTP applications are used to retrieve a group of records and provide them to the end users; for example, a list of computer hardware items sold at a store on a particular day. OLTP is used in airlines, banking, and supermarkets for many applications, which include e-banking, e-commerce, e-trading, payroll registration, point-of-sale system, ticket reservation system, and accounting. A single OLTP system can support thousands of users, and the transactions can be simple or complex. Typical OLTP transactions take few seconds to complete rather than minutes. The main features of OLTP systems are data integrity maintained in multi-access environment and fast query processing and effectiveness in handling transactions per second.

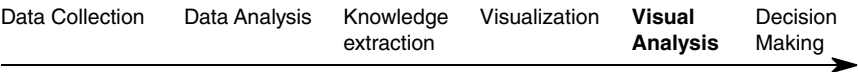


Figure 6.8 Data analysis cycle.

The term “transaction processing” is associated with a process in which an online retail store or e-commerce website processes the payment of a customer in real time for the goods and services purchased. During the OLP the payment system of the merchant will automatically connect to the bank of the customer after which fraud check and other validity checks are performed and the transaction will be authorized if the transaction is found to be legitimate.

6.7.2 Online Analytical Processing (OLAP)

Online analytical processing (OLAP) systems are used to process data analysis queries and perform effective analysis on massive amounts of data. Compared to OLTP, OLAP systems handle relatively smaller numbers of transactions. In other words, OLAP technologies are used for collecting, processing, and presenting the business users with multidimensional data for analysis. Different types of OLAP systems are Multidimensional Online Analytical Processing (MOLAP), Relational Online Analytical Processing (ROLAP), and the combination of MOLAP and ROLAP, the Hybrid Online Analytical Processing (HOLAP). They are referred to by a five-key word definition: Fast Analysis of Shared Multidimensional Information (FASMI).

- *Fast* refers to the speed at which the OLAP system delivers responses to the end users, perhaps within seconds.
- *Analysis* refers to the ability of the system to provide rich analytic functionality. The system is expected to answer most of the queries without programming.
- *Shared* refers to the ability of the system to support sharing and at the time should be able to implement the security requirements for maintaining confidentiality and concurrent access management when multiple write-backs are required.
- *Multidimensional* is the basic requirement of the OLAP system, which refers to the ability of the system to provide a multidimensional view of the data. This multidimensional array of data is commonly referred to as a cube.
- *Information* refers to the ability of the system to handle large volumes of data obtained from the data warehouse.

In an OLAP system the end users are presented with the information rather than the data. OLAP technology is used in forecasting and data mining. They are used to predict current trends in sales and predict future prices of commodities.

6.7.3 Real-Time Analytics Platform (RTAP)

Applying analytic techniques to data in motion transforms data into business insights and actionable information. Streaming computing is crucial in big data analytics to perform in-motion analytics on data from multiple sources at unprecedented speeds and volumes. Streaming computing is essential to process the data at varying velocities and volumes, apply appropriate analytic techniques on that data, and produce actionable insights instantly so that appropriate actions may be taken either manually or automatically.

Real-time analytics platform (RTAP) applications can be used to alert the end users when a situation occurs and also provides the users with the options and recommendations to take appropriate actions. Alerts are suitable in applications where the actions are not to be taken automatically by the RTAP system. For example, a patient-monitoring system would alert a doctor or nurse to take a specific action for a situation. RTAP applications can also be used in failure detection when a data source does not generate data within the stipulated time. Failures in remote locations or problems in networks can be detected using RTAP.

6.8 Big Data Real-Time Analytics Processing

The availability of new data sources like video, images, and social media data provides a great opportunity to gain deeper insights on customer interests, products, and so on. The volume and speed of both traditional and new data generated are significantly higher than before. The traditional data sources include the transactional system data that are stored in RDBMS and flat file formats. These are mostly structured data, such as sales transactions and credit card transactions. To exploit the power of analytics fully, any kind of data—be it unstructured or semi-structured—needs to be captured. The new sources of data, namely, social media data, weblogs, machine data, images and videos captured from surveillance camera and smartphones, application data, and data from sensor devices are all mostly unstructured. Organizations capturing these big data from multiple sources can uncover new insights, predict future events and get recommended actions for specific scenarios, and identify and handle financial and operational risks. Figure 6.7 shows the big data analytics processing architecture with traditional and new data sources, their processing, analysis, actionable insights, and their applications.

Shared operational information includes master and reference data, activity hub, content hub, and metadata catalog. Transactional data are those that describe business events such as selling products to customers, buying products from suppliers, and hiring and managing employees. Master data are the important

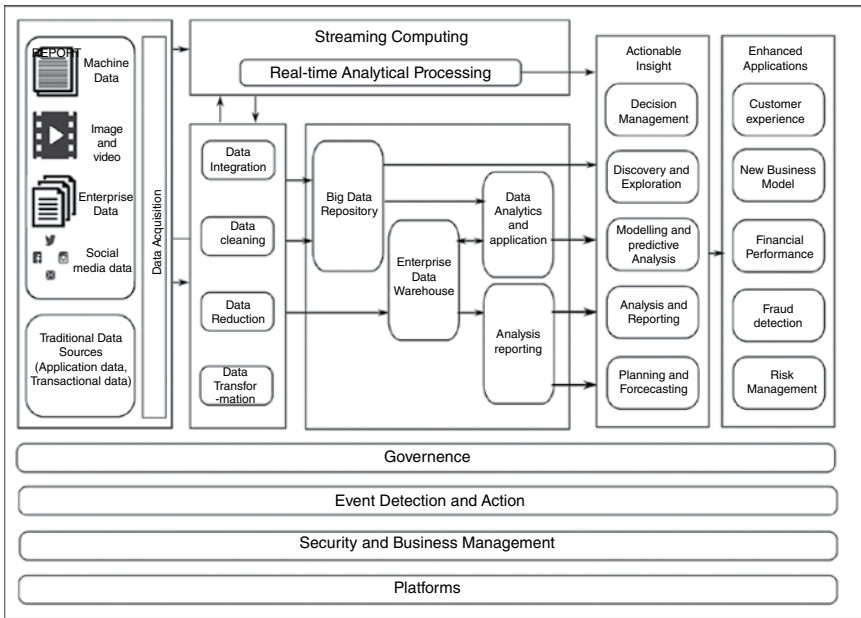


Figure 6.9 Big Data analytics processing.

business information that supports the transaction. Master data are those that describe customers, products, employees, and more involved in the transactions. Reference data are those related to transactions with a set of values, such as the order status of a product, an employee designation, or a product code. Content Hub is a one-stop destination for web users to find social media content or any type of user-generated content in the form of text or multimedia files. Activity hub manages all the information about the recent activity.

6.9 Enterprise Data Warehouse

ETL (Extract, Transform and Load) is used to load data into the data warehouse wherein the data is first transformed before loading, which requires separate expensive hardware. An alternate cost-effective approach is to first load the data into the warehouse and then transform them in the database itself. The Hadoop framework provides a cheap storage and processing platform wherein the raw data can be directly dumped into HDFS, and then transformation techniques are applied on the data.

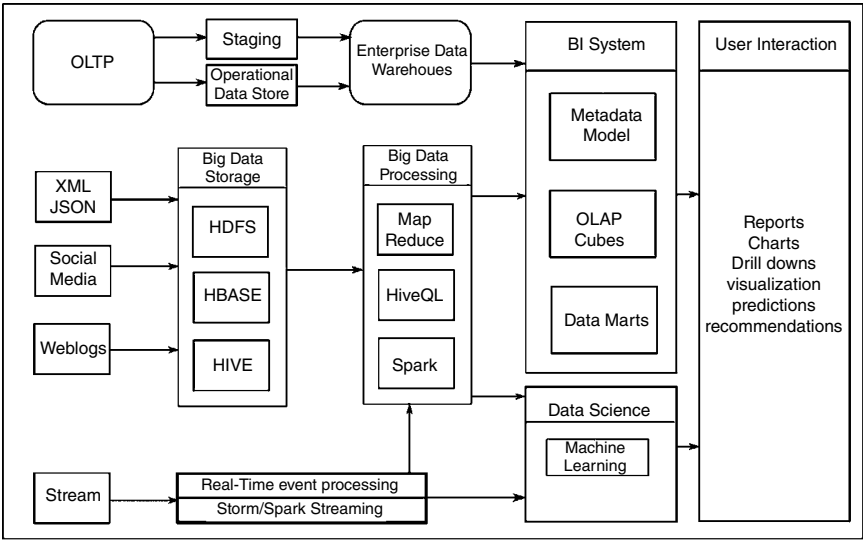


Figure 6.10 Architecture of an integrated EDW with Big Data technologies.

Figure 6.10 shows the architecture of an integrated EDW with big data technologies. The top layer of the diagram shows a traditional business intelligence system with Operational Data Store (ODS), staging database, EDW, and various other components. The middle layer of the diagram shows various big data technologies to store and process large volumes of unstructured data arriving from multiple data sources such as blogs, weblogs, and social media. It is stored in storage paradigms such as HDFS, HBase, and Hive and processed using processing paradigms such as MapReduce and Spark. Processed data are stored in a data warehouse or can be accessed directly through low latency systems. The lower layer of the diagram shows real-time data processing. The organizations use machine learning techniques to understand their customers in a better way, offer better service, and come up with new product recommendations. More data input with better analysis techniques yields better recommendations and predictions. The processed and analyzed data are presented to end users through data visualization. Also, predictions and recommendations are presented to the organizations.

Chapter 6 Refresher

- 1 After acquiring the data, which of the following steps is performed by the data scientist?
A Data cleansing
B Data analysis

- C Data replication
- D All of the above.

Answer: a

Explanation: The data cleansing process fills in the missing values, corrects the errors and inconsistencies, and removes redundancy in the data to improve the data quality.

- 2 Raw data is cleansed only one time.
- A True
 - B False

Answer: b

Explanation: Depending on the extent of dirtiness in the data, the process may be repeated to obtain clean data.

- 3 _____ is the science of extracting meaningful information from the speech and textual data.
- A Semantic analysis
 - B Sentiment analysis
 - C Predictive analysis
 - D Prescriptive analysis

Answer: a

- 4 The full form of OLAP is
- A Online Analytical Processing
 - B Online Advanced Processing
 - C Online Analytical Preparation
 - D Online Analytical Performance

Answer: a

- 5 They are used in transactions where the system is required to respond immediately to the end-user requests.
- A OLAP
 - B OLTP
 - C RTAP
 - D None of the above.

Answer: b

Explanation: In OLTP the applications are processed in real time and not in batch; hence the name OLTP. Hence, they are used in applications where immediate response is required, e.g., ATM transactions.

- 6 _____ is used for collecting, processing, and presenting the business users with multidimensional data for analysis.
- A OLAP
 - B OLTP
 - C RTAP
 - D None of the above.

Answer: a

- 7 _____ is a type of OLAP system.
- A ROLAP
 - B MOLAP
 - C HOLAP
 - D All of the above.

Answer: d

- 8 In a _____ process duplicates are removed.
- A data cleansing
 - B data integration
 - C data transformation
 - D All of the above.

Answer: a

Explanation: The data cleansing process fills in the missing values, corrects the errors and inconsistencies, and removes redundancy in the data to improve the data quality.

- 9 A predictive analysis technique makes use of _____.
- A historical data
 - B current data
 - C assumptions
 - D both current and historical data

Answer: a

Explanation: Predictive analysis exploits patterns from historical data to determine risks and opportunities.

- 10 NLP is the acronym for
- A Natural Level Program
 - B Natural Language Program
 - C National Language Processing
 - D Natural Language Processing

Answer: d

Conceptual Short Questions with Answers

1 What is a data warehouse?

Data warehouse, also termed as Enterprise Data warehouse, is a repository for the data that various organizations and business enterprises collect. It gathers the data from diverse sources to make the data available for unified access and analysis by the data analysts.

2 What is business intelligence?

Business intelligence is the process of analyzing the data and produce a desirable output to the organizations and end users to assist them in decision-making. The benefit of big data analytics is to increase revenue, increase efficiency and performance, and outcompete business rivals by identifying market trends. BI data comprises both data from the storage (previously captured and stored data) and data that are streaming, supporting the organizations to make strategic decisions.

3 What is big data analytics?

Big data analytics is the science of examining or analyzing large data sets with variety of data types, i.e., structured, semi-structured, or unstructured data, which may be streaming or batch data. The objective of big data analytics is to make better decisions, find new business opportunities, compete against business rivals, improve performance and efficiency, and reduce cost.

4 What is descriptive analytics?

Descriptive analytics describes, summarizes, and visualizes massive amounts of raw data into a form that is interpretable by end users. It describes the events that occurred at any point in the past and provides insight into what actually has happened in the past. In descriptive analysis, past data are mined to understand the reason behind failure or success.

5 What is diagnostic analytics?

Diagnostic analytics is a form of analytics that enables users to understand what is happening and why did it happen so that a corrective action can be taken if something went wrong. It benefits the decision-makers of the organizations by giving them actionable insights.

6 What is predictive analytics?

Predictive analytics provides valuable and actionable insights to companies based on the data by predicting what might happen in the future. It analyses the data to determine possible future outcome.

7 What is prescriptive analytics?

Prescriptive analytics provides decision support to benefit from the outcome of the analysis. Thus, prescriptive analytics goes beyond just analyzing the data and predicting future outcome by providing suggestions to extract the benefit and take advantage of the predictions.

8 What is Online Transaction Processing (OLTP)?

OLTP is used to process and manage transaction-oriented applications. The applications are processed in real time and not in batch; hence the name Online Transaction Processing. They are used in transactions where the system is required to respond immediately to the end-user requests.

9 What is Online Analytics Processing (OLAP)?

Online Analytical processing systems are used to process data analysis queries and perform effective analysis on massive amounts of data. Compared to OLTP, OLAP systems handle relatively smaller numbers of transactions. In other words, OLAP technologies are used for collecting, processing, and presenting the business users with multidimensional data for analysis.

10 What is semantic analysis?

Semantic analysis is the science of extracting meaningful information from speech and textual data. For the machines to extract meaningful information from the data, the machines should interpret the data as humans do.

11 What are the types of semantic analysis?

Types of semantics analysis:

- 1) Natural Language Processing
- 2) Text analytics
- 3) Sentiment analysis

12 What is Natural Language Processing?

Natural Language Processing (NLP) is a field of artificial intelligence that helps computers understand human speech and text as understood by humans. NLP is needed when an intelligent system is required perform according to the instructions provided.

13 What is text analytics?

Text analytics is the process of transforming unstructured data into meaningful data by applying machine learning, data mining, and NLP techniques.

7

Big Data Analytics with Machine Learning

CHAPTER OBJECTIVE

This chapter explains the relationship between the concept of big data analytics and machine learning, including various supervised and unsupervised machine learning techniques. Various social applications of big data, namely, health care, social analysis, finance, and security, are investigated with suitable use cases.

7.1 Introduction to Machine Learning

Machine learning is an intersection of Artificial Intelligence and statistics and is the ability of a system to improve its understanding and decision-making with experience. With the ever-increasing data volume, efficient machine learning algorithms are required in many technological applications and have become ubiquitous in every human activity, from automatically recommending which video to watch, what product to buy, listing the friends we may know on Facebook, and much more.

Basically, a machine learning algorithm is a program for pattern recognition and developing intelligence into the machine to make it capable of learning and improving its understanding and decision-making capabilities with experience. Pattern recognition is a program to make the machines understand the environment, learn to differentiate the object of interest from the rest of the objects, and make decisions by categorizing the behavior. Machines are trained in a way that make decisions in a much similar way that humans do. In machine learning, a general algorithm is developed to solve problems.

In the big data context, machine learning algorithms are effective even under circumstances where actionable insights are to be extracted from large and rapidly changing data sets.

Machine learning is performed with two types of data sets. The first data set is prepared manually, and it has multiple input data and the expected output. Each input data provided should have their expected output so as to build a general rule. The second data set has the actual input, and the expected output is to be predicted by applying the rule. The input data set that is provided to build the rule is divided into a training data set, a validation data set, and a testing data set. A training data set is used to train the machine and build a rule-based model. A validation data set is used to validate the model built. A testing data set is used to assess the performance of the model built. There are three phases in machine learning, namely, training phase, validation and test phase, and application phase. In the testing phase the training data set is used to train the machines to recognize patterns or behavior by pairing input with the expected output and build a general rule. In the validation and test phase, the validation data set is used to estimate how well the machine is trained by verifying the data examples against the model built. In the testing phase, the model is exposed to the actual data for which the expected output is to be predicted.

7.2 Machine Learning Use Cases

- *Product recommendation*—Amazon uses machine learning techniques to generate the recommended list for the consumers. This type of machine learning algorithm is known as recommender system wherein the user behaviors are learnt over a period of time, and the products users might be interested in are predicted.
- *Face Recognition*—Another machine learning algorithm is used for face recognition software that identifies a given person from a digital photograph. This is used by Facebook when it provides suggestions to the users to tag their friends in the photographs that are uploaded.
- *Spam Detection*—A machine learning algorithm is used in spam detection by e-mail service providers. A machine learning algorithm categorizes mails as spam based on some predefined rules and moves the mail to the spam folder instead of placing them in the inbox.
- *Fraud Detection*—Credit card frauds can be detected using a machine learning algorithm by detecting the changes in the usage pattern and purchase behavior of the consumer.
- *Speech recognition*—Speech recognition used in call centers is implemented using a machine learning algorithm where the user's speech is interpreted and mapped to a corresponding task for problem solving.
- *Sentiment Analysis*—Sentiment analysis is used for making decisions based on customer opinions. For example, customers leave their comments,

feedback, or suggestions about a product bought in online retail websites such as eBay and Amazon. Customers purchase a product based on these customer opinions.

- **Customer Churn Prevention**—Machine learning is used to predict the behavior of customers, find their interest in other products or services through their comments or likes in social media and predict whether consumers will leave the provider of a service or product. Customer churn prevention is specifically used in the telecommunication industry where the mobile service providers compete for holding back their relatively finite customer base.
- **Customer Segmentation**—Customer segmentation is grouping customers based on their interests. It is used in marketing, where the customer purchasing history is analyzed and ideally matching products are targeted to the customers based on their interests and needs. Thus marketing is transformed into a highly targeted activity

7.3 Types of Machine Learning

There are two types of machine learning algorithms, shown in Figure 7.1:

- 1) Supervised; and
- 2) Unsupervised.

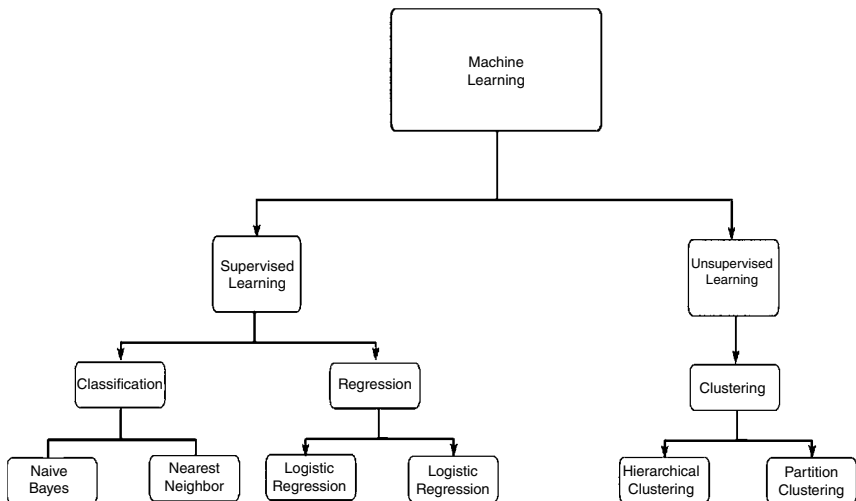


Figure 7.1 Types of machine learning algorithms.

7.3.1 Supervised Machine Learning Algorithm

Supervised or predictive machine learning algorithm (see Figure 7.2) is the most successful type of machine learning algorithm. A machine learning model is built from the input-output pair that forms the training set. This training set trains a model to generate predictions in response to new data. It is the key behind detecting frauds in financial transactions, face recognition in pictures, and voice recognition. Supervised machine learning is used in the applications where the outcome is to be predicted from a given input. Accurate decisions are to be made on never before seen data.

7.3.1.1 Classification

Classification is a machine learning tool to identify groups based on certain attributes. This technique is used to classify things or people into existing groups. A mail is classified as spam by a Mail Service Provider by analyzing the mail account holder’s previous decision in marking certain mail as spam. This classification technique is adopted by Google and Yahoo Mail Service Providers. Similarly, credit card fraud can be detected using a classification technique. Based on historical credit card transactions, a model is built that predicts whether a new transaction is legitimate or fraudulent. Also, from the historical data a customer can be classified as defaulter and can be used by the lenders to make a lending decision. A classification technique is also used in identifying potential customers by analyzing the items purchased and the total money spent. The customers spending

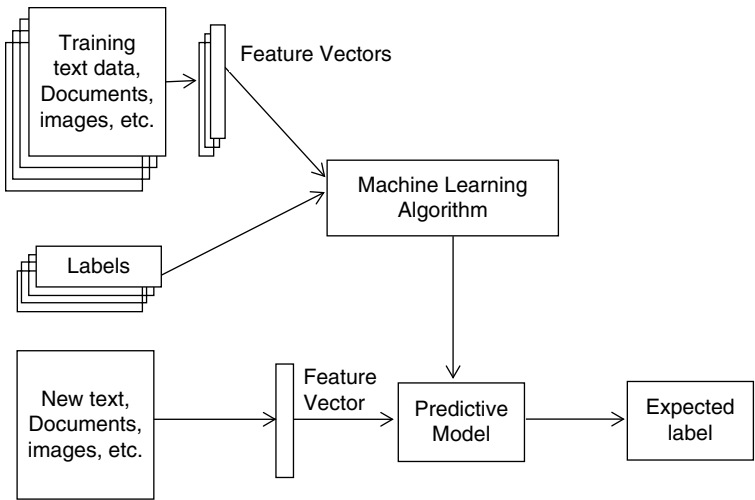


Figure 7.2 Supervised machine learning.

above a specified amount are grouped into one category, and the ones spending below the specified amount are grouped into another category.

7.3.1.2 Regression

A Regression technique is used in predicting future outputs based on experience. Regression is used in predicting values from a continuous set of data. The basic difference between regression and classification is that regression is used in finding the best relationship that represents the set of the given input data, while in classification a known relationship is given as input and the category to which the data belongs is identified. Some of the regression techniques are linear regression, neural networks, and decision trees. There are two types of regressions, namely:

- Linear regression; and
- Logistic regression

Linear Regression A linear regression is a type of supervised machine learning technique used to predict values based on previous history, that is, the value of a variable is determined from another variable whose value is previously known. The variables involved in a linear regression are called dependent and independent variables. The variable whose value is previously known and used for prediction is called independent variable, and the variable whose value is to be determined is called dependent variable. The value of the dependent variable is affected by the changes in the value of the independent variable. For example, if X and Y are related variables, a linear regression is used to predict the value of X from the value of Y and vice versa.

If the value of X is unknown then,

$$X = a + bY,$$

where, a is a constant, b is the regression coefficient, X is the dependent variable, and Y is the independent variable.

If the value of Y is unknown then,

$$Y = c + dX,$$

where, c is a constant, d is the regression coefficient, Y is the dependent variable, and X is the independent variable.

7.3.1.2.1 Logistic Regression A logistic regression is a machine learning technique where there are one or more independent variables, which determine the value of a dependent variable. The main objective of a logistic regression is to find the best-fitting model that describes the relationship between the dependent variable and a set of independent variables. The basic difference between linear

regression and logistic regression is the outcome of a linear regression is continuous. It can have an infinite number of values for the outcomes while a logistic regression has a limited number of values for the outcome.

7.3.2 Support Vector Machines (SVM)

Support vector machines (SVM) are one of the supervised machine learning techniques. SVM can perform regression, outlier detection, and linear and nonlinear classification. SVM build a highly accurate model and overcomes the local optima. The major limitation of SVM is the speed and the size. It is not suitable to construct classification model for large data sets.

SVM will develop a model with the training data set in a way that the data points that belong to different groups are separated by a distinct gap. The data samples that lie on the margin are called the support vectors. The center of the margins separating the two groups is called the separating hyperplane. Figure 7.3 shows SVM.

SVM linear classifiers are simple classifiers where the data points are linearly separated. Data points with several features cannot be linearly separated. Under such cases, several kernels are used to separate the data points, which are the nonlinear classifiers.

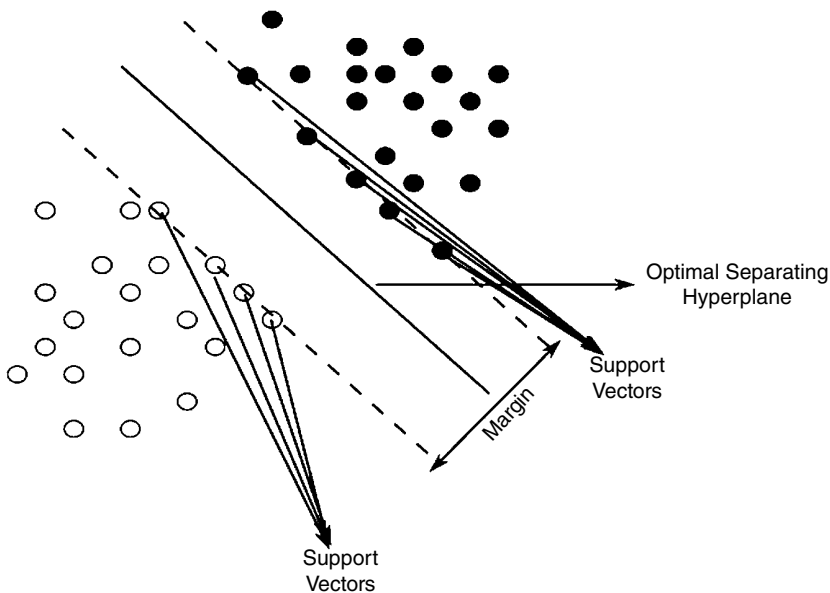


Figure 7.3 Support vector machines.

SVM perform the classification using an N-dimensional separating hyperplane that maximizes the margin width that separates the data points into two classes. The goal of SVM modeling is to find an optimal hyperplane to separate the vector points into two classes. The data points close to the hyperplane are called support vectors.

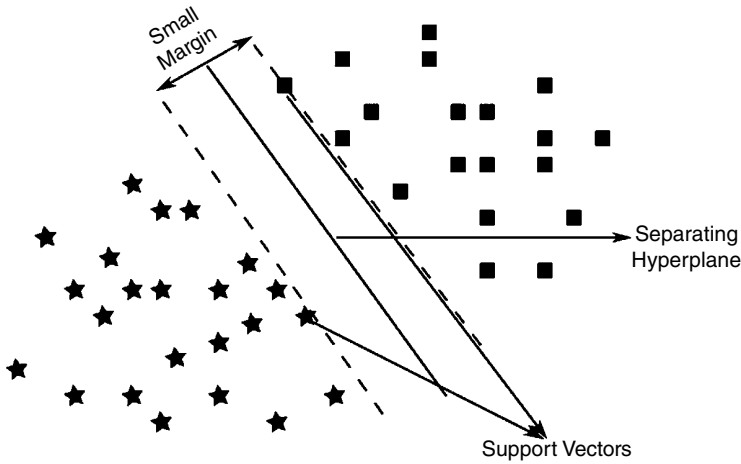
Figure 7.4a shows SVM in a two-dimensional plane. The classification is to be performed on two categories of variables represented by stars and rectangles with one category of variables lying in the lower left corner and the other category lying in the upper right corner.

The classification attempts to find a line that separates the two categories. In a two-dimensional space, the data points can be separated by a line whereas with higher dimensions a hyperplane is required. The dashed lines that are drawn parallel to the separating line is the distance between the hyperplane and the vectors closest to the line. The distance between the dashed lines drawn parallel is called the margin. The vector points that determine the width of the margin are called the support vectors. Support vectors are critical elements that would change the position of the separating hyperplane if removed. The analysis finds a hyperplane that is oriented such that the distance between the dashed lines, that is, the margin distance between the support vectors, is maximized. The quality of classification by SVM depends on the distance between the different classes of data points, which is known as margin. The accuracy of the classification increases with the increase in the margin. Figure 7.4a shows a hyperplane where the distance between the vector points is minimal while Figure 7.4b shows a hyperplane where the distance between the vector points is maximized. Thus the hyperplane in the Figure 7.4b is optimal compared to the hyperplane in Figure 7.4a.

A margin that separates the observations into two distinct classes or groups is called hard margin. This type of hard margin is possible only in separable cases where the observations can easily be segregated into two distinct classes. There are cases where the observations will be non-separable. In such cases the margin is called soft margin. In a non-separable case the support vectors cannot completely separate the data points into two distinct classes. Under such cases the data points or the outliers that lie away from their respective support vectors are penalized. Figure 7.5 shows non-separable SVM. A slack variable is also known as penalty variable (ξ). The value of the slack variable increases with the increase in the distance of the outlier from the support vectors.

The observations belonging to the classes are not penalized. Only the observations that are located beyond the corresponding support vectors are penalized, and the penalty variable ξ increases as the observations of one class get closer to the support vectors of the other class and goes beyond the support vectors of the other class.

(a)



(b)

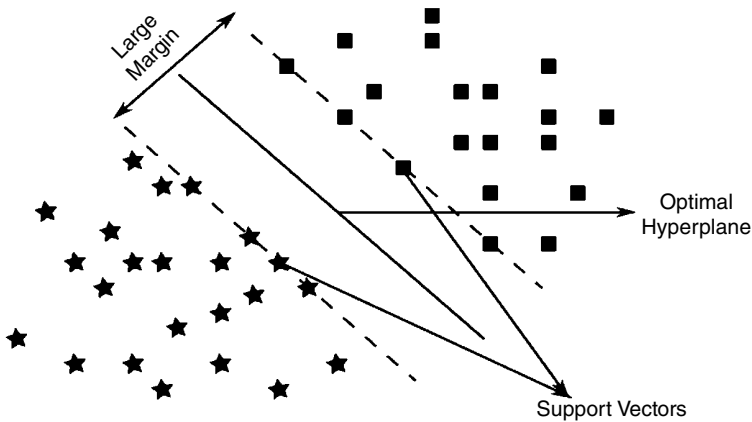


Figure 7.4 (a) Support vectors with small margin. (b) Support vectors with an optimal hyperplane.

7.3.3 Unsupervised Machine Learning

Unsupervised machine learning is a technique where input data has no labels, so it has not training set to predict the output and it rather has to find the data structures based on their relationships; this forms the basic difference between supervised and unsupervised learning. In other words, unsupervised machine learning is to learn without explicit supervision. The main objective of this type of learning is to find the relationships existing between the variables under study, not to find

Figure 7.5 Non-separable support vector machines.

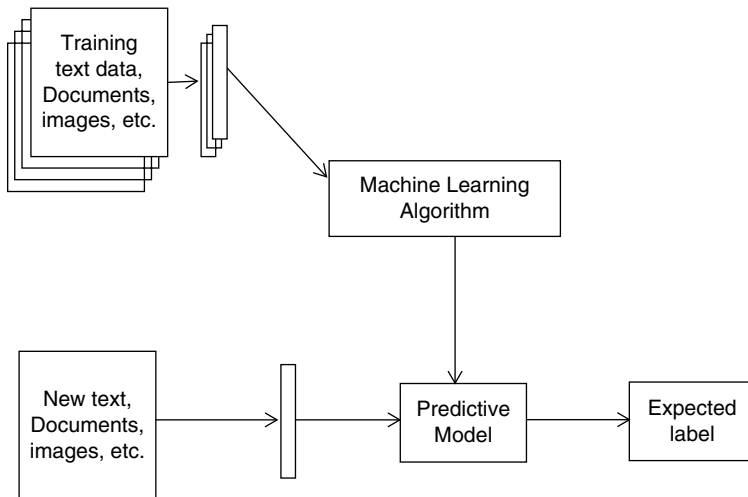
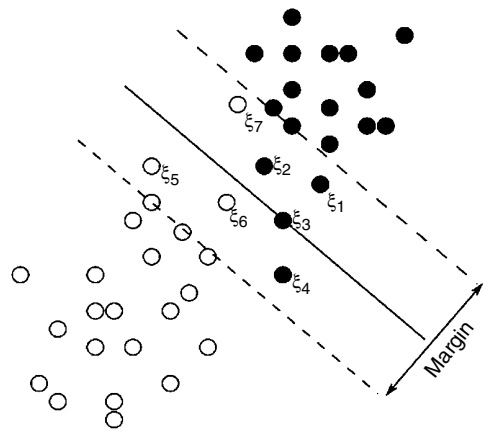


Figure 7.6 Unsupervised machine learning.

the relationship between these study variables and a target variable. Figure 7.6 shows an unsupervised machine learning algorithm.

7.3.4 Clustering

A clustering technique is used when the specific target or the expected output is not known to the data analyst. It is popularly termed as unsupervised classification. In a clustering technique, the data within each group are remarkably similar in their characteristics. The basic difference between classification and

clustering is that the outcome of the problem in hand is not known beforehand in clustering while in classification the historical data groups the class to which the data belongs. Under classification the results will be the same in grouping different objects based on certain criteria. But under clustering where the target required is not known, the results may not be the same every time the clustering technique is performed on the same data. A detailed view on clustering is discussed in Chapter 9.

Chapter 7 Refresher

- 1 _____ is the ability of the system to improve its understanding and decision-making with experience.
- A Machine learning
 - B Data mining
 - C Business intelligence
 - D Semantics

Answer: a

- 2 A _____ technique is used to find groupings of customers, users, products, etc.
- A classification
 - B clustering
 - C regression
 - D None of the above.

Answer: b

- 3 _____ is a type of machine learning
- A Supervised machine learning
 - B Unsupervised machine learning
 - C Both a) and b)
 - D None of the above.

Answer: c

- 4 _____ or its square is a commonly used measure of similarity.
- A Euclidean distance
 - B City-block distance
 - C Chebyshev's distance
 - D Manhattan distance

Answer: a

- 5 In _____, labels are predefined, and the new incoming data is categorized based on the labels.
- A classification
 - B clustering
 - C regression
 - D semantics

Answer: a

- 6 _____ is a clustering technique that starts in one giant cluster dividing the cluster into smaller clusters.
- A Hierarchical clustering
 - B Agglomerative clustering
 - C Divisive clustering
 - D Non-hierarchical clustering

Answer: c

- 7 _____ is a clustering technique that results in the development of a tree-like structure.
- A Hierarchical clustering
 - B Agglomerative clustering
 - C Divisive clustering
 - D Non-hierarchical clustering

Answer: a

- 8 Once the hierarchical clustering is completed the results are visualized with a graph or a tree diagram called _____.
- A Dendrogram
 - B Scatter graph
 - C Tree graph
 - D None of the above

Answer: a

- 9 A _____ technique is used when the specific target or the expected output is not known to the data analyst.
- A clustering
 - B classification
 - C regression
 - D None of the above.

Answer: a

- 10 A machine learning technique is used in _____.
A face recognition
B spam detection in e-mail
C speech recognition
D All of the above.

Answer: d

Conceptual Short Questions with Answers

1 What is machine learning?

Machine learning is an intersection of Artificial Intelligence and statistics and is the ability of the system to improve its understanding and decision-making with experience. Basically, a machine learning algorithm is a program for pattern recognition and developing intelligence into the machine to make it capable of learning and improve its understanding and decision-making capabilities with experience.

2 What are the applications of machine learning?

The applications of machine learning are product recommendation, face recognition, spam detection, fraud detection, speech recognition, sentiment analysis, customer churn prevention, and customer segmentation.

3 What are the types of machine learning?

There are two types of machine learning:

- Supervised machine learning; and
- Unsupervised machine learning.

4 What is clustering?

Clustering is a machine learning tool used to cluster similar data based on the similarities in its characteristics. The clusters are characterized by high intra-cluster similarity and low inter-cluster similarity.

5 What is hierarchical clustering? What are its types?

Hierarchical cluster is a series of partitions running from a single cluster or reversely a single large cluster can be iteratively divided into smaller clusters. Agglomerative clustering and divisive clustering are the types of hierarchical clustering.

6 What is an agglomerative clustering?

Agglomerative clustering is done by merging several smaller clusters into a single larger cluster from the bottom up. It reduces the data into a single large cluster containing all individual data groups.

7 What is a divisive clustering?

Divisive clustering is done by dividing a single large cluster into smaller clusters. The entire data set is split into n number of groups, and the optimal number cluster to stop clustering is decided by the user.

8 What is partition clustering?

Partitional clustering is the method of partitioning a data set into a set of clusters. Given a data set with N data points, partitional clustering partitions N data points into K number of clusters where $N \geq K$. The partitioning is performed by satisfying two conditions: each cluster should have at least one data point, and each of the N data points should belong to at least one of the K clusters.

9 What is a k-means clustering?

K-means clustering is a type of partition clustering. A K-means clustering algorithm partitions the data points into K number of clusters in which each data point belongs to its nearest centroid. The value of K , which is the number of clusters, is given as the input parameter.

10 What is classification?

Classification is a machine learning tool to identify groups based on certain attributes. This technique is used to classify things or people into the existing groups.

11 What is regression?

A regression technique is used in predicting future output based on experience. Regression is used in predicting values from a continuous set of data. The basic difference between regression and classification is that regression is used in finding the best relationship that represents the set of given input data, while in classification a known relationship is given as input and the category to which the data belongs is identified.

12 What is simulation?

Simulation is the technique used in modeling a system or process in the real world. The new system modeled represents the characteristic features or functions of the system or process based on which the system is modeled.

8

Mining Data Streams and Frequent Itemset

CHAPTER OBJECTIVE

Frequent itemset mining is a branch of data mining that deals with the sequences of action. In this chapter, we focus on various itemset mining algorithms, namely nearest neighbor, similarity measure: the distance metric, artificial neural networks (ANNs), support vector machines, linear regression, logistic regression, time-series forecasting, big data and stream analytics, data stream mining. Also, various data mining methods, namely, prediction, classification, decision trees, association, and apriori algorithms, are elaborated.

8.1 Itemset Mining

A collection of all items in a database is represented by

$$I = \{i_1, i_2, i_3, i_4, i_5 \dots i_n\}$$

A collection of all transactions is represented by

$$T = \{t_1, t_2, t_3, t_4, t_5 \dots t_n\}$$

Table 8.1 shows a collection of transactions with a collection of items in each transaction.

Itemset—A collection of one or more items from I is called an itemset. If an itemset has n items, then it is represented as n -itemset. For example, in the transaction with transaction_id 1 in Table 8.1 with Itemset {Rice, Milk, Bread, Jam, Butter} is a 5-itemset.

The strength of the association rule is measured by two important terms, namely, the support and confidence.

Table 8.1 Market basket data.

Transaction_Id	Products_purchased
1	{Rice, Milk, Bread, Jam, Butter}
2	{Diaper, Baby oil, Baby lotion, Milk, Curd}
3	{Cola, Milk, Bread, Chocolates}
4	{Bread, Butter, Milk, Curd, Cheese}
5	{Milk, Bread, Butter, Jam}
6	{Diaper, Baby Shampoo, Baby oil, Bread, Milk}

Support—Support S is the ratio of transactions that contain an itemset to the total number of transactions.

$$\text{Support } S = \frac{\text{Number of transactions that contain an itemset}}{\text{Total number of transactions}}$$

For example, let us consider the number of transactions that contain the itemset {Milk, Bread, Butter}

$$\text{Support } S = \frac{\text{Number of transactions that contain } \{\text{Milk, Bread, Butter}\}}{\text{Total number of transactions}}$$

$$\text{Support } S = \frac{3}{6} = \frac{1}{2} = 50\%$$

Confidence—Let us consider two itemsets X and Y where X is {Milk, Bread} and Y is {butter}. Confidence is a term that measures how often the items in the itemset Y appear in the transactions that contain itemset X.

$$\text{Confidence } C = \frac{\text{Number of transactions that contain } X \text{ and } Y}{\text{Number of transactions that contain } X}$$

$$\{\text{Milk, Bread}\} \rightarrow \{\text{Butter}\}$$

$$\text{Confidence } C = \frac{\text{Number of transaction that contain Milk, Bread, Butter}}{\text{Number of transactions that contain Milk, Bread}}$$

Itemset frequency—An itemset frequency is the number of transactions that contain a particular itemset.

Frequent Itemset—A frequent itemset is an itemset that occurs at least for a minimum number of times with itemset frequency greater than a preset support

threshold. For example, if the support threshold is 3, an itemset is called frequent itemset if the itemset frequency > 3 .

Frequent itemsets play an important role in several data mining tasks such as association rules, classification, clustering, and correlation, which are used for finding interesting patterns from databases. It is most popularly used in association rules problems. The most common problem in frequent itemset mining is the market basket problem. Here, a set of items that are present in multiple baskets are said to be frequent. Formally, let s be the support threshold and I be the set of items; then support is the number of baskets in which I is a subset. A set of items I is said to be frequent if its support is equal to or greater than the support threshold s , which is called minimum support or the MinSup. Suppose the support threshold of an itemset, MinSup, is 2, then for the itemset to be frequent it should be present in at least two of the transactions.

For frequent itemset generation the general rule is,

$$\text{Support} \geq \text{MinSup}$$

Table 8.2 shows the itemset in a transaction and Table 8.3 shows the corresponding support for each item in the itemset. A occurs in 3 transactions and hence its support is 3; similarly the support for B is 1 since it occurs in only one transaction, C, D, and E occurs in two transactions and hence its support is 2. Let us assume the

Table 8.2 Itemset in a transaction.

Transaction Id	Itemset in the transaction
1	{a,b,c,d}
2	{a,e}
3	{a,d}
4	{c,e}

Table 8.3 Support of each items in a transaction.

Item	Support	Frequency for $S = 2$
A	3	Frequent
B	1	Infrequent
C	2	Frequent
D	2	Frequent
e	2	Frequent

support threshold S is 2. Then A, C, D, and E are frequent since their support is $\geq \text{MinSup}$, which is 2 in this case, while B is infrequent as its support is 1.

Exercise 1: Frequent Itemset Mining Using R

The package that has to be installed to implement frequent itemset mining is “arules.” Use the command `install.packages('arules')` to install the arules package. Once installed let us use the file available default in arules package “Groceries.csv.”

```
library(arules)
data(Groceries)
```

The function `data()` is used to load the available dataset. Arules package also has some other functions such as `inspect()`, which is used to display associations and transactions.

```
inspect(Groceries[1:10])
      items
[1] {citrus fruit,
    semi-finished bread,
    margarine,
    ready soups}
[2] {tropical fruit,
    yogurt,
    coffee}
[3] {whole milk}
[4] {pip fruit,
    yogurt,
    cream cheese ,
    meat spreads}
[5] {other vegetables,
    whole milk,
    condensed milk,
    long life bakery product}
[6] {whole milk,
    butter,
    yogurt,
    rice,
    abrasive cleaner}
[7] {rolls/buns}
[8] {other vegetables,
    UHT-milk,
    rolls/buns,
    bottled beer,
    liquor (appetizer)}
```

```
[9] {pot plants}
[10] {whole milk,
      cereals}
```

The frequency of an item occurring in a database can be found using command `itemfrequency()`. The command returns the support of the items if the type is given as relative, while it returns the item count if the type is given as absolute. To find the items highest frequency and item count, the items can be sorted using `sort()`. This command sorts the items in increasing order of frequency and item count by default, by using `decreasing=TRUE`, the items can be sorted in decreasing order.

```
sort(itemFrequency(Groceries[,1:5], type= "relative" ),
decreasing = TRUE)
      sausage frankfurter   ham    meat    liver loaf
0.093950178 0.058973055 0.026029487 0.025826131 0.005083884
> sort(itemFrequency(Groceries[,1:5], type= "absolute" ),
decreasing = TRUE)
      sausage frankfurter   ham    meat    liver loaf
          924          580    256    254          50
```

These statistics can be visually presented using the function `itemfrequencyplot()`. Graph can be plotted either based on relative value or absolute value. Frequency plot with relative value plots the graph based on the support count while frequency plot with absolute value plots the graph based on item count. as shown in fig 8.1 while frequency plot with absolute value plots the graph based on item count as shown in fig 8.2.

```
itemFrequencyPlot(Groceries,
+                 type="relative",
+                 topN=20)
```

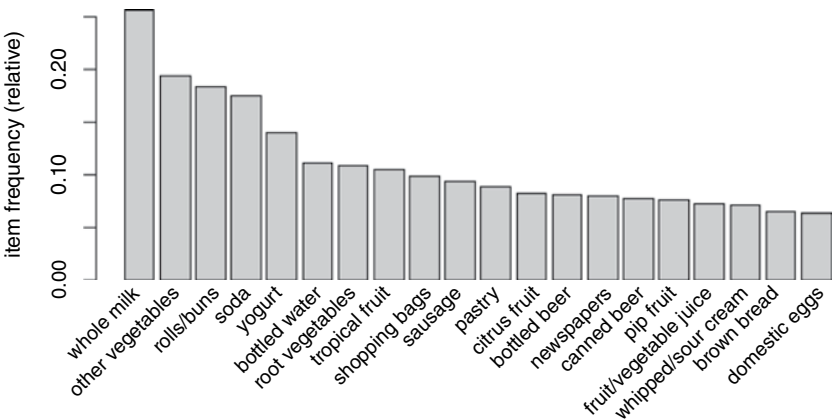


Fig.8.1 Frequency plot with relative value

```
itemFrequencyPlot (Groceries,
+                  type="absolute",
+                  topN=20)
```

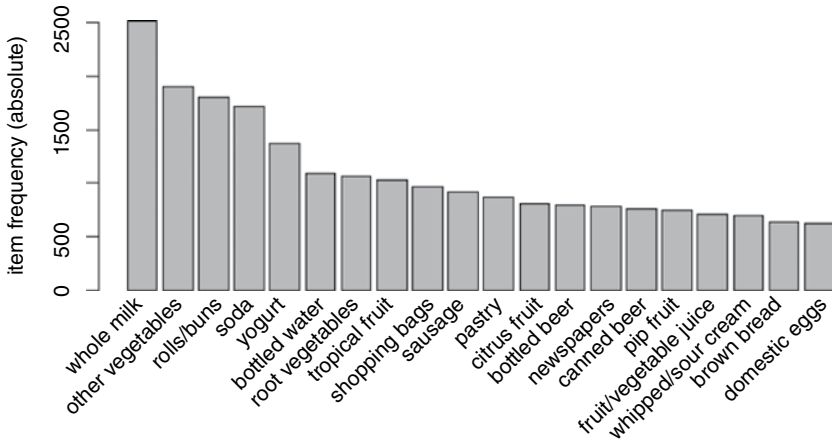


Fig.8.1 Frequency plot with absolute value

8.2 Association Rules

The association rule is framed by a set of transactions with each transaction consisting of a set of items. An association rule is represented by

$$X \rightarrow Y$$

Where X and Y are itemsets of a transaction I , that is, $X, Y \subseteq I$ and they are disjoint: $X \cap Y = \emptyset$. The strength of an association rule in a transaction is measured in terms of its confidence and support. Support is the number of transactions which contain both X and Y given the total number of transactions

$$\text{Support } S, = \frac{\sigma(X \cup Y)}{N}$$

Confidence is a term that measures how often the items in the itemset Y appear in the transactions that contain itemset X .

$$\text{Confidence } C, = \frac{\sigma(X \cup Y)}{\sigma(X)}$$

Support and confidence are important measures to determine the strength of the inference made by the rule. A rule with low support may have occurred by chance.

Also, such rules with low support will not be beneficial from a business perspective because promoting the items that are seldom bought together may not be profitable. Confidence, on the other hand is the reliability measure of the inference made by the rule. The higher the confidence, the higher the number of transactions that contains both X and Y. The higher the number of transactions with X and Y occurring together, the higher the reliability of the inference made by the rule.

In a given set of transactions, find the rules that have

$$\text{Support} \geq \text{Minsup}$$

$$\text{Confidence} \geq \text{Minconf}$$

Where, Minsup and Minconf are support threshold and confidence threshold, respectively.

In association rule mining there are two subtasks, namely, frequent itemset generation and rule generation. Frequent itemset generation is to find the itemsets where $\text{Support} \geq \text{Minsup}$. Itemsets that satisfy this condition are called frequent itemsets. Rule generation is to find the itemsets that satisfy $\text{Confidence} \geq \text{Minconf}$ from the frequent itemsets extracted from frequent itemset generation. The task of finding frequent itemsets will be sensible only when Minsup is set to a larger value.

- For example, if Minsup = 0, then all subsets of the dataset *I* will be frequent making size of the collection of frequent itemsets very large.
- The task of finding the frequent itemsets is interesting and profitable only for large values of Minsup.

Organizations gather large amounts of data from the transactions or activities in which they participate. A large customer transaction data is collected at the grocery stores. Table 8.4 shows a customer purchase data of a grocery store where each row corresponds to purchases by individual customers identified by unique Transaction_id and the list of products bought by individual customers. These data are gathered and analyzed to gain insight about the purchasing behavior of the customers to promote their business, market their newly launched products to right customers, and organize their products in the grocery store based on product that are frequently bought together such as organizing a baby lotion near baby oil to promote sales so that a customer who buys baby lotion will also buy baby oil. Association analysis finds its application in medical diagnosis, bioinformatics, and so forth. One of the most common applications of association analysis, namely, market basket transaction, is illustrated below.

The algorithm that is used to uncover the interesting relationship underlying in large data sets is known as association analysis. The underlying relationship between two unrelated objects is discovered using association analysis. They are

used to find the relationship between the items that are frequently used together. The relationship uncovered is represented by association rules or frequent itemset. The following rule can be formulated from Table 8.4.

$$\{\text{Milk}\} \rightarrow \{\text{Bread}\}$$

The rule implies that a strong relationship exists between the sale of milk and bread because many customers who buy milk also buy bread. This kind of relationship thus uncovered can be used by the retailers for cross-selling their products.

Table 8.5 represents binary database of the market basket data represented in Table 8.4 where the rows represent individual transactions and each column represent the items in the market basket transaction. Items are represented in binary values: zeroes and ones. An item is represented by a one if it is present in a transaction and represented by zero if it is not present in a transaction. However, the important aspects of a transaction, namely, the quantity of items purchased and

Table 8.4 Market basket data.

Transaction_Id	Products_purchased
1	{Rice, Milk, Bread, Jam, Butter}
2	{Diaper, Baby oil, Baby lotion, Milk, Curd}
3	{Cola, Milk, Bread, Chocolates}
4	{Bread, Butter, Milk, Curd, Cheese}
5	{Milk, Bread, Butter, Jam}
6	{Diaper, Baby Shampoo, Baby oil, Bread, Milk}

Table 8.5 Binary database.

T_Id	Milk	Bread	Butter	Jam	Diaper	Baby Oil	Baby Lotion	Rice	Cola	Curd	Egg	Cheese
1	1	1	1	1	0	0	0	0	0	0	0	1
2	1	0	0	0	1	1	1	1	0	1	0	0
3	1	1	0	0	0	0	0	0	1	0	0	0
4	1	1	1	0	0	0	0	0	0	1	1	1
5	1	1	1	1	0	0	0	0	0	0	0	0
6	1	1	0	0	1	1	0	1	0	0	1	0

Table 8.6 Vertical database.

X	Milk	Bread	Butter	Jam	Diaper	Baby Oil	Baby Lotion	Rice	Cola	Curd	Egg	Cheese
t(x)	1	1	1	1	2	2	2	2	3	2	4	1
	2	3	4	5	6	6		6		4	6	4
	3	4	5									
	4	5										
	5	6										
	6											

price of each item, are all ignored in this type of representation. This method is used when an association rule is used to find the frequency of itemsets.

Table 8.6 shows the vertical database where the items are represented by the transaction id's of each items corresponding to the transaction in which the items appear.

Exercise 8.1

Determine the support and confidence of the transactions below for the rule {Milk, Bread} $\rightarrow$ {Butter}.

Transaction_Id	Products_purchased
1	{Rice, Milk, Bread, Jam, Butter}
2	{Diaper, Baby oil, Baby lotion, Milk, Curd, Chocolates}
3	{Cola, Milk, Bread, Chocolates, Rice}
4	{Bread, Butter, Milk, Curd, Cheese}
5	{Milk, Bread, Butter, Jam, Chocolates}
6	{Cola, Baby Shampoo, Baby oil, Bread, Milk}

The number of transactions that contain the itemset {Milk, Bread, Butter} is 3.

$$\begin{aligned}
 \text{Support } S, &= \frac{\sigma(X \cup Y)}{N} \\
 &= \frac{\text{Number of transaction that contain Milk, Bread, Butter}}{\text{Total number of transactions}} \\
 &= \frac{3}{6}
 \end{aligned}$$

$$\begin{aligned} \text{Coffidence } C, &= \frac{\sigma(X \cup Y)}{\sigma(X)} \\ &= \frac{\text{Number of transaction that contain Milk,Bread,Butter}}{\text{Number of transactions that contain Milk,Bread}} \\ &= \frac{3}{5} \end{aligned}$$

8.3 Frequent Itemset Generation

A dataset with n elements can generate up to $2^n - 1$ frequent itemsets. For example, for a dataset with items {a,b,c,d,e} can generate $2^5 - 1 = 31$ frequent itemsets. The lattice structure of the dataset {a,b,c,d,e} with all possible itemsets is represented in Figure 8.1. Frequent itemsets can be found by using a brute-force algorithm. As per the algorithm, frequent itemsets can be determined by calculating the support count for each itemset in the lattice structure. If the support is greater than the Minsup, then itemset is reported as a frequent itemset. Calculating support count for each itemset can be expensive for large datasets. The number of itemset and the number of transactions have to be reduced to speed up the brute-force algorithm. An apriori principle is an effective way that eliminates the need for calculating the support count for every itemset in the lattice structure and thus reduces the number of itemsets.

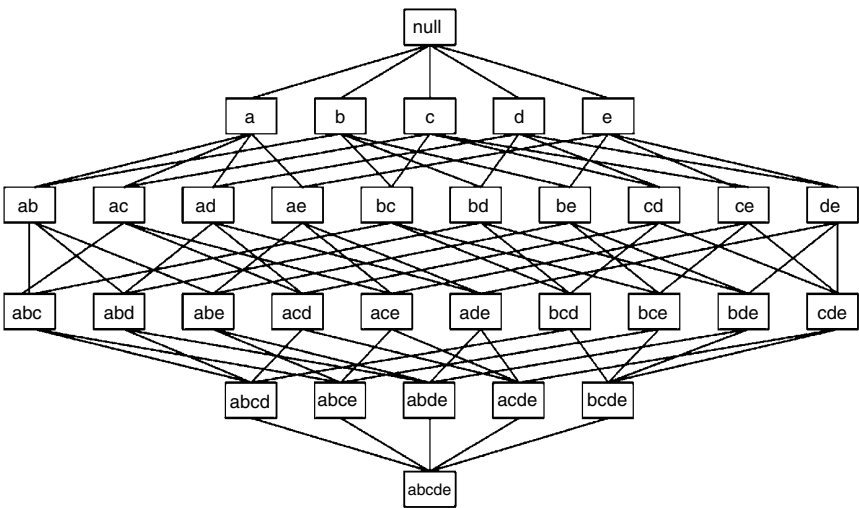


Figure 8.1 Lattice structure of data set {a,b,c,d,e}.

8.4 Itemset Mining Algorithms

Several algorithms have been proposed to solve the frequent itemset problem. Some of the important itemset mining algorithms are:

- Apriori algorithm
- Eclat algorithm (equivalence class transformation algorithm)
- FP growth algorithm

8.4.1 Apriori Algorithm

Apriori principle—The apriori principle states that if an itemset X is frequent, then all the subsets of the itemset are also frequent. Conversely, if an itemset X is not frequent then adding an item “i” will not make the itemset frequent. All its supersets will also be infrequent.

The apriori principle is illustrated in Figure 8.2. Suppose $\{b, c, d\}$ is a frequent itemset, which implies that the transactions that contain $\{b, c, d\}$ and its subsets $\{b\}$, $\{c\}$, $\{d\}$, $\{b, c\}$, $\{c, d\}$, and $\{b, d\}$ are frequent. Thus if $\{b, c, d\}$ is frequent, then all subsets of $\{b, c, d\}$ must also be frequent. This is illustrated by the shaded rectangles in the figure.

Conversely if an itemset $\{a, b\}$ is infrequent, then all its supersets are also infrequent. All the transactions that are a superset of an infrequent itemset are also infrequent, as illustrated in Figure 8.3. This approach is called support-based

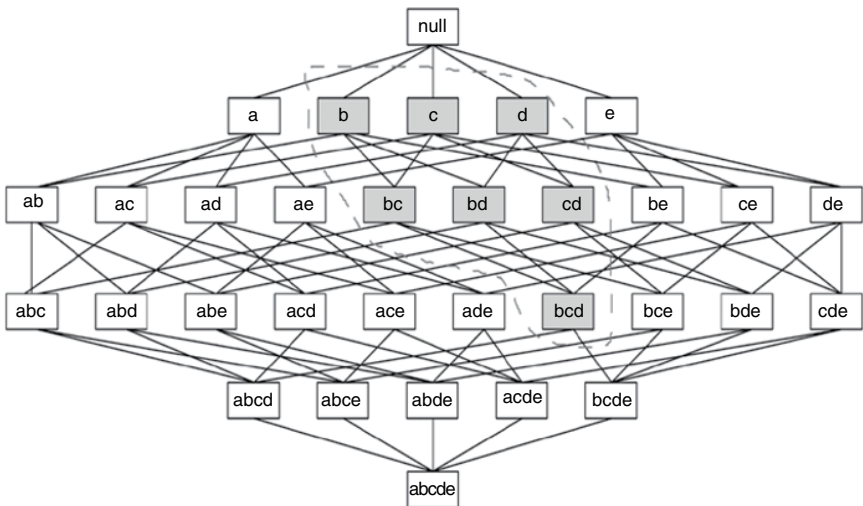


Figure 8.2 Apriori algorithm—frequent itemsets.

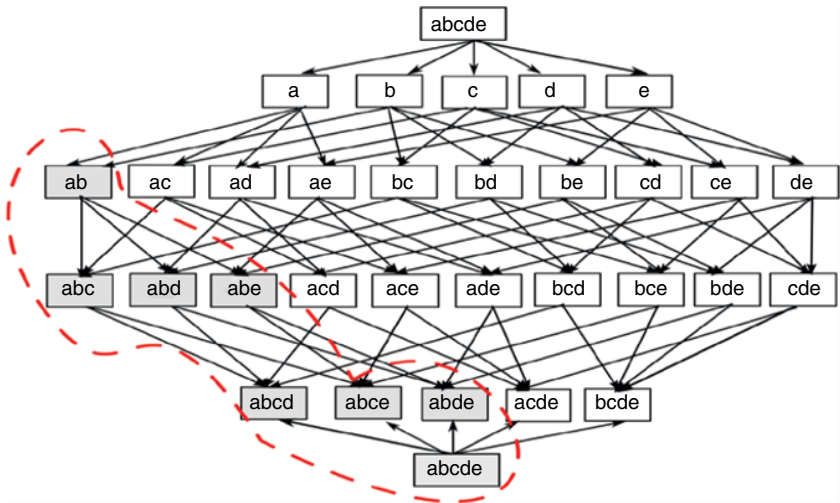


Figure 8.3 Apriori algorithm—Every superset of an infrequent itemset is also infrequent.

pruning. Also, the support of an itemset is always less than the support of its subsets. This property is called anti-monotone property of support.

$$X \subseteq Y \rightarrow S(Y) \leq S(X)$$

The above relation indicates that if Y is a superset of X, then the support of Y, S(Y) never exceeds the support of X, S(X). For example consider Table 8.7 where the support of an itemset is always less than the support of its subsets.

From the table, the anti-monotone property of support can be inferred.

- S (Bread) > S (Milk, Bread)
- S (Cola) > S (Cola, Beer)
- S (Milk, Bread) > S (Milk, Bread, Butter)

Exercise—Implementation of Apriori Algorithm Using R

Function `apriori()` has the following syntax,

```
apriori(data, parameter = NULL, appearance = NULL, control = NULL)
```

Arguments

data	Transactional data which may be a binary matrix or a data frame
parameter	Lists minlen, support, and confidence. The default minimum support is 0.1, minimum confidence is 0.8, maximum of 10 items (maxlen).

Table 8.7 Market Basket data.

Transaction_Id	Items
1	{Milk, Bread, Butter}
2	{Cola, Milk, Bread, Beer, egg, Rice}
3	{Bread, Milk, Diaper, Cola, Beer}
4	{Milk, Butter, Jam, Chocolates}
5	{Cola, Bread, Milk, Butter}
6	{Rice, egg, Diaper, Beer}

The challenge in generating rules for the Apriori algorithm is to set appropriate values for these three parameters, namely, minlen, support, and confidence so as to obtain a maximum set of meaningful rules. The value for these parameters has to be set by trial and error. Support and confidence values that are not appropriate either don't generate rules or generate too many rules. When too many rules are generated, it may have the default items that are frequently purchased together, such as bread and butter. Moving these items close to each other may not increase the revenue. Let us consider various trial-and-error values for the three parameters to see how rules are generated.

```
rules<-rules <- apriori(Groceries,parameter = list(supp
  = 0.1, conf = 0.5, minlen=2))
summary(rules)
set of 0 rules
```

Zero rule are generated, and this is because the support value is too high. A higher support value indicates that the item should have appeared in a greater number of transactions. Confidence 0.5 indicates that the rule should be true at least 50% of the time. Minlen = 2 indicates that rules with less than 2 items are to be eliminated. Let us consider a lower value for support, say, 0.01.

```
rules<-rules <- apriori(Groceries,parameter = list(supp
  = 0.01, conf = 0.5, minlen=2))
summary(rules)
set of 15 rules
rule length distribution (lhs + rhs):sizes
3
15
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
3	3	3	3	3	3

A set of 15 rules are generated, which is still low. Let us further reduce the value of support, say to 0.001. Rule length distribution indicates the number of items in each rule. The above rule length distribution indicates that 15 rules are generated with 3 items in each.

```
rules<-rules <- apriori(Groceries,parameter = list(supp
  = 0.001, conf = 0.5, minlen=2))
summary(rules)
set of 5668 rules
rule length distribution (lhs + rhs):sizes
  2    3    4    5    6
11 1461 3211  939  46
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  2.00   3.00   4.00   3.92   4.00   6.00
```

A set of 5668 rules are generated, which indicates that more generalized rules may be generated, so let us increase the value for support, say 0.005.

```
rules<-rules <- apriori(Groceries,parameter = list(supp
  = 0.003, conf = 0.5, minlen=2))
summary(rules)
set of 421 rules
rule length distribution (lhs + rhs):sizes
  2    3    4    5
  5 281 128   7

  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
2.000   3.000   3.000   3.325   4.000   5.000
```

```
summary of quality measures:
      support      confidence      lift
Min.   :0.003050  Min.   :0.5000  Min.   :1.957
1st Qu.:0.003355  1st Qu.:0.5238  1st Qu.:2.135
Median :0.003965  Median :0.5556  Median :2.426
Mean   :0.004754  Mean   :0.5715  Mean   :2.522
3rd Qu.:0.005186  3rd Qu.:0.6094  3rd Qu.:2.766
Max.   :0.022267  Max.   :0.8857  Max.   :5.804
mining info:
      data ntransactions support confidence
Groceries      9835      0.003      0.5
```

A set of 421 rules are obtained, out of which 5 rules have only 2 items, 281 rules have 3 items, 128 rules have 4 items, and 7 rules have 5 items. The summary of quality measures has three terms, and out of the three terms we are already aware of two terms support and confidence. The third term lift is the ratio of confidence

to that of expected confidence of the rule. It indicates the importance of the rule. The larger the value of lift, the more important is the rule. A larger value of lift indicates true connections existing between the items in a transaction. Mining info indicates the total number of transactions present in the groceries data is 9835, and the support and confidence are 0.003 and 0.5, respectively. Let us now inspect the rules generated.

```
> inspect(rules[1:10])
```

	lhs	rhs	support	confidence	lift
[1]	{cereals}	=> {whole milk}	0.003660397	0.6428571	2.515917
[2]	{specialty cheese}	=> {other vegetables}	0.004270463	0.5000000	2.584078
[3]	{rice}	=> {other vegetables}	0.003965430	0.5200000	2.687441
[4]	{rice}	=> {whole milk}	0.004677173	0.6133333	2.400371
[5]	{baking powder}	=> {whole milk}	0.009252669	0.5229885	2.046793
[6]	{root vegetable, herbs}	=> {other vegetables}	0.003863752	0.5507246	2.846231
[7]	{herbs, other vegetables}	=> {root vegetables}	0.003863752	0.5000000	4.587220
[8]	{root vegetables herbs}	=> {whole milk}	0.004168785	0.5942029	2.325502
[9]	{herbs, whole milk}	=> {root vegetables}	0.004168785	0.5394737	4.949369
[10]	{herbs, other vegetables}	=> {whole milk}	0.004067107	0.5263158	2.059815

The rule {herbs, whole milk} => {root vegetables} 0.004168785 0.5394737 4.949369 has to be read as, if a customer purchases herbs and whole milk, he will also purchase root vegetables. The confidence value 0.5394737 indicates that the rule is true 53% of the time, and a support of 0.004168785 indicates that the itemset is present in 0.41% of the transactions. Support indicates how frequently an item appears in the database while confidence indicates the number of times the rules is found true. It is calculated as below.

Support S , =

Number of transaction that contain herbs, whole milk, root vegetables

Total number of transactions

$$0.004168785 = \frac{\text{Number of transaction that contain herbs, whole milk, root vegetables}}{9835}$$

Number of transaction that contain herbs, whole milk, root vegetables

$$= 9835 \times 0.004168785$$

$$= 41$$

$$\text{Confidence } C = \frac{\text{Number of transaction that contain herbs, whole milk, root vegetables}}{\text{Number of transactions that contain herbs, whole milk}}$$

To verify the confidence let us find the number of transactions in which herbs and whole milk have been bought together. Let us create a table using crossTable() function.

```
table[1:5,1:5]
      frankfurter sausage liver loaf  ham  meat
frankfurter      580      99          7   25   32
sausage           99     924          10   49   52
liver loaf        7      10          50    3    0
ham               25      49           3  256    9
meat              32      52           0   9   254
table['root vegetables', 'herbs']
[1] 69
```

So the number of transactions in which the root vegetables and herbs are purchased together is 69. Now let us calculate the number of transactions where herbs, root vegetables, and whole milk are shopped together.

$$0.5394737 = \frac{\text{Number of transaction that contain herbs, whole milk, root vegetables}}{69}$$
$$\begin{aligned} \text{Number of transaction that contain herbs, whole milk, root vegetable} \\ &= 69 \times .5394737 \\ &= 37.22 \end{aligned}$$

Thus, the rule {herbs, whole milk} => {root vegetables} is true 53% of the time. The object of market basket analysis is to advertise and promote their products, cross-sell their products, for better organization of racks, and so forth. To do this, let us use the function subset() to determine the items that are frequently bought with a specific item.

Let us inspect the items that are frequently bought with domestic eggs using the subset() function.

```
inspect(subset(groceryrules, items %in% "domestic eggs"))
      lhs                rhs                support  confidence lift
[1] {other vegetables, domestic eggs} => {root vegetables} 0.007320793 0.3287671 3.016254
[2] {root vegetables, domestic eggs} => {other vegetables} 0.007320793 0.5106383 2.639058
[3] {whole milk, domestic eggs} => {root vegetables} 0.008540925 0.2847458 2.612383
[4] {tropical fruit, domestic eggs} => {whole milk} 0.006914082 0.6071429 2.376144
```

[5]	{root vegetables, domestic eggs}	=> {whole milk}	0.008540925	0.5957447	2.331536
[6]	{other vegetables, domestic eggs}	=> {whole milk}	0.012302999	0.5525114	2.162336
[7]	{whole milk, domestic eggs}	=> {other vegetables}	0.012302999	0.4101695	2.119820
[8]	{yogurt, domestic eggs}	=> {whole milk}	0.007727504	0.5390071	2.109485
[9]	{domestic eggs}	=> {whole milk}	0.029994916	0.4727564	1.850203
[10]	{whole milk, domestic eggs}	=> {yogurt}	0.007727504	0.2576271	1.846766
[11]	{domestic eggs}	=> {other vegetables}	0.022267412	0.3509615	1.813824
[12]	{domestic eggs, rolls/buns}	=> {whole milk}	0.006609049	0.4220779	1.651865

Customers frequently bought root vegetables and other vegetables with domestic eggs.

Exercise 8.1

Illustrate the Apriori algorithm for frequent itemset {a,b,c,d} for a data set {a,b,c,d,e}.

8.4.1.1 Frequent Itemset Generation Using the Apriori Algorithm

Figure 8.4 shows the illustration of generation of the candidate itemsets and frequent itemsets with minimum support count = 3. Candidate itemset is the frequent itemset. Itemsets that appear in less than three transactions are eliminated

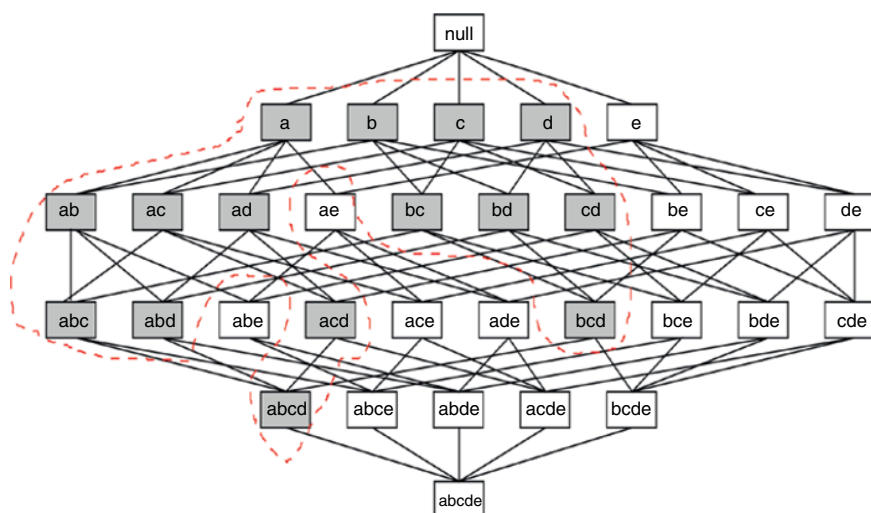


Figure 8.4 Apriori algorithm—frequent itemsets.

from candidate 1 itemset. Egg, Rice, Diaper, Jam, Chocolates appear in less than three transactions. In the next scan, candidate 2 itemsets are generated only with the itemsets that are frequent in the candidate 1 itemset since the Apriori algorithm states that supersets of the infrequent itemsets must also be infrequent. In candidate 2 itemsets {Milk, Beer}, {Bread, Butter}, {Bread, Beer}, {Butter, Cola}, {Cola, Beer} are eliminated since they appear in less than three transactions. With the rest of the frequent itemsets in candidate 2 itemset, the candidate itemset 3 is generated where the itemset {Milk, Bread, Cola} with support count 3 is found to be frequent.

Transaction_Id	Items
1	{Milk, Bread, Butter}
2	{Cola, Milk, Bread, Beer, egg, Rice}
3	{Bread, Milk, Diaper, Cola, Beer}
4	{Milk, Butter, Jam, Chocolates}
5	{Cola, Bread, Milk, Butter}
6	{Rice, egg, Diaper, Beer}

Database

Item	Support Count
Milk	5
Bread	4
Butter	3
Cola	3
Beer	3
Egg	2
Rice	2
Diaper	2
Jam	1
Chocolates	1

Candidate 1

Itemset	Count
{Milk, Bread}	4
{Milk, Butter}	3

Itemset	Count
{Milk, Cola}	3
{Milk, Beer}	2
{Bread, Butter}	2
{Bread, Cola}	3
{Bread, Beer}	2
{Butter, Cola}	1
{Cola, Beer}	2

Candidate 2

Itemset	Count
{Milk, Bread, Butter}	2
{Milk, Bread, Cola}	3
{Milk, Bread, Beer}	2
{Milk, Butter, Cola}	1
{Milk, Cola, Beer}	2
{Bread, Butter, Cola}	1
{Bread, Cola, Beer}	2

Candidate 3

8.4.2 The Eclat Algorithm—Equivalence Class Transformation Algorithm

The Equivalence Class Transformation (Eclat) algorithm uses a horizontal data layout in contrast to the Apriori algorithm, which uses a vertical data layout.

Transaction_Id	Itemset
1	{a, b, c}
2	{a,b, c, d, e}
3	{a, b, c, d, e}
4	{c, e}
5	{d, e}
6	{b, c, d, e}

Horizontal Data layout – Apriori Algorithm

Transaction_Id	Items
1	{Milk, Bread, Butter}
2	{Cola, Milk, Bread, Beer, egg, Rice}
3	{Bread, Milk, Diaper, Cola, Beer}
4	{Milk, Butter, Jam, Chocolates}
5	{Cola, Bread, Milk, Butter}
6	{Rice, egg, Diaper, Beer}

Database



Item	Support Count
Milk	5
Bread	4
Butter	3
Cola	3
Beer	3
Egg	2
Rice	2
Diaper	2
Jam	1
Chocolates	1

Candidate 1

Figure 8.5 Generation of the candidate itemsets and frequent itemsets with minimum support count = 3.



Itemset	Count
{Milk, Bread}	4
{Milk, Butter}	3
{Milk, Cola}	3
{Milk, Beer}	2
{Bread, Butter}	2
{Bread, Cola}	3
{Bread, Beer}	2
{Butter, Cola}	1
{Cola, Beer}	2

Candidate 2



Itemset	Count
{Milk, Bread, Butter}	2
{Milk, Bread, Cola}	3
{Milk, Bread, Beer}	2
{Milk, Butter, Cola}	1
{Milk, Cola, Beer}	2
{Bread, Butter, Cola}	1
{Bread, Cola, Beer}	2

Candidate 3

Figure 8.5 (Continued)

A	B	c	d	E
1	1	1	2	2
2	2	2	3	3
3	3	3	5	4
—	6	4	6	5
—	—	6	—	6

Vertical Data layout—Eclat Algorithm

Despite the Apriori algorithm being easy to understand and straightforward, it involves several scans of the database and generates huge candidate itemsets. The Equivalence Class Transformation algorithm is an algorithm based on an in-depth

Transaction_Id	Itemset
1	{a, b, c}
2	{a,b, c, d, e}
3	{a, b, c, d, e}
4	{c, e}
5	{d, e}
6	{b, c, d, e}

Horizontal Data layout–Apriori Algorithm

A	B	c	d	E
1	1	1	2	2
2	2	2	3	3
3	3	3	5	4
	6	4	6	5
		6		6

Vertical Data layout–Eclat Algorithm

Figure 8.6 Eclat algorithm illustration.

first search. The Eclat algorithm sets an intersection between the items, which improves the speed of support counting.

Figure 8.7 shows that intersecting itemset c and itemset will determine the support of the resulting itemset.

Figure 8.8 illustrates the frequent itemset generation based on the Eclat algorithm with minimum support count as 3. The transaction id's of a is {1, 2, 3} and b is {1, 2, 3, 6}. The support of ab can be determined by intersecting the transaction id's of a and b to obtain the transaction id of ab which is {1,2,3} and the corresponding support count is 3. Similarly, the support count of the rest of the itemsets is calculated and the frequent itemset is generated.

Exercise- Eclat Algorithm Implementation Using R

Frequent itemset mining can be implemented using the eclat() function. This algorithm uses intersection operations for equivalence class clustering and bottom-up lattice traversal.

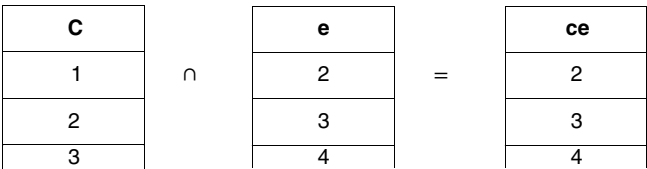


Figure 8.7 Intersection of two itemsets.

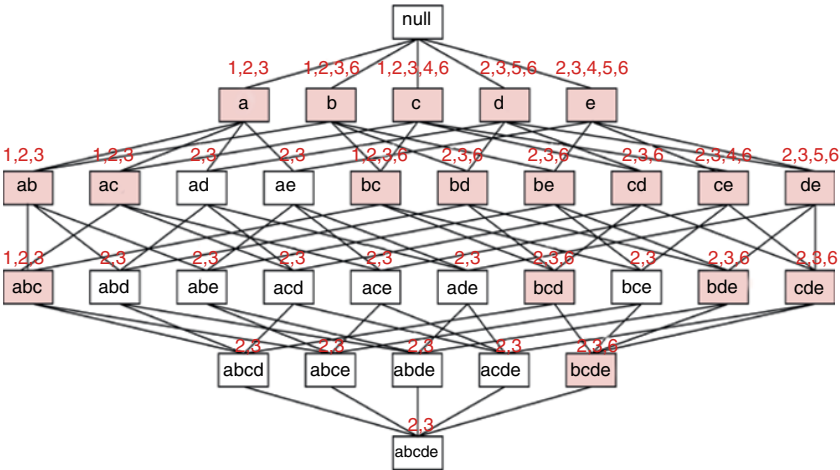


Figure 8.8 Eclat algorithm.

```
> frequentitemsets<-eclat(Groceries, parameter = list(supp = 0.1))
Eclat
```

parameter specification:

```
tidLists support minlen maxlen          target  ext
FALSE      0.1      1      10 frequent itemsets FALSE
```

algorithmic control:

```
sparse sort verbose
7      -2      TRUE
```

Absolute minimum support count: 983

create itemset ...

set transactions ... [169 item(s), 9835 transaction(s)] done [0.05s].

sorting and recoding items ... [8 item(s)] done [0.00s].

creating bit matrix ... [8 row(s), 9835 column(s)] done [0.00s].

writing ... [8 set(s)] done [0.00s].

Creating S4 object ... done [0.00s]

Sparse—a numeric threshold value for sparse representation, its default value is 7.

Sort – can have values 1,–1,0,2,–1.

- 1 implies ascending
- –1 implies descending
- 0 implies do not start
- 2 implies ascending
- –2 implies descending with transaction sum

Verbose—a logical value indicating if progress information is displayed.

Let us see the summary of frequent items generated using `summary()`. The most frequent items are tropical fruit, root vegetables, other vegetables, whole milk, and yogurt. A set of eight itemsets are generated, each of length 1. The summary of quality measures indicates the minimum and maximum support of the items generated.

```
> summary(frequentitemsets)
```

set of 8 itemsets

most frequent items:

```
tropical fruit  root vegetables  other vegetables  whole milk
              1              1              1              1
yogurt          (Other)
              1              3
```

element (itemset/transaction) length distribution:sizes

```

1
8
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
      1      1      1      1      1      1

```

summary of quality measures:

```

      support
Min.      :0.1049
1st Qu.   :0.1101
Median    :0.1569
Mean      :0.1589
3rd Qu.   :0.1863
Max.      :0.2555

```

includes transaction ID lists: FALSE

mining info:

```

      data ntransactions support
Groceries 9835 0.1

```

Let us inspect the frequent itemset generated with minimum support 0.1.

```

> inspect(frequentitemsets)
      items      support
[1] {whole milk} 0.2555160
[2] {other vegetables} 0.1934926
[3] {rolls/buns} 0.1839349
[4] {yogurt} 0.1395018
[5] {soda} 0.1743772
[6] {root vegetables} 0.1089985
[7] {tropical fruit} 0.1049314
[8] {bottled water} 0.1105236

```

8.4.3 The FP Growth Algorithm

The FP growth algorithm is another important frequent itemset mining method, which is used to generate frequent itemset without candidate itemset.

The FP growth algorithm is performed with several steps

Step 1: Find the frequency of occurrence.

With minimum support count as 3, find the frequency of occurrence of items in the database. For example, the item a is present in transaction 1, 2, 3, 6. Hence, the frequency of occurrence of A is 4. Similarly, calculate the frequency of occurrence of each item in the database. Table 8.9 shows the frequency of occurrence of each item.

Table 8.8 Database.

Transaction_id	Items
1	A, E, B, D
2	B, E, C, A, D
3	C, E, D, A
4	D, E, B
5	B, f
6	B,D
7	E, B, A
8	B, D, C

Table 8.9 Frequency of occurrence.

Items	Frequency
a	4
b	7
c	3
d	6
e	5
f	1

Table 8.10 Priority of the items.

Items	Priority
A	4
B	1
C	5
D	2
E	3

Step 2: Prioritize the items.

Prioritize the items according to the frequency of occurrence of each item. Item b has the highest number of occurrences, so it is given the highest priority as 1. Item f has the lowest occurrence, but it does not satisfy the minimum support requirement and hence it is dropped in Table 8.10. The item with highest frequency

of occurrence next to b is given the next highest priority, which is 2. Similarly, all the items are given priority according to their frequency of occurrences. Table 8.10 shows the priority of the items according to their frequency of occurrences.

Step 3: Order items according to the priority.

The items in each transaction are ordered according to its priority. For example, ordering the items in transaction 1 is done by placing item b with highest priority in the first place and after that d, e, and a, respectively. Table 8.10 shows the items ordered according to their priority. In transaction 5 f is dropped since it does not satisfy the minimum support threshold.

Transaction_id	Items	Ordered items
1	a, e, b, d	b, d, e, a
2	b, e, c, a, d	b, d, e, a, c
3	b, d, c	b, d, c
4	e, b, a	b, e, a
5	c, e, d, a	d, e, a, c
6	d, e, b	b, d, e
7	b	B
8	b, d	b, d

Step 4: Draw the FP tree.

Transaction 1:

The root node of all the FP trees is a null node. A tree is started with a null node, and each item of the transaction is attached one by one as shown in Figure 8.9a.

Transaction 2:

The FP tree for transaction 1 is updated by attaching the items of transaction 2. The items of transaction 2 are b, d, e, a, c. Since the previous transaction has the same order, without creating a new branch the same branch can be updated, increasing the count of each item. A new item c can be attached to the same branch as shown on Figure 8.9b below.

Transaction 3:

Transaction 3 has items b, d, c. Since there is no existing branch for the path b, d, c, a new branch is created starting from b, as shown in Figure 8.9c.

Transaction 4:

Transaction 4 has items b, e, a. Similar to transaction 3, a new branch is created, and items e and a are attached to item b, as shown in Figure 8.9d.

Transaction 5:

Transaction 5 has items d, e, a, c. The same branch is updated by increasing the count of the items d : 2 to d : 3, e : 2 to e : 3, a : 2 to a : 3, and c : 1 to c : 2, as shown in Figure 8.9e.

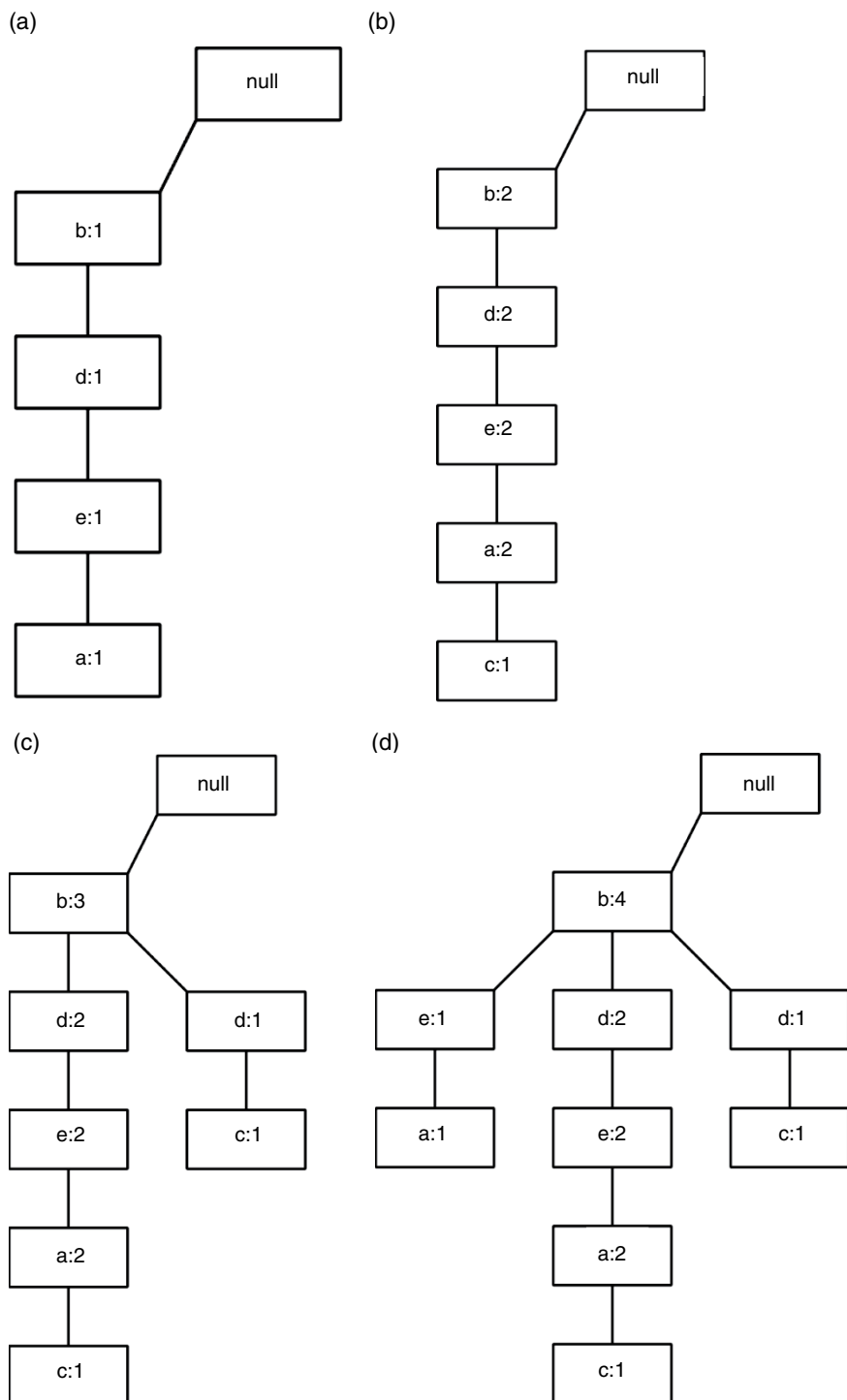


Figure 8.9 (a) FP tree for transaction 1. (b) FP tree for transaction 2. (c) FP tree for transaction 3. (d) FP tree for transaction 4. (e) FP tree for transaction 5. (f) FP tree for transaction 6, 7, 8.

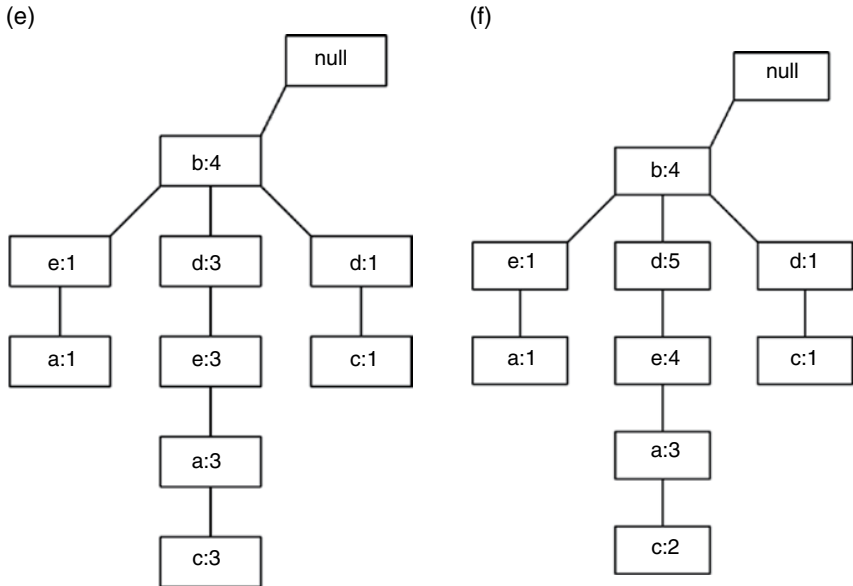


Figure 8.9 (Continued)

Transaction 6, 7, 8:

Transaction 6 has items b, d, e. The items can be updated by increasing the count from b : 4 to b : 5, d : 3 to d : 4, and e : 3 to e : 4.

Transaction 7 has item b, and hence b will be increased from b : 5 to b : 6; similarly, transaction 7 has b, d, and items b and d will be increased from b : 6 to b : 7 and d4: to d : 5, as shown in Figure 8.9e.

8.5 Maximal and Closed Frequent Itemset

A Frequent itemset I, is called maximal frequent itemset when none of the immediate supersets of the itemset is frequent.

A frequent itemset I, is called closed frequent itemset if it is closed and its support count is equal to or greater than the MinSup. An itemset is said to be closed if there is no superset with the same support count as the original itemset.

Table 8.11 shows a transaction and the corresponding itemset in the transaction. Table 8.12 shows the support count of the itemset and its corresponding frequency. From the table it is evident that only the items that are frequent are closed and only the items which are closed are maximal, i.e., all the items that are maximal are closed and all the items that are closed are frequent. But all the items

Table 8.11 Itemset in a transaction.

Transaction Id	Itemset in the transaction
1	abc
2	abcd
3	abd
4	acde
5	ce

Table 8.12 Maximal/closed frequent itemset.

Item	Support count	Frequency for S = 2	Maximal/Closed
a	4	Frequent	Closed
b	3	Frequent	–
c	4	Frequent	Closed
d	3	Frequent	–
e	2	Frequent	–
ab	3	Frequent	Closed
ac	3	Frequent	Closed
ad	3	Frequent	Closed
ae	1	infrequent	–
bc	2	Frequent	–
bd	2	Frequent	–
be	0	infrequent	–
cd	2	Frequent	–
ce	2	Frequent	Maximal and closed
de	1	infrequent	–
abc	2	Frequent	Maximal and closed
abd	2	Frequent	Maximal and closed
abe	0	infrequent	–
acd	2	Frequent	–
ace	1	infrequent	–
ade	1	infrequent	–
bcd	1	infrequent	–

Table 8.12 (Continued)

Item	Support count	Frequency for S = 2	Maximal/Closed
bce	0	infrequent	–
bde	0	infrequent	–
cde	1	infrequent	–
abcd	1	infrequent	–
abce	0	infrequent	–
Abde	0	infrequent	–
acde	1	infrequent	–
bcde	0	infrequent	–
abcde	0	infrequent	–

that are frequent are not closed and all the items that are closed are not maximal, which means all the closed itemsets form a subset of frequent itemsets and all the maximal itemsets form a subset of the closed itemsets.

Figure 8.10 shows the itemset and their corresponding support count. It gives a clear picture of the immediate superset of the itemset and their frequency. Figure 8.11 shows the itemsets that are closed and those that are both closed and maximal.

Figure 8.12 shows that both maximal frequent itemset and closed frequent itemset are subsets of frequent itemsets. Further, every maximal frequent itemset is a subset of a closed frequent itemset.

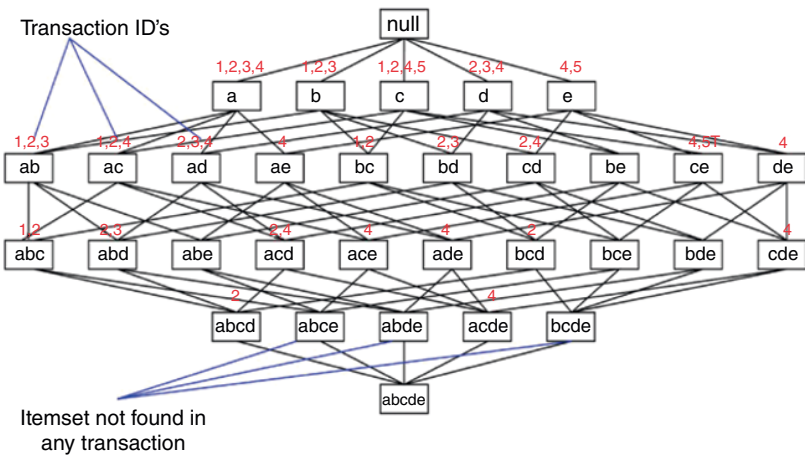


Figure 8.10 Itemset and their corresponding support count.

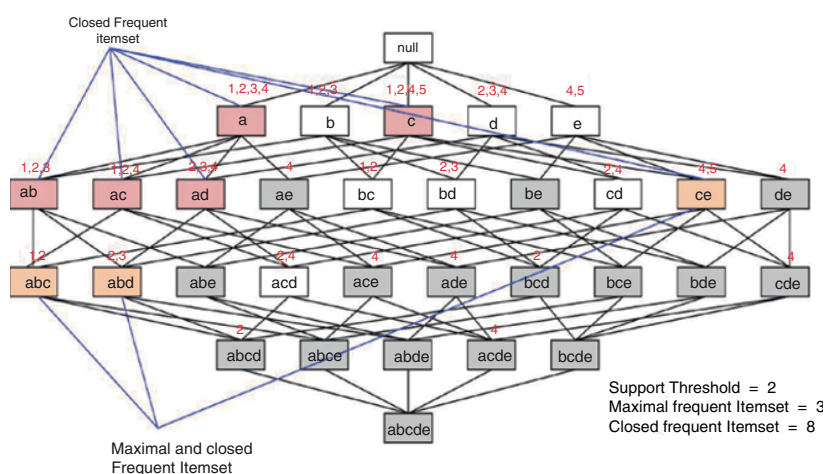


Figure 8.11 Maximal and closed frequent itemset.

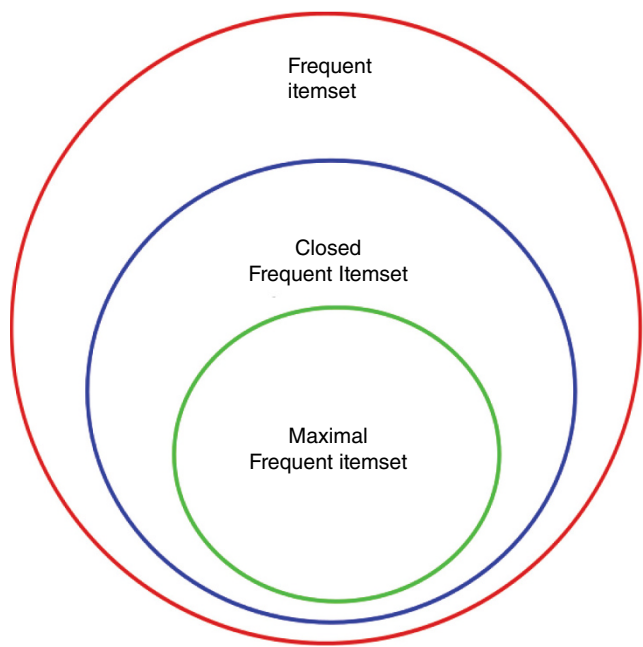


Figure 8.12 Maximal and closed frequent itemset – subset of frequent itemset.

Exercise 8.2

Determine the maximal and closed itemset for the given itemset in a transaction.

Transaction Id	Itemset in the transaction
1	abc
2	Abde
3	Bce
4	Bcde
5	De

8.6 Mining Maximal Frequent Itemsets: the GenMax Algorithm

The GenMax Algorithm is a highly efficient algorithm to determine the exact maximal frequent itemsets. It is basically an algorithm based on backtracking search for mining maximal frequent itemset. Maximality checking, that is, eliminating non-maximal itemsets, is performed by progressive focusing, and fast frequency computation is performed by diffset propagation.

Let $I = \{i_1, i_2, i_3, \dots, i_m\}$ be the set of distinct items and D be the database of transactions with a unique transaction identifier (tid) for each transaction. The

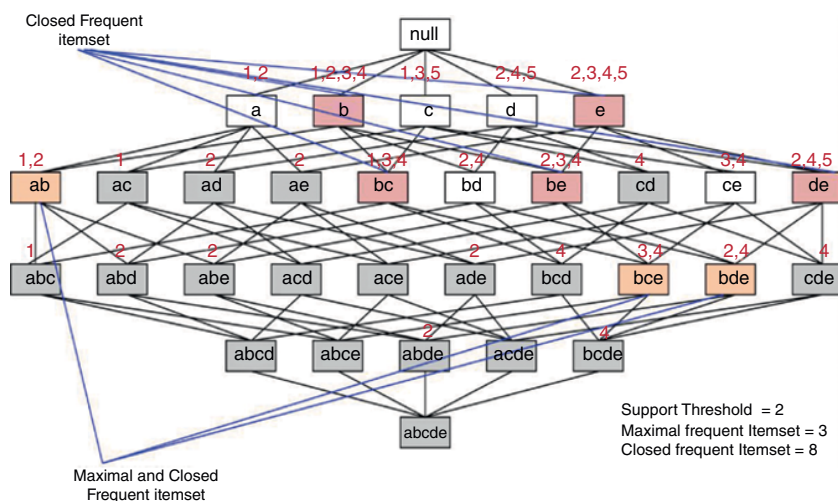


Figure 8.13 Maximal and closed frequent itemset – subset of frequent itemset.

transaction identifier tid is denoted by $T = \{t_1, t_2, t_3, \dots, t_n\}$ for n transactions. Let $X \subseteq I$ be an itemset. The set $t(X) \subseteq T$ that has all the transaction ids with X as subset is known the tidset of X . For example, let $X = \{A, B, C\}$ be the itemset and when X is the subset of transactions 2,3,4,5 then $t(x) = \{2,3,4,5\}$ is the tidset of itemset X . The support count $\sigma(x) = |t(x)|$, is the number of transactions in which the itemset occurs as subset. An itemset is said to be maximally frequent if it does not have any superset that is frequent. A frequent itemset is a subset of maximal frequent itemset.

Let us consider an example with items $I = \{A, B, C, D, E\}$ and $T = \{1, 2, 3, 4, 5, 6\}$. Table 8.14 shows frequent itemsets with minimum support count 3. Table 8.15 shows frequent itemsets with the transaction list in which the itemsets occur and the corresponding support count.

Figure 8.14 shows implementation of the GenMax algorithm. The frequent itemsets that are extended from A are AB , AD , and AE . The next extension of AB which is frequent is ABD . Since it has no further extensions that are frequent, ABD is added to set of maximal frequent itemsets. The search backtracks one level and processes AD . The next extension of AD that is frequent is ADE . Since it has no further extensions that are frequent, ADE is added to the set of maximal frequent itemsets. Now, all maximal itemsets that are the extensions of A are identified.

Table 8.13 Transaction database.

Tid	Itemset
1	ABCDE
2	ADE
3	ABD
4	ACDE
5	BCDE
6	ABDE

Table 8.14 Frequent itemsets with minsup = 3.

Support	Itemsets
6	D
5	A,E,AD,DE,
4	B,BD,AE,ADE,
3	C,AB,ABD,BE,CD,CE,BDE,CDE

Table 8.15 Frequent itemsets with tidset.

Frequent Itemset	Tidset	Support Count
A	12 346	5
B	1356	4
C	145	3
D	123456	6
E	12456	5
AB	136	3
AD	12 346	5
AE	1246	4
BD	1356	4
BE	156	3
CD	145	3
CE	145	3
ABD	136	3
ADE	1245	4
BDE	156	3
CDE	145	3

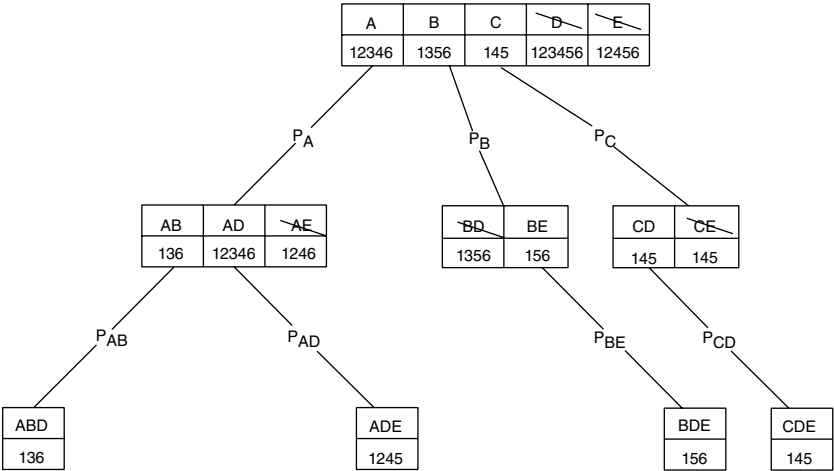


Figure 8.14 GenMax Algorithm implementation.

So the next step is to process branch B. BD and BE are the frequent itemsets. Since BD is already contained in ABD, which is identified as maximal frequent itemset, BD is pruned. The extension of BD that is frequent is BDE. Since BDE has no further extension that is frequent, BDE is added to the maximal frequent itemset. Similarly, branch C is processed where the frequent itemsets which are extensions of C are CD and CE. The extension of CD that is frequent is CDE, and since it has no further extensions that are frequent, CDE is added to the set of maximal frequent itemsets. Since CE is already contained in CE, it is pruned. Subsequently, all other branches are contained in one of the maximal frequent itemsets, and hence D and E are pruned.

8.7 Mining Closed Frequent Itemsets: the Charm Algorithm

Charm is an efficient algorithm for mining the set of all closed frequent itemsets. Instead of enumerating non-closed subsets, it skips many levels to quickly locate closed frequent itemsets. The fundamental operation used in this algorithm is the union of two itemsets and the intersection of the corresponding transaction lists.

The basic rules of charm algorithm are:

- i) If $t(x_1) = t(x_2)$, then $t(x_1 \cup x_2) = t(x_1) \cap t(x_2) = t(x_1) = t(x_2)$. Thus every occurrence of x_1 can be replaced with $x_1 \cup x_2$, and x_2 can be removed from further consideration. This is because the closure of x_2 is identical to the closure of $x_1 \cup x_2$.
- ii) If $t(x_1) \subset t(x_2)$, then $t(x_1 \cup x_2) = t(x_1) \cap t(x_2) = t(x_1) \neq t(x_2)$. Thus every occurrence of x_1 can be replaced with $x_1 \cup x_2$, because whenever x_1 occurs then x_2 will always occur. Since $t(x_1) \neq t(x_2)$, x_2 cannot be removed from further consideration as it has a different closure.
- iii) If $t(x_1) \supset t(x_2)$, then $t(x_1 \cup x_2) = t(x_1) \cap t(x_2) = t(x_2) \neq t(x_1)$. Here every occurrence of x_1 can be replaced with $x_1 \cup x_2$ because if x_2 occurs in any transaction then x_1 will always occur. Since $t(x_2) \neq t(x_1)$, x_1 cannot be removed from further consideration as it has a different closure.
- iv) If $t(x_1) \neq t(x_2)$, $t(x_1 \cup x_2) = t(x_1) \cap t(x_2) \neq t(x_2) \neq t(x_1)$. Here neither x_1 nor x_2 can be eliminated as both lead to different closure.

8.8 CHARM Algorithm Implementation

Consider the transaction database below, shown in Table 8.16, to implement the CHARM algorithm for mining closed frequent itemsets.

Let the minimum support be 3. Table 8.17 shows the itemsets that are frequent and their corresponding support count.

Table 8.16 Transaction database.

Transaction	Itemset
1	ABDE
2	BCE
3	ABDE
4	ABCE
5	ABCDE
6	BCD

Table 8.17 Frequent Itemset with minsup = 3.

Support	Itemset
6	B
5	E, BE
4	A, C, D, AB, AE, BD, ABE, BC
3	AD, CE, ABD, BCE, ABDE, BDE

Table 8.18 shows the transactions in which the frequent itemsets occur and their corresponding support counts.

Figure 8.15 shows the implementation of the CHARM algorithm. Initially the children of A are generated by combining with other items. When x_1 with its transaction $t(x_1)$ is paired with x_2 and $t(x_2)$, the resulting itemset and tidset pair will be $x_1 \cup x_2$ and $t(x_1) \cap t(x_2)$. In other words, the union of itemsets and intersection of tidset has to be performed. When A is extended with rule number (ii) is true, i.e., $t(A) = 1345 \subseteq 123456 = t(B)$. Thus, A can be replaced with AB. Combining A with C produces ABC, which is infrequent; hence, it is pruned. Combination with D produces ABD with tidset 135. Here rule (iv) holds true, and hence none of them are pruned. When A is combined with E, $t(A) \subseteq t(E)$, so according to rule (ii) all unpruned occurrences of A are replaced with AE. Thus, AB is replaced by ABE, and ABD is replaced by ABDE. The branch A is completely processed, and processing of branch B is started.

When B is combined with C, property 3 becomes true, i.e., $t(B) \supset t(C)$. Wherever C occurs, B always occurs. Thus, C can be removed from further consideration, and hence C is pruned. BC replaces C. D and E are pruned in similar fashion and replaced by BD and BE as children of B. Next, BC node is processed further: combining with D generates an infrequent itemset BCD; hence, it is pruned. Combining BC with E generates BCE with tidset 245, where rule (iv) holds true; hence,

Table 8.18 Tidset of the frequent itemset.

Frequent Itemset	Tidset	Support
A	1345	4
B	123456	6
C	2456	4
D	1356	4
E	12 345	5
AB	1345	4
AD	1345	4
AE	1345	4
BC	2456	4
BD	1356	4
BE	12 345	5
CE	245	3
ABD	135	3
ABE	1345	4
BCE	245	3
BDE	135	3
ABDE	135	3

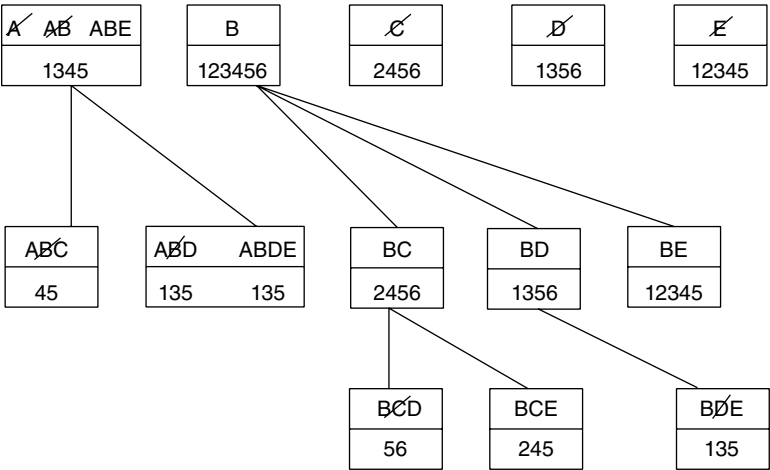


Figure 8.15 CHARM algorithm implementation.

nothing can be pruned. Combining BD with E, BDE with tidset 135 will be generated. BDE is removed since it is contained in ABDE with same tidset 135.

8.9 Data Mining Methods

The large volume of data collected by the organizations is of no great benefit until the raw data is converted into useful information. Once the data is converted into information, it must be analyzed using data analysis techniques to support decision-making. Data mining is the method of discovering the underlying pattern in large data sets to establish relationships and to predict outcomes through data analysis. Data mining is also known as knowledge discovery or knowledge mining. Data mining tools are used to predict future trends and behavior, which allows organizations to make knowledge-driven decisions. Data mining techniques answer business questions that were traditionally time consuming to resolve. Figure 8.16 shows various techniques for knowledge discovery in data mining. Various applications of data mining are:

- **Marketing**—To gather comprehensive data about the customers, to target their product to the right customer. For example, by knowing the items in a

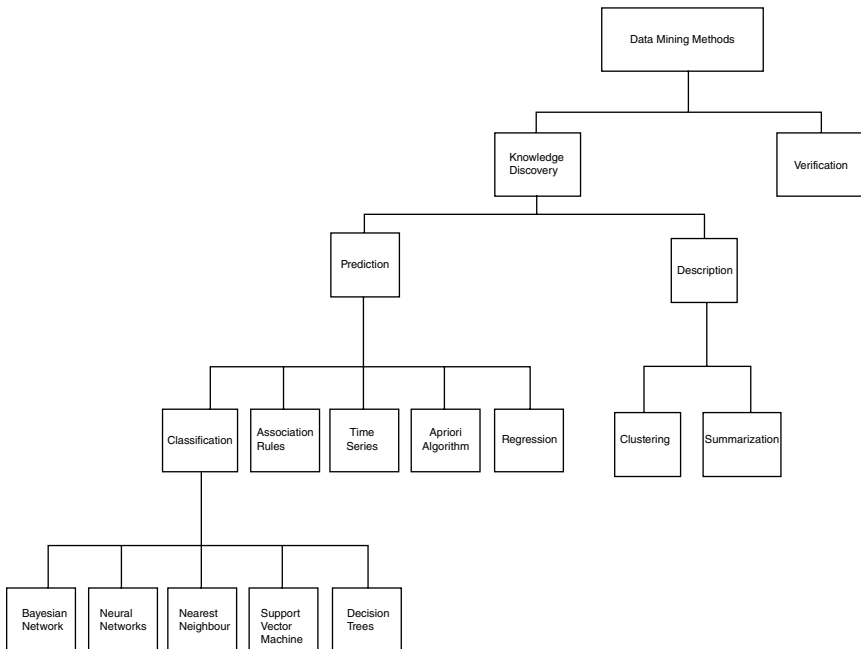


Figure 8.16 Data mining methods.

customer's shopping cart, it can be analyzed if the customer is likely to be expecting a baby so as to begin targeting promotions for muslin clothes, nappies, and other baby care products.

- **E-Commerce**—E-commerce sites such as eBay and Amazon use data mining techniques to cross-sell and upsell their products. Based on the products viewed by the customers they are provided suggestions to buy related products. Tags such as “Frequently bought together,” “Customers who viewed this item also viewed” can be found in the e-commerce websites to cross-sell and upsell their products.
- **Retail**—Retailers segment their existing customers into three categories, namely, recency, frequency, and monetary (RFM) based on their purchasing behavior. RFM analysis is a marketing approach used to determine the customer value. This customer analysis technique examines how recently they have purchased (recency), how often they purchase (frequency), and how much they spend (monetary). Based on the purchasing habit of the customers, retailers offer different deals to different customers to encourage them to shop.

8.10 Prediction

Prediction is used to determine an ordered-valued or a continuous-valued function. For example, an analyst is to predict how much a customer will shop for when a sale is put up in the company. Here a model or predictor is built to predict the value. Various prediction algorithms are shown in Figure 8.16.

Applications of prediction are:

- Loan approval;
- To diagnose if a tumor is benign or malignant;
- To detect if a transaction is fraudulent;
- Customer churn.

8.10.1 Classification Techniques

Classification is the most widely used technique in data mining to classify or group the data among various classes. Classification techniques are frequently used to identify the group or class to which a particular data item belongs. For example, classification may be used to predict the weather of the day and classify them into “sunny,” “cloudy,” or a “rainy” day. Initially the classification model is built, which contains a set of predetermined classes. Each data item in the data is assumed to belong to a predetermined class. The set of data items used to build model is called the training set. The constructed model is then used to classify the

unknown objects. The known labels from the new data item are compared with the labels of the training set to determine the class label of the unknown data item. There are several algorithms in data mining that are used to classify the data. Some of the important algorithms are:

- Decision tree classifier;
- Nearest neighbor classifier;
- Bayesian classifier;
- Support vector machines;
- Artificial neural networks;
- Ensemble classifier;
- Rule based classifier.

8.10.1.1 Bayesian Network

A graphical model represents the joint probability distribution of random variables in a compact way. There are two major types of graphical models, namely, directed and undirected. A commonly used directed graphical model is called Bayesian network. A Bayesian network is a powerful reasoning and knowledge representation mechanism for an uncertain domain. The nodes of the Bayesian network represent the random variables from the domain, and the edges between the nodes encode the probabilistic relationship between the variables. Directed arcs or links are used to connect a pair of nodes.

The Bayesian classification technique is named after Thomas Bayes, who formulated Bayes theorem. It is a supervised learning method for classification. It can be used to solve diagnostic as well as predictive problems. Some of the applications of Bayesian classification technique are:

- Naïve Bayes text classification;
- Spam filtering in emails.

8.11 Important Terms Used in Bayesian Network

8.11.1 Random Variable

A random variable is a variable whose values are the outcome of a random phenomenon. For example, tossing a coin is a random phenomenon, and the possible outcomes are heads or tails. Let the values of heads be assigned “0” and tails be assigned “1” and let the random variable be “X.” When the outcome of the event is a tail then the random variable X will be assigned “1.” Random variables can be discrete or continuous. A discrete random variable has only a finite number of values. A random variable that represents the outcome of tossing a coin can have

only two values, a head or a tail. A continuous random variable can take an infinite number of values. A random variable that represents the speed of a car can take an infinite number of values.

8.11.2 Probability Distribution

The random variable X that represents the outcome of any event can be assigned some value for each of the possible outcomes of X . The value assigned to the outcome of a random variable indicates how probable it is, and it is known as the probability distribution $P(X)$ of the random variable. For example, let X be the random variable representing the outcome of tossing a coin. It can take the values {head, tail}. $P(X)$ represents the probability distribution of random variable X . If $X = \text{tail}$, then $P(X = \text{tail}) = 0.5$ and $X = \text{head}$, then $P(X = \text{head}) = 0.5$ which means, when tossing a coin there is 50% chance for head to occur and 50% chance for tail to occur.

8.11.3 Joint Probability Distribution

The joint probability distribution is a probability distribution over a combination of attributes. For example, selecting a restaurant depends on various attributes such as quality and taste of the food, cost of the food, locality of the restaurant, size of the restaurant, and much more. A probability distribution over these attributes is called joint probability distribution. Let the random variable for the quality and taste of the food be T and the random variable for cost of the food be C . T can have three possible outcomes {good, average, bad}, and C can have two possible outcomes {high, low}. Generally, if the taste and quality of the food is good, then the cost of the food will also be high; conversely, if the taste and quality of the food is low, the cost of the food will also be low. Hence, the cost and quality of the food are dependent variables; thus, the change in one quantity affects the other. So the joint probability distribution for taste and cost $P(T, C)$ can have the possible combinations of the outcomes $P(T = \text{good}, C = \text{high})$, which represents the probability of good food with high cost, and $P(T = \text{bad}, C = \text{low})$, which represents the probability of bad food with low cost. The variables or the attributes may not always depend on each other; for example, there is no relation between the size of the restaurant and the quality of the food.

8.11.4 Conditional Probability

A conditional probability of an event Y is the probability that that event will occur with the knowledge of the event X , which has already occurred.

The probability of an event A is represented by $P(A)$. For example, the probability of occurrence of “5” when rolling a die is $1/6$ since the sample space has 6 possible outcomes that have equal probability to occur. Similarly, if we toss a fair coin three times, the probability of head occurring at least twice is $2/4$. The sample space of tossing three coins is {HHH, HHT, HTH, THH, TTT, TTH, THT, HTT}. The number events where head occurred is 4, and the total number of events is 8, and hence the probability is $4/8 = 2/4$.

Consider an example where there are eight balls in a bag out of which three are black and five are red. The probability of selecting a black ball out of the bag is $3/8$. Now, let the balls be split into two separate bags A and B. Bag A has two black and three red balls and bag B has one black and two red balls. Now the conditional probability is the probability of selecting a black ball from bag B which is represented by $P(\text{black} | \text{bag B})$ read out as “the probability of black given bag B.”

$P(\text{black} | \text{bag B}) = P(\text{black and bag B})/P(\text{bag B})$,
where, $P(\text{black and bag B}) = 1/8$,
since the total number of balls in both the bags is eight and the number of black balls in bag B is one.

$P(\text{bag B}) = 1/2$ as there are two bags and one bag selected from it.

So, $P(\text{black} | \text{bag B}) = P(\text{black and bag B})/P(\text{bag B})$
 $= (1/8)/(1/2)$
 $= 1/4$

Thus, the formal definition of conditional probability is “Conditional probability of an event B in relationship to an event A is the probability that event B occurs given that event A has already occurred.” The notation for conditional probability is $P(B|A)$, read as *the probability of B given A*.

Exercise Problem:

Conditional probability:

In an exam with two subjects, English and mathematics, 25% of the total number of students passed both subjects, and 42% of the total number of students passed English. What percent of those who passed English also passed mathematics?

Answer:

$$\begin{aligned} P(B|A) &= \frac{P(A \text{ and } B)}{P(A)} \\ &= \frac{P(A) \cdot P(B|A)}{P(A)} \\ &= .25 / .42 = 6 \end{aligned}$$

Thus, 60% of the students passed both the subjects.

8.11.5 Independence

Two events are said to be independent if the knowledge of one event that has occurred already does not affect the probability of occurrence of the other event. This is represented by:

A is independent of B iff $P(A|B) = P(A)$.

That is, the knowledge of the event Y that has occurred does not affect the probability of event X.

8.11.6 Bayes Rule

Bayes rule is named after Thomas Bayes. It connects to the conditional probability inversely. The probability of the events A and B, $P(A \cap B)$, occurring is the probability of A ($P(A)$), times the probability of B given that the event A has occurred $P(B|A)$.

$$P(A \cap B) = P(A)P(B|A) \quad (8.1)$$

Similarly, the probability of the events A and B, $P(A \cap B)$, occurring is the probability of B ($P(B)$), times the probability of A given that the event B has occurred $P(A|B)$.

$$P(A \cap B) = P(B)P(A|B) \quad (8.2)$$

Equating RHS of Eqs. (8.1) and (8.2),

$$\begin{aligned} P(B)P(A|B) &= P(A)P(B|A) \\ P(A|B) &= \frac{P(A \text{ and } B)}{P(A)} \quad , \\ &= \frac{P(A).P(B|A)}{P(B)} \end{aligned}$$

where $P(A)$ and $P(B)$ are the probabilities of events A and B, respectively, and $P(B|A)$ is the probability of B given A. Here, A represents the hypothesis, and B represents observed evidence. Hence, the formula can be rewritten as:

$$P(H|E) = \frac{P(H)P(E|H)}{P(E)}$$

The posterior probability $P(H|E)$ of a random event is the conditional probability assigned after getting relevant evidence. The prior probability $P(H)$ of a random event is the probability of the event computed before the evidence is taken

into account. The likelihood ratio is the factor that relates $P(E)$ and $P(E|H)$, that is, $\frac{P(E|H)}{P(E)}$.

If a single card is drawn from a deck of playing cards, the probability that the card drawn is a queen is $\frac{4}{52}$, i.e., $P(Queen) = \frac{4}{52} = \frac{1}{13}$. If evidence is provided that the single card drawn is a face card, then $P(Queen | Face)$ the posterior probability can be calculated using Bayes theorem,

$$P(Queen|Face) = \frac{P(Face|Queen)P(Queen)}{P(Face)} \quad (8.3)$$

Since every queen is also a face card, the probability $P(Face | Queen) = 1$. In each suit there are three face cards, Jack, king, and the queen, and there are 4 suits, so the total number of face cards is 12. The probability that card drawn is a face card is, $P(Face) = \frac{12}{52} = \frac{3}{13}$. Substituting the values in Eq. (8.3) gives,

$$\begin{aligned} P(Queen|Face) &= \frac{1 * \frac{1}{13}}{\frac{3}{13}} \\ &= \frac{1}{13} * \frac{13}{3} \\ &= \frac{1}{3} \end{aligned}$$

8.11.6.1 K-Nearest Neighbor Algorithm

Nearest neighbor algorithm is the simplest of all existing machine learning algorithms. It is used for classification and regression and is an instance-based algorithm where a training data set is given, based on which the new input data may be classified by simply comparing with the data point in the training data set.

To demonstrate the K-nearest neighbor algorithm, let us consider the example in the Figure 8.17 to classifying a new data where several known data points exist.

The new data is represented by a circle that should be classified either as rectangle or a star based on the k-nearest neighbor technique. Let us evaluate the outcome of k-nearest neighbor with 1-nearest neighbor, i.e., $k = 1$, and is represented by the innermost circle. It is evident that the new outcome of the new data point will be a star as the nearest neighbor to it is star. Now let us evaluate the outcome of 3-nearest neighbor, i.e., with $k = 3$, which is represented by the dotted circle.

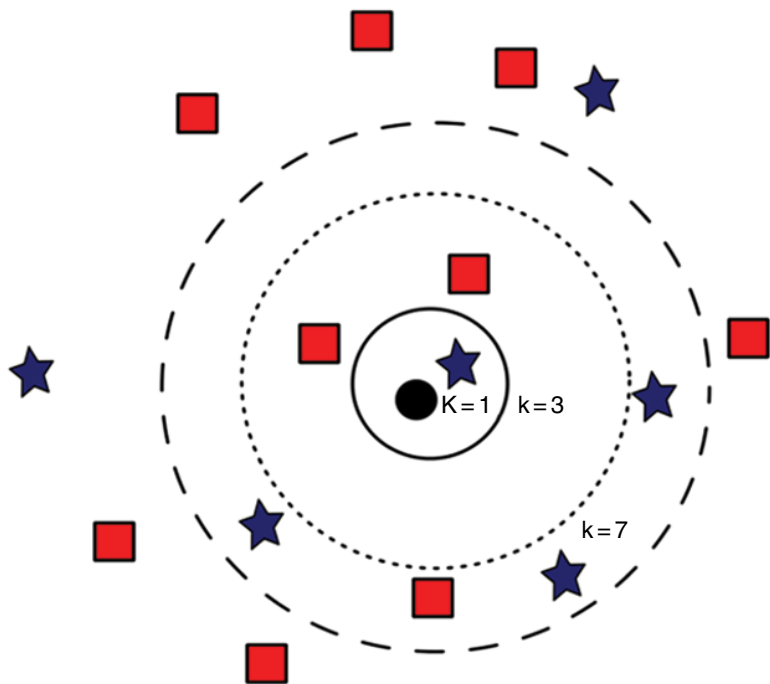


Figure 8.17 K-Nearest neighbor – classification.

Here the outcome will be a square as the number of squares is greater than the number of stars. Evaluation of 7-nearest neighbor with $k = 7$, which is represented by dashed circle, will result in a star as the number of stars within the circle is four while the number of squares is three. Classification is not possible if the number of squares and the number of stars are equal for a given k .

Regression is the method of predicting the outcome of a dependent variable with the given independent variable. In the Figure 8.18 where a set of (x,y) points are given, the k -nearest neighbor technique is used to predict the outcome of X . To predict the outcome of 1-nearest neighbor where $k = 1$, the point closest to X is located. The outcome will be (x_4, y_4) , i.e., $Y = y_4$. Similarly, for $k = 2$ the nearest neighbor will be the average of y_3 and y_4 . Thus, the outcome of the dependent variable is predicted by taking the average of the nearest neighbors.

8.11.6.1.1 The Distance Metric Performing the k -nearest neighbor algorithm requires the analysts to make two crucial decisions, namely, determining the value of k and determining the similarity measure. The similarity is determined by the mathematically calculated distance metric, i.e., the distance has to be measured between the new data points and the data points that already

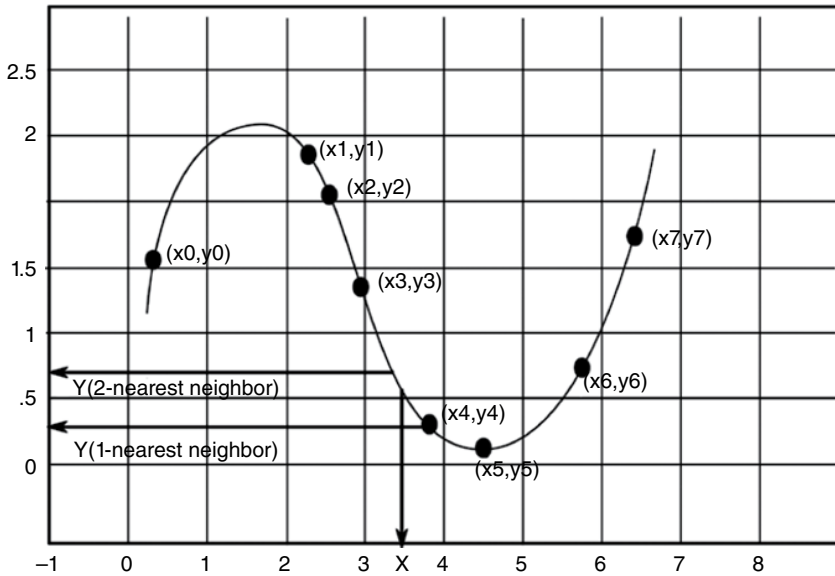


Figure 8.18 k-nearest neighbor – regression.

exists in the sample. The distance is measured using distance measurement methods, namely, Euclidean and Manhattan distance.

8.11.6.1.2 The Parameter Selection – Cross Validation The value of k is determined using a technique called cross-validation. The value of k is chosen to minimize the prediction errors. The original set of data is divided into a training set T and a validation set V . The objects in the training set are used as neighbors and the objects in the validation set are used as objects that need to be classified. The average of the data in V is taken to determine the prediction error. This method is extended to cross-validate all of the observations in the original set of data. V -fold cross validation technique is adopted where the original data set is divided into V number of subsets where the V -th set is used as the validation set and $V-1$ sets are used as the training set and the error is evaluated. The procedure is repeated until all the subsets are tested against remaining $V-1$ sets. Once the V numbers of cycles is completed the computed errors are accumulated. The k value that yields the smallest error values is chosen as the optimal k value.

8.11.6.2 Decision Tree Classifier

A decision tree is a method of classification in machine learning that is represented by a tree-like graph with nodes connected by branches starting from the root node and that extends until it reaches the leaf node to terminate. The root

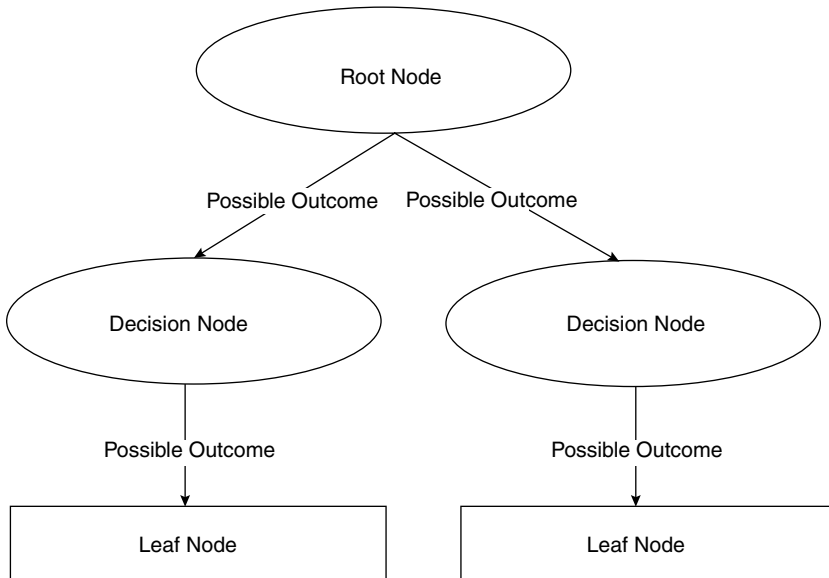


Figure 8.19 Decision tree diagram.

node is placed at the beginning of the decision tree diagram. The attributes are tested in each node, and the possible outcome of the test results are represented in the branches. Each branch will then connect to another decision node or it will terminate in a leaf node. Figure 8.19 shows a basic decision tree diagram.

A simple scenario may be considered to better understand the flow of a decision tree diagram. In Figure 8.20 a scenario is considered where a decision is made based on the day of a week.

- If it is a weekday then go to the office.
(Or)
- If it is a weekend and it is a sunny day and you need comfort, then go to watch movie sitting in the box.
(Or)
- If it is a weekend and it is a sunny day and you do not need comfort, then go to watch movie sitting in the first class.
(Or)
- If it is a weekend and it is a windy day and you need comfort, then go shopping by car.
(Or)
- If it is a weekend and it is a windy day and you do not need comfort, then go shopping by bus.
(Or)
- If it is a weekend and it is rainy, then stay at home.

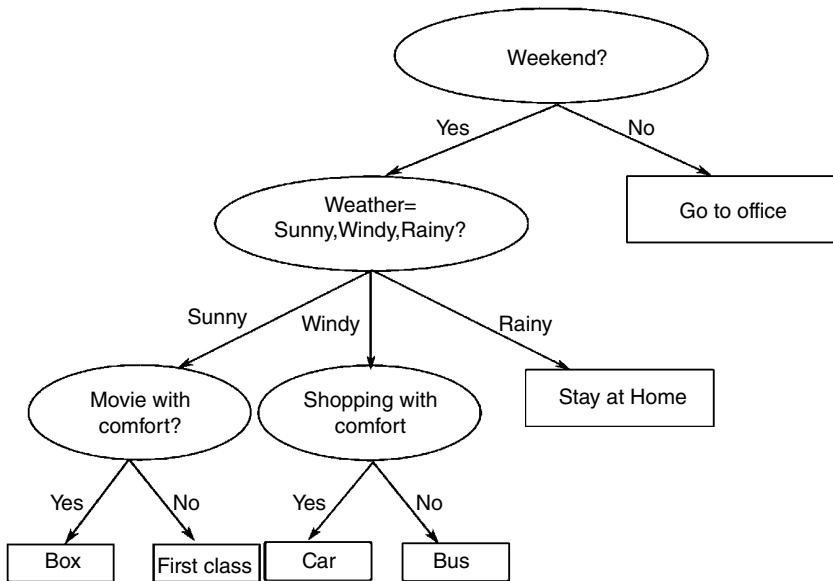


Figure 8.20 Decision tree – Weekend plan.

8.12 Density Based Clustering Algorithm

If points are distributed in space, the clustering concept suggests that there will be areas in the space where the points will be clustered with high density and also areas with low density clusters, which may be spherical or non-spherical. Several techniques have been developed to find clusters that are spherical and non-spherical. The popular approach to discover non-spherical shape clusters is the density-based clustering algorithm. A representative method of density-based clustering algorithm is Density Based Spatial Clustering of Applications with Noise (DBSCAN), which is discussed in the section below.

8.13 DBSCAN

DBSCAN is one of the most commonly used density-based clustering algorithms. The main objective of the density-based clustering approach is to find high dense regions in the space where the data points are distributed. The density of a data point can be measured by the number of data points closer to it. DBSCAN finds the objects or the data points that have a dense neighborhood. The object or the point with dense neighborhood is called the core object or the core point. The data point and their neighborhood are connected together to form dense clusters. The distance between two points in a cluster is controlled by a parameter called

epsilon (ϵ). No two points in a cluster should have a distance greater than epsilon. The major advantage of using epsilon parameter is that outliers can be easily eliminated. Thus, a point lying in a low density area will be classified as outlier.

The density can be measured with the number of objects in the neighborhood. The greater the number of objects in the neighborhood, the denser is the cluster. There is minimum threshold for a region to be identified as dense. This parameter is specified by the user and is called MinPts. A point is defined as the core object if the neighborhood of the object has at least the MinPts.

Given a set of objects, all the core objects can be identified with the epsilon ϵ and MinPts. Thus, clustering is performed by identifying the core objects and their neighborhood. The core objects and their neighborhood together form a dense region, which is the cluster.

DBSCAN uses the concept of density connectivity and density reachability. A point p is said to be in density reachability from a point q if p is within epsilon from point q and q has MinPts within the epsilon distance. Points p and q are said to be in density connectivity if there exists a point r which has the MinPts and the points p and q are within the epsilon distance. This is a chain process. So if point q is the neighbor of point r , point r is the neighbor of point s , point s is the neighbor of point t , and t in turn is the neighbor of point p , then point p is the neighbor of point q .

Figure 8.21a shows points distributed in space. The two parameters epsilon and MinPts are chosen to be 1 and 4, respectively. Epsilon is a positive number and MinPts is a natural number. A point is arbitrarily selected, and if the number of points is more than MinPts within epsilon distance from the selected point, then all the points are considered to be in that cluster. The cluster is grown recursively by choosing a new point and checking if they have points more than the MinPts within the epsilon. And a new arbitrary point is selected and the same process is repeated. There may be points that do not belong to any cluster, and such points are called noise points.

Figure 8.21c shows the DBSCAN algorithm performed on the same set of data points but with different values of the epsilon and MinPts parameters. Here epsilon is taken as 1.20 and MinPts as 3. A larger number of clusters are identified as the MinPts is reduced from 4 to 3 and the epsilon value is increased from 1.0 to 1.2, so the points that are little farther apart as compared to previous iteration also will be considered in the cluster.

8.14 Kernel Density Estimation

The major drawback of the DBSCAN algorithm is that the density of the cluster varies greatly with the change in radius parameter epsilon. To overcome this drawback the Kernel Density Estimation is used. Kernel Density Estimation is a non-parametric approach.

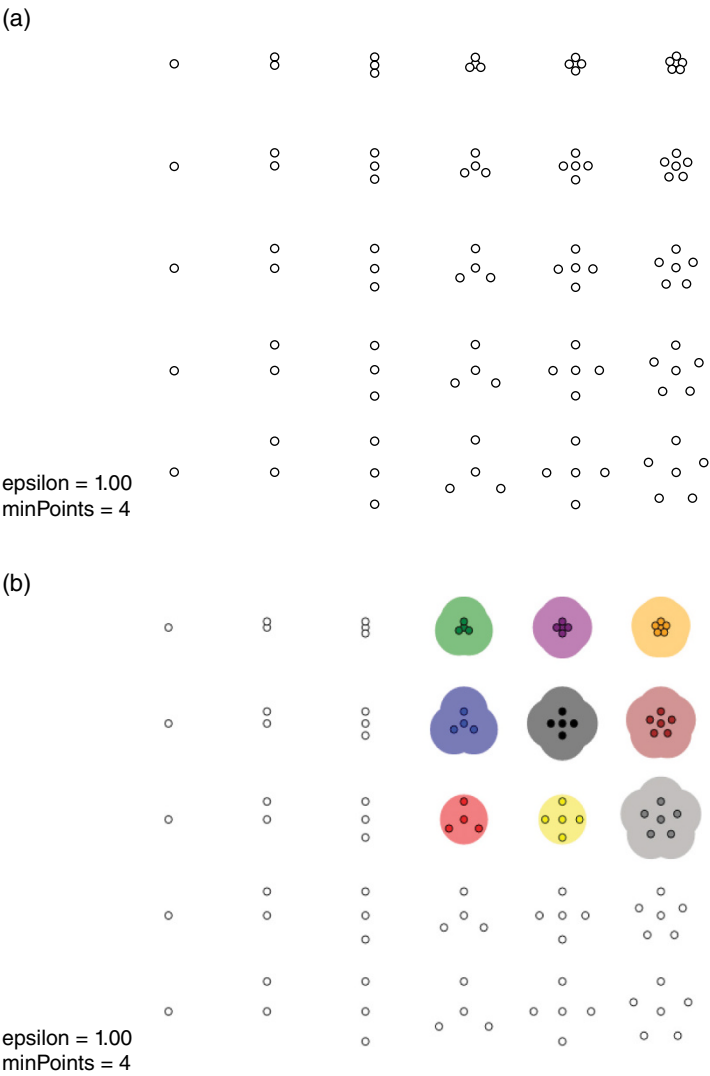


Figure 8.21 (a) DBSCAN with $\epsilon = 1.00$ and $\text{MinPts} = 4$. (b) DBSCAN with $\epsilon = 1.00$ and $\text{MinPts} = 4$. (c) DBSCAN with $\epsilon = 1.00$ and $\text{MinPts} = 4$. (d) DBSCAN output with $\epsilon = 1.00$ and $\text{MinPts} = 4$.

8.14.1 Artificial Neural Network

ANN is a computational system composed of interconnected processing elements and is modeled based on the structure, processing method, and learning ability of a biological neural network. An ANN gains knowledge through a learning process. The learning process may be supervised learning, unsupervised learning, or

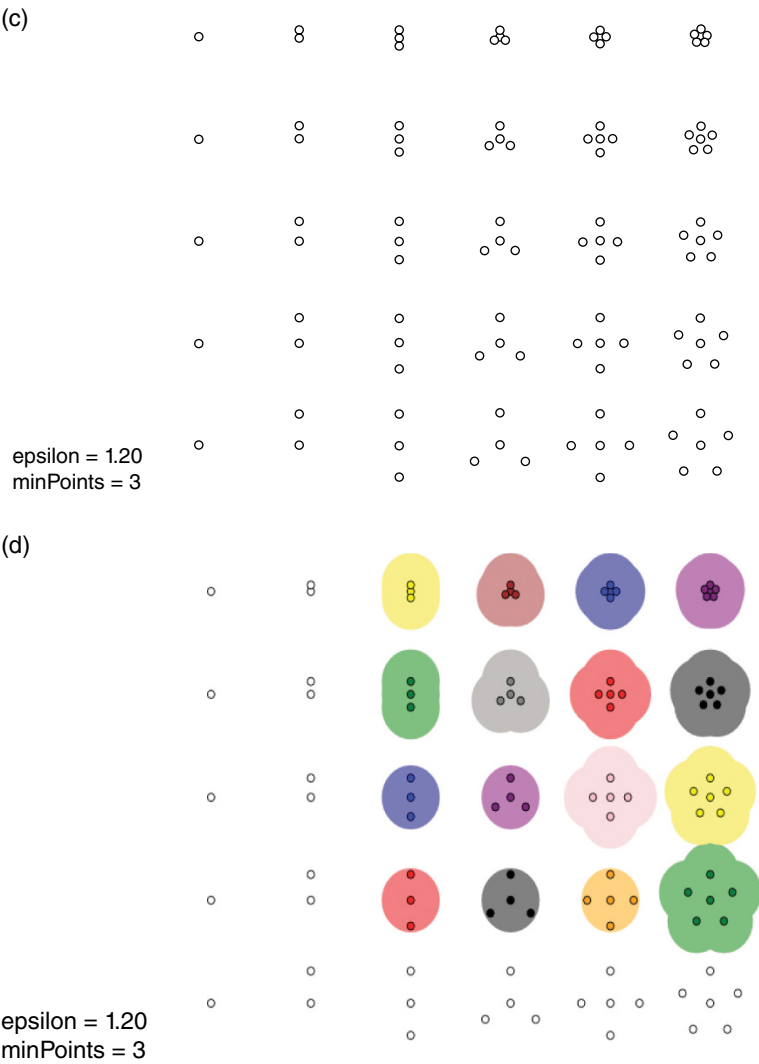


Figure 8.21 (Continued)

a combination of supervised and unsupervised learning. ANN are versatile and ideal to handle complex machine learning tasks such as image classification, speech recognition, recommendation engines used in used in social medias, fraud detection, Zip code recognition, text-to-voice translation, pattern classification, and so forth.

The primary objective of ANNs is to implement massively parallel network to perform complex computation with an efficiency equivalent to the human brain. These are generally modeled based on the interconnection of the neurons present in the human nervous systems. The neurons interconnected together are responsible for transmitting various signals within the brain. A human brain has billions of neurons responsible for processing information, which makes the human body react to heat, light, and so forth. Similar to a human brain, an ANN has thousands of processing units. The most important similarity between an ANN and a biological neural network is the learning capability and the neurons, which are the fundamental building blocks of the neural network. The nodes of an ANN are referred to as the processing elements or the “neurons.” To have a better understanding on the ANN let us take a closer look on the biological neural network.

8.14.2 The Biological Neural Network

Figure 8.22 shows a biological neural network as described in researchgate.net. The biological neural network is composed of billions of interconnected nerve cells called neurons. The projection of a neuron that transmits the electrical impulse to other neurons and glands are called axons. Axons typically connect the neurons together and transmit the information. One of the most important structures of neurons is called dendrites, which are a branch-like structure projecting from the neurons and are responsible for receiving signals. Dendrites receive

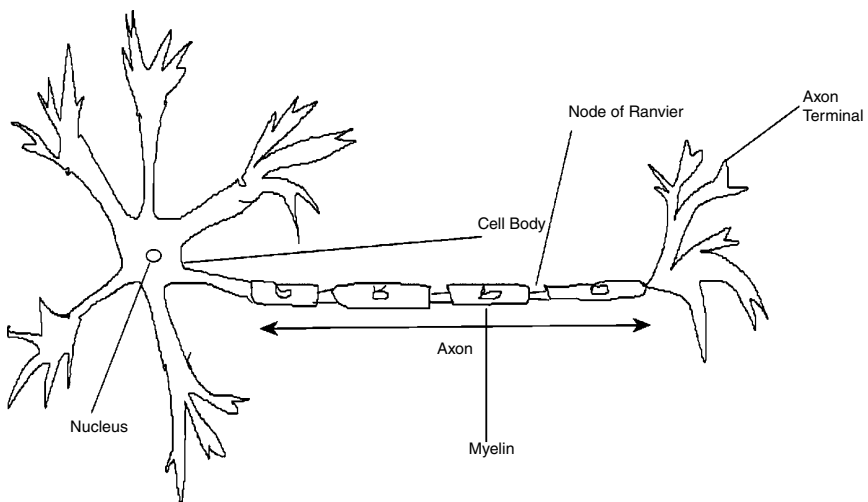


Figure 8.22 Biological neural network.

external stimuli or inputs from sensory organs. These inputs are passed to other neurons. The axon connects with a dendrite of another neuron via a structure called synapse.

ANNs are designed similar to the functionality of a human neural network. An ANN is designed with thousands of elementary processing network, the nodes imitating biological neurons of human brain. The nodes in the ANN are called neurons and they are interconnected to each other. The neurons receive the input and perform operations on the input, the results of which are passed to other neurons. The ANN also performs storage of information, automatic training, and learning.

8.15 Mining Data Streams

The data generated in audio, video, and text format are flowing from one node to another node in an uninterrupted fashion, which are continuous and dynamic in nature with no defined format. By definition, “Data Stream is an ordered sequence of data arriving at a rate which does not permit them to be stored in a memory permanently.” The 3 Vs, namely the volume, velocity, and variety, are the important characteristics of data streams. Because of their potentially unbound size, most of the data mining approaches are not capable of processing them. The speed and volume of the data poses a great challenge in mining them. The other important challenges posed by the data streams to the data mining community are concept-drift, concept-evolution, infinite length, limited labeled data, and feature evolution.

- **Infinite length**—Infinite length of the data is because the amount of data in the data streams has no bounds. This problem is handled by a hybrid batch incremental processing technique, which splits up the data in blocks of equal size.
- **Concept drift**—Concept drift occurs when the underlying concept of the data in the data streams changes over time, i.e., class or target value to be predicted, goal of prediction, and so forth, changes over time.
- **Concept evolution**—Concept evolution occurs due to evolution of new class in the streams.
- **Feature evolution**—Feature evolution occurs due the variations in the feature set over time, i.e., regression of old features and evolution of new features in the data streams. Feature evolution is due to concept drift and concept evolution.
- **Limited labeled data**—Labeled data in the data streams are limited since it is impossible to manually label all the data in the data stream.

Data arriving in streams, if not stored or processed immediately will be lost forever. But it is not possible to store all the data that are entering the system. The

speed at which the data arrives mandates the processing of each instance to be in real time and then discarded. The number of streams entering a system is not uniform. It may have different data types and data rates. Some of the examples of stream sources are sensor data, image data produced by satellites, surveillance cameras, Internet search queries, and so forth.

Mining data streams is the process of extracting the underlying knowledge from the data streams that are arriving at high speed. Following are the characteristics in which mining data streams differ from traditional data mining concepts.

The major goal for most of the data stream mining techniques is to predict the class of the new instances arriving in the data stream with the knowledge about class of the instances that are already present in the data stream. Machine learning techniques are applied to automate the process of learning from labeled instances and predict the class of new instances.

8.16 Time Series Forecasting

Time series is a series of observations measured in chronological order. The measurement can be made every hour, every day, every week, every month, every year, or at any regular time interval. For example, sales of a specific product in consecutive months or increase in price of gold every year,. Figure 8.23 shows increase of gold rate every year from 1990 to 2016. A time series is an ordered sequence of real valued variables.

$$T = (t_1, t_2, t_3, t_4, \dots, t_n),$$

Table 8.19 Comparison between Traditional data mining technique and mining data streams.

S. No	Traditional Data Mining	Data stream mining
1.	Data instances arrive in batches.	Data instances arrive in real-time.
2.	Processing time is unlimited.	Processing time is limited.
3.	Memory usage is unlimited.	Memory usage is limited.
4.	Has control over the order in which the data arrive.	Has no control over the order in which the data arrive.
5.	Data is not discarded after processing.	Data is discarded or archived after processing.
6.	Random access.	Sequential access.
7.	Multiple Scan.	Single scan, i.e. data is read only once.

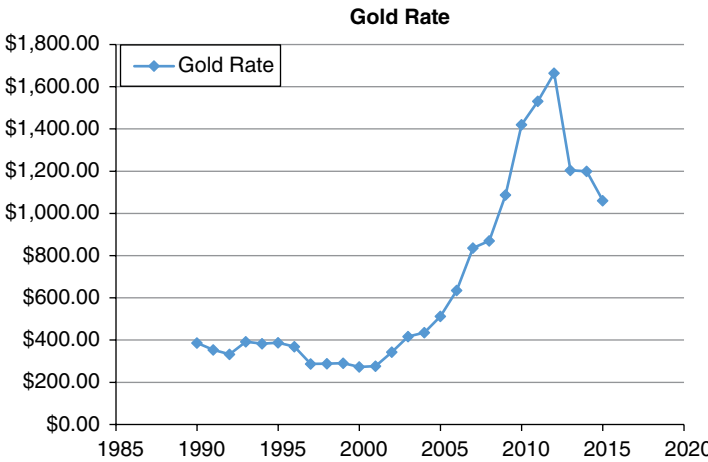


Figure 8.23 Time series forecasting.

Where $t_i \in \mathbb{R}$

Forecasting is the process of using a model to predict the future value of an observation based on historical values. Time series forecasting is an important predictive analytics technique in machine learning, where forecasting is made on time series data where observations are made in specific time interval. Thus, in time series forecasting we are aware of how the attribute or the target variable has changed over time in past so that it can be predicted as how it will change over time in future.

Applications of time series forecasting include:

- Sales forecasting;
- Pattern recognition;
- Earthquake prediction;
- Weather forecasting;
- Budgeting;
- Forecasting demand for a product.

One of the famous examples of time series forecasting is weather forecasting, where future weather is predicted based on the changes in the pattern in the past. In this case the predictor variable (the independent variable, which is used to predict the target variable) and the target variable are same. This type of forecasting technique where there is no difference between independent variable and the target variable is called data-driven forecasting method.

Another technique of time series forecasting is a model-driven method, where the predictor variable and the target variable are two different attributes. Here, the independent or the predictor variable is the time. The target variable can be predicted using the model below:

$$y(t) = a + b * t,$$

where $y(t)$ is the target variable at given time instant t . The values of coefficients a and b are predicted to forecast $y(t)$.

9

Cluster Analysis

9.1 Clustering

Clustering is a machine learning tool used to cluster similar data based on the similarities in its characteristics. The major difference between classification and clustering is that in classification, labels are predefined and the new incoming data is categorized based on the labels, whereas in clustering, data are categorized based on their similarities into clusters and then the clusters are labeled. The clusters are characterized by high intra-cluster similarity and low inter-cluster similarity. Clustering techniques play a major role in pattern recognition, marketing, biometrics, YouTube, online retails, and so forth. Online retailers use clustering to group items based on the clustering algorithm. For example, TVs, fridges, and washing machines are all clustered together since they all belong to the same category: electronics; similarly kids' toys and accessories are grouped under toys and baby products to make better online shopping experience for the consumers. YouTube utilizes clustering techniques to evolve a list of videos that the user might be interested in, to increase the time span spent by the user on the site. In marketing, clustering technology is basically used to group customers based on their behavior to boost their customer base. For example, a supermarket would group customers based on their buying patterns to reach the right group of customers to promote their products. Cluster analysis splits data objects into groups that are useful and meaningful, and this grouping is done in a specialized approach that objects belonging to the same group (cluster) have more similar characteristics than objects belonging to different groups. The greater the homogeneity within a group and the greater the dissimilarity between different groups, the better the clustering.

Clustering techniques are used when the specific target or the expected output is not known to the data analyst. It is popularly termed as unsupervised

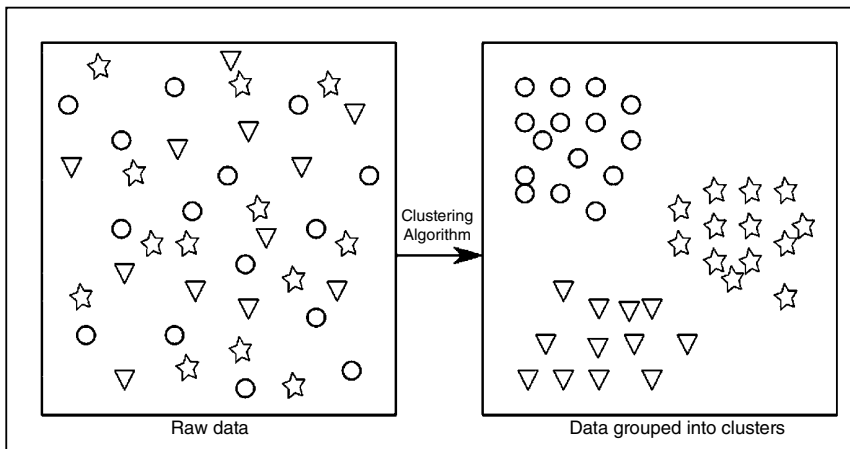


Figure 9.1 Clustering algorithm.

classification. In clustering techniques, the data within each group are very similar in their characteristics. The basic difference between classification and clustering is that the outcome of the problem in hand is not known beforehand in clustering while in classification the historical data groups the data into the class to which it belongs. Under classification, the results will be the same in grouping different objects based on certain criteria, but under clustering, where the target required is not known, the results may not be the same every time a clustering technique is performed on the same data.

Figure 9.1 depicts the clustering algorithm where the circles are grouped together forming a cluster, triangles are grouped together forming a cluster, and stars are grouped together to form a cluster. Thus, all the data points with similar shapes are grouped together to form individual clusters.

The clustering algorithm typically involves gathering the study variables, pre-processing them, finding and interpreting the clusters, and framing a conclusion based on the interpretation. To achieve clustering, data points must be classified by measuring the similarity between the target objects. Similarity is measured by two factors, namely, similarity by correlation and similarity by distance, which means the target objects are grouped based on their distance from the centroid or based on the correlation in their characteristic features. Figure 9.2 shows clustering based on distance where intra-cluster distances are minimized and inter-cluster distances are maximized. A centroid is a terminology in the clustering algorithm that is the center point of the cluster. The distance between each data point and the centroid is measured using one of the following measuring

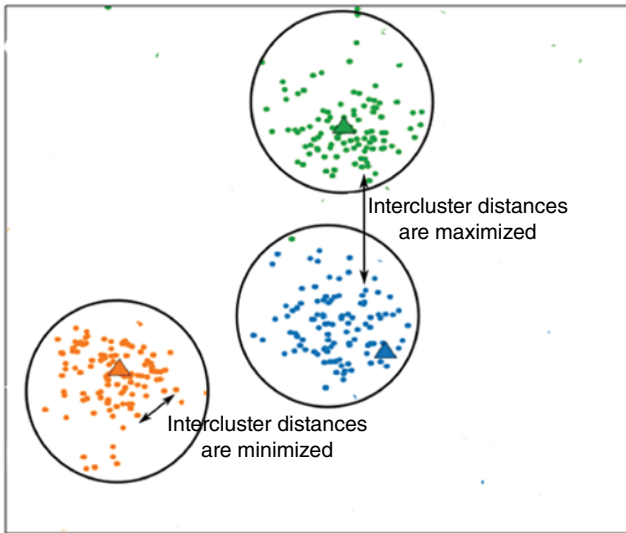


Figure 9.2 Clustering based on distance.

approaches: Euclidean distance, Manhattan distance, cosine distance, Tanimoto distance, or squared Euclidean distance.

9.2 Distance Measurement Techniques

A vector is a mathematical quantity or phenomenon with magnitude and direction as its properties. Figure 9.3 illustrates a vector.

Euclidean distance—is the length of the line connecting two points in Euclidean space. Mathematically, the Euclidean distance between two n -dimensional vectors is:

$$\text{Euclidean distance } d = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_n - y_n)^2}$$

Manhattan distance—is the length of the line connecting two points measured along the axes at right angles. Mathematically, the Manhattan distance between two n -dimensional vectors is:

$$\text{Manhattan distance } d = |x_1 - y_1| + |x_2 - y_2| + \cdots + |x_n - y_n|$$

Figure 9.4 illustrates that the shortest path to calculate the Manhattan distance is not a straight line; rather, it follows the grid path.

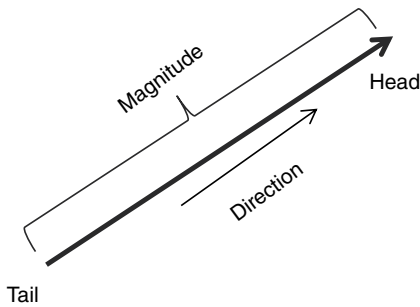


Figure 9.3 A vector in space.

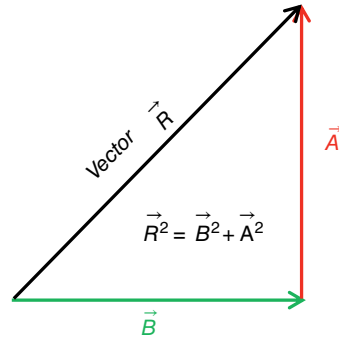
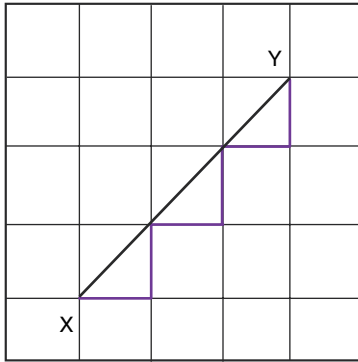


Figure 9.4 Manhattan distance.



Cosine Similarity—The cosine similarity between two n -dimensional vectors is a mathematical quantity that measures the cosine angle between them.

$$\text{Cosine similarity} = \frac{(x_1y_1 + x_2y_2 + \dots + x_ny_n)}{\left(\sqrt{(x_1^2 + x_2^2 + \dots + x_n^2)}\sqrt{(y_1^2 + y_2^2 + \dots + y_n^2)}\right)}$$

$$\text{Cosine Distance } d = 1 - \frac{(x_1y_1 + x_2y_2 + \dots + x_ny_n)}{\left(\sqrt{(x_1^2 + x_2^2 + \dots + x_n^2)}\sqrt{(y_1^2 + y_2^2 + \dots + y_n^2)}\right)}$$

Clustering techniques are classified into:

- 1) Hierarchical clustering algorithm
 - a) Agglomerative
 - b) Divisive
- 2) Partition clustering algorithm

Clustering techniques basically have two classes of algorithms, namely, partition clustering and hierarchical clustering. Hierarchical clustering is further subdivided into agglomerative and divisive.

9.3 Hierarchical Clustering

Hierarchical clustering is a series of partitions running from a single cluster or reversely a single large cluster can be iteratively divided into smaller clusters. In the hierarchical method the division, once made, is irrevocable. A hierarchical cluster is formed by merging two similar groups that started as a single item. The distance between the groups is calculated in all the iterations, and the closest groups are merged, forming a new group. This procedure is repeated until all the groups are merged into a single group. Figure 9.5 shows hierarchical clustering.

In the above figure, similarity is measured by calculating the distance between the data points. The closer the data points, the more similar they are. Initially numbers 1, 2, 3, 4, and 5 are individual data points; next, they are grouped

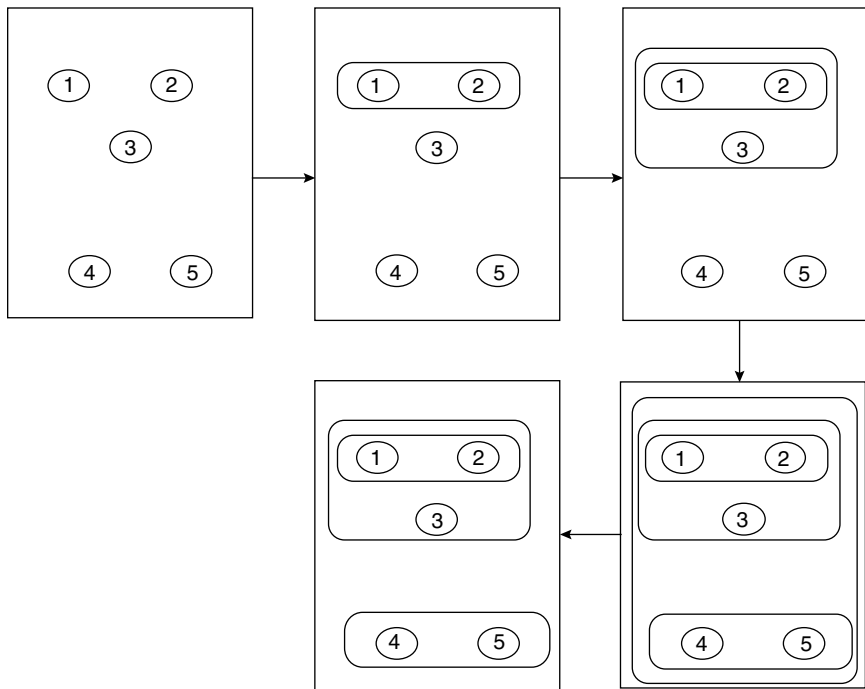


Figure 9.5 Hierarchical clustering.

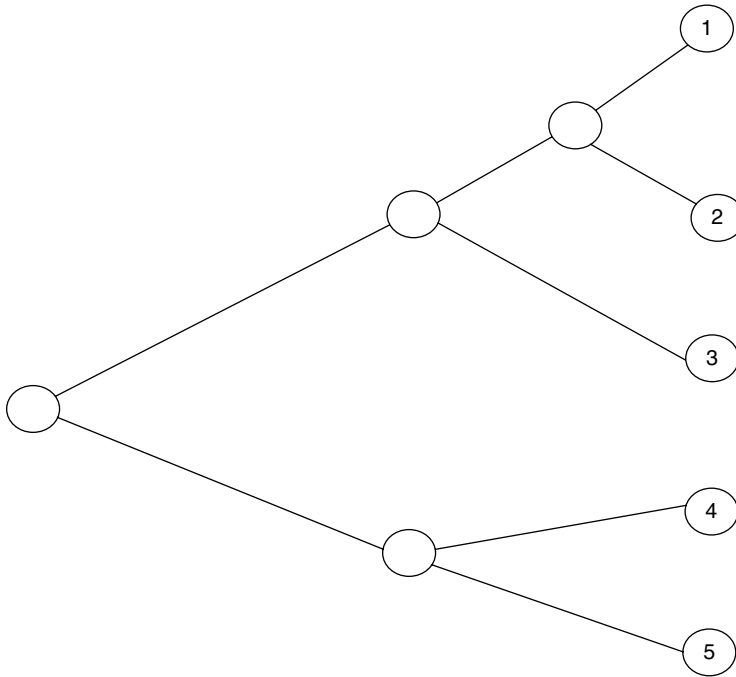


Figure 9.6 Dendrogram graph.

together based on the distance between them. One and two are grouped together since they are close to each other, to form the first group. The new group thus formed is merged with 3 to form a single new group. Since 4 and 5 are close to each other, they form a new group. Finally, the two groups are merged into one unified group. Once the hierarchical clustering is completed, the results are visualized with a graph or a tree diagram called dendrogram, which depicts the way in which the data point are sequentially merged to form a single larger group. The dendrogram of the above explained hierarchical clustering is depicted below, in Figure 9.5. The dendrogram is also used to represent the distance between the smaller groups or clusters that are grouped together to form the single large cluster.

There are two types of hierarchical clustering:

- 1) Agglomerative clustering;
- 2) Divisive clustering.

Agglomerative clustering—Agglomerative clustering is one of the most widely adopted methods of hierarchical clustering. Agglomerative clustering is done

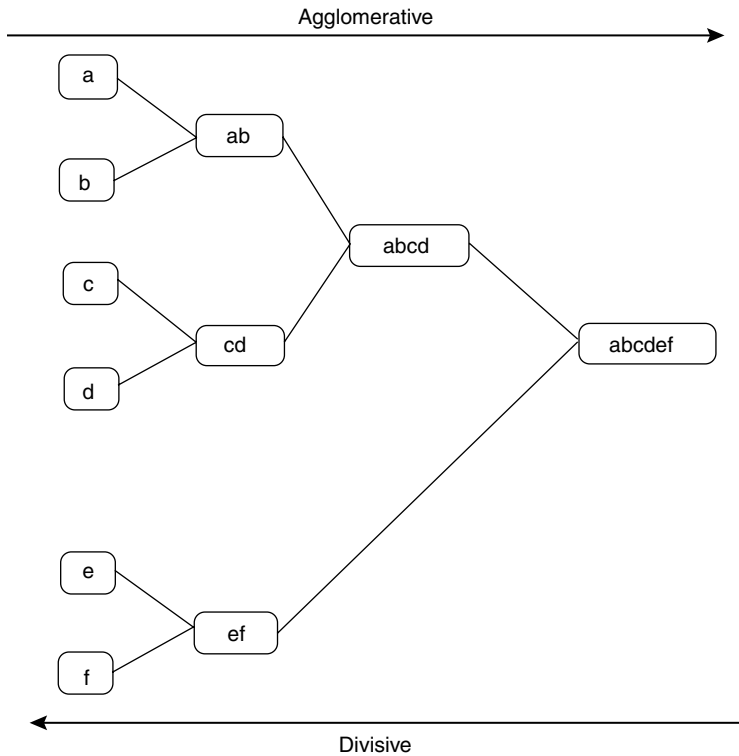


Figure 9.7 Agglomerative and divisive clustering.

by merging several smaller clusters into a single larger cluster from the bottom up. Ultimately the agglomerative clustering reduces the data into a single large cluster containing all individual data groups. Fusions once made are irrevocable, i.e., when smaller clusters are merged by agglomerative clustering, they cannot be separated. Fusions are made by combining clusters or group of clusters that are closest or similar.

Divisive clustering—Divisive clustering is done by dividing a single large cluster into smaller clusters. The entire data set is split into n groups, and the optimal number cluster to stop clustering is decided by the user. Divisions once made are irrevocable, i.e., when a large cluster is split by divisive clustering, they cannot be merged again. The clustering output produced by both the agglomerative and divisive clustering are represented by two-dimensional dendrogram diagrams. Figure 9.7 depicts that agglomerative merges several small clusters into one large cluster while divisive does the reverse of it by successively splitting the large cluster into several small clusters.

9.3.1 Application of Hierarchical Methods

The hierarchical clustering algorithm is a powerful algorithm on multivariate data analysis and is often used to identify natural clusters. It renders a graphical representation, a hierarchy or dendrogram of the resulting partition. It does not require the number of clusters to be specified a priori. Groups of related data are identified, which can be used to explore further relationships. The hierarchical clustering algorithm finds its application in the medical field to identify diseases. There are two ways by which diseases can be identified from biomedical data. One way is to identify the disease using a training data set. When the training data set is unavailable, then the task would be to explore the underlying pattern and to mine the samples into meaningful groups. One of the important applications of hierarchical clustering is the analysis of protein patterns in the human cancer-associated liver. An investigation of the proteomic (a large-scale study of proteins) profiles of a fraction of human liver is performed using two-dimensional electrophoresis. Two-dimensional electrophoresis, abbreviated as 2DE, is a form of gel electrophoresis used to analyze proteins. Samples were resected from surgical treatment of hepatic metastases. Unsupervised hierarchical clustering on the 2DE images revealed clusters that provided a rationale for personalized treatment. Other applications of hierarchical clustering include:

- Recognition using biometrics of hands;
- Regionalization;
- Demographic based customer segmentation;
- Text analytics to derive high quality information from the text data;
- Image analysis; and
- Bioinformatics.

9.4 Analysis of Protein Patterns in the Human Cancer-Associated Liver

There are two ways by which diseases can be identified from biomedical data. One way is to identify the disease using a training data set. When the training data set is unavailable, then the task would be to explore the underlying pattern and to mine the samples into meaningful groups. An investigation of the proteomic (a large scale study of proteins) profiles of a fraction of human liver is performed using two-dimensional electrophoresis. Two-dimensional electrophoresis abbreviated as 2DE is a form of gel electrophoresis used to analyze proteins. Samples were resected from surgical treatment of hepatic metastases. Unsupervised hierarchical clustering on the 2DE images revealed clusters which provided a rationale for personalized treatment.

9.5 Recognition Using Biometrics of Hands

9.5.1 Partitional Clustering

Partitional clustering is the method of partitioning a data set into a set of clusters. Given a data set with N data points, partitional clustering partitions N data points into K number of clusters, where $N \geq K$. The partitioning is performed by satisfying two conditions, each cluster should have at least one data point, and each of the N data points should belong to at least one of the K clusters. In case of fuzzy partitioning algorithm, a point can belong to more than one group. The function to group data points to clusters is:

$$\sum_{m=1}^K \sum_{n=1}^{C_m} Dist(x_n, Center(m)),$$

where K is the total number of clusters, c_m is the total number of points in the cluster m , and $Dist(x_n, Center(m))$ is the distance between the point x_n and the center m . One of the commonly used partition clustering, K-means clustering, is explained in this chapter.

9.5.2 K-Means Algorithm

The K-means cluster was proposed by MacQueen. It is a widely adopted clustering methodology because of its simplicity. It is conceptually simple and computationally cheap. On the downside it may get stuck in the local optima and sometimes misses an optimal solution. The K-means clustering algorithm partitions the data points into K number of clusters in which each data point belongs to nearest centroid. The value of K , which is the number of clusters, is given as the input parameter.

In K-means clustering, initially a set of data points are given; let the set of data points be $d = \{x_1, x_2, x_3, \dots, x_n\}$. The K-means clustering algorithm partitions the given data points into K number of clusters with a center called centroid for each cluster. Random but logical number of K centroids are selected, and the location of the K centroids are refined iteratively by assigning each data point x to its closest centroid, and the centroid position is updated by calculating the mean distance of all the data points belonging to that centroid. The iteration continues until there are no changes in assignments of data points to the centroid, in other words, until there are no or very little changes in the position of the centroid. Figure 9.8 shows the K-means clustering flowchart.

The final result depends on the initial position of centroids and the number of centroids. Changes in the initial position of the centroid yield a different output for the same set of data points. Consider the following example, where in

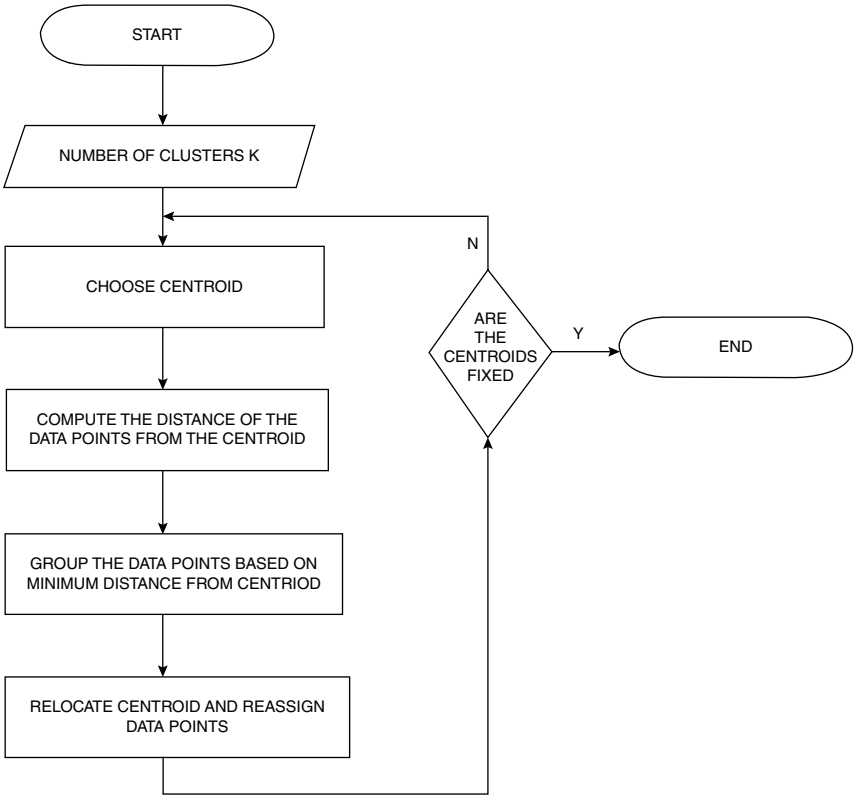


Figure 9.8 K-means clustering flowchart.

Figure 9.10b and d the results are different for the same set of data points. Figure 9.10b is an optimal result compared to Figure 9.10d.

The fundamental step in cluster analysis is to estimate the number of clusters, which has a deterministic effect on the results of cluster analysis. The number of clusters must be specified before the cluster analysis is performed. The result of cluster analysis is highly dependent on the number of clusters. The solutions to cluster analysis may vary with the difference in the number of clusters specified. The problem here is to determine the value of K appropriately. For example, if the K-means algorithm is run with $K = 3$, the data points will be split up into three groups, but the modeling may be better with $K = 2$ or $K = 4$. The number of clusters is ambiguous because the inherent meaning of the data is different for different clusters. For example, the speed of different cars on the road and the customer base of an online store are two different types of data sets that have to be interpreted differently. Gap statistics is one of the popular methods in determining the value of K .

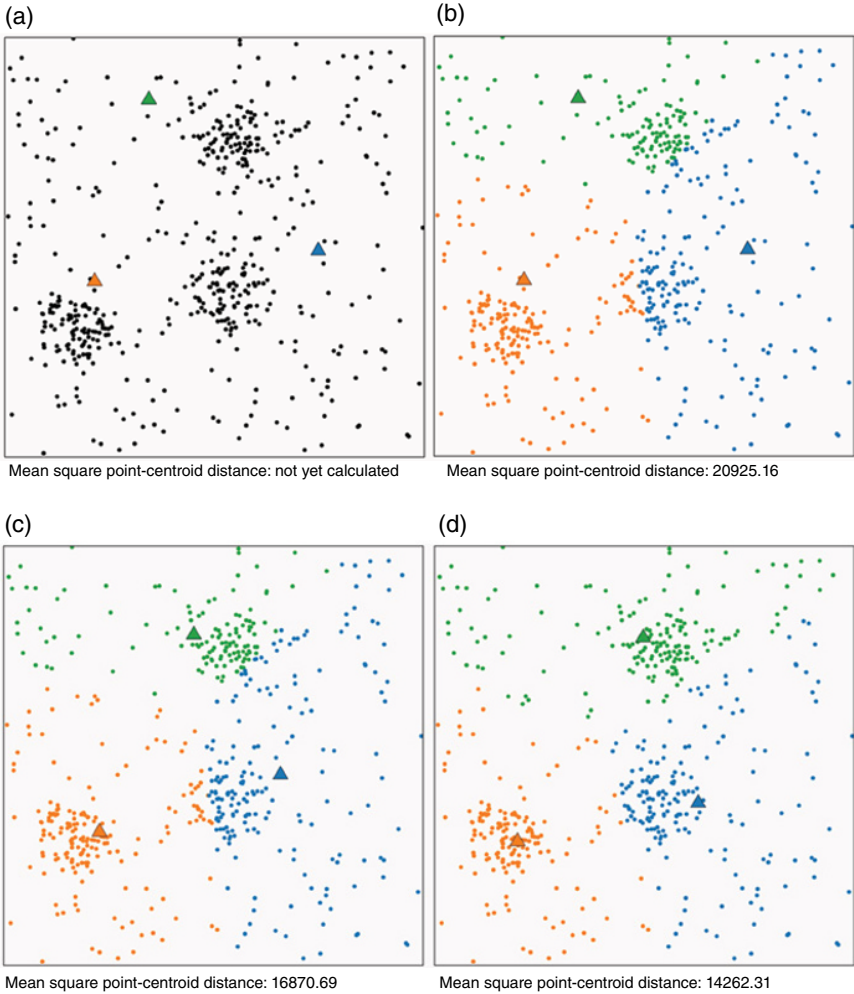


Figure 9.9 (a) Initial clustered points with random centroids (b) Iteration 1: Centroid distance calculated, and data points assigned to each centroid. (c) Iteration 2: Centroids are recomputed, and clusters are reassigned (d) Iteration 3: Centroids are recomputed, and clusters are reassigned. (e) Iteration 4: Centroids are recomputed, and clusters are reassigned. (f) Iteration 5: Changes in the position of the centroid and assignment of clusters are minimal. (g) Iteration 6: Changes in the position of the centroid and assignment of clusters are minimal. (h) Iteration 7: There is no change in the position of the centroid and assignment of clusters, and hence the process is terminated.

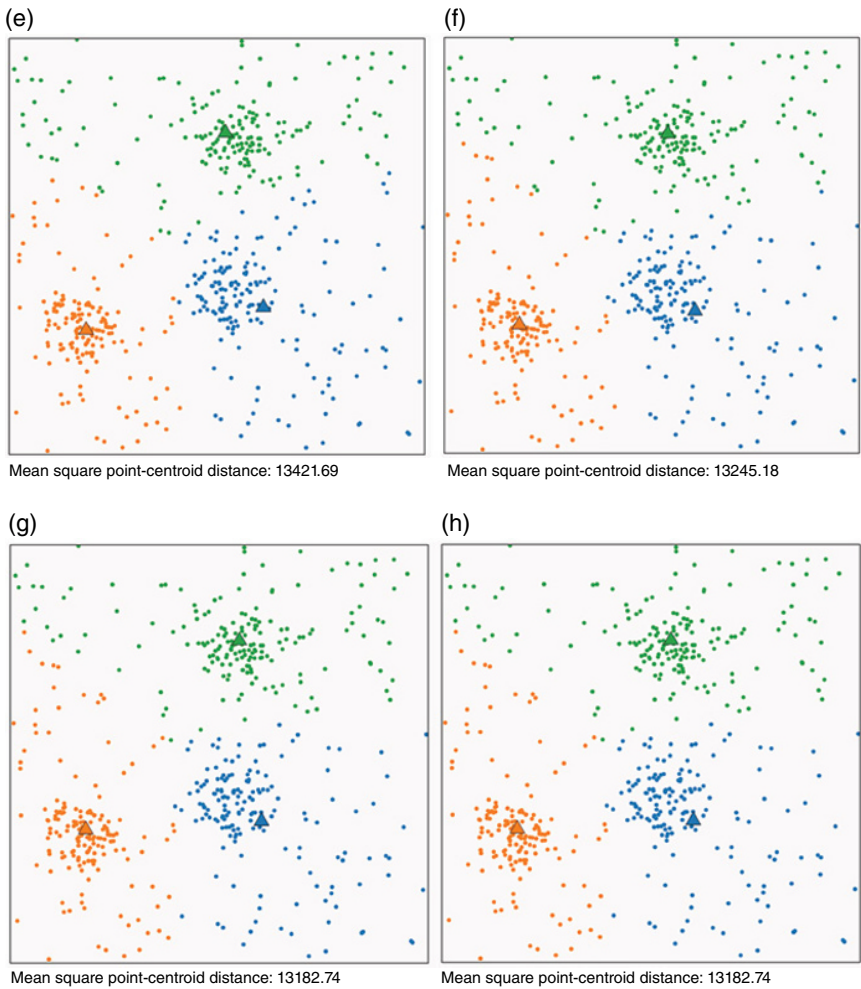


Figure 9.9 (Continued)

9.5.3 Kernel K-Means Clustering

K-means is a widely adopted method in cluster analysis. It just requires the data set and a pre-specified value for K; then the algorithm that minimized the sum of squared errors is applied to obtain the desired result. K-means works perfectly if the clusters are linearly separable, as shown in Figure 9.11. But when the clusters are arbitrarily shaped and not linearly separable, as shown in Figure 9.12, then the kernel K-means technique may be adopted.

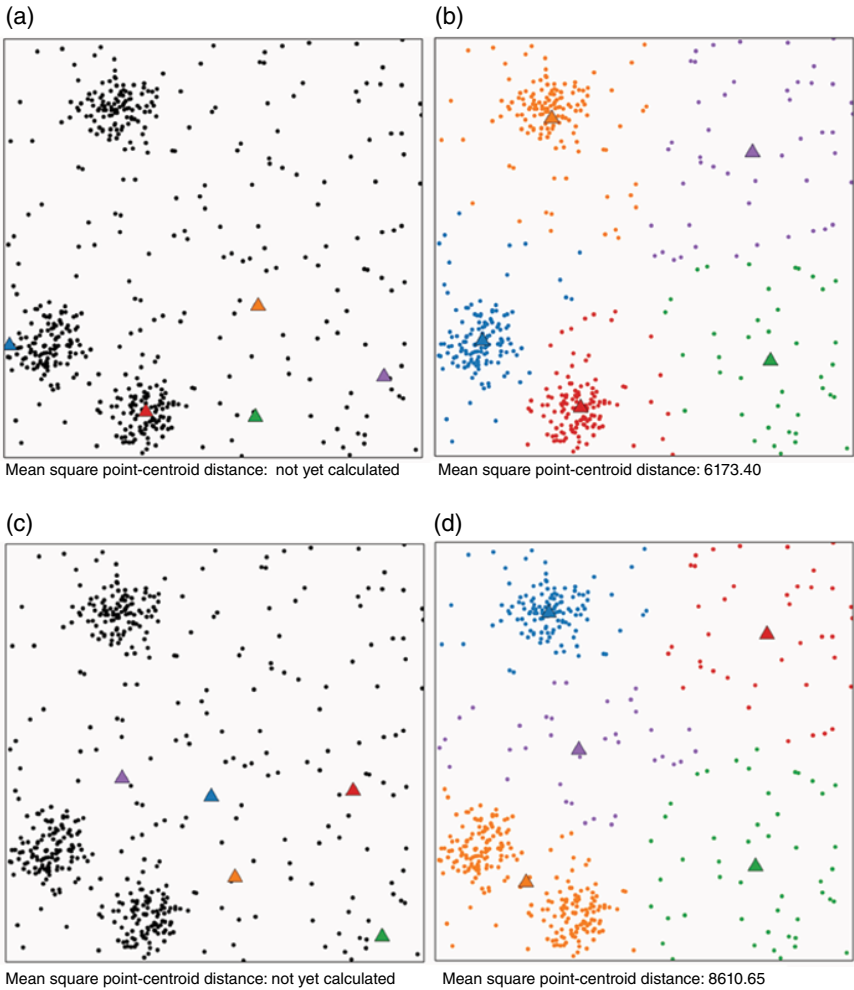


Figure 9.10 (a) Initial clustered points with random centroids (b) Final Iteration (c) same clustered points with different centroids (d) Final Iteration, which is different from 9.10b.

K-means performs well in the data set shown in Figure 9.11 whereas it performs poorly in the data set shown in Figure 9.12.

On Figure 9.13 it is evident that the data points belong to two distinct groups. With K-means the data points are grouped as shown in Figure 9.13b, which is not the desired output. Hence, we go for kernel K-means (KK means) where the data points are grouped as shown in Figure 9.13c.

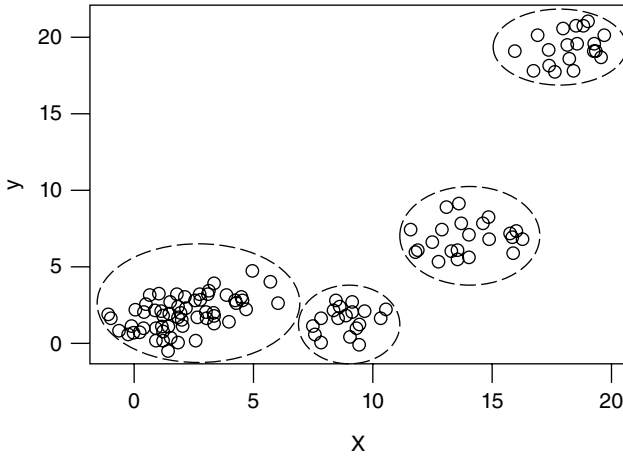


Figure 9.11 Linearly separable clusters.

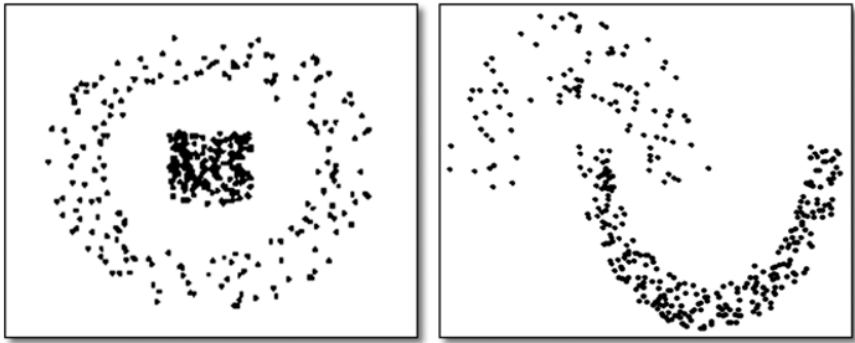


Figure 9.12 Arbitrarily shaped clusters.

Let $X = \{x_1, x_2, x_3, \dots, x_n\}$ be the data points and c be the cluster center. Randomly initialize the cluster centers. Compute the distance between the cluster centers and each data point in the space.

The goal of kernel K-means is to minimize the sum of square errors:

$$\min \sum_{i=1}^n \sum_{j=1}^m u_{ij} \|x_i - c_j\|^2, \quad (9.1)$$

where

$$u_{ij} \in \{0, 1\}$$

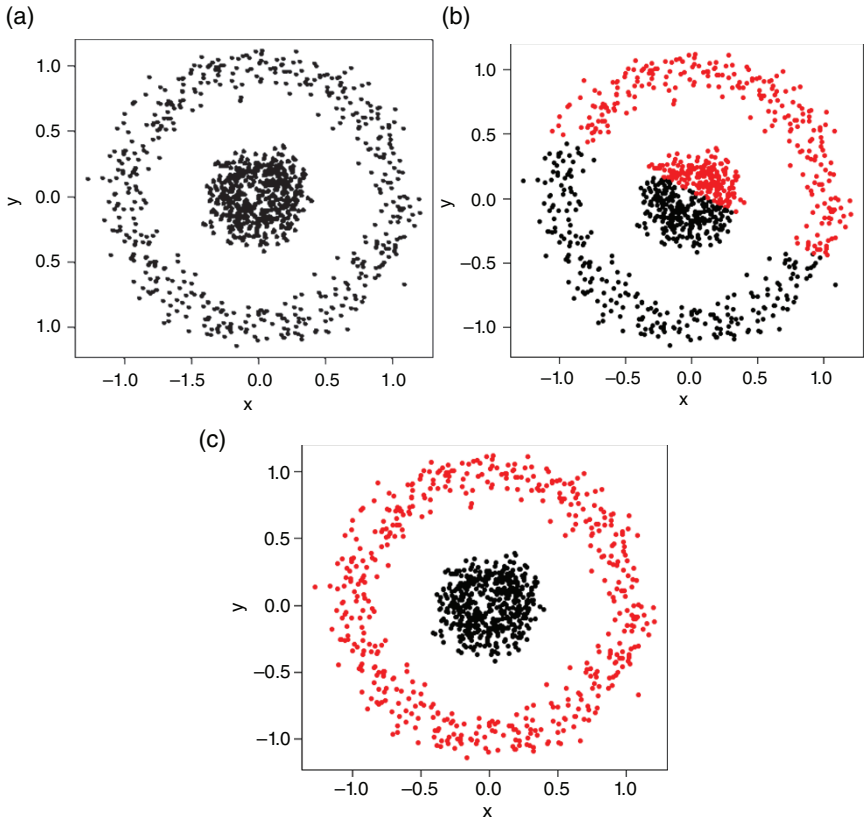


Figure 9.13 (a) Original data set (b) K means (c) KK means.

- The cluster center $c_j = \frac{1}{n_j} \sum_{i=1}^n u_{ij} \phi(x_i)$,
- x_i is the data point,
- n_j is the total number of data points.

Replacing x_i with $\phi(x)$, which is the data point in the transformed space, and c_j with $\frac{1}{n_j} \sum_{i=1}^n u_{ij} \phi(x_i)$ in equation (9.1), we get:

$$\min \sum_{i=1}^n \sum_{j=1}^m u_{ij} \|\phi(x_i) - \frac{1}{n_j} \sum_{i=1}^n u_{ij} \phi(x_i)\|^2$$

Assign the data points to the cluster center such that the distance between the cluster center and data point is minimum.

9.6 Expectation Maximization Clustering Algorithm

Basically, there are two types of clustering, namely,

- Hard clustering—Clusters do not overlap: each element of the cluster belongs to only one cluster.
- Soft Clustering—Clusters may overlap: the elements of the cluster can belong to more than one cluster. Data points are assigned based on certain probabilities.

The K-means algorithm performs hard clustering—the data points are assigned to only one cluster based on their distances from the centroid of the cluster. In case of soft clustering, instead of assigning data points to the closest cluster centers, data points can be assigned partially or probabilistically based on distances. This can be implemented by:

- Assuming a probability distribution (the model) for each cluster, typically a mixture of Gaussian distributions. Figure 9.14 shows a univariate Gaussian distribution, $N(\mu, \sigma^2)$,

where,

μ = mean, the center of the mass

σ^2 = variance.

- And computing the probability that each data point corresponds to each cluster.

The expectation maximization algorithm is used to infer the values of the parameters μ and σ^2 . Let us consider an example to see how the expectation maximization algorithm works. Let us consider the data points shown in Figure 9.15, which comes from two different models: gray Gaussian distribution and white Gaussian

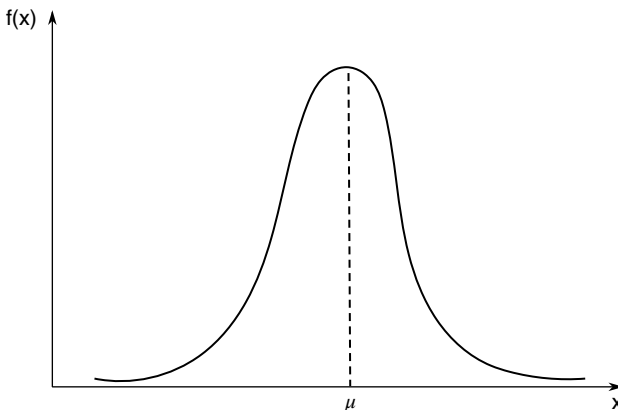


Figure 9.14 Univariate Gaussian distribution.



Figure 9.15 Data points from two different models.

distribution. Since it is evident which points came from which Gaussian, it is easy to estimate the mean, μ , and variance, σ^2 .

$$\mu = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n} \quad (9.2)$$

$$\sigma^2 = \frac{(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2 + \dots + (x_n - \mu_n)^2}{n} \quad (9.3)$$

To calculate the mean and variance for the gray Gaussian distribution use (9.4) and (9.5) and to evaluate the mean and variance for the white Gaussian distribution use Eqs. (9.6) and (9.7).

$$\mu_g = \frac{x_{1g} + x_{2g} + x_{3g} + x_{4g} + x_{5g}}{n_g} \quad (9.4)$$

$$\sigma_g^2 = \frac{(x_{1g} - \mu_{1g})^2 + (x_{2g} - \mu_{2g})^2 + (x_{3g} - \mu_{3g})^2 + (x_{4g} - \mu_{4g})^2 + (x_{5g} - \mu_{5g})^2}{n_g} \quad (9.5)$$

$$\mu_w = \frac{x_{1w} + x_{2w} + x_{3w} + x_{4w} + x_{5w}}{n_w} \quad (9.6)$$

$$\sigma_w^2 = \frac{(x_{1w} - \mu_{1w})^2 + (x_{2w} - \mu_{2w})^2 + (x_{3w} - \mu_{3w})^2 + (x_{4w} - \mu_{4w})^2 + (x_{5w} - \mu_{5w})^2}{n_w} \quad (9.7)$$

Evaluating the parameters we will get the Gaussian distributions, as shown in Figure 9.16.

Since the source of the data points was evident, the mean and variance were calculated, and we arrived at the Gaussian distribution. If the source of the data

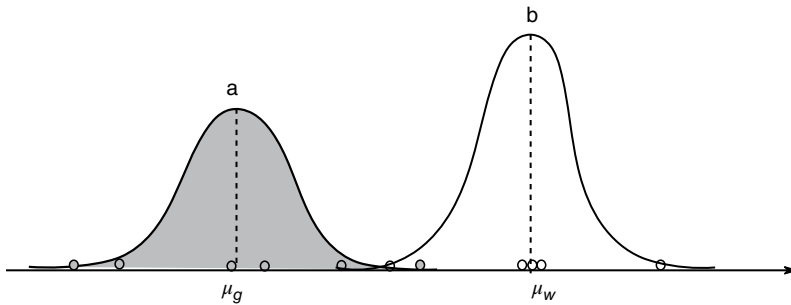


Figure 9.16 Gaussian distribution.

points was not known, as shown below, but we still know that the data points came from two different Gaussians and the parameters mean and variance are also known, then it is possible to guess whether the data point belongs more likely to a or b using the formulas:

$$P(b|x_i) = \frac{P(x_i|b)P(b)}{P(x_i)P(b) + P(x_i|a)P(a)} \quad (9.8)$$

$$P(x_i|b) = \frac{1}{\sqrt{2\pi\sigma_b^2}} \exp \left[-\frac{(x_i - \mu_b)^2}{\sigma_b^2} \right] \quad (9.9)$$

$$P(x_i|a) = 1 - P(x_i|b) \quad (9.10)$$

Thus, we should know either the source to estimate the mean and variance or the mean and variance to guess the source points. When the source, mean, and variance are not known and the only data in hand is that they came from two Gaussians, then the expectation maximization (EM) algorithm is used. To begin, place Gaussians at random positions, as shown in Figure 9.17 and estimate (μ_a, σ_a) and (μ_b, σ_b) . Unlike K-means, the EM algorithm does not make any hard assignments, i.e., it does not assign any data point deterministically to one cluster. Rather, for each data point, the EM algorithm estimates the probabilities that the data point belongs to a or b Gaussian.

Let us consider the point shown in Figure 9.18 and estimate the probabilities $P(b|x_i)$ and $P(a|x_i)$ for the randomly placed Gaussians. The probability $P(b|x_i)$ will be very less since the point is very far from the b Gaussian while the probability $P(a|x_i)$ will be even lower than the probability $(b|x_i)$. Thus, the point will be assigned for the b Gaussian.

Similarly estimate the probabilities for all other points. Re-estimate the mean and variance with the computed probabilities using the formulae (9.11), (9.12), (9.13), and (9.14).

$$\mu_a = \frac{P(a|x_1)x_1 + P(a|x_2)x_2 + P(a|x_3)x_3 + \dots + P(a|x_n)x_n}{P(a|x_1) + P(a|x_2) + P(a|x_3) + \dots + P(a|x_n)} \quad (9.11)$$

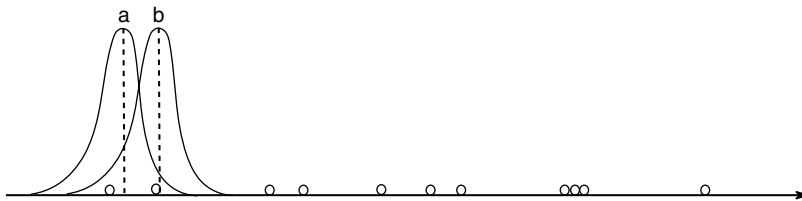


Figure 9.17 Gaussians placed in random positions.

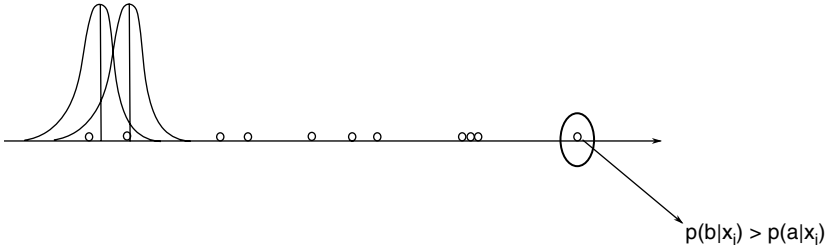


Figure 9.18 Probability estimation for the randomly placed Gaussians.

$$\sigma_a^2 = \frac{P(a|x_1)(x_1 - \mu_1)^2 + \dots + P(a|x_n)(x_n - \mu_n)^2}{P(a|x_1) + P(a|x_2) + P(a|x_3) + \dots + P(a|x_n)} \quad (9.12)$$

$$\mu_b = \frac{P(b|x_1)x_1 + P(b|x_2)x_2 + P(b|x_3)x_3 + \dots + P(b|x_n)x_n}{P(b|x_1) + P(b|x_2) + P(b|x_3) + \dots + P(b|x_n)} \quad (9.13)$$

$$\sigma_b^2 = \frac{P(b|x_1)(x_1 - \mu_1)^2 + \dots + P(b|x_n)(x_n - \mu_n)^2}{P(b|x_1) + P(b|x_2) + P(b|x_3) + \dots + P(b|x_n)} \quad (9.14)$$

Eventually, after a few iterations, the actual Gaussian distribution for the data points will be obtained.

9.7 Representative-Based Clustering

Representative-based clustering partitions the given data set with n data points in an N -dimensional space. The data set is partitioned into K number of clusters, where K is determined by the user.

9.8 Methods of Determining the Number of Clusters

9.8.1 Outlier Detection

An outlier is a data point that lies outside the pattern of a distribution, i.e., a data point that lies farther away from other points or observations that deviate from the normal observations. It is considered as an abnormal data point. Outlier detection or anomaly detection is the process of detecting and removing the anomalous data points from other normal data points, i.e., observations with significantly different characteristics are to be identified and removed. Once the outliers are removed,

the variations of the data points in a given data set have to be minimal. It is an important step in data cleansing where the data is cleansed before applying the data mining algorithms to the data. Removal of outliers is important for an algorithm to be successfully executed. Outliers in case of clustering are the data points that do not conform to any of the clusters. In this case, for a successful implementation of the clustering algorithm, outliers are to be removed.

Outlier detection finds its application in fraud detection, where abnormal transactions or activities are detected. Its other applications are stock market analysis, email spam detection, marketing, and so forth. Outlier detection is used for failure prevention, cost savings, fraud detections, health care, customer segmentation, and so forth. Fraud detection, specifically financial fraud, is the major application of outlier detection. It provides warning to the financial institutions by detecting the abnormal behavior before any financial loss occurs. In health care, patients with abnormal symptoms are detected and treated immediately. Outliers are detected to identify the faults before the issues result in disastrous consequences. The data points or objects deviating from other data points in the given data set are detected.

The several methods used in detecting anomalies include clustering-based methods, proximity-based methods, distance-based method, and deviation-based method. In proximity-based methods, outliers are detected based on their relationship with other data objects. Distance-based methods are a type of proximity-based method. In distance-based methods, outliers are detected based on the distance from their neighbors, and normal data points have crowded neighborhoods. Outliers have neighbors that are far apart, as shown in Figure 9.19. In a deviation-based method, outliers are detected by analyzing the characteristics of

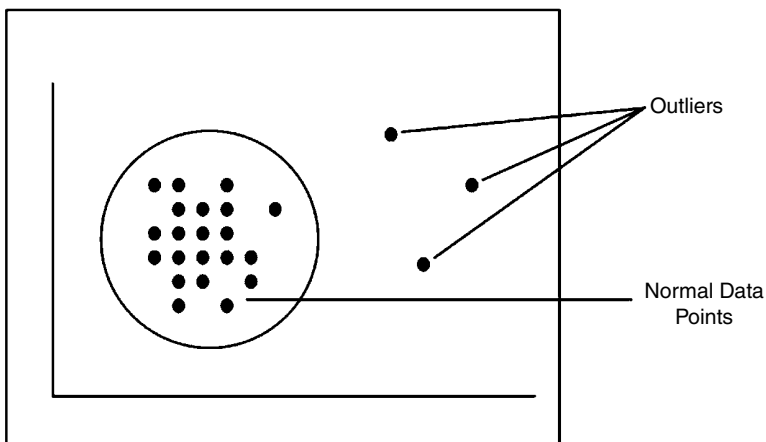


Figure 9.19 Outliers.

the data objects. The object that deviates from the main features of the other objects in a group is identified as an outlier. The abnormality is detected by comparing the new data with a normal data or an abnormal data or it is classified as normal or abnormal data. More techniques of detecting outliers are discussed in detail under the outlier detection techniques.

Outlier detection in big data is more complex due to the increasing complexity, variety, volume, and velocity of data. Additionally, there are requirements where outliers are to be detected in real time and provide instantaneous decisions. Hence, the outlier detectors are designed in a way to cope with these complexities. Algorithms are to be specifically designed to handle the large volume of heterogeneous data. Also, existing algorithms to detect outliers such as binary KD-tree are taken and parallelized for distributed processing. Though big data poses multiple challenges, it also helps in detecting rare patterns by exploiting a broader range of outliers and increases the robustness of the outlier detector.

Anomalies detected are to be prioritized in the order of its criticalities. Financial frauds, hack attacks, and machine faults are all critical anomalies that need to be detected and addressed immediately. Also, there are cases where some anomalies detected may be false positives. Thus, the anomalies are to be ranked so they can be analyzed in their order of priority so that the critical anomalies may not be ignored amid the false positives. Data points may be categorized as outliers even if they are not.

9.8.2 Types of Outliers

There are three types of outliers:

- Point or global outliers;
- Contextual outliers;
- Collective outliers.

Point Outlier—An individual data object considered as outlier that significantly deviates from the rest of the data objects in the set is the point outlier. Figure 9.20 shows a graph with temperature recorded at different months, where 18° is detected as outlier as it deviates from other data points. In a credit card transaction, an outlier is detected with the amount spent by the individual. If the amount spent is too high compared to the usual range of expenditure by the individual, it will be considered as a point outlier.

Contextual outlier—An object in a given set is identified as contextual outlier if the outlier object is anomalous based on a specific context. To detect the outlier, the context has to be specified in the problem definition. For example, the temperature of a day depends on several attributes such as time and location. A temperature of 25 °C could be considered an outlier, depending on time and location.

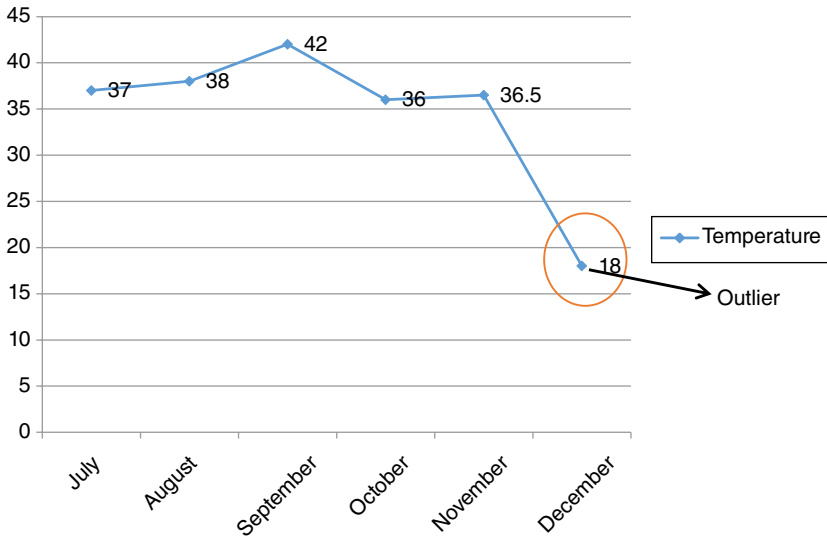


Figure 9.20 Point outlier.

In summer in California, a temperature recorded as 25 °C is not identified as an outlier, whereas if it is winter with a temperature of 25 °C, then it will be identified as an outlier. The attributes of each object are divided into contextual and behavioral attributes. Contextual attributes are used to determine the context for that object, such as time and location. Behavioral attributes are the characteristics of the objects used in outlier detection, such as temperature. Thus, contextual outlier detection depends both on contextual and behavioral attributes. Therefore, the analysts are provided with the flexibility to analyze in the objects in different contexts. Figure 9.19 shows a graph with temperature recorded at different months, where 18° is detected as outlier, as it deviates from other data points.

Collective Outliers—A collection of related data objects that are anomalous with respect to the rest of the data objects in the entire data set is called collective outlier. The behavior of the individual objects as well the behavior of the objects as a group are considered to detect the collective outliers. The odd object itself may not be an outlier, but the repetitive occurrence of similar objects makes them collective outliers.

Figure 9.22 shows white color and black color data points distributed in a two-dimensional space where there is a group of data points clustered together forming an outlier. Though the individual data points by themselves are not outliers, the cluster as a whole makes them a collective outlier based on the distance between the data points, since the rest of the data points don't have a dense neighborhood.

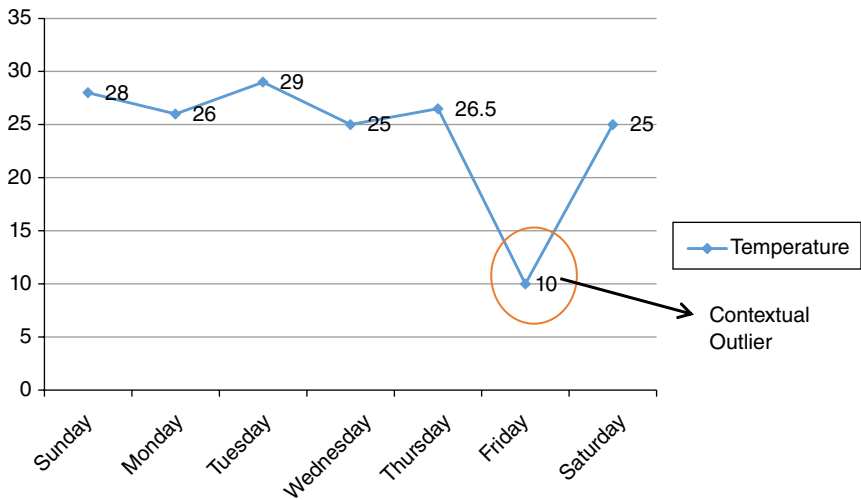
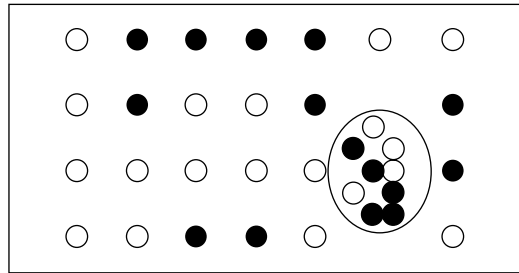


Figure 9.21 Contextual outlier.

Figure 9.22 Collective outlier.



9.8.3 Outlier Detection Techniques

Outliers can be detected based on two approaches. In the first approach, the analysts are provided with a labeled training data set where the anomalies are labeled. Obtaining such labeled anomalies is expensive and difficult as the anomalies are often labeled manually by experts. Also, the outliers are dynamic in nature, where new types of outliers may arise for which there may not be a labeled training data set available. The second approach is based on assumptions where a training data set is not available. Based on the availability of a training data set, outlier detection can be performed with one of three techniques, namely supervised outlier detection, unsupervised outlier detection, and semi-supervised outlier detection methods. Based on the assumptions, outlier detection can be performed with statistical, clustering, and proximity-based methods.

9.8.4 Training Dataset–Based Outlier Detection

There are three types of outlier detection techniques based on the availability of training data set. They are:

- Supervised outlier detection;
- Semi-supervised outlier detection; and
- Unsupervised outlier detection.

Supervised outlier detection—Supervised outlier detection is performed with the availability of training data set where the data objects that are normal as well as the outliers are labeled. A predictive model is built for normal and outliers are built, and any unseen data is compared against the predictive model to determine if the new data is a normal object or an outlier. Obtaining all types of outliers in a training data set is difficult as normal objects will be more than the number of outliers in a given data set. Since the outlier and normal classes are imbalanced because of the presence of more normal class objects, the training data set will be insufficient for outlier detection. Thus, artificial outliers are injected among normal data objects to obtain a labeled training data set.

Semi-supervised Outlier Detection—Semi-supervised outlier detection has a training set with labels only for normal data points. A predictive model is built for normal behavior, and this model is used to identify the outlier objects in the test data. Since the labels for outlier classes is not required, they are more widely adopted than supervised outlier detection. Semi-supervised outlier detection also has training sets where there are only a very small set of normal and outlier objects labeled with most of the data objects left unlabeled. A predictive model is built by labeling the unlabeled objects. An unlabeled object is labeled by evaluating its similarity with a normal labeled object. The model thus built can be used to identify outliers, by detecting the objects that do not fit with the model of normal objects.

Unsupervised Outlier detection—Unsupervised outlier detection is used under the scenarios where the labels for both normal and outlier class are unavailable. Unsupervised outlier detection is performed by assuming that the frequency of normal class objects is far more than the outlier class objects. The major drawback of unsupervised outlier detection is that a normal object may be labeled as outlier and the outliers may go undetected.

Unsupervised outlier detection can be performed using a clustering technique where the clusters are first identified, and the data points that do not belong to the cluster are identified as outliers. But the actual process of identifying the outliers is performed after the clusters are identified, which makes this method of outlier detection expensive.

9.8.5 Assumption-Based Outlier Detection

There are three types of outlier detection techniques based on assumptions, namely:

- Statistical method;
- Proximity-based method; and
- Clustering-based method.

Statistical method—The statistical method of outlier detection is performed by assuming that normal data objects follow some statistical method, and the data objects that do not follow the model are classified as outliers. The normal data points follow a known distribution, and their occurrence is found in high probability regions of the model. Outliers deviate from this distribution.

Proximity-based method—In the proximity-based method, outliers deviate from the rest of the objects in the given data set. There are two types of proximity-based methods, namely, distance-based methods and deviation-based methods. In distance-based methods, outliers are detected based on the distance from their neighbors, and normal data points have crowded neighborhoods. Outliers have neighbors that are far apart. In a density-based method, outliers are detected based on the density of the neighbors. The density of the outlier is relatively much lower than the density of its neighbors.

Clustering based method—Clustering-based outlier detecting is performed using three approaches. The first approach is executed by detecting if an object belongs to any cluster; if it does not belong to a cluster, then it is identified as outlier. Second, an outlier is detected with the distance between an object and the nearest cluster. If the distance is large, then the object is identified as an outlier. Third, it is determined whether the data object belongs to a large or a small cluster. If the cluster is very small compared to the rest of the clusters, all the objects in the cluster are classified as outliers.

9.8.6 Applications of Outlier Detection

Intrusion detection—Intrusion detection is the method of detecting malicious activities such as hacking from the security perspective of a system. Outlier detection techniques are applied to identify abnormal system behavior.

Fraud Detection—Fraud detection is used to detect criminal activities occurring in financial institutions such as banks.

Insurance claim fraud detection—Insurance claimants propose an unauthorized and illegal claim, which is very common in automobile insurance. The documents submitted by the claimant are analyzed for detecting fake documents.

Healthcare—Patient record with details such as patient age, illness, blood group, and so forth, are provided. Abnormal patient conditions or any errors in the

instrument are identified as outliers. Also, electroencephalograms (EEG) and electrocardiograms (ECG) are monitored, and any abnormality is detected as outlier.

Industries—The damages due to continuous usage and other defects in the machineries must be detected early to prevent heavy financial losses. The data is recorded and collected by the sensors and are used for analysis.

9.9 Optimization Algorithm

An optimization algorithm is an iterative procedure that compares various solutions until an optimum solution is found. Figure 9.23 shows an example of an optimization algorithm, the gradient descent optimization algorithm, which is used to find the value of coefficients of a function that minimizes the cost function. The goal is to iteratively change the values for the coefficient and evaluate the cost of the new coefficient. The coefficients that have lower cost are the best set of coefficients.

The particle swarm optimization algorithm is an efficient optimization algorithm proposed by James Kennedy and Russell Eberhart in 1995. The “particle swarm algorithm imitates human (or insects) social behavior. Individuals interact with one another while learning from their own experience, and gradually the population members move into better regions of the problem space.” The basic

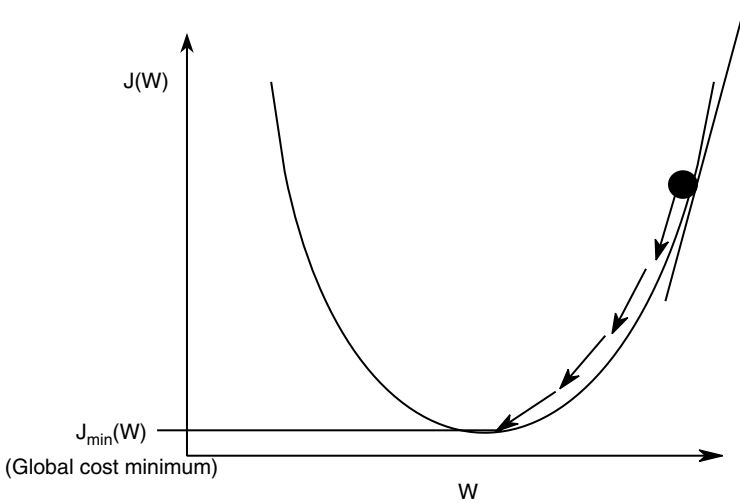


Figure 9.23 Optimization algorithm.

idea behind particle swarm algorithm is bird flocking or fish schooling. Each bird or fish is treated as a particle. Birds or fish exploring the environment in search of food is mimicked to explore the objective space in search of optimal function values.

In the particle swarm optimization algorithm, the particles are placed in the search space of a problem or a function to evaluate the objective function at its current position. Each particle in the search space then determines its movement by combining some aspects of its own best-fitness locations with those of the members of the swarm. After all the particles have moved, the next iteration takes place. For every iteration, the solution is evaluated by a target function to determine the fitness. The particles swarm through the search space to move close to the optimum value. Eventually, like birds flocking together searching for food, the particles as a whole are likely to move toward the optimum of the fitness function.

Each particle in the search space maintains:

- Its current position in the search space, x_i ;
- Velocity, v_i ; and
- Individual best position, p_i .

In addition to this, the swarm as a whole maintains its global best position gp_i . Figure 9.24 shows the particle swarm algorithm. The current position in each iteration is evaluated as a solution to the problem. If the current position x_i is found to be better than the previous position p_i , then the current values of the coordinates are stored in p_i . The values of p_i and gp_{best} are continuously updated to find the optimum value. The new position p_i is updated by adjusting the velocity v_i .

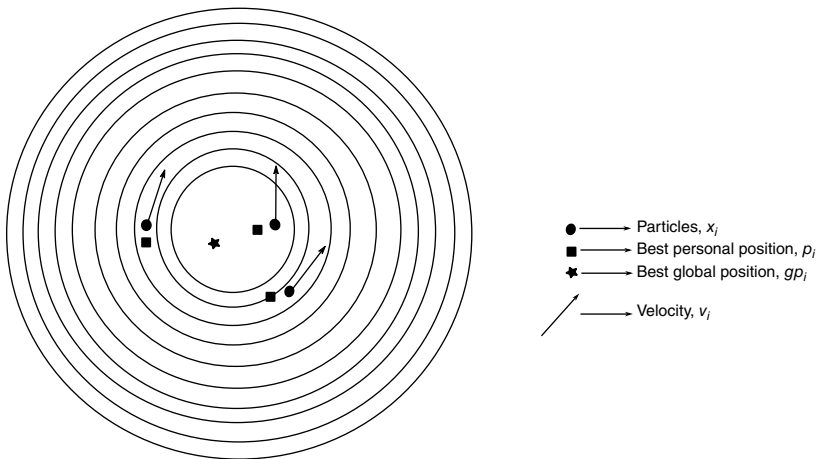


Figure 9.24 Particle swarm algorithm.

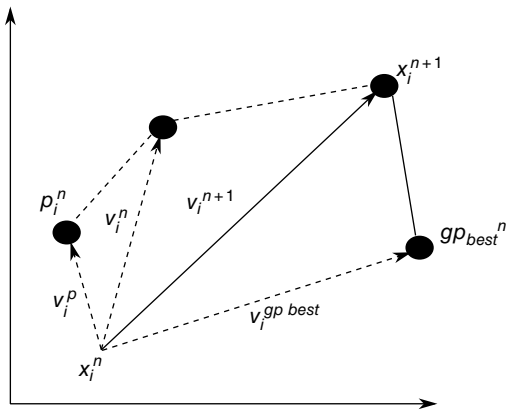


Figure 9.25 Individual particle.

Figure 9.25 shows an individual particle and its movement, its global best position, personal best position, and the corresponding velocities.

- x_i^n is the current position of the particle and the current velocity v_i^n ,
- p_i^n is the previous best position of the particle and its corresponding velocity is v_i^p ,
- x_i^{n+1} is the next position of the particle and its corresponding velocity is v_i^{n+1} ,
- gp_{best}^n is the global best position and its corresponding velocity is $v_i^{gp_{best}}$.

Figure 9.26 shows the flowchart of the particle swarm optimization algorithm. Particles are initially assigned random positions and random velocity vectors. The fitness function for the current positions is calculated for each particle. The current fitness value is compared with the best individual fitness value. If it is found to be better than the previous best fitness value, then the previous individual best fitness value is replaced by the current value. If it's not better than the previous best value, then no changes are made. The best fitness values of all the particles are compared, and the best of all the values is assigned as the global best fitness value. Update the position and velocity, and if the termination criterion is met, stop the iterations; otherwise, evaluate the fitness function.

Applications of particle swarm optimization include:

- Neural network training—Parkinson's disease identification, image recognition, etc.;
- Telecommunication;
- Signal processing;
- Data mining;
- Optimization of electric power distribution networks;
- Structural optimization;
- Transportation network design; and
- Data clustering.

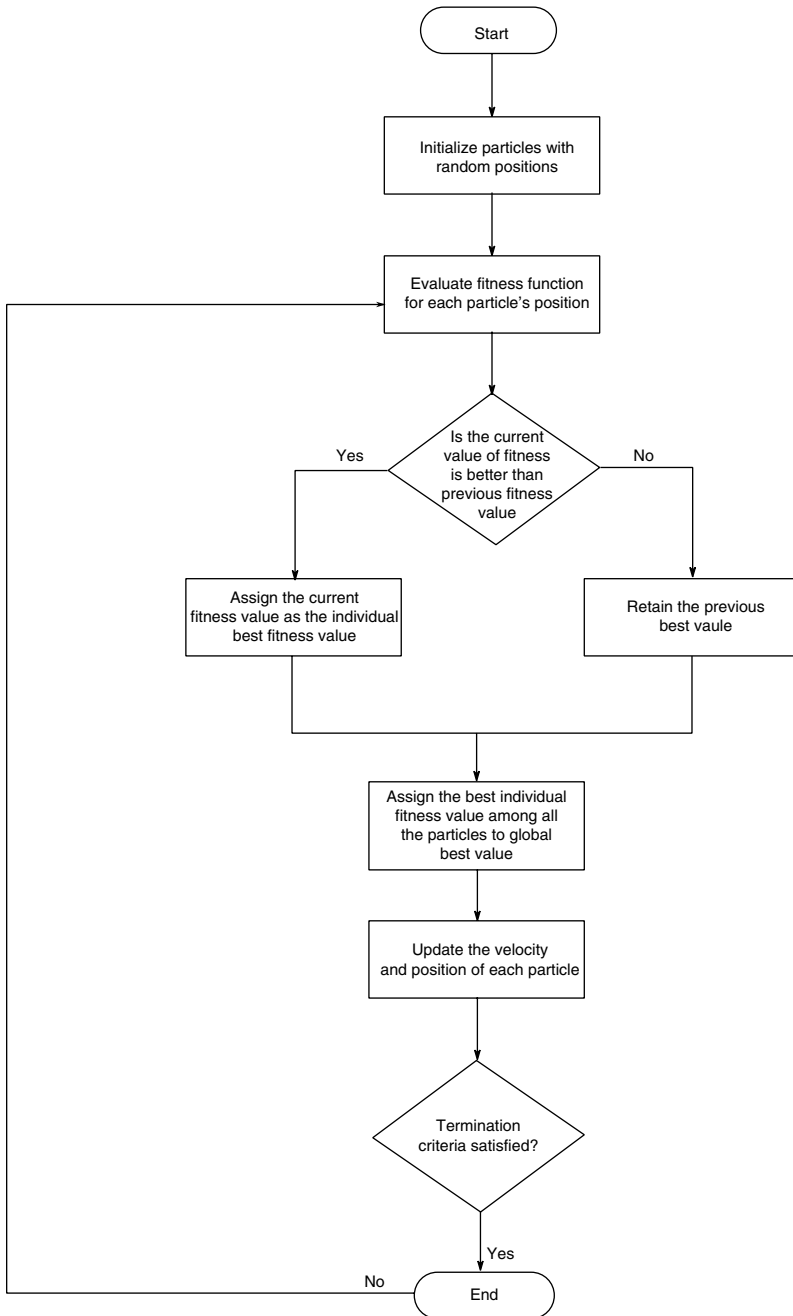


Figure 9.26 Particle swarm optimization algorithm flowchart.

9.10 Choosing the Number of Clusters

Choosing the optimal number of clusters in the clustering technique is the most challenging task. The most frequently used method for choosing the number of clusters is choosing it manually by glancing at the visualizations. However, this method results in ambiguous values for K , as some of the analysts might see four clusters in the data, which suggests $K = 4$, while some others may see two clusters, which suggests $K = 2$, or for some it may even look like the number of clusters is three. Hence, this is not always a clear-cut answer as how many numbers of clusters do exist in the data. To overcome this ambiguity, the elbow method, a method to validate the number of clusters, is used. The elbow method is implemented in the following four steps:

Step 1: Choose a range of values for K , say 1–10.

Step 2: Run the K-means clustering algorithm.

Step 3: For each value of K , evaluate sum of squared errors.

Step 4: Plot a line chart; if the line charts appears like an arm, then the value of K near the elbow is the optimum K value.

The basic idea is that the sum of squared errors should be small, but as the number of clusters K increases, the value of sum of squared errors approaches zero. The sum of squared errors is equal to zero when the number of clusters K is equal to the number of data points in the cluster. This is because each data point lies in its own cluster and the distance between the data point and the center of the cluster becomes zero. Hence, the sum of square errors also becomes zero. Hence, the goal here is to have a small value for K , and the elbow usually represents the K value where the sum of square errors diminishes when the value of K increases.

An R implementation for validating the number of clusters using the elbow method is shown below. A random number of clusters are generated with $m = 50$ data points. Figure 9.27 shows random numbers of clusters generated.

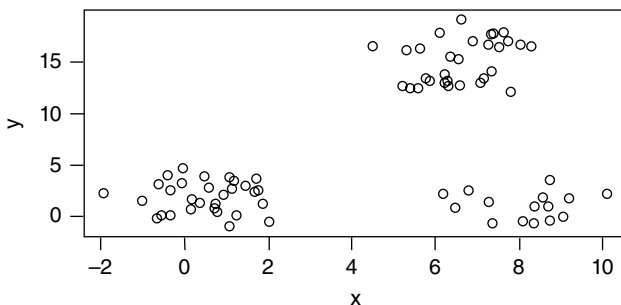


Figure 9.27 Generating random numbers of clusters.

```

> m = 50
> n = 5
> set.seed(n)
> mydata <- data.frame(x = unlist(lapply(1:n, function(i)
  rnorm(m/n, runif(1)*i^2))),
+                       y = unlist(lapply(1:n, function(i)
  rnorm(m/n, runif(1)*i^2))))
> plot(mydata, pch=1, cex=1)

```

Figure 9.28 shows the implementation of k-means clustering with $k = 3$.

```

set.seed(5)
> kmean = kmeans(mydata, 3, nstart=100)
> plot(mydata, col = (kmean$cluster + 1) , main="K-Means
with k=3", pch=1, cex=1)

```

Figure 9.29 shows Elbow method is implemented using R. It is evident from the plot that $K = 3$ is the optimum value for the number of clusters.

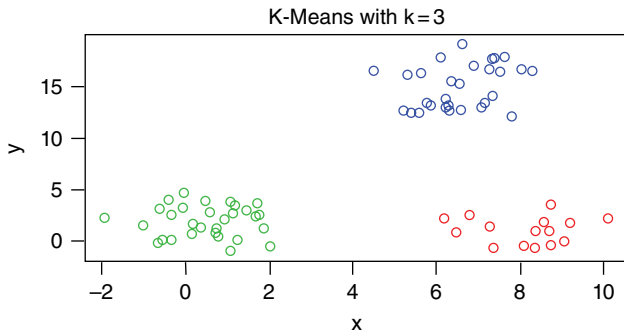


Figure 9.28 K-means clustering.

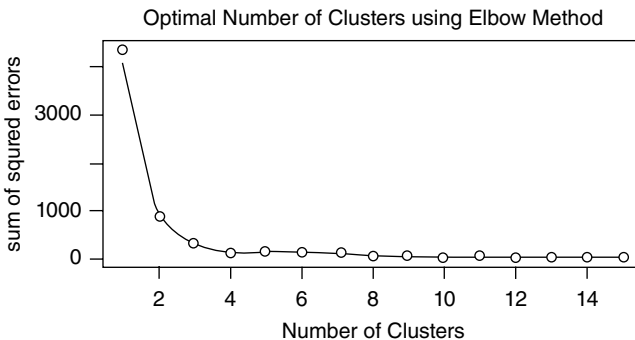


Figure 9.29 Implementation of elbow method.

```

> wss <- (nrow(mydata)-1)*sum(apply(mydata,2,var))
> for (i in 2:15) wss[i] <- sum(kmeans(mydata,
+                               centers=i)$withinss)
> plot(1:15, wss, type="b", xlab="Number of Clusters",
+      ylab="sum of squared errors",
+      main="Optimal Number of Clusters using Elbow Method",
+      pch=1, cex=1)

```

9.11 Bayesian Analysis of Mixtures

A mixture model is used to represent the subpopulation present within an overall population, for example, describing the distribution of heights in a human population that is a mixture of a male and female subpopulations.

9.12 Fuzzy Clustering

Clustering is the technique of dividing the given data objects into clusters, such that data objects in the same clusters are highly similar and data objects in different clusters are highly dissimilar. It is not an automatic process, but it is an iterative process of discovering knowledge. It is often required to modify the clustering parameters such as the number of clusters to achieve the desired result. Clustering in general is classified into conventional hard clustering and soft fuzzy clustering. In conventional clustering, each data object belongs to only one cluster, whereas in fuzzy clustering each data object belongs to more than one cluster. Fuzzy set theory, which was first proposed by Zadeh, gave the idea of uncertainty of belonging described by a membership function. It paved the way to the integration of fuzzy logic and data mining techniques in handling the challenges posed by a large collection of natural data. The basic idea behind fuzzy clustering techniques is the non-unique partition of a large data set into a collection of clusters. Each data point in a cluster is associated with membership value for each cluster it belongs to.

Fuzzy clustering is applied when there is uncertainty or ambiguity in a partition. In real-time applications there is often no sharp boundary between the classes; hence, fuzzy clustering is better suited for such data. Fuzzy clustering is a technique that is capable of capturing the uncertainty of real data and obtains a robust result as compared to that of conventional clustering techniques. Fuzzy clustering uses membership degrees instead of assigning a data object specific to a cluster. Fuzzy clustering algorithms are basically of two types. Figure 9.30 shows the types of fuzzy clustering. The most common fuzzy clustering algorithm is fuzzy c-means algorithm.

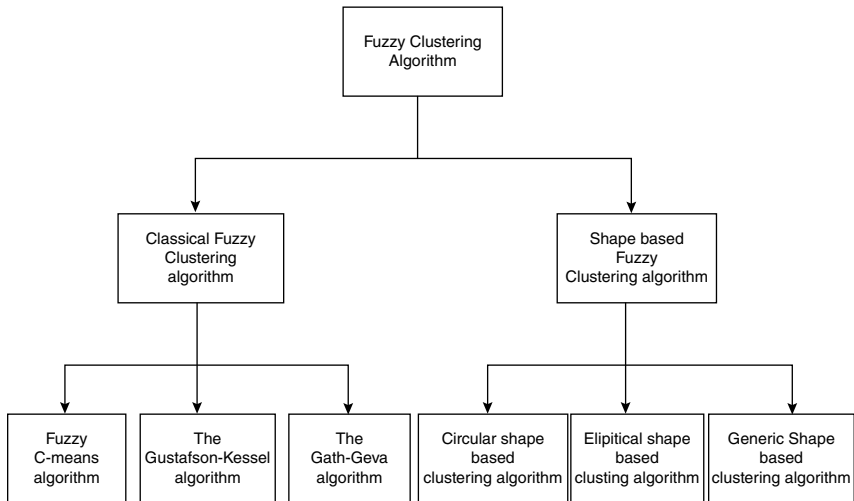


Figure 9.30 Types of fuzzy clustering.

Conventional hard clustering classifies the given data objects as exclusive subsets, i.e., it clearly segregates the data points indicating the cluster to which the data point belongs to. However, in real-time situations such a partition is not sufficient. Fuzzy clustering techniques allow the objects to belong to more than one cluster simultaneously, with different membership degrees. Objects that lie on the boundaries between different classes are not forced to completely belong to one particular class; rather, they are assigned membership degrees ranging from 0 to 1 indicating their partial membership. Thus, uncertainties are more efficiently handled in fuzzy clustering than traditional clustering techniques.

Fuzzy clustering techniques can be used in segmenting customers by generating a fuzzy score for individual customers. This approach provides more profitability to the company and improves the decision-making process by delivering value to the customer. Also, with fuzzy clustering techniques the data analyst gains in-depth knowledge into the data mining model.

A fuzzy clustering algorithm is used for target selection in finding groups of customers for targeting their products through direct marketing. In direct marketing the companies try to contact the customers directly to market their product offers and maximize their profit. Fuzzy clustering also finds its applications in the medical field.

9.13 Fuzzy C-Means Clustering

Fuzzy C-means clustering iteratively searches for the fuzzy clusters and their associated centers. The fuzzy C-means clustering algorithm requires the user to

specify the value of C , the number of clusters that are present in the data set to be clustered. The algorithm performs clustering by assigning a membership degree to each data object corresponding to each cluster center. The membership degree is assigned based on the distance of the data object from the cluster center. The more the distance from the cluster, the lower the membership toward the corresponding cluster center and vice versa. Summation of all the membership degrees corresponding to a single data object should be equal to one. After each iteration, with the change in the cluster centers, the membership degrees also change.

The major limitations of the fuzzy C-means algorithm are:

- It is sensitive to noises;
- It easily gets stuck to the local minima; and
- Its long computational time.

Since the constraint in fuzzy C-means clustering is that the membership degree of every data object to all the clusters must be one, noises are also considered the same as points that are closer to the cluster centers. However, in reality the noises are to be assigned a low or even a zero membership degree. In order to overcome the drawback of the fuzzy C-means algorithm, a new clustering model called probabilistic clustering algorithm was proposed where the column sum constraint is relaxed. Another method of overcoming the drawbacks of the fuzzy C-means algorithm is to incorporate the kernel method with the fuzzy C-means clustering algorithm, which has been proved to be robust to the noises in the data set.

10

Big Data Visualization

CHAPTER OBJECTIVE

Data Visualization, the easiest way for the end users to interpret the business analysis, is explained with various types of conventional data visualization techniques, namely, line graphs, bar charts, pie charts, and scatterplots. Data visualization, which assists in identifying the business sectors that need improvement, predicting sales volume, and more, is explained through visualization techniques, namely Pentaho, Tableau, and datameer.

10.1 Big Data Visualization

Data visualization is the process that makes the analyzed data results to be visually presented to the business users for effective interpretation. Without data visualization tools and techniques, the entire analysis life cycle carries only a meager value as the analysis results could only be interpreted by the analysts. Organizations should be able to interpret the analysis results to obtain value from the entire analysis process and to perform visual analysis and derive valuable business insights from the massive data.

Visualization makes the life cycle of Big Data complete assisting the end users to gain insights from the data. Everyone, from executives to call center employees, wants to extract knowledge from the data collected to assist them in making better decisions. Regardless of the volume of data, one of the best methods to discern relationships and make crucial decisions is to adopt advanced data analysis and visualization tools.

Data visualization is a technique where the data are represented in a systematic form for easy interpretation of the business users. It can be interpreted as the front end of big data. The benefits of data visualization techniques are improved

decision-making, enabling the end users to interpret the results without the assistance of the data analysts, increased profitability, better data analysis, and much more. Visualization techniques use tables, diagrams, graphs, and images as the ways to represent data to the users. Big data has mostly unstructured data, and due to bandwidth limitations, visualization should be moved closer to the data to efficiently extract meaningful information.

10.2 Conventional Data Visualization Techniques

There are many conventional data visualization techniques available, and they are line graphs, bar charts, scatterplots, bubble plots, and pie charts. Line graphs are used to depict the relationship between one variable and another. Bar charts are used to compare the values of data belonging to different categories represented by horizontal or vertical bars, the height of which represents the actual value. Scatterplots are similar to line graphs and are used to show the relationship between two variables (X and Y). A bubble plot is a variation of a scatterplot where the relationship of X and Y is displayed in addition to the data value associated with the size of the bubble. Pie charts are used where parts of a whole phenomenon are to be compared.

10.2.1 Line Chart

A line chart has vertical and horizontal axes where the numeric data points are plotted and connected, which results in a simple and straightforward way to visualize the data. The vertical Y axis displays some numeric value, and the horizontal X axis displays time or other category. Line graphs are specifically useful in viewing trends over a period of time, for instance, the change in stock price over a period of 5 years, the increase in gold price in the past 10 years, or revenue growth of a company in a quarter.

Figure 10.1 depicts the line graph depicting the increase in gold rate from the year 2004 to 2011.

10.2.2 Bar Chart

Bar charts are the most commonly used data visualization techniques as they reveal the ups and downs at a glance. The data can be either discrete or continuous. The numeric values are represented along the horizontal X axis, and the time series runs along the vertical Y axis. Bar charts are used to visualize data such as percentage of expenditure by each department in a company or monthly sales of a company. Figure 10.2 shows the bar chart with monthly sales of a company.

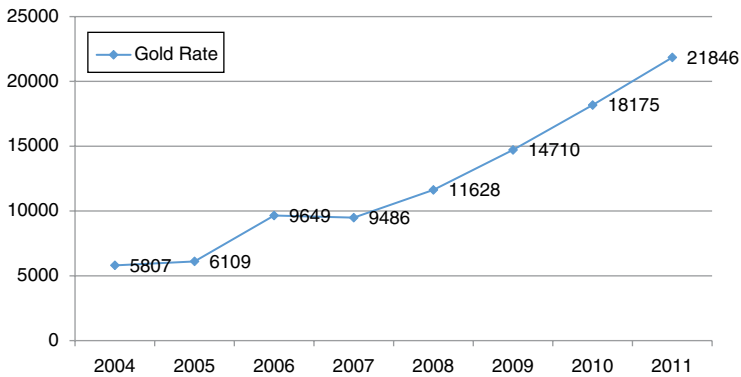


Figure 10.1 Increase in gold rate from 2004 to 2011.

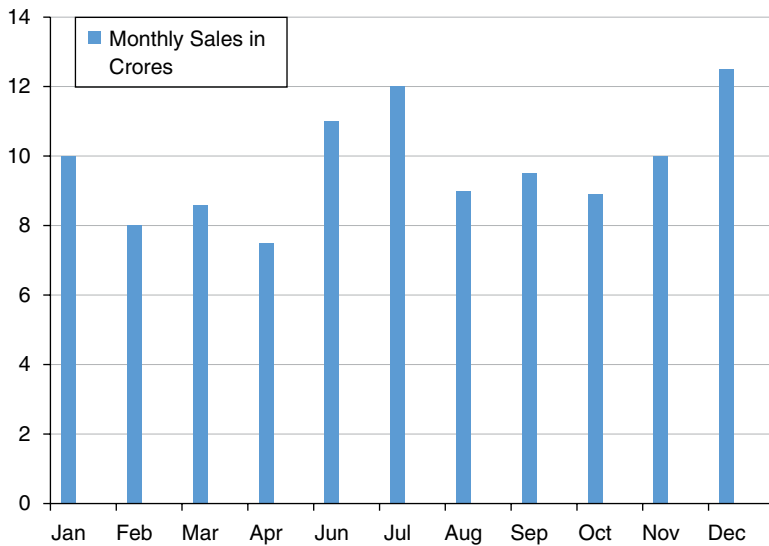


Figure 10.2 Bar chart—monthly sales of a company.

10.2.3 Pie Chart

Pie charts are the best visualization technique used for part-to-whole comparison. The pie chart can also be in a donut shape with either the design element or the total value in the center. The drawback with pie charts is that it is difficult to differentiate the values when there are too many slices, which decreases the effectiveness of visualization. If the small slices are less significant, they can be grouped together tagging them as a miscellaneous category. The total of all percentages should be equal to 100%, and the size of the slices should be proportionate to their percentages.

Activities

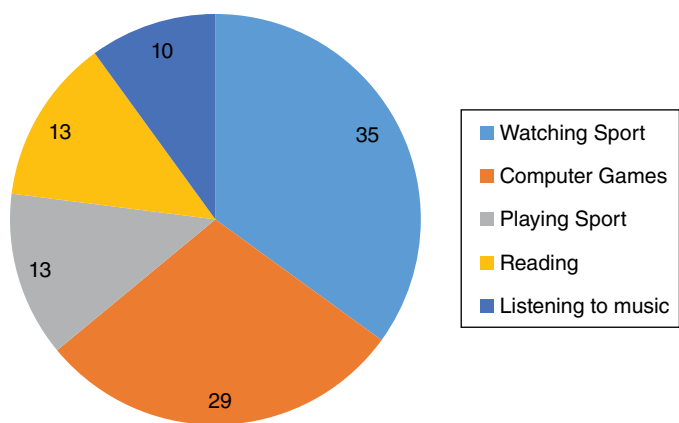


Figure 10.3 Pie chart—favorite activities of teenagers.

10.2.4 Scatterplot

A scatterplot is used to show the relationship between two groups of variables. The relationship between the variables is called correlation.

Figure 10.4 depicts a scatterplot. In a scatterplot both axes represent values.

10.2.5 Bubble Plot

A bubble plot is a variation of a scatterplot where bubbles replace the data points. Similar to scatterplots, both the X and Y axes represent values. In

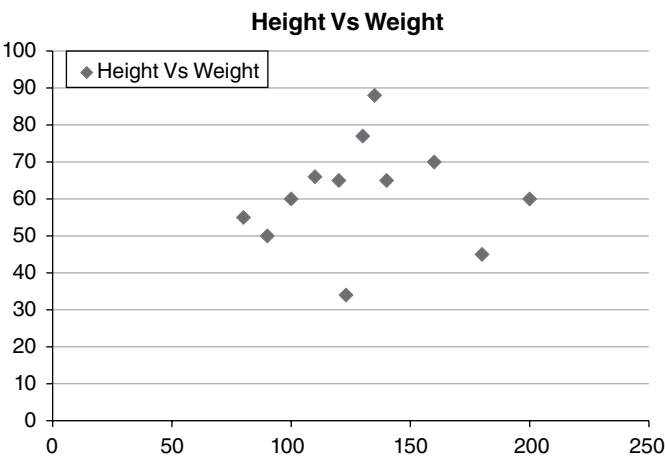


Figure 10.4 Scatter plot—height vs. weight.

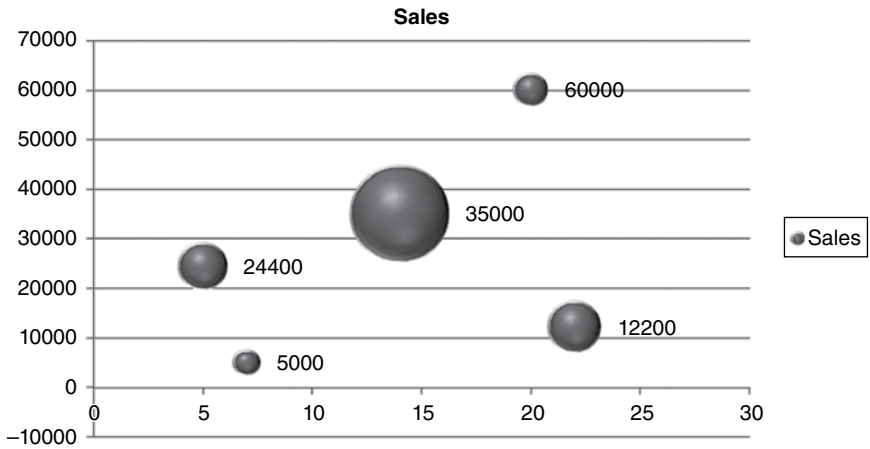


Figure 10.5 Bubble plot—Industry market share study.

In addition to X and Y values plotted in a scatterplot, a bubble plot represents X, Y, and size values.

Figure 10.5 depicts a bubble plot where the X axis represents the number of products, the Y axis represents the sales, and the size of the bubbles represents the market share percentage.

10.3 Tableau

Tableau is a data analysis software that is used to communicate data to the end users. Tableau is capable of connecting to the files, relational databases, and other big data sources to acquire the data and process them. Tableau mission statement is, “We help people see and understand data.” VizQL, a visual query language, is used to convert the drag-and-drop actions by the users into queries. This permits the users to understand and share the underlying knowledge in the data. Tableau is used by business analysts and academic researchers to perform visual data analysis.

Tableau has many unique features, and its drag-and-drop interface is user-friendly and allows the users to explore and visualize the data. The major advantages of Tableau are:

- It does not require any expertise in programming, and anyone with access to the required data can start using the tool to explore and discover the underlying value from the data.
- Tableau does not require any big software setup to run it. The desktop version of the Tableau, which is the most frequently used Tableau product, is easy to install and to perform data analysis with it.

- It does not require any complex scripts to be written as almost everything can be performed by drag-and-drop actions by the users.
- Tableau is capable of blending data from various data sources in real time, thus saving the integration cost in unifying the data.
- One centralized data storage location is provided by the Tableau Server to organize all the data of a particular organization.

With Tableau the analyst first connects to the data that are stored in the files, warehouses, databases such as HDFS, and other data storage platforms. The analyst then interacts with Tableau to query the data and view the results in the form of charts, graphs, and so forth. The results can be arranged on a dashboard. Tableau is used both as a communication tool and a tool for data discovery, that is, to find the insight underlying in the data.

There are four types of Tableau products, namely:

- 1) Tableau Desktop;
- 2) Tableau Server;
- 3) Tableau Online;
- 4) Tableau Public.

Tableau Desktop—Tableau Desktop comes in two versions: a personal version and a professional version. The major difference between the personal and professional versions is the amount of data sources that can be connected to Tableau. The personal version of Tableau Desktop allows the users to connect only to the local files, whereas the professional version allows the users to connect to a variety of data sources and save the data to the user's server of the Tableau Server.

Tableau Public—Tableau Public is a free to download software, and by the word “public,” it means that the visualizations can be viewed by anyone, but there is no option to save the workbooks locally to the user's personal computer. Though it can be visualized by other users, the workbook cannot be saved or downloaded to their personal computer. Tableau Public has all the features similar to that of Tableau Desktop, and it can be opted when the user wants to share the data. Thus, it is used both for development and sharing and is suitable for journalists and bloggers.

Tableau Server—Tableau Server is used to interact with visualizations and share them across the organization securely. To share the workbook with the organization, Tableau Desktop must be used to publish them in the Tableau Server. Once the visualizations are published, licensed users can access them through a web browser. For enterprise wide deployment, users who require access to the visualizations should possess individual licenses.

Tableau Online—Tableau online has functionalities similar to Tableau Server, but it is hosted in their cloud by Tableau. This is used by companies and is the solution for storing and accessing the data in the cloud.

All the four products of Tableau incorporate the same visualization user interface. The basic difference is in the types of data sources the users can connect and the method of sharing the visualizations with other users.

Two other minor products with Tableau are:

- Tableau Public Premium; and
- Tableau Reader.

Tableau Public Premium—It is an annual premium subscription that allows the users to prevent the viewers of visualizations that are hosted on Tableau Public from downloading the workbook.

Tableau Reader—Tableau Reader is an open-source Windows application that allows customers to open a saved Tableau workbook. It also allows the users to interact with the visualizations that were created and saved locally with the desktop version of Tableau or interact with workbooks that are downloaded from Tableau Public. However, it restricts the creation of new visualizations or modification of the existing ones.

Tableau connects to the data engines directly, and the data can be extracted locally. Visual Query (VizQL) language was developed as a research project by Stanford University. VizQL allows the users to translate their drag-and-drop actions in a visual environment into a query language. Thus, VizQL does not require the users write any lengthy code to query the database.

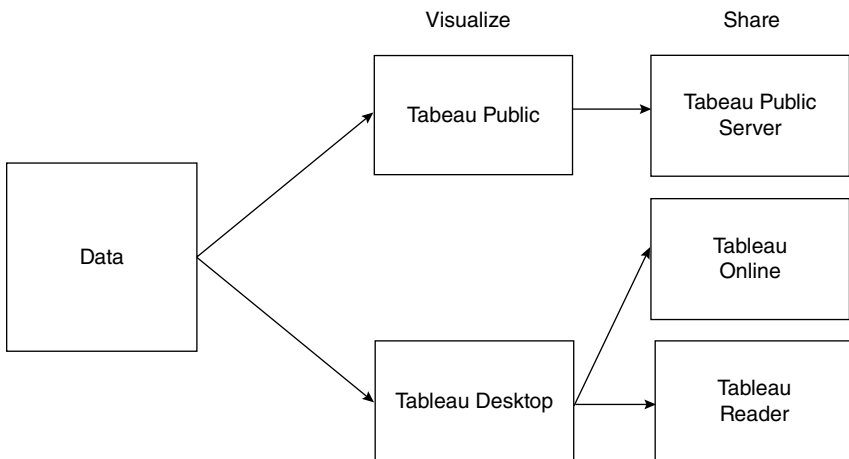


Figure 10.6 Visualizing and sharing with Tableau.

10.3.1 Connecting to Data

A connection in Tableau means a connection made to only one set of data, which may be a database or files in a database or tables. A data source in Tableau can have more than one connection. The connections in the data source can be joined.

Figure 10.7 shows a Tableau Public workbook. The left side panel of the workbook has the options to make connections to a server or to files such as Excel, text file, Access, JSON files, and statistical files. An Excel file is a file with extension .xls, .xslm, or .xlsx, created in Excel. A text file has extensions .txt or .tab. An Access file is a file with extension .mdb or .accdB that was created in Access. A statistical file is a file with extension .sav, .sas7bdat, or .rda that was created by statistical tools.

Database servers such as an SQL server host data on server machines and use database engines to store data. These data are served to client applications based on the queries. Tableau retrieves data from the servers for visualization and analysis. Also, data can be extracted from the servers and stored in Tableau Data Extract.

Connection to an SQL server requires authentication information and server name. A database administrator can use an SQL server username and password to gain access. With SQL server users can read uncommitted data to improved performance. However, it may produce unpredicted results if the data is altered at the same time a Tableau query is performed.

Once the database is selected, the users have several options:

- The user can select a table that already existed in the selected database;
- The user can write new SQL scripts to add new tables; and
- Stored procedures that returns tables may be used.

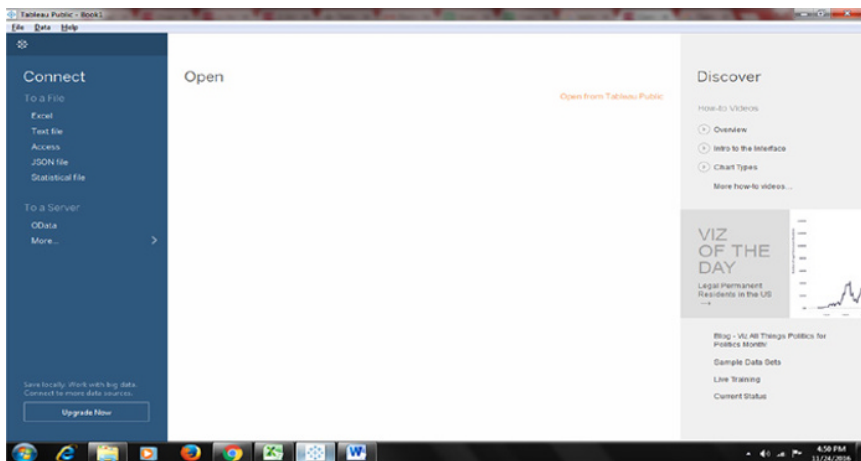


Figure 10.7 Tableau workbook.

10.3.2 Connecting to Data in the Cloud

Data connections can be made to the data that are hosted in a cloud environment such as Google Sheets. Figure 10.8 shows the Tableau start page.

When the Google Sheets tab is selected, a pop-up screen appears requesting the user to provide the login credentials. The user can login with the Google account, and once the user allows the Tableau with appropriate permissions, the user will be presented with the list of all available Google Sheets associated with the Google account.

10.3.3 Connect to a File

To connect to a file in Tableau, navigate to the appropriate option (Excel/Text File/Access/JSON file/PDF file/Spatial file/Statistical file) from the left panel of the Tableau start page. Select the file you want to connect to and then click open. Figure 10.9 shows a CSV file with US cities, county, area code, zip code, population of each city, land area, and water area.

Tableau categorizes the attributes automatically into the different data types shown in Table 10.1. The data type reflects the type of information stored in the field. The data type is identified in the data pane as shown in Figure 10.9 by one of the symbols in Table 10.1.

Navigate to sheet1 tab to view the screen as shown in Figure 10.10. Tableau categorizes the attributes into two groups: dimensions and measures. The attributes with discrete categorical information where the values are Boolean values or strings are assigned to dimensions. Examples of attributes that may be assigned to dimensions are employee id, year of birth, name, and geographic data such as states and cities. Those attributes that contain any quantitative or numerical

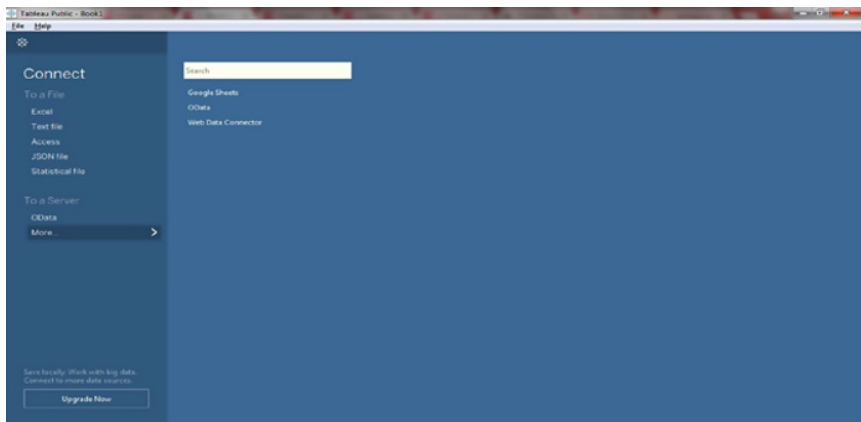


Figure 10.8 Tableau start page.

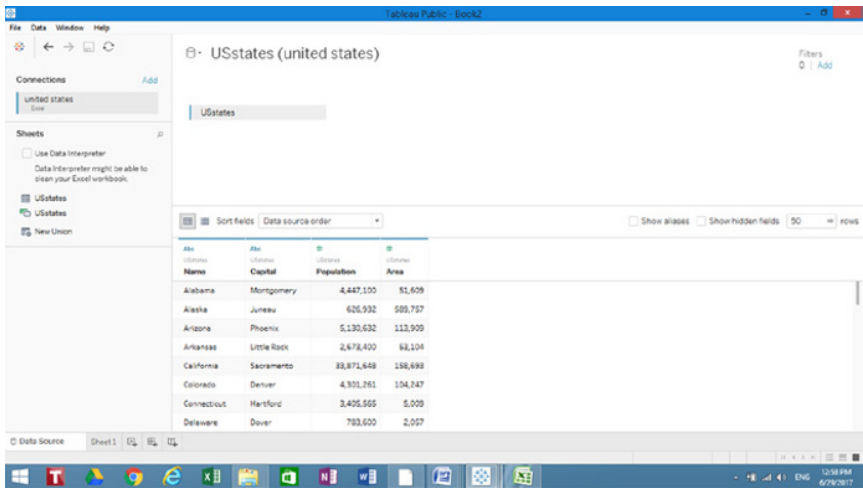
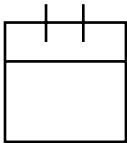
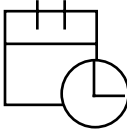
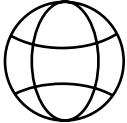


Figure 10.9 CSV file connected to Tableau.

Table 10.1 Tableau data types.

Abc	String Values
	Date values
	Date and time values
	Numerical values
	Boolean values
	Geographic values

information are assigned to measures. Examples of attributes that may be assigned to measures area are average sales, age, and crime rate. Tableau is not capable of aggregating the values of the fields under the dimension area. If the values of the field have to be aggregated, it must be a measure. In such cases the field can be

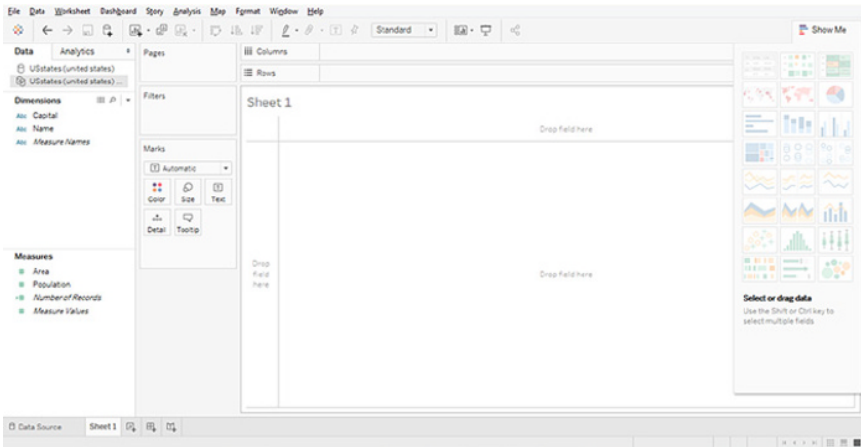


Figure 10.10 Tableau worksheet.

converted into a measure. Once the field is converted, Tableau will prompt the user to assign an aggregation such as count or average.

In some cases, Tableau may interpret the data incorrectly but then the data type of the field can be modified. For example, in Figure 10.9 the data type of the name field is wrongly interpreted as string. It can be modified by clicking on the data type icon of the Name field as shown in Figure 10.11. The name of the attributes can also be modified in a similar fashion. Here the Name field is changed to state for better clarity.

The attributes can be dragged to either the row or columns shelf in the Tableau interface based on the requirement. Here, to create a summary table with states in

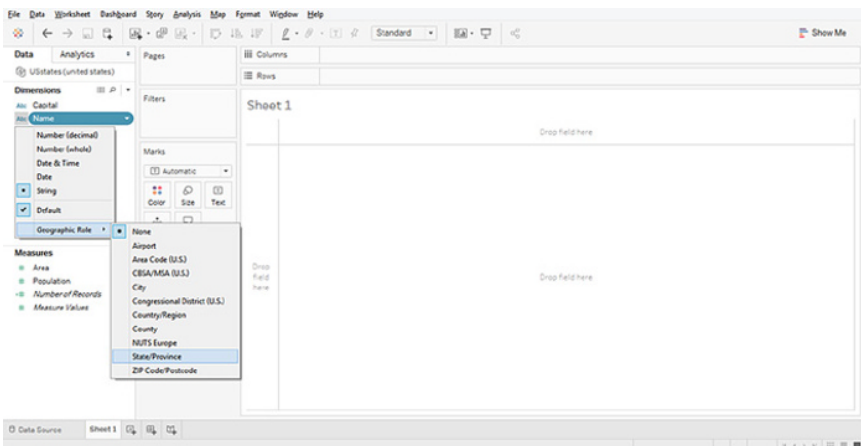
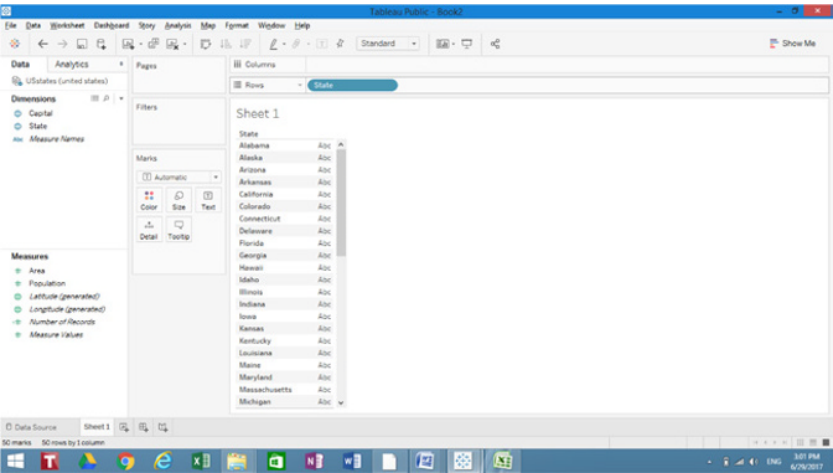
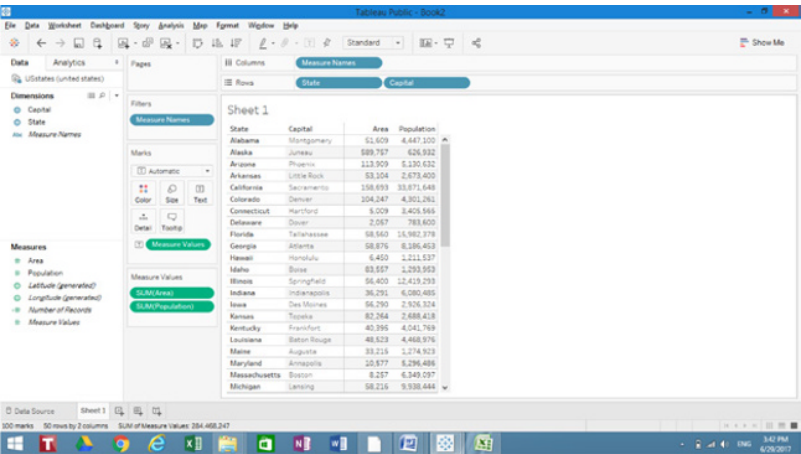


Figure 10.11 Modifying the data type in tableau.

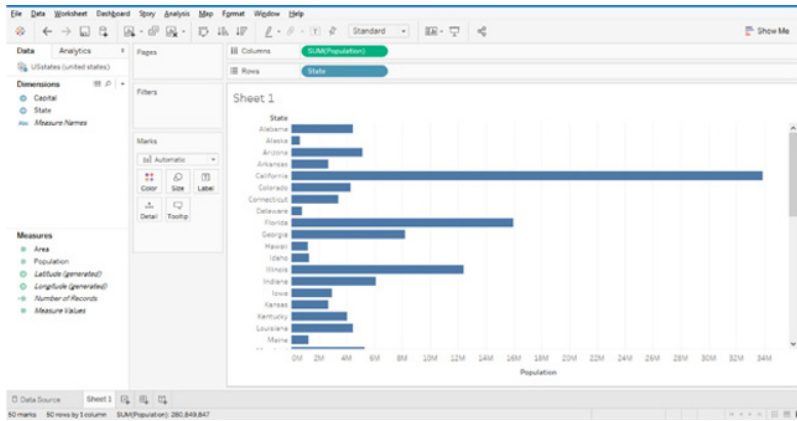
USA and their capitals and the population of each state, the state field is dragged and dropped in the rows shelf, as shown below.



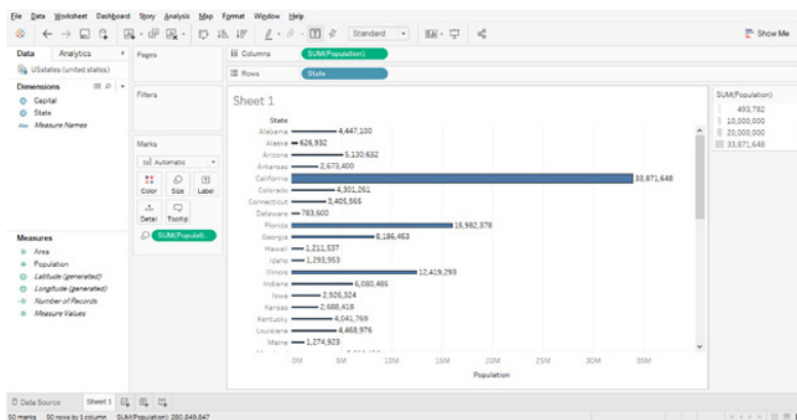
To display the population of each state, drag the population field either over the text tab in the marks shelf or drag it over the “Abc” in the summary table. Both will yield the same result. Similarly, the capital field can also be dragged to the summary table. The actions performed can be reverted by using ctrl+z or the backward arrow in the Tableau interface. When we don’t want any of these fields to be displayed in the summary table, it can be dragged and dropped back to the data pane. The data pane is the area that has the dimension and measure classification. New sheets can be added by clicking on the icon near the sheet1 tab.



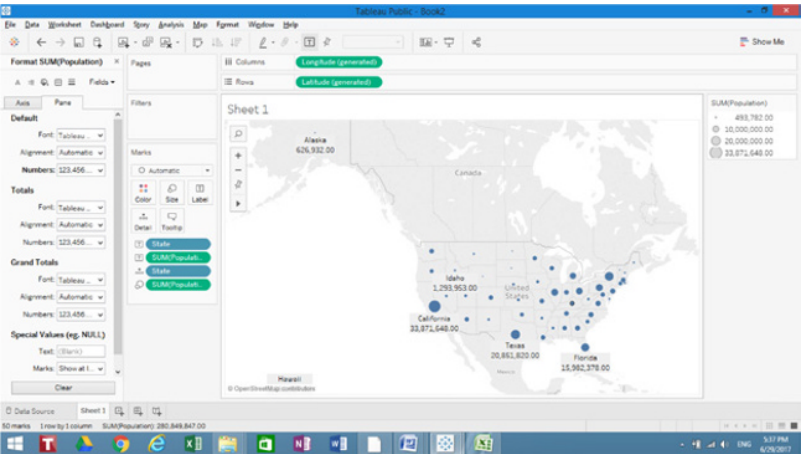
The summary table can be converted to a visual format. Now to create a bar chart out of this summary table, click on the “Show Me” option in the top right corner. It will display the various possible options. Select the horizontal bars. Let us display the horizontal bars only for the population of each state.



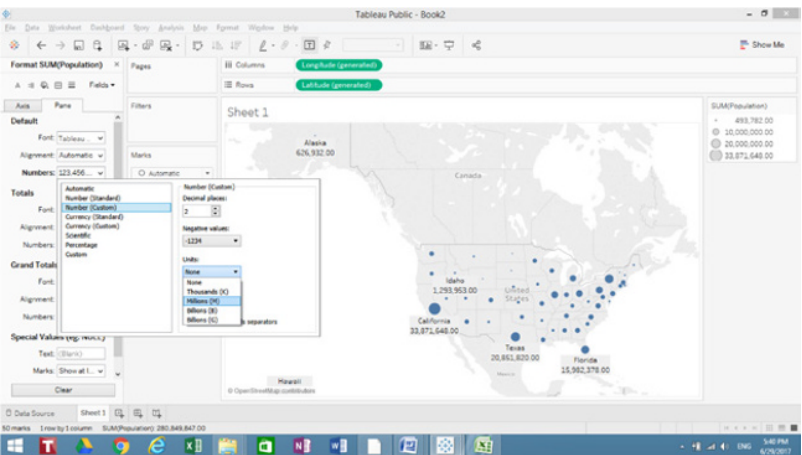
The marks shelf has various formatting options to make the visualization more appealing. The “Color” option is used to change the color of the horizontal bars. When the state field is dragged and dropped over the color tab, each state will be represented by different colored bars. And to change the width of the bars according to the density of the population, drag the population field over the size tab of the marks shelf. Labels can be added to display the population of each state by using the “Label” option in the marks shelf. Click on the “Label” option and check the “Show Marks Label” option. Different options of the marks shelf can be experimented to make changes in the visualization to suit our requirements.



The same details can be displayed using the map in the “Show Me” option. The size of the circle shows the density of the population. The larger the circle, the greater the density of the population.



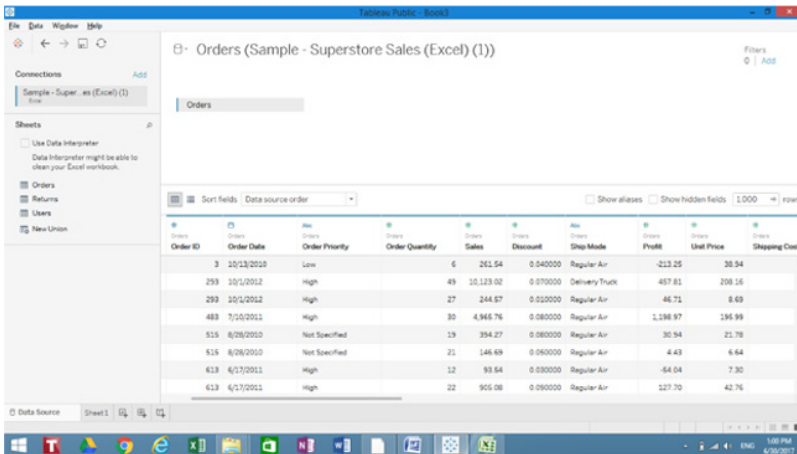
The population displayed in large number can be formatted to show in millions by formatting the units of population. Right-click on the Sum(Population) tab of the “Text Label” option in the marks shelf. Make the selection as shown below to change the unit to millions.



10.3.4 Scatterplot in Tableau

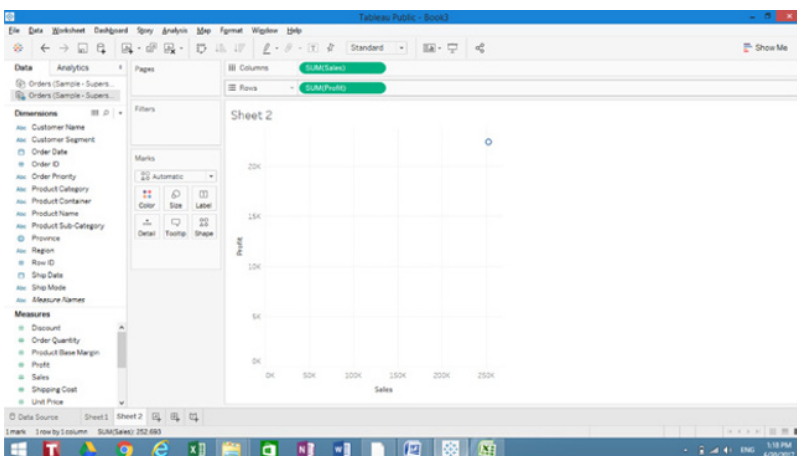
A scatterplot can be created where at least one measure has to be placed in the columns shelf and at least one measure has to be placed in the rows shelf. Let us consider supermarket data with several attributes such as customer name,

customer segment, order date, order id, order priority, and product category, but let's take into consideration only those attributes that are under our scope. This file is available as a Tableau sample file “Sample – Superstore Sales(Excel).xls.” For better understanding, we have considered only the first 125 rows of data.



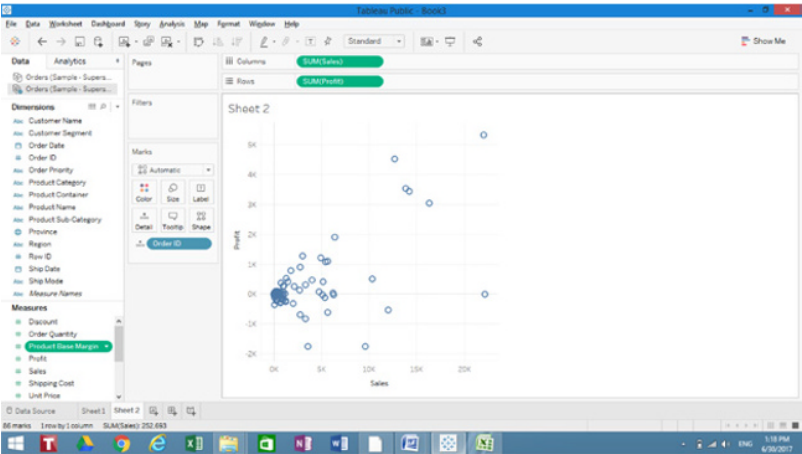
Order ID	Order Date	Order Priority	Order Quantity	Sales	Discount	Ship Mode	Profit	Unit Price	Shipping Cost
3	10/13/2010	Low	6	261.54	0.040000	Regular Air	-213.25	38.54	
293	10/12/2012	High	49	10,123.02	0.070000	Delivery Truck	487.81	208.16	
293	10/12/2012	High	27	244.87	0.020000	Regular Air	46.71	8.69	
483	7/10/2011	High	30	4,946.76	0.080000	Regular Air	1,186.97	166.99	
515	8/28/2010	Not Specified	19	394.27	0.080000	Regular Air	30.94	21.78	
515	8/28/2010	Not Specified	21	146.49	0.060000	Regular Air	4.43	6.64	
613	6/17/2011	High	12	93.64	0.030000	Regular Air	64.04	7.30	
613	6/17/2011	High	22	905.08	0.030000	Regular Air	127.70	42.76	

Let us investigate the relationship between sales and profits by a scatterplot. Drag the sales to the columns shelf and profits to the rows shelf.

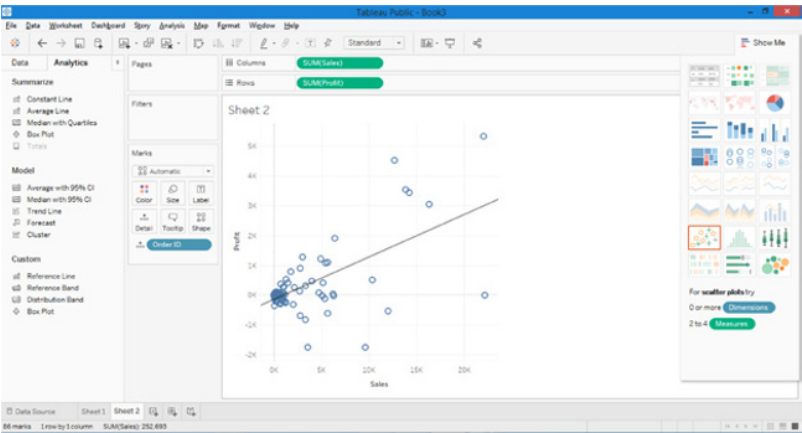


We will get only one circle; this is because Tableau has summed up the profits and sales, so eventually we will get one sum for profit and one sum for sales, and the intersection of this summed-up value is represented by a small circle. This is not what is expected: we are investigating the relationship between sales and profits. We want to investigate the relationship for all the orders, and the orders are identified by the order id. Hence, drag the order id and place over the “Detail” tab

in the marks shelf. The same can be obtained by two other ways. One way is to drag the order id directly over the scatterplot. Another way is clear the sheet and select the three fields order id, profit, and sales by pressing the ctrl key, use the “Show Me” option in the top right corner, and select the scatterplot. The same result can be obtained by using either of these ways.



To better interpret the relationship, let us add a trend line. A trend line renders a statistical definition of the relationship between two values. To add a trend line, navigate to the “Analysis” tab and click on the trend line under measure.

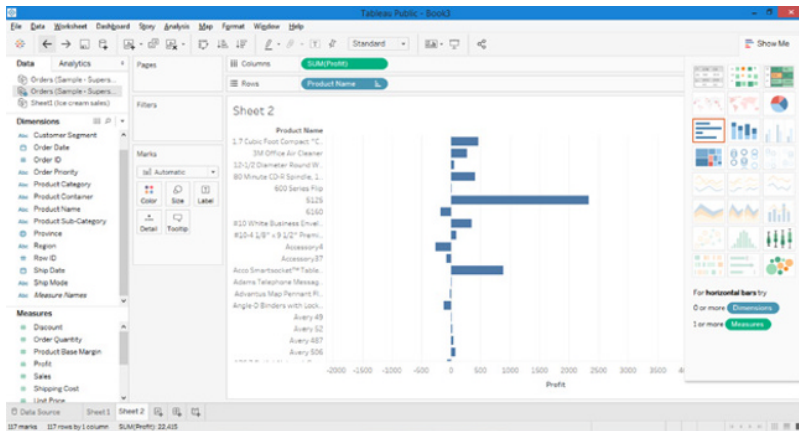


10.3.5 Histogram Using Tableau

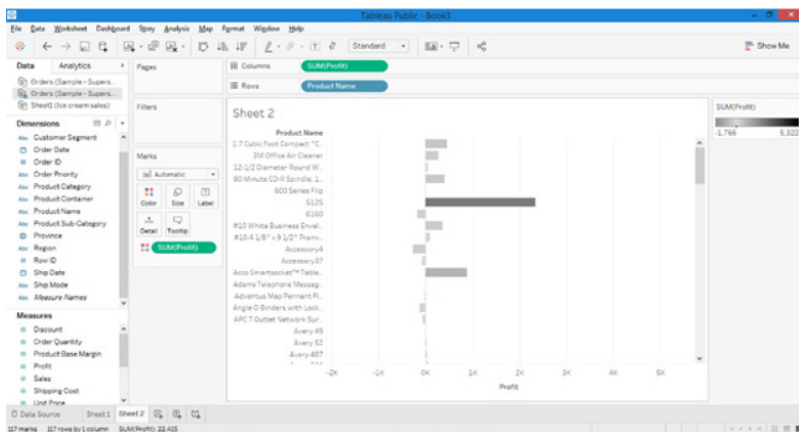
A histogram is a graphical representation of data with bars of different heights. Results of continuous data such as height or weight can be effectively represented by histograms.

10.4 Bar Chart in Tableau

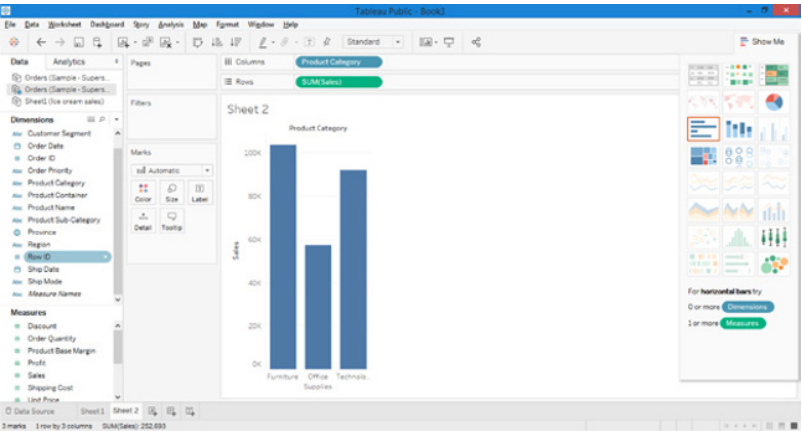
Bar charts are graphs with rectangular bars. The height of the bars is proportional to the values that the bars represent. Bar charts can be created in Tableau by placing one attribute in the rows shelf and one attribute in the columns shelf. Tableau automatically produces a bar chart if appropriate attributes are placed in the row and column shelves. “Bar chart” can also be chosen from the “Show Me” option. If data is not appropriate, then the bar “Chart” option from the “Show Me” button will automatically be grayed out. Let us create a bar chart to profit or loss for each product using the bar chart option. Drag profit from measures and drop it to the columns shelf and drag product name from dimensions and drop it to rows shelf.



The Color can be applied to the bars from the marks shelf based on their ranges. The longer bars are applied darker shades, and the smaller bars are applied lighter shades by Tableau.

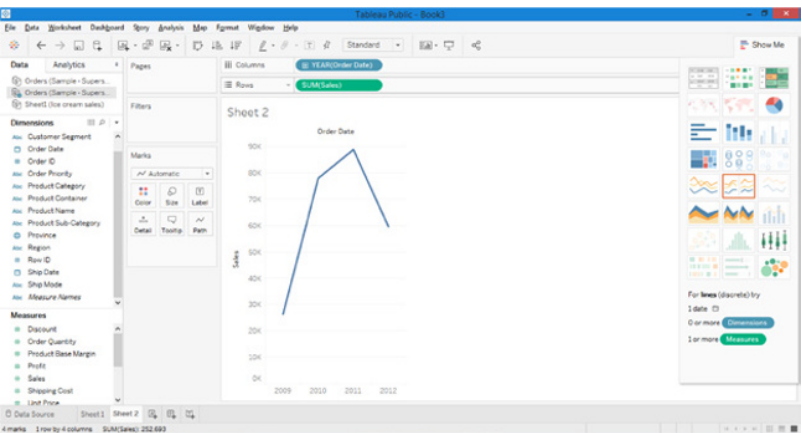


Similarly, a bar chart can be created for product category and the corresponding sales. Drag the product category from dimensions to the columns shelf and sales to the rows shelf. A bar chart will be automatically created by Tableau.

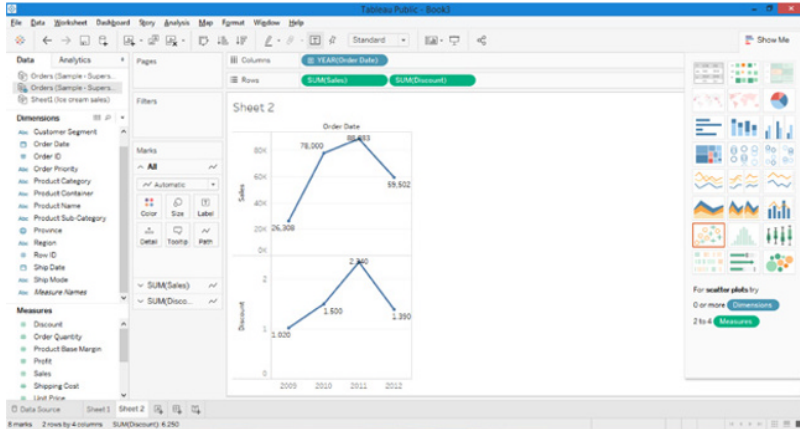


10.5 Line Chart

A line chart is a type of chart that represents a series of data points connected with a straight line. A line chart can be created in Tableau by placing zero or more dimensions and 1 or more measures in the rows and columns shelves. Let us create a line chart by placing the order date from dimensions into the columns shelf and sales from the measures to the rows shelf. A line chart will automatically be created depicting the sales for every year. It shows that peak sales occurred in the year 2011.

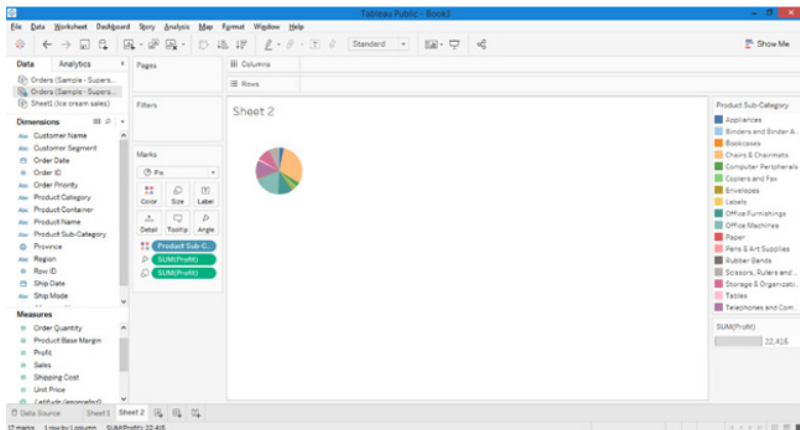


A line chart can also be created by using one dimension and two measures to generate multiple line charts, each in one pane. Line charts in each pane represent the variations corresponding to one measure. Line charts can be created with labels using show mark labels option from label in the marks shelf.

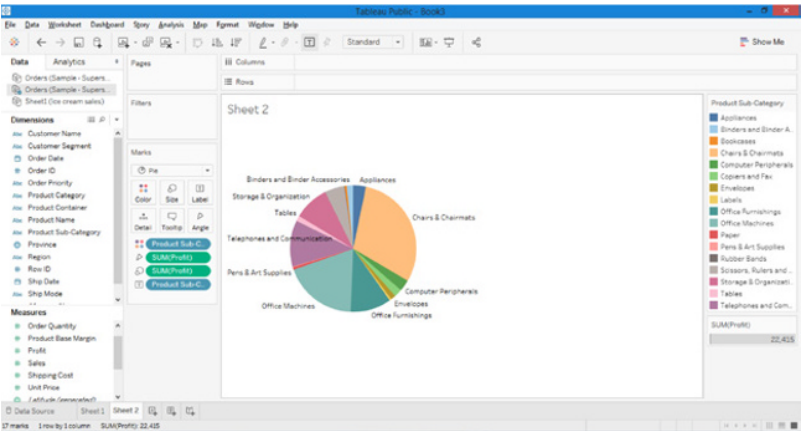


10.6 Pie Chart

A pie chart is a type of graph used in statistics where a circle is divided into slices, with each slice representing a numerical portion. A pie chart can be created by using one or more dimensions and one or two measures. Let us create a pie chart to visualize the profit for different product subcategories.

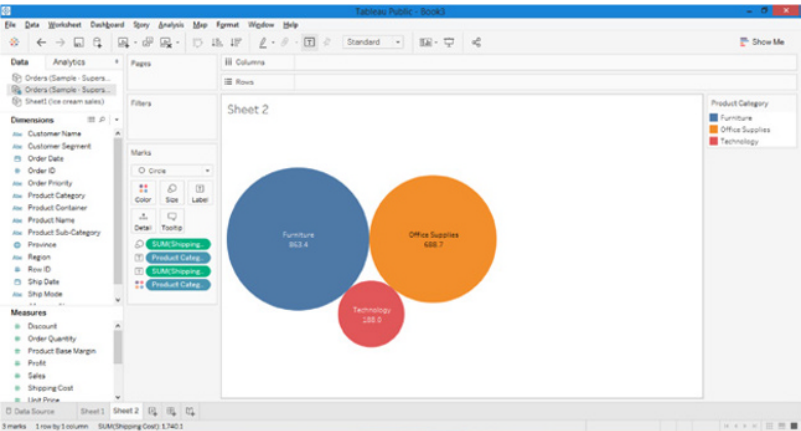


The size of the pie chart can be increased by using ctrl + shift + b. And product sub-category can be dragged to label in marks shelf to display the name of the products.



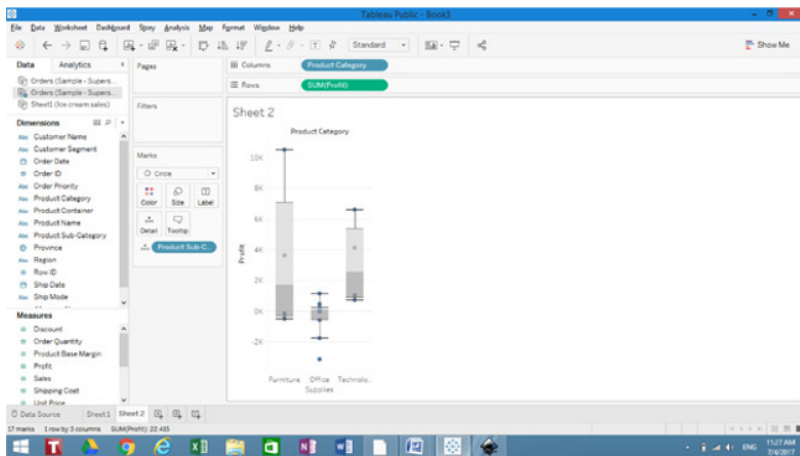
10.7 Bubble Chart

A bubble chart is a chart where the data points are represented as bubbles. The values of the measure are represented by the size of each circle. Bubble charts can be created by dragging the attributes to the rows and column shelves or by dragging the attributes to the size and label in the marks shelf. Let us create a bubble chart to visualize the shipping cost of different product categories such as furniture, office supplies, and technology. Drag the shipping cost to the size and products category to label in the marks shelf. Shipping cost can again be dragged to label in the marks shelf to display the shipping cost.



10.8 Box Plot

A box plot, also known as a box-and-whisker plot, is used to represent statistical data based on the minimum, first quartile, median, third quartile, and maximum. A rectangle in the box plot indicates 50% of the data, and the remaining 50% of the data is represented by lines called whiskers on both sides of the box. Figure 10.12 shows a box plot.



10.9 Tableau Use Cases

10.9.1 Airlines

Let us consider the airlines data set with three attributes, namely, region, period (financial year), and revenue. Data sets for practicing Tableau may be downloaded from Tableau official website: <https://public.tableau.com/en-us/s/resources>.

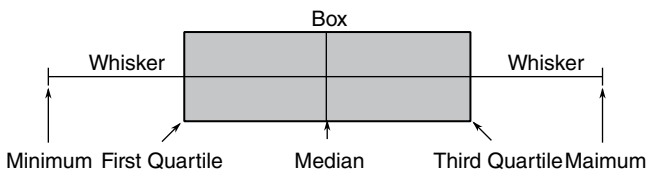
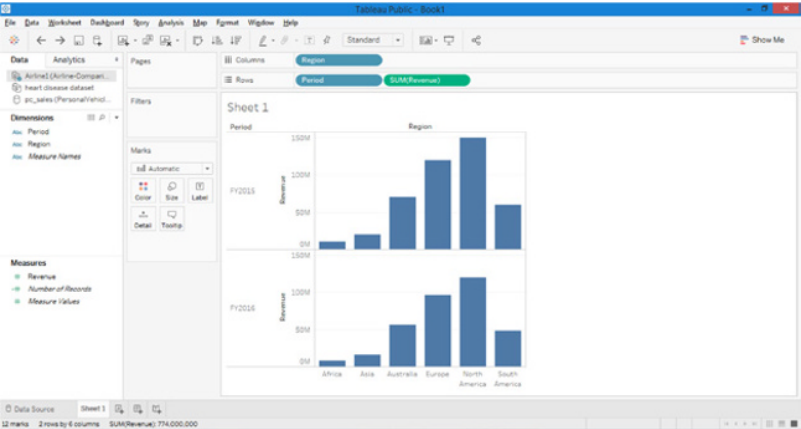


Figure 10.12 Box plot.

Let us visualize the revenue made up in different continents during the financial years 2015 and 2016. To create the visualization, drag the Region dimension to the columns shelf and period and revenue to the rows dimension. The visualization clearly shows that the revenue yielded by North America is the highest, and the revenue yielded by Africa is the lowest.



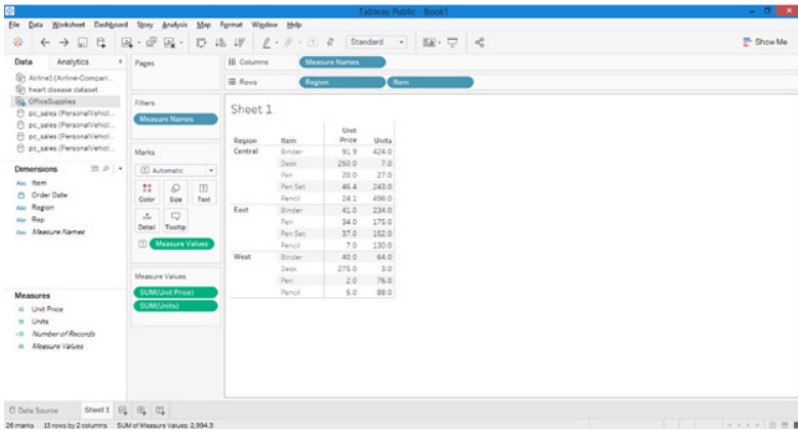
10.9.2 Office Supplies

Let us consider the file below showing the stationery orders placed by a company from East, West, and Central and the number of units and the unit price of each item.

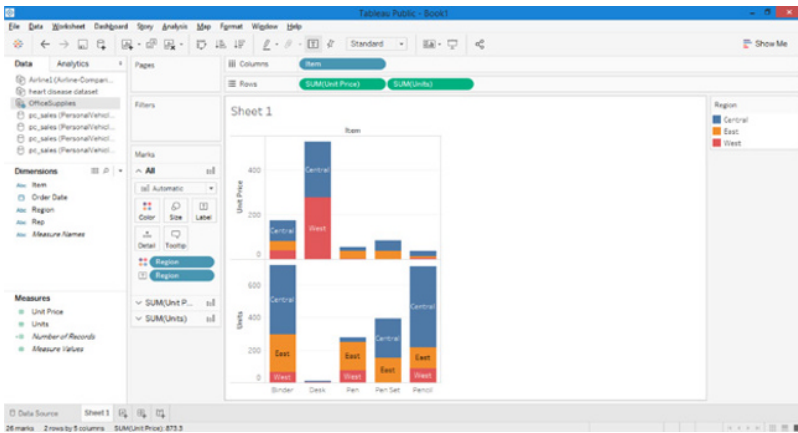
The screenshot shows the Tableau Public interface with a table titled 'OfficeSupplies'. The table has columns: Order Date, Region, Rep, Item, Units, and Unit Price. The data is sorted by Order Date.

Order Date	Region	Rep	Item	Units	Unit Price
7/4/2014	East	Richard	Pen Set	62	4.990
7/12/2014	East	Nick	Binder	29	1.990
7/21/2014	Central	Morgan	Pen Set	55	12.490
7/29/2014	East	Susan	Binder	81	19.990
8/7/2014	Central	Matthew	Pen Set	42	23.990
8/15/2014	East	Richard	Pencil	35	4.990
8/24/2014	West	James	Desk	9	279.000
9/1/2014	Central	Smith	Desk	2	128.000

To create a summary table with region, item, unit price, and the number of units, drag each field to the worksheet.

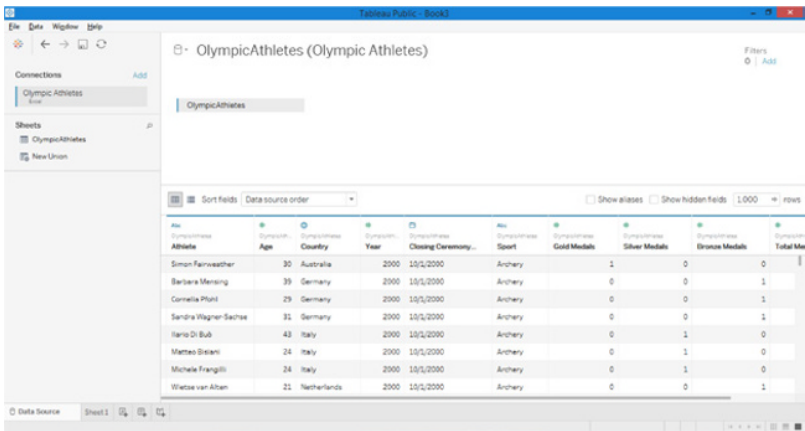


Using the “Show Me” option, select “Stacked Bars” to depict the demand for each item and the total sum of unit prices of each item. Visualization shows that the demand for binder and pencil are high, and the unit price for the desk is the highest of all.

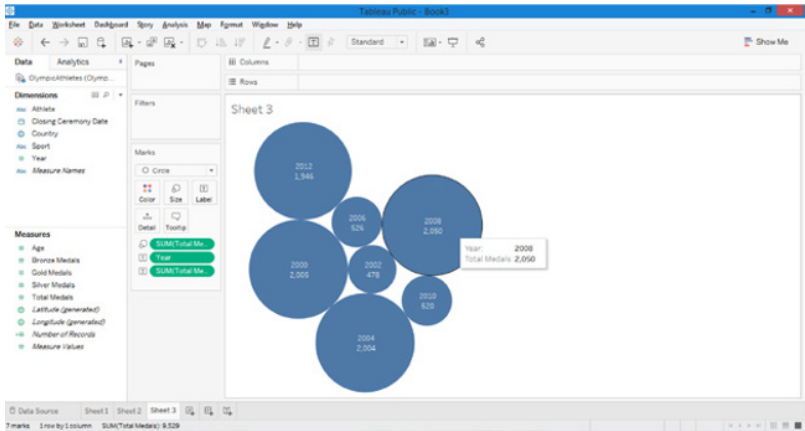


10.9.3 Sports

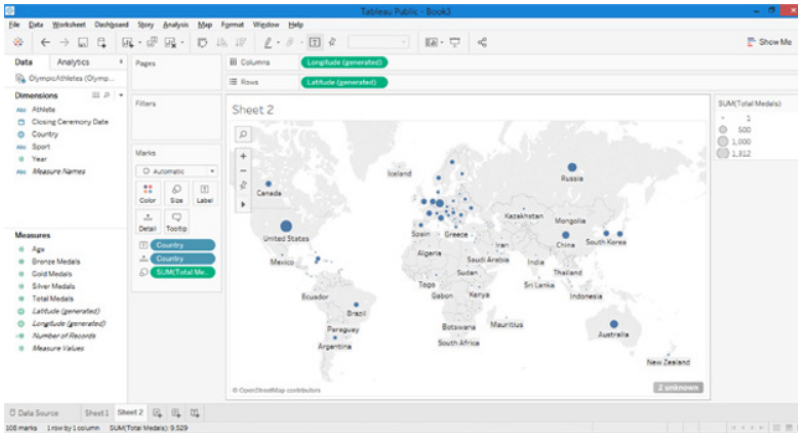
Tableau can be applied in sports to analyze the number of medals won by each country, number of medals won each year, and so forth.



Let us create packed bubbles by dragging the year to the columns shelf and total medals to the rows shelf. The bubbles represent the medals won every year. The larger the size of the bubble, the higher the number of medals won.

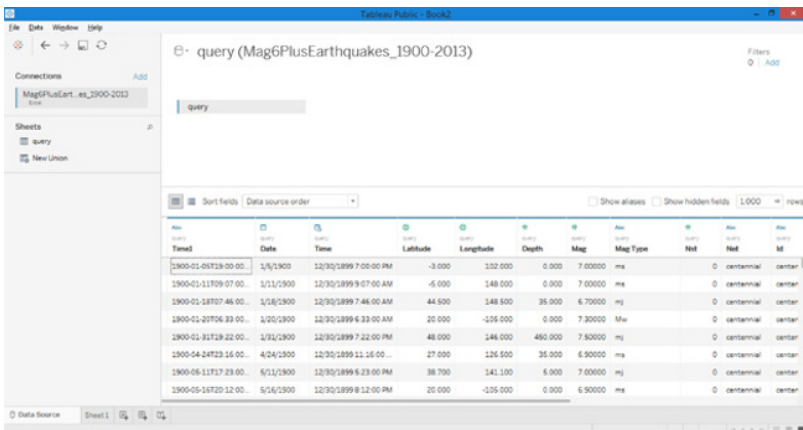


The number of medals won by each country can be represented by using symbol maps in tableau. The circles represent the total medals won by each country. The size of the circles represents the number of medals: a larger size represents a higher number of medals.

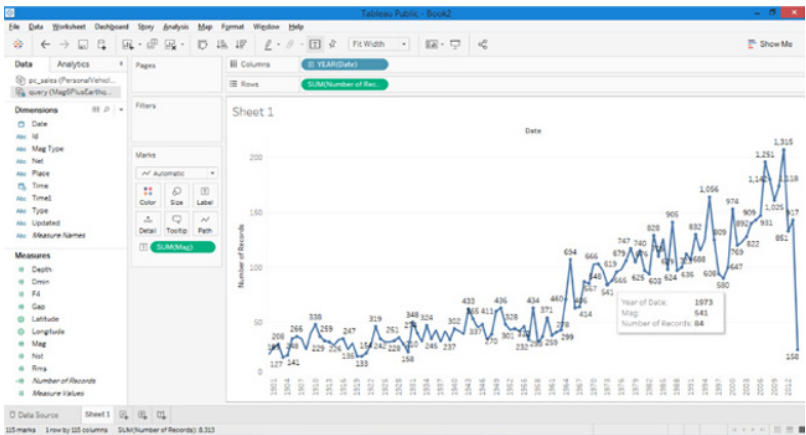


10.9.4 Science – Earthquake Analysis

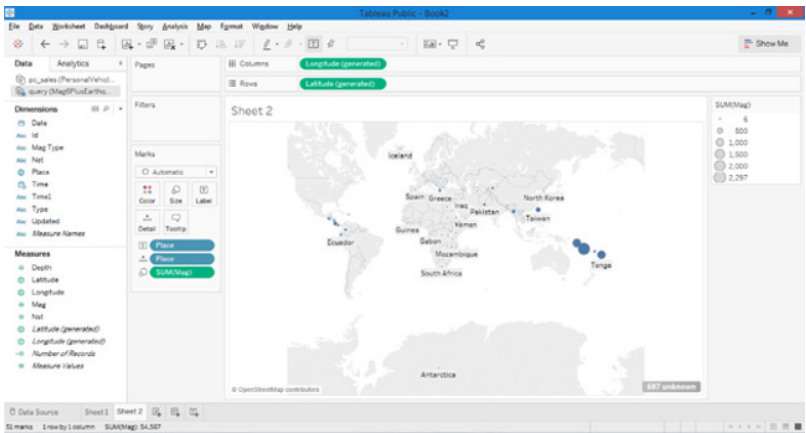
Tableau is used to analyze the magnitude of earthquakes and the frequency of occurrence over the years.



Let us visualize the total number of earthquakes occurred, their magnitudes, and the year of occurrence using continuous lines. Drag time from dimensions to columns and number of records from measures to rows.



Let us visualize the places affected by earthquakes and the magnitude of the earthquakes using symbol maps. Drag and drop place to the worksheet and drag magnitude to worksheet and drop near the place column. Now use the “Show Me” option to select the symbol map to visualize earthquakes that occurred at different places.



10.10 Installing R and Getting Ready

R studio can be downloaded from <http://www.rstudio.com>. Once installed, launch R Studio and start working with R Studio. An example of R Studio GUI in Windows is shown in Figure 10.13.

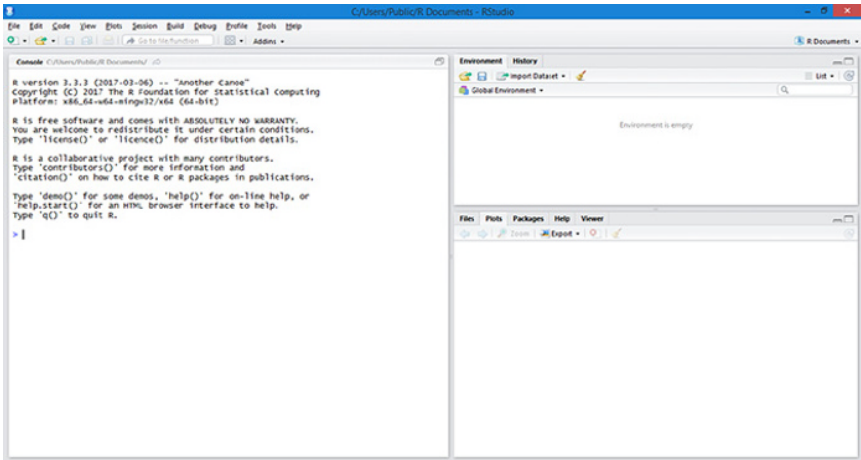


Figure 10.13 R Studio interface on windows.

There are several built-in packages in R. https://cran.r-project.org/web/packages/available_packages_by_name.html provides the list of packages available in R with a short description about each package. A package can be installed using the function `install.packages()`. The installed package has to be loaded in the active R session to use the package. To load the package, the function `library()` is used.

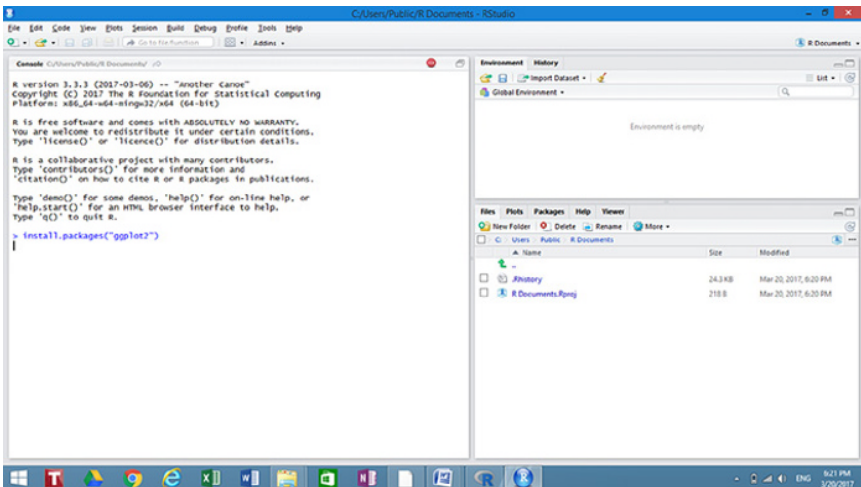


Figure 10.14

10.10.1 R Basic Commands

A numeric sequence can be generated using the function `seq()`. For example, the following code generates a sequence of numbers from 5 to 15 incremented by 2.

```
> seq(5,15,2)
[1] 5 7 9 11 13 15
```

Another numeric vector with length 6 starting from 5 can be generated as shown below.

```
> seq(5,length.out = 6)
[1] 5 6 7 8 9 10
```

10.10.2 Assigning Value to a Variable

Assigning a value to an object in R uses “<-” or “=” as the assignment operators. Programmers often follow the “<-” notation as the assignment operator. A simple example for assigning a value to an object and displaying the value is shown below.

```
> a<-5
> a
[1] 5
```

Here, ‘a’ is the object, and the value 5 is assigned to it. “[1]” in the output indicates that first element is the element 1 of the output. For example, if there are five elements in row 1, the second row would be labeled [6] indicating that the first element in that row is the sixth element of the entire data collection. If an object holds a value, and if a new value is assigned to the object, then the previous value is erased and the new value gets stored.

```
> x<-5
> x
[1] 5
> x<-"a"
> x
[1] "a"
```

An expression can be simply typed, and its result can be displayed without assigning it to an object.

```
> (5-2)*10
[1] 30
```

10.11 Data Structures in R

Data structures are the objects that are capable of holding the data in R. Various data structures in R are:

- Vectors;
- Matrices;
- Arrays;
- Data Frames; and
- Lists.

10.11.1 Vector

A vector is a row or column of alphabets or numbers. For example, to create a numeric vector of length 20, the expression shown below is used.

```
> x<-1:20
> x
[1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
17 18 19 20
```

R can handle five classes of objects:

- Character;
- Numeric;
- Integer;
- Complex (imaginary); and
- Logical (True or False).

The function `c()` combines its arguments into vectors. The function `Class()` tells the class of the object.

```
> x<-c(1,2,3)
> class(x)
[1] "numeric"
> x
[1] 1 2 3
> a<-c("a","b","c","d")
> class(a)
[1] "character"
> a
[1] "a" "b" "c" "d"

> i<-c(&minus;3.5 + 2i,1.2 + 3i)
> class(i)
```

```
[1] "complex"
> i
[1] -3.5+2i 1.2+3i

> logi<-c(TRUE,FALSE,FALSE,FALSE)
> class(logi)
[1] "logical"
> logi
[1] TRUE FALSE FALSE FALSE
```

The class of an object can be ensured using the function `is.*`.

```
> is.numeric(logi)
[1] FALSE
> a<-c("a","b","c","d")
> is.character(a)
[1] TRUE
> i<-c(&minus;3.5 + 2i,1.2 + 3i)
> is.complex(i)
[1] TRUE
```

10.11.2 Coercion

Objects can be coerced from one class to another using the `as.*` function. For example, an object created as numeric can be type-converted into a character.

```
> a<-c(0,1,2,3,4,5.5,-6)
> class(a)
[1] "numeric"
> as.integer(a)
[1] 0 1 2 3 4 5 -6
> as.character(a)
[1] "0" "1" "2" "3" "4" "5.5" "-6"
> as.logical(a)
[1] FALSE TRUE TRUE TRUE TRUE TRUE TRUE
> as.complex(a)
[1] 0.0+0i 1.0+0i 2.0+0i 3.0+0i 4.0+0i
5.5+0i -6.0+0i
```

When it is not possible to coerce an object from one class to another, then it will result in NAs being introduced with a warning message.

```
> x<-c("a","b","c","d","e")
> class(x)
```

```

[1] "character"
> as.integer(x)
[1] NA NA NA NA NA
Warning message:
NAs introduced by coercion
> as.logical(x)
[1] NA NA NA NA NA
> as.complex(x)
[1] NA NA NA NA NA
Warning message:
NAs introduced by coercion
> as.numeric(x)
[1] NA NA NA NA NA
Warning message:
NAs introduced by coercion

```

10.11.3 Length, Mean, and Median

The length of an object can be found using the function `length()`, and the average and median can be found using the functions `mean()` and `median()`, respectively.

```

> a<-c(1,2,3,4,5)
> length(a)
[1] 5
> age<-c(10,12,13,15,16,18)
> mean(age)
[1] 14
> x<-c(1,2,3,4,5,6,7)
> median(x)
[1] 4

```

With the basic commands learnt let us write a simple program to find the correlation between age in years and height in centimeters.

```

> age<-c(10,12,13,15,16,18)
> height<-c(137.5,147,153,166,173,181)
> cor(age,height)
[1] 0.9966404

```

The value of the correlation (Figure 10.15) shows that there exists a strong positive correlation, and the relationship can be shown using a scatterplot.

```

> plot(age,height)

```

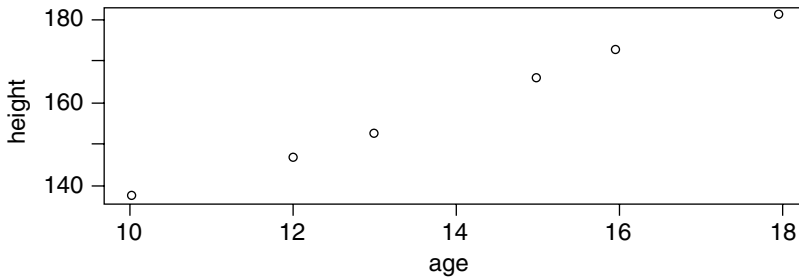


Figure 10.15 Correlation between age and height.

10.11.4 Matrix

The `matrix()` function is used to create a matrix by specifying either of its dimensions row or a column.

```
> matrix(c(1,3,5,7,9,11,13,15,17),ncol = 3)
```

```
      [,1] [,2] [,3]
[1,]    1    7   13
[2,]    3    9   15
[3,]    5   11   17
```

```
> matrix(c(1,3,5,7,9,11,13,15,17),nrow = 3)
```

```
      [,1] [,2] [,3]
[1,]    1    7   13
[2,]    3    9   15
[3,]    5   11   17
```

By specifying `ncol = 3`, a matrix with three columns is created, and the rows are determined automatically based on the number of columns. Similarly, by specifying `nrow = 3`, a matrix with three rows is created, and the columns are determined automatically based on the number of rows.

```
> matrix(c(1,3,5,7,9,11,13,15,17),nrow = 3, byrow = FALSE)
```

```
      [,1] [,2] [,3]
[1,]    1    7   13
[2,]    3    9   15
[3,]    5   11   17
```

```
> matrix(c(1,3,5,7,9,11,13,15,17),nrow = 3, byrow = TRUE)
```

```
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    7    9   11
[3,]   13   15   17
```

By setting `byrow = FALSE`, a matrix will be filled in by columns, while by setting `byrow = TRUE`, it will be filled in by rows. The default value is `byrow = FALSE`, i.e., when it is not specified, the matrix will be filled by columns.

A diagonal matrix can be created by using the function `diag()` specifying the number of rows.

```
> diag(5, nrow = 5)
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	5	0	0	0	0
[2,]	0	5	0	0	0
[3,]	0	0	5	0	0
[4,]	0	0	0	5	0
[5,]	0	0	0	0	5

The rows and columns in a matrix can be named while creating the matrix to make it clear what actually the rows and columns mean.

```
matrix(1:20,nrow = 5,byrow = TRUE,dimnames = list(c("r1"
,"r2","r3","r4","r5"),c("c1","c2","c3","c4")))
```

	c1	c2	c3	c4
r1	1	2	3	4
r2	5	6	7	8
r3	9	10	11	12
r4	13	14	15	16
r5	17	18	19	20

The rows and columns can also be named after creating the matrix using the alternative approach shown below.

```
cells<-1:6
> rnames<-c("r1","r2")
> cnames<-c("c1","c2","c3")
> newmatrix<-matrix(cells,nrow=2, byrow = TRUE, dimnames
= list(rnames,cnames))
> newmatrix
```

	c1	c2	c3
r1	1	2	3
r2	4	5	6

A matrix element can be selected using the subscripts of the matrix. For example, in a 3×3 matrix `A`, `A[1,]` refers to 1st row of the matrix. Similarly `A[,2]` refers to second column of the matrix. `A[1,2]` refers to second element of the first row in the matrix. `A[c(1,2),3]` refers to the `A[1,3]` and `A[2,3]` elements.

```

> A<-matrix(1:9,nrow = 3,byrow = TRUE)
> A
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
> A[1,]
[1]    1    2    3
> A[,2]
[1]    2    5    8
> A[1,2]
[1]    2
> A[c(1,2),3]
[1]    3    6

```

A specific row or a column of a matrix can be omitted using negative numbers. For example

A[-1,] omits first row of that matrix and displays the rest of the elements while
A[, -2] omits second column.

```

      [,1] [,2] [,3]
[1,]    4    5    6
[2,]    7    8    9
> A[, -2]
      [,1] [,2]
[1,]    1    3
[2,]    4    6
[3,]    7    9

```

Matrix addition, subtraction, multiplication, and division can be performed using arithmetic operators of that matrix.

```

> A[-1,]
> B<-matrix(11:19,nrow = 3, byrow = T)
> B
      [,1] [,2] [,3]
[1,]   11   12   13
[2,]   14   15   16
[3,]   17   18   19
> A+B
      [,1] [,2] [,3]
[1,]   12   14   16
[2,]   18   20   22
[3,]   24   26   28
> B-A

```



```

      [,1] [,2] [,3]
[1,]    10    10    10
[2,]    10    10    10
[3,]    10    10    10
> A*B
      [,1] [,2] [,3]
[1,]    11    24    39
[2,]    56    75    96
[3,]   119   144   171
> B/A
      [,1] [,2] [,3]
[1,] 11.000000 6.00 4.333333
[2,]  3.500000 3.00 2.666667
[3,]  2.428571 2.25 2.111111

```

10.11.5 Arrays

Arrays are multidimensional data structures capable of storing only one data type. Arrays are similar to matrices where data can be stored in more than two dimensions. For example, if an array with dimension (3,3,4) is created, four rectangular matrices each with three rows and three columns will be created.

```

> x<-c(1,3,5)
> y<-c(2,4,6)
> z<-c(7,8,9)
> arr<-array(c(x,y,z),dim=c(3,3,4))
> arr
, , 1
      [,1] [,2] [,3]
[1,]     1     2     7
[2,]     3     4     8
[3,]     5     6     9
, , 2
      [,1] [,2] [,3]
[1,]     1     2     7
[2,]     3     4     8
[3,]     5     6     9
, , 3
      [,1] [,2] [,3]
[1,]     1     2     7
[2,]     3     4     8
[3,]     5     6     9

```

```
, , 4
      [,1] [,2] [,3]
[1,]    1    2    7
[2,]    3    4    8
[3,]    5    6    9
```

When only the number of rows and columns ($m \times n$) is specified, only one $m \times n$ matrix is created.

```
> arr<-array(c(x,y,z),dim=c(3,3))
> arr
      [,1] [,2] [,3]
[1,]    1    2    7
[2,]    3    4    8
[3,]    5    6    9
```

```
arr<-array(c(x,y,z),dim=c(2,2))
> arr
      [,1] [,2]
[1,]    1    5
[2,]    3    2
```

```
arr<-array(c(x,y,z),dim=c(4,4))
> arr
      [,1] [,2] [,3] [,4]
[1,]    1    4    9    2
[2,]    3    6    1    4
[3,]    5    7    3    6
[4,]    2    8    5    7
```

10.11.6 Naming the Arrays

Each dimension of the array can be named, and the rows and columns in each dimension can be named as shown below.

```
> row<-c("R1","R2","R3")
> col<-c("C1","C2","C3")
> mat<-c("M1","M2","M3")
> newarr<-array(1:27,c(3,3,3),dimnames=list(row,col,mat))
> newarr
, , M1
      C1    C2    C3
R1     1     4     7
R2     2     5     8
R3     3     6     9
```

```
, , M2
      C1    C2    C3
R1    10    13    16
R2    11    14    17
R3    12    15    18

, , M3
      C1    C2    C3
R1    19    22    25
R2    20    23    26
R3    21    24    27
```

10.11.7 Data Frames

Data frame is the most commonly used way of storing data in R and makes the data analysis easier. Each column in a data frame holds the value for a single attribute while each row has one set of values of each attribute. Let us create a data frame with employee details of a company.

```
> empid <- c (139,140,151,159,160)
> empname <- c ("John","Joseph","Mitchell","Tom","George")
> JoiningDate <- as.Date(c("2013-11-01", "2014-09-20",
"2014-12-16", "2015-02-10", "2016-06-25"))
> age <- c(23,35,35,40,22)
> salary<- c(1900,1800,2000,1700,1500)
> empdata<-data.frame(empid,empname,JoiningDate,age,
salary)
> empdata
      empid  empname  JoiningDate    age    salary
1      139    John   2013-11-01     23     1900
2      140   Joseph   2014-09-20     35     1800
3      151 Mitchell   2014-12-16     35     2000
4      159     Tom    2015-02-10     40     1700
5      160   George   2016-06-25     22     1500
```

The command `str(empdata)` displays the structure of the data frame.

```
> str(empdata)
'data.frame' :5      obs.   of      5      variables:
 $ empid      :num   139    140    151    159    160
 $ empname    :Factor w/ 5 levels "George","John",...:
2 3 4 5 1
 $ JoiningDate: Date, format: "2013-11-01" "2014-09-20"
"2014-12-16" ...
```

```
$ age      : num  23 35 35 40 22
$ salary   : num  1900 1800 2000 1700 1500
```

Specific column can be extracted from the data frame.

```
> data.frame(empdata$empid,empdata$salary)
  empdata.empid  empdata.salary
1           139           1900
2           140           1800
3           151           2000
4           159           1700
5           160           1500

> empdata[c("empid", "salary")]
  empid  salary
1   139   1900
2   140   1800
3   151   2000
4   159   1700
5   160   1500

> empdata[c(1,5)]
  empid  salary
1   139   1900
2   140   1800
3   151   2000
4   159   1700
5   160   1500
```

To extract a specific column and row of a data frame, both rows and columns of interest have to be specified. Here empid and salary of rows 2 and 4 are fetched.

```
> empdata[c(2,4),c(1,5)]
  empid  salary
2   140   1800
4   159   1700
```

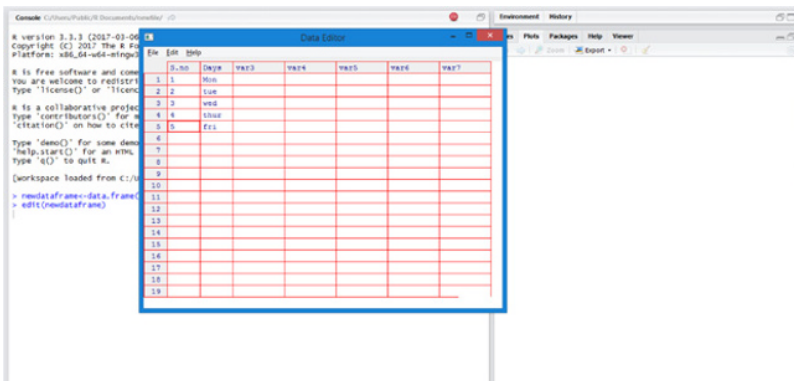
To add a row to an existing data frame, the rbind() function is used whereas to add a column to an existing data frame, empdata\$newcolumn is used.

```
> empid <- c (161,165,166,170)
> empname <- c ("Mathew","Muller","Sam","Garry")
> JoiningDate <- as.Date(c("2016-08-01", "2016-09-21",
"2017-02-10", "2017-04-12"))
> age <- c(24,48,32,41)
> salary<- c(1900,1600,1200,900)
> new.empdata<-data.frame(empid,empname,JoiningDate,age,salary)
> emp.data<-rbind(empdata,new.empdata)
```

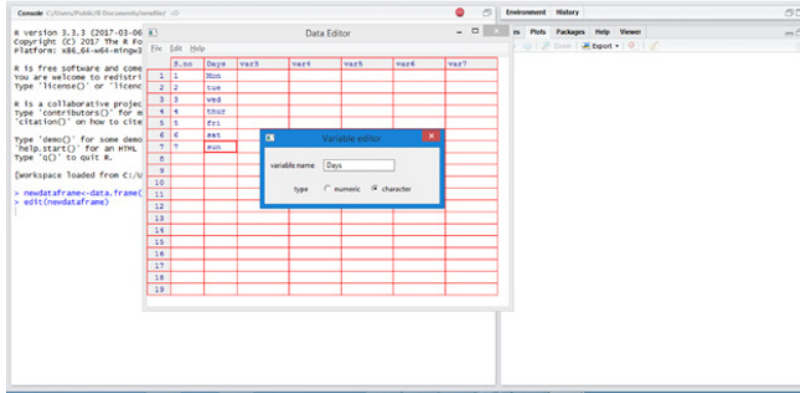
```
> emp.data
  empid  empname  JoiningDate  age  salary
1   139    John   2013-11-01   23   1900
2   140   Joseph   2014-09-20   35   1800
3   151 Mitchell   2014-12-16   35   2000
4   159    Tom    2015-02-10   40   1700
5   160   George   2016-06-25   22   1500
6   161   Mathew   2016-08-01   24   1900
7   165   Muller   2016-09-21   48   1600
8   166    Sam     2017-02-10   32   1200
9   170   Garry    2017-04-12   41    900

> emp.data$address<-c("Irving","California ","Texas",-
  "Huntsville","Orlando","Atlanta","Chicago","Boston","Liv
  ington")
> emp.data
  empid  empname  JoiningDate  age  salary  address
1   139    John   2013-11-01   23   1900   Irving
2   140   Joseph   2014-09-20   35   1800 California
3   151 Mitchell   2014-12-16   35   2000    Texas
4   159    Tom    2015-02-10   40   1700 Huntsville
5   160   George   2016-06-25   22   1500   Orlando
6   161   Mathew   2016-08-01   24   1900   Atlanta
7   165   Muller   2016-09-21   48   1600   Chicago
8   166    Sam     2017-02-10   32   1200   Boston
9   170   Garry    2017-04-12   41    900 Livingston
```

A data frame can be edited using the `edit()` function. Text editor can be invoked using the `edit()` function. Even an empty data frame can be created and data entered using the text editor.



Variable names can be edited by clicking on the variable name column. Also, the type can be modified as numeric or character. Additional columns can be added by editing the unused columns. Upon closing the text editor, the data entered in the editor gets saved into the object.



10.11.8 Lists

List is a combination of unrelated elements such as vectors, strings, numbers, logical values, and other lists.

```
> num<-c(1:5)
> str<-c("a","b","c","d","e")
> logi<-c(TRUE,FALSE,FALSE,TRUE,TRUE)
> x<-list(num,str,logi)
> x
[[1]]
[1] 1 2 3 4 5

[[2]]
[1] "a" "b" "c" "d" "e"

[[3]]
[1] TRUE FALSE FALSE TRUE TRUE
```

The elements in the list can also be named after creating the list.

```
> newlist<-list(c(1:5),matrix(c(1:9),nrow = 3),list(c("sun",
"mon","tue"),"False","11"))
> names(newlist)<-c("Numbers","3x3 Matrix","List inside
a list")
> newlist
```

```

$Numbers
[1] 1 2 3 4 5

$`3x3 Matrix`
      [,1] [,2] [,3]
[1,] 1 4 7
[2,] 2 5 8
[3,] 3 6 9

$`List inside a list`
$`List inside a list`[[1]]
[1] "sun" "mon" "tue"

$`List inside a list`[[2]]
[1] "False"

$`List inside a list`[[3]]
[1] "11"

```

The elements in a list can be accessed using their index.

```

> newlist[1]
$Numbers
      [,1] [,2] [,3]
[1,] 1 2 3 4 5

> newlist[3]
$`List inside a list`
$`List inside a list`[[1]]
[1] "sun" "mon" "tue"
$`List inside a list`[[2]]
[1] "False"
$`List inside a list`[[3]]
[1] "11"

> newlist$`3x3 Matrix`
      [,1] [,2] [,3]
[1,] 1 4 7
[2,] 2 5 8
[3,] 3 6 9

```

The elements in a list can be deleted, updated, or new elements can be added using their indexes.

```

> newlist[4] <- "new appended element"
> newlist
$Numbers
[1] 1 2 3 4 5

```

```

$`3x3 Matrix`
      [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9

$`List inside a list`
$`List inside a list`[[1]]
[1] "sun" "mon" "tue"

$`List inside a list`[[2]]
[1] "False"

$`List inside a list`[[3]]
[1] "11"

[[4]]
[1] "new appended element"
> newlist[2]<-"Updated element"
> newlist[3]<-NULL
> newlist
$Numbers
[1] 1 2 3 4 5
$`3x3 Matrix`
[1] "Updated element"

[[3]]
[1] "new appended element"
  Several lists can be merged into one list.

> names(newlist[2])<-"Matrix updated to a string"
> newlist1<-list(c(1,2,3),c("red","green","blue"))
> newlist2<-list(c("TRUE","FALSE"),matrix((1:6),nr
ow = 2))
> mergedList<-c(newlist1,newlist2)
> mergedList

[[1]]
[1] 1 2 3

[[2]]
[1] "red"      "green" "blue"

[[3]]
[1] "TRUE"     "FALSE"

```



```
[4]
```

	[, 1]	[, 2]	[, 3]
[1,]	1	3	5
[2,]	2	4	6

10.12 Importing Data from a File

R is associated with a working directory where R will read data from files and save the results into files. To know the current working directory, the command `getwd()` is used, and to change the existing path for the working directory, `setwd()` is used.

```
> setwd("C:/Users/Public/R Documents")
> getwd()
[1] "C:/Users/Public/R Documents"
```

Note that R always uses a forward slash “/” to set path and a backward slash “\” as escape character. In case “\” is used, it throws an error. The function `setwd()` is not used to create a directory. If a new directory has to be created, the `dir.create()` function is used and the `setwd()` function is then used to change the path for an existing directory.

```
> dir.create("C:/Users/Public/R Documents/newfile")
> setwd("C:/Users/Public/R Documents/newfile")
> getwd()
[1] "C:/Users/Public/R Documents/newfile"
```

If the directory already exists, `dir.create()` throws a warning message that the directory already exists.

```
> dir.create("C:/Users/Public/R Documents/newfile")
Warning message:
In dir.create("C:/Users/Public/R Documents/newfile") :
  'C:\Users\Public\R Documents\newfile' already exists
```

R command to read a csv file is `newdata<-read.csv("Sales.csv")`. If the file does not exist in the working directory, it is necessary to mention the path where the file resides.

Figure 10.16 illustrates reading a csv file and displaying the data. In the command `newdata<-read.csv("Sales.csv")` the csv file `Sales.csv` is read and stored in the object `newdata`.

```
dim(newdata)
[1] 998      12
```

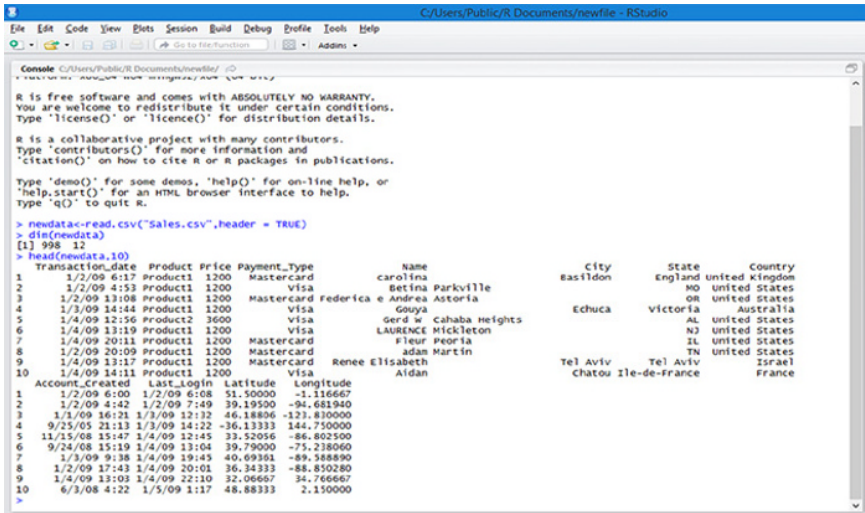


Figure 10.16 Importing data from csv file and displaying the data.

The function `dim()` is used to display the dimension of the file. The output shows that the file has 998 rows and 12 columns, and sample records of the file are displayed using the function `head(newdata,10)`, which displays first 10 records of the file, while `tail(newdata,10)` displays the last 10 records of the file.

10.13 Importing Data from a Delimited Text File

Data from a delimited text file can be imported using the function `read.table()`. In the syntax the delimiter and the header are to be specified. The delimiter may be a space (`sep=" "`), comma (`sep=","`), or tab (`sep="\t"`). `header = FALSE` denotes that the first row of the file does not contain the variable names, while `header = TRUE` denotes that the first row of the file contains the variable names. The file will be fetched by default from the current working directory. If the file does not exist in the current working directory, the location of the file has to be specified.

```
> mydata<-read.table(file="C:/processed.switzerland.
data.txt", sep=" ", header=FALSE)
> head(mydata,10)
```

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13	V14
1	32	1	1	95	0	?	0	127	0	.7	1	?	?	1
2	34	1	4	115	0	?	?	154	0	.2	1	?	?	1

3	35	1	4	?	0	?	0	130	1	?	?	?	7	3
4	36	1	4	110	0	?	0	125	1	1	2	?	6	1
5	38	0	4	105	0	?	0	166	0	2.8	1	?	?	2
6	38	0	4	110	0	0	0	156	0	0	2	?	3	1
7	38	1	3	100	0	?	0	179	0	-1.1	1	?	?	0
8	38	1	3	115	0	0	0	128	1	0	2	?	7	1
9	38	1	4	135	0	?	0	150	0	0	?	?	3	2
10	38	1	4	150	0	?	0	120	1	?	?	?	3	1

10.14 Control Structures in R

Control structures are used to control the flow of execution of a series of statements in R. The control structures that are frequently used in R are:

- if and else;
- for;
- while;
- repeat;
- break; and
- next.

10.14.1 If-else

```
If(condition)
{Statement/statements will be executed if the expression
is true.}
else
{Statement/statements will be execute if the expression
is false.}
```

Example:

```
> x <- 5
>
> if(x>4) {
+   print("x is greater than 4")
+ } else {
+   print("x is less than 4")
+ }
[1] "x is greater than 4"
```

```

> x <- 5
>
> if(x>6) {
+   print("x is greater than 6")
+ } else {
+   print("x is less than 6")
+ }
[1] "x is less than 6"

```

10.14.2 Nested if-Else

```

If(expression 1)
{
Statement/statements will be executed if expression
1 is true
}
else If(expression 2)
{
Statement/statements will be executed if expression
2 is true
}
else
{
Statement/statements will be executed if both expression
1 and expression 2 are false)
}

```

Example:

```

x <- c(1,2,3,4,5)
if(7 %in% x)
{
    print("the vector x has 7")
}
else if(8 %in% x)
{
    print("the vector x has 8")
}
else
{
    print("the vector x does not have 7 and 8")
}
[1] "the vector x does not have 7 and 8"

```

10.14.3 For Loops

Frequently certain block of statements is to be executed repeatedly. Under such circumstances for loops are commonly used. The following example shows a simple execution of a for loop:

Example

```
for(x in 1:10)
{
  print(x)
}
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9
[1] 10
```

In the above example the value of x is displayed in each iteration of the loop. The for loop can also be used with arrays.

Example

```
x<-c("Sunday","Monday","Tuesday","Wednesday","Thursday",
"Friday","Saturday")
for(i in 1:7)
{
  print (x[i])
}
[1] "Sunday"
[1] "Monday"
[1] "Tuesday"
[1] "Wednesday"
[1] "Thursday"
[1] "Friday"
[1] "Saturday"
```

A for loop can be repeated using the elements in the vector x , and the same code can be executed in each element.

Example

```

for (days_in_a_week in x)
{
  print(days_in_a_week)
}
[1] "Sunday"
[1] "Monday"
[1] "Tuesday"
[1] "Wednesday"
[1] "Thursday"
[1] "Friday"
[1] "Saturday"
print(days_in_a_week[1])
[1] "Saturday"

```

10.14.4 While Loops

While loops begin with testing a condition: if the condition is true, then the body of the loop is executed. The testing repeats, and the loop executes until the condition fails.

```

While(condition)
{
  Statements to be executed
}

```

Example

```

> x <- 5
> while(x < 15) {
+   print(x)
+   x <- x + 1
+ }
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9
[1] 10
[1] 11
[1] 12
[1] 13
[1] 14

```

10.14.5 Break

```
> i <- 1:5
> for (value in i) {
+   if (value == 3){
+     break
+   }
+   print(value)
+ }
[1] 1
[1] 2
```

In the example, the loop is iterated over the vector `i`, which has numerical sequence from 1 to 5. If condition is used to break the loop, then the value of `i` becomes 3.

10.15 Basic Graphs in R

Graphs are the basic tools for data visualization that are frequently used while analyzing the data. Several libraries are available in R to create these graphs. Pie charts, bar charts, boxplots, histograms, line graphs, and scatterplots are the different charts and graphs discussed below.

10.15.1 Pie Charts

Pie charts are used to represent the frequency distribution graphically. The syntax of pie chart is:

```
pie(x, labels = names(x), edges = 200, radius = 0.8, clock-
wise = FALSE, init.angle = if(clockwise) 90 else 0, density
= NULL, angle = 45, col = NULL, main = NULL, ...)
```

`x` – is a vector containing non-negative numeric values that are the slices of pie chart.

Labels – describe the slices of pie chart.

Radius – is the radius of the circle of the pie chart.

Clockwise – indicates if the slices are drawn clockwise or anticlockwise.

Init.angle – represents the starting angle of the slices.

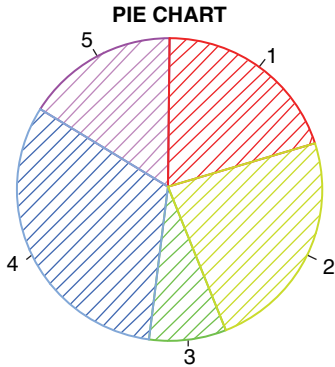
Density – density of the lines used for shading the circle. The density is represented in lines per inch.

Angle – represents the slope of shading lines. Vertical shading lines are drawn when the angle is given as 90.

Col – represents the colors to be used when shading the slices.

Main – represents the title of the chart.

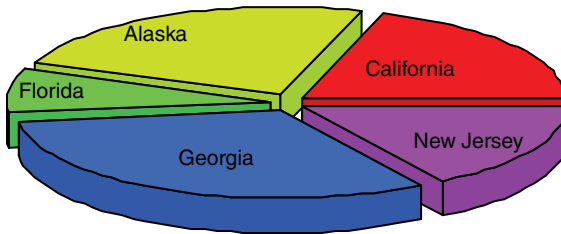
```
pie(x, labels = names(x), edges = 200, radius = 1,
+   clockwise = TRUE, init.angle = 90, density = 20, angle
= 45, col = rainbow(length(x)), main = "PIE CHART")
```



10.15.2 3D – Pie Charts

A three-dimensional pie chart can be drawn using the package `plotrix` and the function `pie3D`.

```
pie3D(x, labels = labelnames, explode = 0.1, radius =
1.5, main = "PIE CHART OF STATES OF AMERICA")
```



10.15.3 Bar Charts

A bar chart represents the value of the variables with the height of the bar. The syntax of bar chart is:

```
barplot (height, main, xlab, ylab, border, density)
```

height – is either a vector or a matrix which make up the plot.

main – is the title of the plot

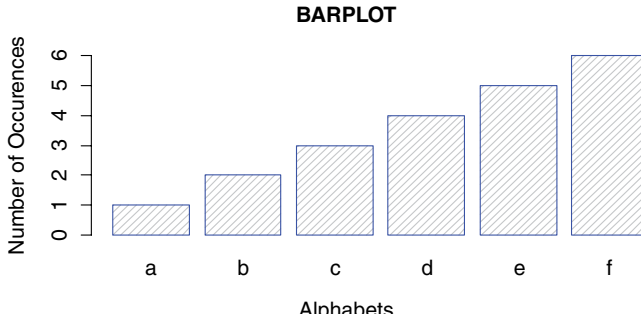
xlab – is the label for x-axis

ylab – is the label for y-axis

border – is the border color of the bars

density – is the density of the lines used for shading the bars. The density is represented in lines per inch.

```
> x<-c('a','b','b','c','c','c','d','d','d','d','e','e','e',
'e','e','e','f','f','f','f','f','f')
> no_of_occurrence<-table(x)
> barplot(no_of_occurrence,main="BARPLOT", xlab = "Alphabets",
ylab = "Number of Occurences",border = "BLUE", density = 20 )
```

10.15.4 Boxplots

Boxplots can be used for single variable or group of variables. Boxplot represents minimum value, maximum value, median value (50th percentile), upper quantile (75th percentile) and the lower quantile (25th percentile). The basic syntax for boxplots is,

`boxplot(x, data = NULL, ..., subset, na.action = NULL, main)`

`x` – is a formula.

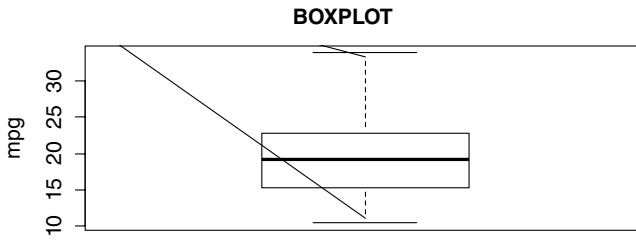
`data` – represents the data frame.

The dataset `mtcars` available in R is used. The first 10 rows of the data are displayed below.

```
> head(mtcars,10)
function dim() is used to display the
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4

```
> median(mtcars$mpg)
[1] 19.2
> min(mtcars$mpg)
[1] 10.4
> max(mtcars$mpg)
[1] 33.9
> quantile(mtcars$mpg)
 0%    25%   50%   75%  100%
10.400 15.425 19.200 22.800 33.900
```

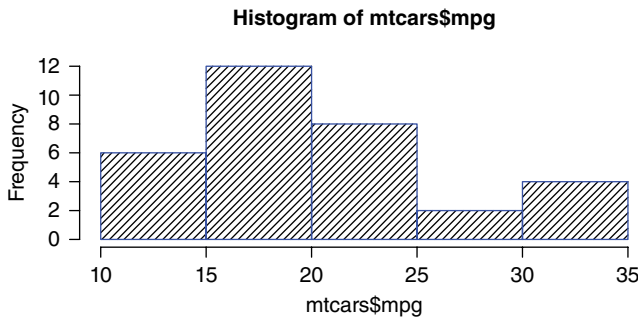


Boxplot represents the median value, 19.2, the upper quartile, 22.8, the lower quartile, 15.425, the largest value, 33.9, and the smallest value, 10.4.

10.15.5 Histograms

Histograms can be created by the function `hist()`. The basic difference between bar charts and histograms is that histograms plot the values in continuous range. The basic syntax of histogram is,

```
hist(x, main, density, border)
> hist(mtcars$mpg, density = 20, border = 'blue')
```



10.15.6 Line Charts

Line charts can be created using either of the two functions `plot(x, y, type)` or `lines(x, y, type)`.

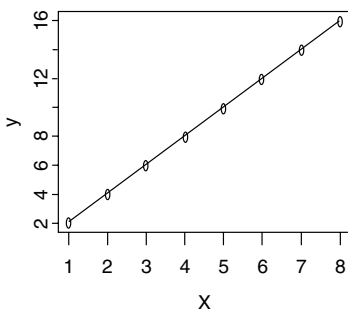
The basic syntax for lines is:

```
lines(x, y, type = )
```

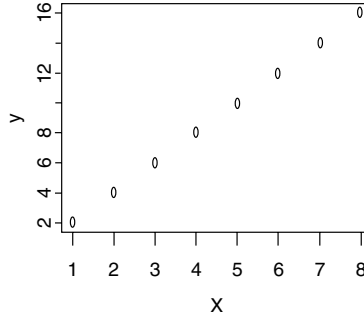
Possible types of plots are,

- “p” for points,
- “l” for lines,
- “b” for both,
- “c” for the lines part alone of “b,”

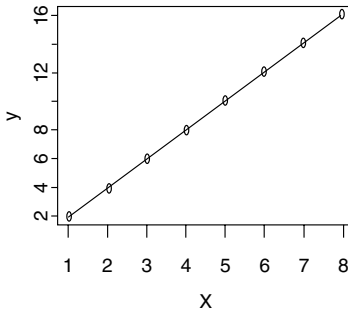
- “h” for “histogram” like vertical lines,
- “s” for stair steps,
- “S” for other steps,
- “n” for no plotting.



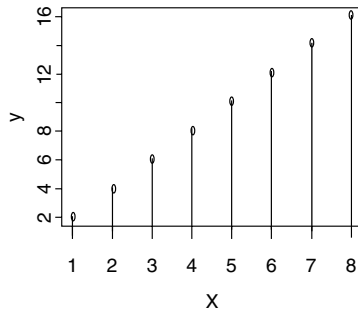
(i) lines (x,y)



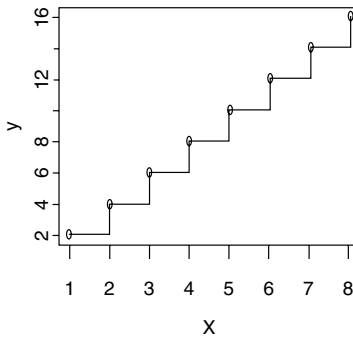
(ii) lines (x,y, type='p')



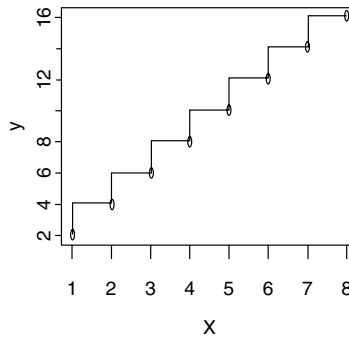
(iii) lines (x,y, type='l')



(iv) lines (x,y, type='h')



(iii) lines (x,y, type='s')



(iv) lines (x,y, type='S')

10.15.7 Scatterplots

Scatterplots are used to represent the points scattered in the Cartesian plane. Similar to line charts, scatterplots can be created using the function `plot(x, y)`. Points in the scatterplot that are connected through lines form the line chart. An example showing a scatter plot for age and the corresponding weight is shown below,

```
> age<-c(4,5,6,7,8,9,10)
> weight<-c(13.5,14.8,16.3,18,19.7,21.5,23.5)
> plot(age,weight,main='SCATTER PLOT')
```



Index

a

A/B testing 172
 accelerometer sensors 7
 ACID 56
 activity hub 181
 agglomerative clustering 264–265
 Amazon DynamoDB 61
 Amazon Elastic MapReduce (Amazon EMR) 153
 Apache Avro 144–145
 Apache Cassandra 63–64, 141
 Apache Hadoop 11, 18, 111
 architecture of 112
 ecosystem components 112–113
 storage 114–119
 Apache Hive
 architecture 151–152
 data organization 150–151
 primitive data types 149
 Apache Mahout 146
 Apache Oozie 146–147
 Apache Pig 145–146
 ApplicationMaster failure 137
 apriori algorithm
 frequent itemset generation
 217–219
 implementation of 212–217
 arbitrarily shaped clusters 272
 artificial neural network 251–253
 association rules

algorithm 207
 binary database 208
 market basket data 208
 support and confidence 206–207
 vertical database 209
 assumption-based outlier detection 283
 asymmetric clusters 35, 36
 atomicity (A) 56
 attributes/fields 43
 availability 54
 availability and partition tolerance (AP) 56

b

bar charts 342–343
 BASE 56–57
 basically available database 57
 batch processing 88
 Bayesian network
 Bayes rule 244–249
 classification technique 241
 conditional probability 242–243
 independence 244
 joint probability distribution 242
 probability distribution 242
 random variable 241–242
 big data 1
 applications 21
 black box 7
 characteristics 4
 vs. data mining 3, 4

- big data (*cont'd*)
 - evolution of 2, 3
 - financial services 23–24
 - handling, traditional database
 - limitations in 3
 - in health care 7, 21–22
 - infrastructure 11–12
 - life cycle
 - data aggregation phase 14
 - data generation 12
 - data preprocessing 14–17
 - schematic representation 12, 13
 - and organizational data 8
 - and RDMS attributes 3, 4
 - in sensors 7
 - sources of 7–8
 - storage architecture 31, 32
 - technology
 - Apache Hadoop 18
 - challenges 19
 - data privacy 20–21
 - data storage 20
 - Hadoop common 19
 - heterogeneity and incompleteness 19–20
 - volume and velocity of data 20
 - YARN 19
 - in telecom 22–23
 - types of 8–11
 - variety 6–7
 - velocity 5–6
 - visualization 17–18
 - volume 5
 - and web data 8
- big data analytics 17
 - applications of 163, 170
 - business intelligence 162, 178–180
 - data analytics 162
 - data warehouse 161
 - description 162
 - descriptive analytics 163–164
 - diagnostic analytics 164
 - enterprise data warehouse 181–182
 - life cycle

- business case evaluation 166–167
- confirmatory data analysis 169
- data extraction and transformation 169
- data preparation 168
- data visualization 169–170
- exploratory data analysis 169
- source data identification 166–167
- predictive analytics 165
- prescriptive analytics 165–166
- qualitative analysis 171
- quantitative analysis 170–171
- real-time analytics processing
 - 180–181
- semantic analysis 175–177
- statistical analysis techniques
 - 172–175
- visual analysis 178
- big data visualization
 - benefits of 293
 - conventional data visualization techniques
 - bar charts 294–295
 - bubble plot 296–297
 - line chart 294
 - pie charts 295–296
 - scatterplot 296
 - Tableau (*see* Tableau)
- binary database 208
- biological neural network 253–254
- biometrics 259
- black box data 7
- boxplots 343–344
- bucket testing 172
- bundle jobs 147
- business case evaluation 166–167
- business intelligence (BI) 162
 - online analytical processing 179
 - online transaction processing
 - 178–179
 - real-time analytics platform 180
- business support services (BSS) 103

C

- capacity scheduler 136
- CAP theorem 54–56

- client-server architecture 84
- clinical data repository 7
- cloud architecture 101–103
- cloud computing 93–94
 - challenges 103
 - computing performance 103
 - Internet-based computing 101
 - interoperability 103
 - portability 103
 - reliability and availability 103
 - security and privacy 103
 - types 94–95
- Cloudera Hadoop distribution (CDH) 152
- cloud services
 - infrastructure as a service (IaaS) 96
 - platform as a service (PaaS) 96
 - software as a service (SaaS) 95–96
- cloud storage 96
 - Google File System architecture 97–101
- cluster analysis
 - Bayesian analysis of mixtures 290
 - and classification 259, 260
 - data point and centroid 260
 - on distance 261
 - distance measurement techniques
 - cosine similarity 262
 - Euclidean distance 261
 - hierarchical clustering algorithm 262
 - Manhattan distance 261–262
 - partition clustering algorithm 262
 - expectation maximization (EM)
 - algorithm 276
 - fuzzy clustering 290–291
 - fuzzy C-means clustering 291–292
 - Gaussian distribution 275
 - hard clustering 274
 - hierarchical clustering
 - agglomerative clustering 264–265
 - applications 266
 - dendrogram graph 264
 - divisive clustering 265
 - intra-cluster distances 260
 - Kernel K-means clustering 270–273
 - K-means algorithms 267–270
 - number of clusters 288–290
 - outlier detection
 - application 283–284
 - assumption-based outlier detection 283
 - semi-supervised outlier detection 282
 - supervised outlier detection 282
 - unsupervised outlier detection 282
 - partitional clustering 267
 - protein patterns 266
 - representative-based clustering 277
 - role of 259
 - soft clustering 274
 - study variables 260
 - univariate Gaussian distribution 274, 275
- cluster computing
 - cluster structure 35, 36
 - cluster types 33–35
 - description 32
 - schematic illustration 33
- clustering based method 283
- clustering technique 195–196
- cluster structure 35, 36
- Clustrix 46
- collective outliers 280–281
- column-store database
 - Apache Cassandra 63–64
 - working method of 62
- compiler 152
- confirmatory data analysis 169
- consistency (C) 54, 56
- consistency and availability (CA) 54
- consistency and partition tolerance (CP) 54
- container failure 138
- content analysis 171
- content hub 181
- contextual outlier 279–280
- control structures, in R
 - break 341
 - if and else 337–338
 - for loops 339–340
 - nested if-else 338
 - while loops 340

- coordinator jobs 147
- corporate cloud 95
- CouchDB 65
- cross-validation 247
- customer churn prevention 189
- customer segmentation 189
- Cypher Query Language (CQL)
 - 66–72

d

- data aggregation phase 14
- data analytics 162
- database transactions, properties
 - related to 56
- data-cleaning process 16
- data definition language (DDL) 149
- data extraction and transformation 169
- data generation 12
- data import
 - from delimited text file 336–337
 - from file 335–336
- data integration 15
- data mining methods
 - vs. big data 3, 4
 - E-commerce sites 240
 - marketing 239–240
 - retailers 240
- DataNode 115–117
- data preparation 168
- data preprocessing
 - data-cleaning process 16
 - data integration 15
 - data reduction 16
 - data transformation 16–17
 - description 14
- data privacy 20–21
- data processing
 - centralized 83
 - defined 83
 - distributed 84
- data reduction 16
- data replication process 39–41
- data storage 20
- data structures, in R

- arrays 327–328
- coercion 322–323
- data frames 329–332
- length, mean, and median 323–324
- lists 332–335
- matrix() function 324–327
- naming arrays 328–329
- vector 321–322
- data transformation
 - aggregation 17
 - challenge 16–17
 - description 16
 - discretization 17
 - generalization 17
 - smoothing 17
- data virtualization *see* virtualization
- data visualization 169–170
- data warehouse 161
- decision tree classifier 247–249
- Density Based Spatial Clustering of
 - Applications with Noise (DBSCAN) 249–250
- descriptive analytics 163–164
- diagnostic analytics 164
- discourse analysis 171
- discretization 17
- distance metric 246–247
- distributed computing 60, 90
- distributed file system 43
- distributed shared memory 86, 87
- distribution models
 - data replication process 39–41
 - sharding 37–39
 - sharding and replication,
 - combination of 41–42
- divisive clustering 265
- document-oriented database 64–65
- durability (D) 56

e

- E-commerce sites 240
- elbow method 288, 289
- electronic health records (EHRs) 7
- encapsulation technique 91

- enterprise data warehouse (EDW)
 - 161, 181–182
- Equivalence Class Transformation (Eclat)
 - algorithm
 - implementation of 223–225
 - vertical data layout 222–223
- ETL (extract, transform and load) 181
- Euclidean distance 261
- eventual consistency 57
- exploratory data analysis 169
- externally hosted private cloud 95

f

- face recognition 188
- failover 32–33
- fair scheduler 136
- fast analysis of shared multidimensional information (FASMI) 179
- FIFO (first in, first out) scheduler
 - 135–136
- file system, distributed 43
- flat database 43
- flume 143–144
- foreign key 45
- FP growth algorithm
 - FP trees 227–229
 - frequency of occurrence 225–226
 - order items 227
 - prioritize items 226–227
- framework analysis 171
- fraud detection 188, 283
- frequent itemset 210
- fuzzy clustering 290–291
- fuzzy C-means clustering 291–292

g

- Gaussian distribution 275
- GenMax algorithm
 - frequent itemsets with tidset 235
 - implementation. 235
 - minimum support count 234
- Google File System architecture 97–101
- graph-oriented database 65

- Cypher Query Language (CQL)
 - 66–72
- general representation 66
- Neo4J 66

- graphs, in R
 - bar charts 342–343
 - boxplots 343–344
 - 3D-pie charts 342
 - histograms 344
 - line charts 344–345
 - pie charts 341–342
 - scatterplots 346
- grounded theory 171

h

- Hadoop 11, 31, 96, 111
 - architecture of 112
 - clusters 112
 - computation (*see* MapReduce)
 - ecosystem components 112–113
 - storage 114–119
- Hadoop 2.0
 - architectural design 129
 - features of 130–131
 - vs. Hadoop 1.0 129, 130
 - YARN 131, 132
- Hadoop common 19
- Hadoop distributed file system (HDFS)
 - 11, 43, 141
 - architecture 115–116
 - cost-effective 118–119
 - data replication 119
 - description 114
 - distributed storage 119
 - features of 118–119
 - rack awareness 118
 - read/write operation 116–118
 - vs. single machine 114
- Hadoop distributions
 - Amazon Elastic MapReduce (Amazon EMR) 153
 - Cloudera Hadoop distribution (CDH) 152
 - Hortonworks data platform 152
 - MapR 152

- hard clustering 274
- HBase
 - automatic failover 140
 - auto sharding 140–141
 - column oriented 141
 - features of 140–141
 - HFiles 140
 - HMaster 139
 - horizontal scalability 141
 - master-slave architecture 138, 139
 - MemStore 140
 - regions 140
 - RegionServer 139, 140
 - write-ahead log technique 138, 140
 - Zookeeper 139
- Healthcare 283–284
- HFiles 140
- hierarchical clustering algorithm 262
- high availability clusters 34
- histograms 344
- Hive
 - architecture 151–152
 - data organization 150–151
 - metastore 151
 - primitive data types 149
- Hive Query Language (HQL) 151
- horizontal scalability 47–48
- Hortonworks data platform 152
- human-generated data 8–9
- hybrid cloud 95
- hypervisor 91

i

- industries, outlier detection 284
- infrastructure as a service (IaaS) 96, 102
- insurance claim fraud detection 283
- internal cloud 95
- interval data 171
- intra-cluster distances 260
- Intrusion detection 283
- inverted index 129
- isolation (I) 56
- isolation technique 92
- itemset mining
 - apriori algorithm

- frequent itemset generation 217–219
 - implementation of 212–217
- Charm algorithm
 - implementation 236–239
 - rules of 236
- confidence 202
- Equivalence Class Transformation (Eclat)
 - algorithm
 - implementation of 223–225
 - vertical data layout 222–223
- FP growth algorithm
 - FP trees 227–229
 - frequency of occurrence 225–226
 - order items 227
 - prioritize items 226–227
- frequency of item 203–206
- frequent itemset 202–203
- GenMax algorithm
 - frequent itemsets with tidset 235
 - implementation. 235
 - minimum support count 234
- itemset frequency 202
- market basket data 202
- maximal and closed frequent itemset 232
 - corresponding support count. 231
 - subsets of frequent itemset 232
 - support count 230–231
 - transaction 230
 - transaction database 234
- support 202
- support of transaction 203
- in transaction 203

j

- JobTracker 115, 122–123, 131
- joint probability distribution 242

k

- Kernel density estimation
 - artificial neural network 251–253
 - biological neural network 253–254
 - mining data streams 254–255
 - time series forecasting 255–257
- Kernel K-means clustering
 - 270–273

- key-value store database
 - Amazon DynamoDB 61
 - Microsoft Azure Table Storage 62
 - schematic illustration 60, 61
- KeyValueTextInputFormat 124
- K-means algorithms 267–270
- K-means clustering 289
- K-nearest neighbor algorithm 245–246

l

- lexical analysis 177
- linearly separable clusters 272
- line charts 344–345
- load-balancing clusters 34–35

m

- machine-generated data 8–9
- machine learning
 - clustering technique 195–196
 - customer churn prevention 189
 - customer segmentation 189
 - decision-making capabilities 187
 - face recognition 188
 - fraud detection 188
 - general algorithm 187
 - pattern recognition 187
 - product recommendation 188
 - sentiment analysis 188–189
 - spam detection 188
 - speech recognition 188
 - supervised (*see* supervised machine learning)
 - types of data sets 188
 - understanding and decision-making 187
 - unsupervised 194–195
- Mahout 146
- Manhattan distance 261–262
- MapR 152
- MapReduce 12
 - combiner 120–121
 - description 119
 - example 125–126
 - indexing technique 129
 - input formats 123–124

- JobTracker 122–123
- limitations of 129
- mapper 119–120
- processing 126–128
- programs 31
- reducer 121
- TaskTracker 122–123
- market basket data 208
- marketing 239–240, 259
- master data 180–181
- master-slave model 40, 41
- MemSQL 46
- MemStore 140
- Microsoft Azure Table Storage 62
- mining data streams 254–255
- multidimensional online analytical processing (MOLAP) 179

n

- NameNode 115–117, 129–131
- narrative analysis 171
- natural language generation (NLG) 176
- natural language processing (NLP) 175–177
- natural language understanding (NLU) 176
- negative correlation 172–173
- Neo4J 66
- NewSQL databases 46
- NLineInputFormat 124
- NodeManager 133–135
 - failure 137–138
- nodes 32
- nominal data 170
- non-relational databases 45
- non-uniform memory access architecture 86
- NoSQL (Not Only SQL) databases 45, 46, 53
 - ACID 56
 - advantages 77
 - BASE 56–57
 - CAP theorem 54–56
 - distributed computing 60
 - features of 59–60
 - handling massive data growth 60

NoSQL (Not Only SQL) databases (*cont'd*)

- horizontal scalability 59
- lower cost 60
- operations
 - create collection 73–74
 - create database 72–73
 - delete document 75–76
 - drop collection 74
 - drop database 73
 - insert document 74–75
 - query document 76
 - update document 75
- vs. RDBMS 58, 59
- schemaless databases 57, 59
- types of 60–72
- n-tier architecture 84
- NuoDB 46

O

- online retailers 259
- online retails 259
- on-premise private cloud 95
- Oozie 146–147
 - bundles 149
 - coordinators 148–149
 - job types 147
 - workflow 147–148
- operational support services (OSS) 103
- optimization algorithm
 - particle swarm algorithm 285, 287
 - random positions and random velocity
 - vectors 286
- ordinal data 170
- organizational data 8
- outlier detection techniques 281

P

- parallel computing 89–90
- parser 152
- parsing 177
- partitional clustering 267
- partition clustering algorithm 262
- partitioning technique 92

- partition tolerance 54
- patient portals 7
- pattern recognition 187, 259
- Pearson product moment correlation 174
- peer-to-peer architecture 84
- peer-to-peer model 40–42
- pie charts 341–342
- Pig Latin 145, 146
- plan executor 152
- platform as a service (PaaS)
 - 96, 102
- point outlier 279
- positive correlation 172, 173
- pragmatic analysis 177
- prediction 240–241
- predictive analytics 165
- prescriptive analytics 165–166
- private cloud 95
- probability distribution 242
- product recommendation 188
- protein patterns 266
- proximity-based method 283
- proximity sensors 7
- public cloud 94–95

Q

- qualitative analysis 171
- quantitative analysis 170–171

R**r**

- control structures in
 - break 341
 - if and else 337–338
 - for loops 339–340
 - nested if-else 338
 - while loops 340
- data structures in
 - arrays 327–328
 - coercion 322–323
 - data frames 329–332
 - length, mean, and median
 - 323–324

- lists 332–335
- matrix() function 324–327
- naming arrays 328–329
- vector 321–322
- installation
 - basic commands 320
 - R Studio interface on windows 319
 - value, assigning of 320
- random load balancing 35
- random variable 241–242
- ratio data 171
- real-time analytics platform (RTAP) 180
- real-time analytics processing 180–181
- real-time data processing 88–89
- records 43
- reference data 181
- regression technique 174–175
- Relational Database Management Systems (RDBMS) 3, 45
 - and big data, attributes of 3, 4
 - drawbacks 54
 - life cycle 55
 - migration to NoSQL 76–77
 - vs. NoSQL databases 58, 59
- relational databases 43, 45
- relational online analytical processing (ROLAP) 179
- ResourceManager 132–133
 - failure 137
- retailers 240
- round robin load balancing 35

S

- scalability 47
- Scalability of Hadoop 11, 111
- scaling-out storage platforms 47–48
- scaling-up storage platforms 47
- scatterplots 346
- schemaless databases 57, 59
- searching algorithm 128–129
- searching and retrieval process 177
- semantic analysis 177
 - natural language processing 175–177

- sentiment analysis 177
 - text analytics 177
- semi-structured data 6, 10
- semi-supervised outlier detection 282
- sentiment analysis 177, 188–189
- SequenceFileAsTextInputFormat 124
- SequenceFileInputFormat 124
- server virtualization 92
- sharding 37–39
- sharding and replication, combination of 41–42
- shared everything architecture
 - description 85
 - distributed shared memory 86
 - symmetric multiprocessing architecture 86
- shared-nothing architecture 86, 87
- soft clustering 274
- software as a service (SaaS) 95–96, 102
- sorting algorithm 128
- source data identification 166–167
- spam detection 188
- speech recognition 188
- split testing 172
- SQOOP (SQL to Hadoop) 141–143
- statistical analysis techniques
 - A/B testing 172
 - correlation 172–174
 - regression 174–175
- statistical method 283
- streaming computing 180
- structured data 6, 9, 10
- student course registration database 43, 44
- supervised machine learning
 - classification 190–191
 - regression technique 191–192
 - support vector machines 192–194
- supervised outlier detection 282
- support vector machines 192–194
- symmetric clusters 35, 36
- symmetric multiprocessing architecture 86
- syntactic analysis 177

t

Tableau

- airlines data set 313–314
- bar charts 309–310
- box plot 313
- bubble chart 312
- connecting to data 300
 - in Cloud 301
- connect to file 301–306
- earthquakes and frequency 317–318
- histogram 308
- line chart 310–311
- office supplies 314–315
- pie chart 311–312
- scatterplot 306–308
- in sports 315–317
- Tableau Desktop 298
- Tableau Online 299
- Tableau Public 298
- Tableau public 298
- Tableau Public Premium 299
- Tableau Reader 299
- Tableau Server 298
- TaskTracker 115, 122–123
- Term Frequency–Inverse Document
 - Frequency (TF-IDF) 128, 129
- text analytics 12, 177
- TextInputFormat 123–124
- text mining 177
- 3D-pie charts 342
- three-tier architecture 84
- time series forecasting 255–257
- traditional relational database, drawbacks
 - of 76–77
- transactional data 180
- two-dimensional electrophoresis 266

u

- uniform memory access 86
- univariate Gaussian distribution 274, 275
- unstructured data 6–7, 9–10
- unsupervised hierarchical clustering 266
- unsupervised machine learning 194–195
- unsupervised outlier detection 282

v

- vertical database 209
- vertical scalability 47
- virtualization
 - attributes of 91–92
 - purpose of 90
 - server virtualization 92
 - system architecture before and after 91
- Virtual Machine Monitor (VMM) 91
- visual analysis 178
- VoltDB 46

w

- web data 8
- weight-based load balancing algorithm 35
- word count algorithm, MapReduce 127, 128
- workflow jobs 147
- write-ahead log (WAL) technique 138, 140

y

- Yet Another Resource Negotiator
 - (YARN) 19, 131, 132
 - core components of 132–135
 - failures 137–138
 - NodeManager 133–135
 - ResourceManager 132–133
 - scheduler 135–136
- YouTube 259

WILEY END USER LICENSE AGREEMENT

Go to www.wiley.com/go/eula to access Wiley's ebook EULA.